

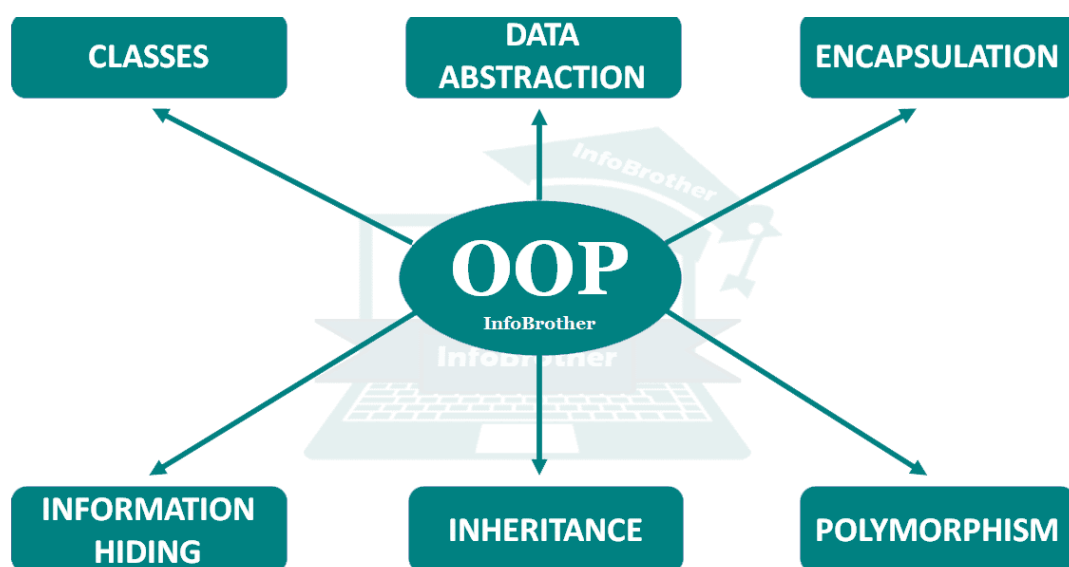
**O‘ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKASIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN’IY INTELLEKT”
KAFEDRASI**



“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN

O‘QUV-USLUBIY MAJMUA



“TASDIQLAYMAN”

O‘R HX va MU AXBOROT-KOMMUNIKATSIYA
TEXNOLOGIYALARI VA HARBIY ALOQA INSTITUTI
BOSHLIG‘INING BIRINCHI O‘RINBOSARI
(O‘QUV VA ILMIY ISHLAR BO‘YICHA)

kapitan

B. To‘rayev

2025-yil “ ____ ” _____

“OBJEKTGA YO'NALTIRILGAN DASTURLASH” FANIDAN

O‘QUV-USLUBIY MAJMUA

“KELISHILGAN”

AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI “AXBOROT TEXNOLOGIYALARI VA SUN‘IY
INTELLEKT” KAFEDRASI BOSHLIG‘I

kapitan

B. Yusupov

2025-yil “ ____ ” _____

AKT VA HAI “AXBOROT TEXNOLOGIYALARI VA SUN‘IY
INTELLEKT” KAFEDRASI “AXBOROT TEXNOLOGIYALARI” SIKLI
KATTA O‘QITUVCHISI

katta leytenant

A. Fayzullayev

2025-yil “ ____ ” _____

Tuzuvchilar:

- | | |
|-----------------------------------|---|
| kapitan
B.K. Yusupov | – PhD, professor, Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti, axborot texnologiyalari va sun'iy intellekt kafedrası boshlig'i, PhD, professori; |
| kapitan
A. Komilov | – Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti, axborot texnologiyalari va sun'iy intellekt kafedrası axborot texnologiyalari sikli boshlig'i. |
| katta leytenant
A. Fayzullayev | – Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti, axborot texnologiyalari va sun'iy intellekt kafedrası axborot texnologiyalari sikli katta o'qituvchisi. |

Taqrizchilar:

- | | |
|--------------------------------|--|
| podpolkovnik
A. Mulladjanov | – O'R QK Bosh shtabi AAT va AH BB Axborot-kommunikatsiya texnologiyalarini rivojlantirish boshqarmasi boshlig'i; |
| podpolkovnik
O. Abdiroziqov | – O'R HX va MU "Istiqbolli harbiy texnologiyalar" kafedrası boshlig'i, PhD, dotsent. |

O'quv-uslubiy majmua Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti "Axborot texnologiyalari va sun'iy intellekt" kafedrasining 2025-yil 15-avgust kunidagi majlisida ko'rib chiqilingan va 2-sonli bayonnomasi bilan o'quv jarayoniga joriy etishga ruxsat berilgan.

№

M U N D A R I J A

1.	Kirish.....	6
2.	Fan bo'yicha o'qitiladigan materiallar to'plami.....	9
3.	Fan bo'yicha mashg'ulotlar o'tish rejalari.....	178
4.	Glossariy.....	211
5.	Tarqatma materiallar to'plami (1-ilova).....	219
6.	Fan bo'yicha yakuniy nazorat qabul qilish uchun uslubiy ishlanma (2-ilova).....	235
8.	Fanni o'qitishda foydalaniladigan interfaol ta'lim metodlari (3-ilova).	241
9.	Fanning o'quv dasturi.....	244
10.	Fanning ishchi o'quv dasturi.....	253
11.	Asosiy va qo'shimcha adabiyotlar va axborot manbalari (4-ilova).....	276

KIRISH

Ushbu “OBYEKTga yo‘naltirilgan dasturlash” faniga tegishli bo‘lgan barcha mavzular kursantlarga Davlat ta‘lim standartlari asosida yetkazilishi shart bo‘lgan asosiy bilimlar va ko‘nikmalarni to‘la qamrab olingan. Ushbu fandagi mavzular kursantlarga komputer kerakli bo‘lgan barcha turdagi dasturlarni yaratish imkoniyatini beruvchi bilimlarni nazariy va amaliy jihatdan o‘rganishlariga qaratilgan.

Fanning maqsad va vazifalari

Fanning asosiy vazifasi kursantlarni axborot texnologiyalari vositalaridan samarali foydalana oladigan yuqori malakali aloqachi ofitserlarini tayyorlashga yo‘naltirilgan bo‘lib, bunda kursantlarda OBYEKTga yo‘naltirilgan dasturlash bo‘yicha nazariy va amaliy ko‘nikmalarini hosil qilishdan iborat.

Fanning maqsadi:

“OBYEKTga yo‘naltirilgan dasturlash” fanining asosiy maqsadi qo‘shilma va harbiy qismlarda avtomatlashtirishning qulay vositalaridan kompyuter texnologiyalaridan keng foydalangan holda dasturlashda ma‘lumotlarning turli tarkibiy tuzilmalarini tadbqiq etish. C# dasturlash tilning sodda va murakkab tuzilmalarini qo‘llash. Algoritmnlarni baholash. Qo‘yilgan masalani yechish algoritmini tanlash, tanlovni asoslash, algoritmnini tadbqiq etish. C# dasturlash tilining asosiy konstruksiyalari, ma‘lumotlar toifalari va tuzilmalarini qo‘llash. C# dasturlash tilida aniq algoritmnlarni tadbqiq etish. C# dasturlash tilining statik va dinamik imkoniyatlarni tadbqiq etish.

Fanning asosiy vazifasi kursantlar tomonidan yangi avlod kompyuter tarmoqlarini loyihalash bosqichlari va usullari to‘g‘risida ma‘lumotga ega bo‘lgan, turli hil mavjud maxsus dasturiy ta‘minotlar yordamida tarmoqni loyihalashning amaliy ko‘nikmasiga ega, shuningdek OBYEKTga yo‘naltirilgan dasturlash asoslarini qo‘llash. Dasturiy ishlanmalarni ishlab chiqarish jarayonini ta‘minlash. Ma‘lumotlarning dinamik informasion tuzilmasini tadbqiq etish. Aniq loyihalarni ishlab chiqish va tadbqiq etish. Dasturiy ishlanmalarni testlash va sozlash. Murakkab dasturiy tizimnlarni tashkil etish. Natijalarni tahlil qilish imkoniyatiga ega bo‘lgan ofitserlarni tayyorlashdan iboratdir.

Fanni o‘qitishdan maqsad – Qurolli Kuchlar tizimida ish faoliyatini to‘liq avtomatlashtirilgan tizimga o‘tkazish. OBYEKTga yo‘naltirilgan dasturlash asosida apparat-dasturiy ishlanmalarni yaratish va uni Qurolli Kuchlar tizimiga tadbqiq qilish.

Fanning vazifasi – OBYEKTga yo‘naltirilgan dasturlashga oid bilim va malakalarni shakllantirish, zamonaviy dasturlash tillarni foydalangan holda mikroprotssesorlarda qo‘llash.

Fan bo‘yicha kursantlarning tasavvur, bilim, ko‘nikma va malakalariga qo‘yiladigan talablar

Fanni o‘rganish natijasida kursantlar quyidagi tushunchalarga ega bo‘lishi lozim:

Quyidagi tasavvurga ega bo‘lishi lozim:

- C# dastrulash tilida asosiy tushunchalarini;
- Shakllar bilan ishlash qoidalar va imkoniyatlari;
- Windows Forms konteynerlari haqida tushuncha;
- TabControl va SplitContainer tab paneli;
- Boshqaruv elementlari haqida tushuncha;

Quyidagi ko‘nikma va malakalarini egallashi lozim:

- Winformda komputer uchun dasturlarni dizaynini yaratish;
- Winformda dastur strukturasini yaratish;
- Winformda dastur strukturasini va dizaynini bog‘lash.

Quyidagi bilim va ko‘nikmalarga ega bo‘lishi kerak:

- Dasturiy ta‘minot yaratish, o‘rnatish va sozlash usullarini ;
- Dasturning mavjud bo‘lishi mumkin bo‘lgan xatoliklarni bartaraf etish;

Bulardan tashqari harbiy sohada ishonchli dasturlarni yaratish uchun foydalaniladigan dasturiy va apparat vositalarini yaxshi bilishi, loyihalay olishi va takomillashtirish bilimlarini egallashi talab qilinadi.

Fanni o‘qitishda zamonaviy axborot va pedagogik texnologiyalar

O‘quv jarayoni bilan bog‘liq ta‘lim sifatini belgilovchi holatlar quyidagilar: yuqori ilmiy-pedagogik darajada dars berish, muammoli ma‘ruzalar o‘qish, darslarni savol-javob tarzida qiziqarli tashkil qilish, ilg‘or pedagogik texnologiyalardan va mul’timedia vositalaridan foydalanish, kursantlarni bilim olishga undaydigan, o‘ylantiradigan muammolarni, ular oldiga qo‘yish, talabchanlik, kursantlar bilan individual ishlash, erkin muloqot yuritishga, ilmiy izlanishga jalb qilish. Kursantlarning “Web texnologiyalar” fanini o‘zlashtirishlari uchun o‘qitishning zamonaviy va ilg‘or usullaridan foydalanish, yangi axborot–pedagogik texnologiyalarini tadbiq qilish muhim ahamiyatga egadir. Fanni o‘zlashtirishda o‘quv qo‘llanmalar, ma‘ruza matnlari, tarqatma materiallar, elektron materiallardan foydalaniladi. Ma‘ruza va amaliy mashg‘ulotlarida mos ravishdagi zamonaviy pedagogik va axborot texnologiyalaridan foydalaniladi.

Mashg‘ulotlarning asosiy qismi amaliy mashg‘ulotlardan iborat bo‘lib, ularda

o‘quv elementlarni mazmunlari va mavzularni bir-birlariga mantiqiy bog‘liqligi o‘rganiladi. Mashg‘ulotlarda kursantlar har bir element va unga qo‘yilgan o‘quv savollari bo‘yicha mustaqil o‘rganib, o‘qituvchi yordamida bilim va ko‘nikmalarini takomillashtirishlari lozim. Mashg‘ulotlarni asosiy turlari zamonaviy pedagogik va innovatsion ta’lim texnologiyalari va interaktiv dars shakllarini qo‘llagan holda olib borilishi va unda kursantlarni faol qatnashuvi fanni o‘rganishning asosiy omillaridan biri hisoblanadi.

**O'ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT”
KAFEDRASI**



“OBJEKTGA YO'NALTIRILGAN DASTURLASH”

**FANI BO'YICHA O'QITILADIGAN
MATERIALLAR TO'PLAMI**

7-MAVZU: C# DASTURLASH TILIDA TARMOQ NAZARIYASI**1-Mashg'ulot: C# va .NET da tarmoq bilan ishlash asoslari.****Reja:**

1. Tarmoq haqida tushuncha va tarmoq protokollari.
2. .NET da adreslar haqida tushuncha(IP adres, URI).
3. Addreslash sxemasi.
4. URI adresslar haqida tushuncha.

Tarmoq haqida tushuncha va tarmoq protokollari

Bugungi kunda millionlab kompyuterlar va qurilmalar global internetga yoki alohida mahalliy pastki tarmoqlarga ulangan. Shu munosabat bilan, tarmoq orqali ma'lumotlarni uzatishning barcha afzalliklaridan foydalanadigan dasturlarni yaratish zarurati tug'iladi. Masalan, tarmoq orqali uzatishni ishlatadigan keng tarqalgan dasturlardan biri bu veb-brauzer. . Net platformasi ham, C# dasturlash tili ham tarmoq orqali o'zaro aloqada bo'lishi va turli xil tarmoq protokollaridan foydalanishi mumkin bo'lgan dasturlarni yaratish uchun barcha kerakli imkoniyatlarni taqdim etadi.

Ammo to'g'ridan-to'g'ri dasturlarni yaratishga o'tishdan oldin, tarmoqdagi aloqa nima ekanligini bir necha so'z bilan aytish kerak.

Butun tarmoq alohida elementlardan iborat - kompyuterlar va boshqa ulangan qurilmalar bo'lgan xostlar. Ular bir-biri bilan aloqa kanallari (Ethernet kabellari, Wi-Fi va boshqalar) va marshrutizatorlar bilan bog'langan. Routerlar kompyuterlarni pastki tarmoqqa birlashtiradi va ular orasidagi ma'lumotlarni uzatishni boshqaradi.

Ammo xost kompyuterlar bir-biri bilan o'zaro ta'sir qilmaydi. Ular protokollarni qo'llaydilar. Protokol-bu ma'lumotlar paketlari aloqa kanallari orqali qanday uzatilishi haqidagi konvensiyalar. Shunday qilib, protokol o'zaro ta'sirni tartibga soladi.

Turli xil protokollar mavjud. Tarmoq orqali ma'lumotlarni uzatish uchun ishlatiladigan protokollar TCP / IP protokollari oilasini tashkil qiladi. Ulardan asosiylari: Internet Protocol (IP), Transmission Control Protocol (TCP) va User Datagram Protocol (UDP). Bundan tashqari, ushbu protokollar qatlam tizimida tashkil etilgan:

IP tarmoq qatlamini ifodalaydi. U ma'lumotlar paketlarini boshqa xostga uzatish uchun jismoniy aloqa kanallari - Ethernet kabellari va boshqalarni ifodalovchi asosiy qatlamlardan foydalanadi.

IP - dan yuqorida TCP va UDP protokollari tashkil etadigan transport qatlami joylashgan. Ushbu protokollar ma'lumotlarni uzatish uchun ma'lum portlardan foydalanadi. TCP sizga paketlarning yo'qolishini va uzatishda ularning takrorlanishini kuzatishga imkon beradi. UDP buni amalga oshirishga imkon bermaydi va oddiy ma'lumotlarni uzatishga qaratilgan.

Biroq, dastur TCP / UDP qatlami bilan to'g'ridan-to'g'ri emas, balki rozetkalar taqdim etadigan maxsus API orqali o'zaro ta'sir qiladi. Soketlar hech qanday protokol

emas, shunchaki operatsion tizimning o'rnatilgan imkoniyatlariga tayanadigan tarmoq ilovalarini yaratish interfeysi.

Amaldagi protokolga qarab, rozetkalarining ikki turi mavjud: oqim rozetkalari (oqim rozetkalari) va Datagram rozetkalari (datagram rozetkalari). Oqim rozetkalari TCP protokolidan, Datagram - UDP protokolidan foydalanadi.

Natijada, dastur boshqa xostda ishlaydigan dasturga xabar yuborganda, dastur TCP / UDP darajasiga ma'lumotlarni uzatish uchun rozetkalarga kiradi. Bundan tashqari, ushbu transport qatlamidan ma'lumotlar tarmoq qatlamiga - IP protokoli qatlamiga uzatiladi. Va bu protokol ma'lumotlarni jismoniy qatlamlarga uzatadi va shundan so'ng ma'lumotlar tarmoq orqali o'tadi.

Tarmoqdagi xostlarni noyob tarzda aniqlash uchun har bir xostda manzil mavjud. Bir nechta turli xil manzil protokollari mavjud. Hozirgi vaqtda eng keng tarqalgan IPv4 protokoli bo'lib, u manzilni 32 bitli raqam sifatida taqdim etishni o'z ichiga oladi, masalan, 37.120.16.63. Bunday manzil nuqta bilan ajratilgan to'rtta raqamni o'z ichiga oladi va har bir raqam 0 dan 255 gacha. Shu bilan birga, so'nggi paytlarda 128 bitli qiymat bo'lgan IPv6 protokoli manzillaridan foydalanish ham aylanmoqda.

Biroq, bunday manzillarni eslab qolish juda qiyin, shuning uchun aslida ular ko'pincha domenlar bilan ishlaydi. Domenlar Internet manzillari uchun ishlatiladigan maxsus nomlarni ifodalaydi. Masalan, domen nomi bor "www.microsoft.com, unga IPv4 2.23.143.150 formatidagi manzil mos keladi. Ammo o'zaro ta'sir o'tadigan IP protokoli uchun domen manzillari mavjud emas. Shuning uchun, domen nomi orqali ma'lumotlarni yuborish yoki uzatishda kompyuter hali ham IPv4 yoki IPv6 formatidagi Internet manzillari va domen nomlari o'rtasida taqqoslashni amalga oshiradigan Domain Name System (DNS) xizmatlariga murojaat qiladi.

Manzildan tashqari, tarmoq o'zaro ta'sirida portlar ishlatiladi. Port 1 dan 65 535 gacha bo'lgan 16 bitli raqamni ifodalaydi. Portlardan foydalanish bitta xostda bir nechta ishlaydigan dasturlarni ajratish imkonini beradi.

Tarmoqning asosiy tarkibiy qismlari mijoz va serverdir. Mijoz so'rovni yuboradi va server so'rovni qabul qiladi, uni qayta ishlaydi va mijozga ba'zi javoblarni yuboradi. Eng oddiy misol-bu veb-brauzer bo'lib, u ma'lum bir saytga so'rov yuborish orqali mijoz sifatida xizmat qiladi. Va sayt server vazifasini bajaradi, brauzerga ba'zi javoblarni yuboradi, keyin brauzer foydalanuvchiga ko'rsatadi. Biroq, aslida, ko'pincha bitta dastur server sifatida ham, mijoz sifatida ham harakat qilishi mumkin.

Aslida, bularning barchasi siz bilishingiz kerak bo'lgan tarmoq orqali o'zaro ta'sirning asosiy printsiplari. Aslida, qoida tariqasida, dasturlarni yaratishda siz ularning barcha protokollari va nuanslarini chuqur bilishingiz shart emas. Agar kamdan-kam hollarda protokollarni batafsilroq bilish zarurati tug'ilsa, unda bu holda siz ixtisoslashgan adabiyotlarga murojaat qilishingiz mumkin, xuddi shu qo'llanmada

biz to'g'ridan-to'g'ri.net ramkasi tarmoq bilan ishlash uchun taqdim etadigan imkoniyatlarga e'tibor qaratamiz.

Tarmoq bilan ishlash uchun. net ramkasining asosiy funktsionalligi paketda mavjud System.Net. qo'shimcha paketlar ham mavjud:

System. Net. Http: HTTP protokoli bilan ishlash funktsiyasini o'z ichiga oladi

System. Net. NetworkInformation: tarmoq trafigi va tarmoq manzillari to'g'risidagi ma'lumotlarga, shuningdek tarmoq xostlari haqidagi boshqa ma'lumotlarga kirishni ta'minlaydi. Shuningdek, Ping funktsiyasini taqdim etadi

System. Net. Security: xostlar o'rtasida xavfsiz aloqa o'rnatish uchun tarmoq oqimlarini ta'minlaydi

System. Net. Sockets: operatsion tizim rozetkalarining funktsionalligiga kirishni ta'minlaydi

System.Net. WebSockets: Websocket infterface dasturiga kirishni ta'minlaydi

System. Net. Quic: RFC 9000 spetsifikatsiyasiga muvofiq QUIC protokolini amalga oshiradigan turlarni o'z ichiga oladi.

.NET da adreslar haqida tushuncha(IP adres, URI).

Tarmoqdagi ma'lum bir qurilmaga xabar yuborish uchun ushbu qurilma noyob manzilga ega bo'lishi kerak. Ip-manzil noyob manzil sifatida ishlaydi. Bir nechta turli xil manzil protokollari mavjud. Hozirgi vaqtda eng keng tarqalgan IPv4 protokoli bo'lib, u manzilni 32 bitli raqam sifatida taqdim etishni o'z ichiga oladi, masalan, 37.120.16.63. Bunday manzil nuqta bilan ajratilgan to'rtta raqamni o'z ichiga oladi va har bir raqam 0 dan 255 gacha. Shu bilan birga, so'nggi paytlarda 128 bitli qiymat bo'lgan IPv6 protokoli manzillaridan foydalanish ham aylanmoqda, masalan, 2001:0db8:85a3:0000:0000:8a2e:0370:7334 (IPv6 formati)..

.Net sinf tizimida ip-manzil kosmosdan IP-manzil sinfi bilan ifodalanadi System.Net. ip-manzilni aniqlash uchun siz ushbu sinfning bir qator dizaynerlaridan foydalanishingiz mumkin. Ulardan ba'zilar:

```
public IPAddress (byte[] address);
public IPAddress (long newAddress);
```

Birinchi konstruktor manzil ta'riflarini baytlar qatori sifatida qabul qiladi, ikkinchisi esa 0 dan 0x00000000ffffffgacha bo'lgan uzun qiymat sifatida qabul qiladi. Ilova:

```
using System.Net;

IPAddress localIp = new IPAddress(new byte[] { 127, 0, 0, 1 });
Console.WriteLine(localIp); // 127.0.0.1

IPAddress someIp = new IPAddress(0x0100007F);
Console.WriteLine(someIp); // 127.0.0.1
```

Long qiymatidan foydalanganda big-endian formati qo'llaniladi, ya'ni, taxminan aytganda, manzil qiymatlari oxiridan aniqlanadi. Masalan, 0x0100007f

qiymati " 7F " (o‘nlik tizimda 127) manzilning birinchi segmentini, 00 va 00 ikkinchi va uchinchi segmentlarni va 01 to‘rtinchi segmentni belgilaydi. Shunday qilib, biz "0x0100007F" dan 127.0.0.1 manzilini olamiz.

IPAddress.Parse va IPAddress.TryParse

Yana bir va qulayroq usul parse () usulini ifodalaydi, u IPv4 yoki IPv6 formatidagi manzilni satr sifatida ifodalaydi va ipaddress ob’ektini qaytaradi:

```
IPAddress ip = IPAddress.Parse("127.0.0.1");
```

Biroq, satrda xatolar bo‘lishi mumkin va IP manziliga o‘tish muvaffaqiyatsiz bo‘lishi mumkin. Bunday holda, istisno paydo bo‘ladi. Bunga yo‘l qo‘ymaslik uchun biz TryParse () usulidan foydalanishimiz mumkin:

```
using System.Net;

IPAddress.TryParse("127.0.0.1rrr", out IPAddress? ip);
Console.WriteLine(ip?.ToString());
```

Agar konvertatsiya muvaffaqiyatsiz bo‘lsa, unda IPAddress.TryParse () false ni qaytaradi va chiqish parametri null bo‘ladi.

O‘rnatilgan manzillar

Shuningdek, ipaddress klassi bir qator statik xususiyatlar orqali bir qator standart manzillarni taqdim etadi:

Statik loopback xususiyati: 127.0.0.1 manzili uchun IPAddress ob’ektini qaytaradi

Statik xususiyat har qanday: 0.0.0.0 manzili uchun ipaddress ob’ektini qaytaradi

Broadcast statik xususiyati: 255.255.255.255 manzili uchun IPAddress ob’ektini qaytaradi

```
IPAddress anyIp = IPAddress.Any;
IPAddress localIp = IPAddress.Loopback;
IPAddress broadcastIp = IPAddress.Broadcast;
```

So‘nggi maqolada ip-manzillar ko‘rib chiqildi. Biroq, bunday manzillarni haqiqiy hayotda ishlatish qiyin, masalan, ularni eslab qolish qiyin. Tarmoqdagi qurilmalarni aniqlash va ularga murojaat qilishni osonlashtirish uchun URI/URL ham ishlatiladi.

Uniform Resource Identifier (URI) tarmoqdagi resursni global va noyob tarzda aniqlaydigan qatorni taqdim etadi

URI ning kichik turi - Uniform Resource Locator (URL) - tarmoqdagi resurslarni joylashtirishni aniqlash uchun universal standart. Biroq, aslida, URI va URL ko‘pincha bir-birini aniqlaydi va bir-birining o‘rnida ishlatiladi. Masalan, satr <https://metanit.com> ushbu saytga kirishingiz mumkin bo‘lgan URL manzilini taqdim etadi. Bunday manzillarni IP-manzillarga qaraganda eslab qolish va ulardan foydalanish osonroq.

URL qaysi qismlardan iboratligini ko'rib chiqing. URL manzilining umumiy shakli quyidagicha:

```
scheme:[//authority/]path[?query] [#fragment]
```

authority domen va portni, shuningdek foydalanuvchi nomi va parol sifatida mumkin bo'lgan foydalanuvchi ma'lumotlarini o'z ichiga oladi:

```
//[access_credentials][@]host_domain[:port]
```

@ Belgisi hisob ma'lumotlarini domendan ajratib turadi. Hisob ma'lumotlarini tavsiflovchi access_credentials parametri quyidagicha berilgan

```
[user_id][:][password]
```

User_id parametri loginni, parol esa manbaga kirish uchun parolni taqdim etadi. Ular bir-biridan ikki nuqta bilan ajralib turadi :

Query komponenti quyidagi shaklga ega:

```
?[parameter1=value2][(;|&)parameter2=value2]...
```

Oxirida # panjara belgisidan keyin fragment ko'rsatilishi mumkin-URL ichidagi komponentlarni aniqlash uchun ixtiyoriy qator. Odatda veb-brauzerlarda veb-sahifaning qismlarini navigatsiya qilish uchun qo'llaniladi.

Ammo o'zaro ta'sir o'tadigan IP protokoli uchun URI manzillari mavjud emas. Shuning uchun, domen nomi orqali ma'lumotlarni yuborish yoki uzatishda kompyuter hali ham IPv4 yoki IPv6 formatidagi Internet manzillari va domen nomlari o'rtasida taqqoslashni amalga oshiradigan domen nomlari tizimiga yoki domen nomlari tizimiga (DNS) kiradi.

System.Uri

Tizim sinfi. net-da URI manzillarini taqdim etish uchun javobgardir.Uri. Ushbu sinfdan bir qator dizaynerlar mavjud. Eng sodda parametr sifatida resursning satr manzilini oladi.

Addrash sxemasi.

Ippaddress xususiyatlari orasida manzil sxemasini ko'rsatadigan manzil oilasi xususiyatini ta'kidlash kerak. Ushbu xususiyat quyidagi qiymatlarni qabul qilishi mumkin bo'lgan manzil oilasining ro'yxatini anglatadi:

AppleTalk: AppleTalk manzili

Atm: ATM xizmatlari manzili

Banyan: Banyan manzili

Ccitt: CCITT protokollari manzillari, masalan, X25 protokoli

Chaos: MIT CHAOS protokollari manzili

Cluster: Microsoft kompaniyasining klaster mahsulotlari manzili

ControllerAreaNetwork: Kontroller hududi tarmoq manzili

DataKit: Datakit protokollari manzili

DataLink: To'g'ridan-to'g'ri ma'lumot uzatish interfeysi manzili

DecNet: DECnet manzili

Ecma: ECMA (European Computer Manufacturers Association — Yevropa Kompyuter Ishlab Chiqaruvchilari Assotsiatsiyasi) manzili

FireFox: FireFox manzili

HyperChannel: NSC Hyperchannel manzili

Ieee12844: IEEE 12844 ishchi guruhi manzili

ImpLink: ARPANET IMP manzili

InterNetwork: IPv4 manzili

InterNetworkV6: IPv6 manzili

Ipx: IPX yoki SPX manzili

Irda: IrDA manzili

Iso: ISO protokollari manzili

Lat: LAT manzili

Max: MAX manzili

NetBios: NetBios manzili

NetworkDesigners: Network Designers OSI shlyuz protokollari manzili

NS: Xerox NS protokollari manzili

Osi: OSI protokollari manzili

Packet: Pastki darajadagi paket manzili

Pup: PUP protokollari manzili

Sna: IBM SNA manzili

Unix: Unix uchun lokal manzil

Unknown: Noma'lum manzillar oilasi

Unspecified: Belgilanmagan manzillar oilasi

VoiceView: VoiceView manzili

URI adresslar haqida tushuncha.

System.Uri — C# tilidagi .NET kutubxonasida mavjud bo'lgan sinf bo'lib, u URL yoki URI (Uniform Resource Identifier) bilan ishlash uchun mo'ljallangan. Uri sinfi veb-sahifalar, fayllar, va boshqa resurslar manzillarini o'qish, tekshirish, va tahlil qilish imkonini beradi.

Uri yaratilishi: Uri obyekti yaratilganda, u qabul qilingan manzilni analiz qilib, tegishli qismlarga ajratadi. Manzilning turli qismlari (protokol, host, port, yo'l va hokazo) obyektidan osonlik bilan olinishi mumkin.

Uri	myUri	=	new
Uri("https://www.example.com:8080/path?query=value#fragment");			

Uri qismlari: Uri sinfi quyidagi qismlarni ajratib beradi:

Scheme: Protokol nomi (masalan, http, https, ftp, mailto, va hokazo).

Host: Domen nomi yoki IP manzili.

Port: Port raqami (agar ko'rsatilgan bo'lsa).

Path: Resursning joylashuvi (yo'l).

Query: So‘rov qismi, parametrlar.

Fragment: URL ning oxiridagi # dan keyin keladigan qism.

```
Console.WriteLine(myUri.Scheme); // https
Console.WriteLine(myUri.Host);    // www.example.com
Console.WriteLine(myUri.Port);    // 8080
Console.WriteLine(myUri.AbsolutePath); // /path
Console.WriteLine(myUri.Query);   // ?query=value
Console.WriteLine(myUri.Fragment); // #fragment
```

AbsoluteUri: Butun URL (https://www.example.com/path).

IsAbsoluteUri: URI absolut ekanligini tekshiradi.

RelativeUri: Nisbiy manzil (masalan, /path

```
Uri baseUri = new Uri("https://www.example.com");
Uri relativeUri = new Uri("/path", UriKind.Relative);
Uri fullUri = new Uri(baseUri, relativeUri);
Console.WriteLine(fullUri); // https://www.example.com/path
```


8-MAVZU: C# DASTURLASH TILIDA TARMOQ AMALIYOTI KOMPONENTALARI

1-Mashg‘ulot: Tarmoq konfiguratsiyasi va Soket klassi.

Reja:

1. DNS.
2. Tarmoq konfiguratsiyasi va tarmoq trafigi haqida ma’lumot olish.
3. Soket klassi.

DNS

C# dasturlash tilida DNS (Domain Name System) bilan ishlash, asosan System.Net namespacesdagi Dns klassi orqali amalga oshiriladi. Ushbu klass orqali siz domen nomlarini IP manzillarga aylantirish (DNS lookup) va teskarisini amalga oshirishingiz (reverse DNS lookup) mumkin. Quyida DNS bilan ishlashning asosiy misollari keltirilgan:

1. Domen nomini IP manziliga aylantirish (DNS Lookup)

Domen nomini uning IP manziliga aylantirish uchun Dns.GetHostAddresses metodidan foydalaniladi. Quyida example.com domenining IP manzilini qanday qilib aniqlash mumkinligi ko‘rsatilgan:

```
using System;
using System.Net;

class Program
{
    static void Main()
    {
        string hostName = "example.com";
        IPHostEntry host = Dns.GetHostEntry(hostName);

        Console.WriteLine($"{hostName} domeni uchun IP manzillar:");
        foreach (IPAddress address in host.AddressList)
        {
            Console.WriteLine(address);
        }
    }
}
```

2. IP manzilini domen nomiga aylantirish (Reverse DNS Lookup)

IP manzilidan unga mos keladigan domen nomini aniqlash uchun Dns.GetHostEntry metodidan foydalaniladi. Quyida IP manzilidan domen nomini qanday qilib aniqlash mumkinligi ko‘rsatilgan:

```

using System;
using System.Net;

class Program
{
    static void Main()
    {
        string ipAddress = "93.184.216.34";
        IPEndPoint hostEntry = Dns.GetHostEntry(ipAddress);

        Console.WriteLine($"{ipAddress} manzili uchun domen nomi:
{hostEntry.HostName}");
    }
}

```

3. Lokal mashina haqidagi ma'lumotni olish

Lokal kompyuter yoki server haqidagi DNS ma'lumotlarini olish uchun `Dns.GetHostName` va `Dns.GetHostEntry` metodlaridan foydalanishingiz mumkin:

```

using System;
using System.Net;

class Program
{
    static void Main()
    {
        string localHostName = Dns.GetHostName();
        Console.WriteLine($"Lokal mashina nomi: {localHostName}");

        IPEndPoint localHostEntry = Dns.GetHostEntry(localHostName);
        Console.WriteLine($"Lokal mashina IP manzillari:");
        foreach (IPAddress address in localHostEntry.AddressList)
        {
            Console.WriteLine(address);
        }
    }
}

```

C# dasturlash tilida tarmoq uchun dasturiy ta'minotlar ishlab chiqish jarayonida tarmoq konfiguratsiyasi va tarmoq trafik ma'lumotlarini olish juda muhimdir. Bunday ma'lumotlarni olish uchun .NET Framework'ning `System.Net.NetworkInformation`

namespacedan foydalaniladi. Quyida ushbu mavzuga oid asosiy tushunchalar va ular bilan ishlash bo'yicha misollar keltirilgan:

Tarmoq konfiguratsiyasi va tarmoq trafigi haqida ma'lumot olish.

Tizimdagi barcha tarmoq interfeyslarini va ularning holatlarini ko'rish uchun NetworkInterface klassidan foydalanishingiz mumkin:

```
using System;
using System.Net.NetworkInformation;

class Program
{
    static void Main()
    {
        NetworkInterface[] interfaces =
        NetworkInterface.GetAllNetworkInterfaces();
        foreach (NetworkInterface ni in interfaces)
        {
            Console.WriteLine("Name: " + ni.Name);
            Console.WriteLine("Description: " + ni.Description);
            Console.WriteLine("Status: " + ni.OperationalStatus);
            Console.WriteLine("Speed: " + ni.Speed);
            Console.WriteLine("Network Interface Type: " +
ni.NetworkInterfaceType);
            Console.WriteLine();
        }
    }
}
```

Ip konfiguratsiyasi haqida ma'lumot olish

Har bir tarmoq interfeysi uchun IP konfiguratsiyasini (IP manzil, subnet niqobi, gateway) olish uchun quyidagi kodni ishlatishingiz mumkin:

```
using System;
using System.Net.NetworkInformation;

class Program
{
    static void Main()
    {
        foreach (NetworkInterface ni in
NetworkInterface.GetAllNetworkInterfaces())
        {
```

```

        Console.WriteLine(ni.Name);
        IPInterfaceProperties properties = ni.GetIPProperties();

        foreach (IPAddressInformation uniCast in properties.UnicastAddresses)
        {
            Console.WriteLine("Unicast Address: " + uniCast.Address);
        }

        foreach (GatewayIPAddressInformation gateway in
properties.GatewayAddresses)
        {
            Console.WriteLine("Gateway Address: " + gateway.Address);
        }

        Console.WriteLine();
    }
}

```

Soket klassi.

Tarmoq interfeysining joriy trafik ma'lumotlarini olish uchun IPv4InterfaceStatistics yoki IPv6InterfaceStatistics kabi klasslardan foydalanishingiz mumkin. Quyida IPv4 statistikasini ko'rish misoli keltirilgan:

```

using System;
using System.Net.NetworkInformation;

class Program
{
    static void Main()
    {
        foreach (NetworkInterface ni in
NetworkInterface.GetAllNetworkInterfaces())
        {
            if (ni.OperationalStatus == OperationalStatus.Up)
            {
                Console.WriteLine(ni.Name);
                IPv4InterfaceStatistics stats = ni.GetIPv4Statistics();
                Console.WriteLine("Bytes Received: " + stats.BytesReceived);
                Console.WriteLine("Bytes Sent: " + stats.BytesSent);
                Console.WriteLine();
            }
        }
    }
}

```

```

    }
  }
}

```

C# dasturlash tilida System.Net.Sockets namespace ostidagi Socket klassi tarmoqlar orqali ma’lumot uzatish uchun juda muhimdir. Socket klassi past darajali tarmoq interfeysini taqdim etadi va dasturchilarga TCP, UDP kabi protokollar orqali ma’lumot almashish imkonini beradi.

Socket klassining asosiy xususiyatlari:

Transport protokollari: Socket klassi TCP (Transmission Control Protocol) va UDP (User Datagram Protocol) kabi turli transport protokollarini qo‘llab-quvvatlaydi.

Blocking va Non-blocking rejimlar: Siz socketlarni bloklovchi yoki bloklanmaydigan (asinxron) tarzda ishlata olasiz.

Ma’lumot yuborish va qabul qilish: Ma’lumotlarni yuborish va qabul qilish uchun Send va Receive metodlaridan foydalaniladi.

Qo‘shilish va tinglash: TCP protokoli uchun, server tomonida socketlar Bind va Listen metodlari orqali portni tinglashi va Accept metodi orqali keluvchi ulanishlarni qabul qilishi kerak.

TCP Server va Client Misollari:

TCP Server:

TCP server yaratish uchun, avvalo, Socket obyektini yaratish, uni ma’lum bir portga bog‘lash va ulanishlarni qabul qilish kerak.

```

using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

class TCPServer
{
    public static void Main()
    {
        IPAddress ipAddr = IPAddress.Any;
        IPEndPoint localEndPoint = new IPEndPoint(ipAddr, 11000);
        Socket listener = new Socket(ipAddr.AddressFamily, SocketType.Stream,
        ProtocolType.Tcp);

        try
        {
            listener.Bind(localEndPoint);

```

```

listener.Listen(10);

Console.WriteLine("Waiting for a connection...");
Socket handler = listener.Accept();

string data = null;
byte[] bytes = null;

while (true)
{
    bytes = new byte[1024];
    int bytesRec = handler.Receive(bytes);
    data += Encoding.ASCII.GetString(bytes, 0, bytesRec);
    if (data.IndexOf("<EOF>") > -1)
    {
        break;
    }
}

Console.WriteLine("Text received : {0}", data);
byte[] msg = Encoding.ASCII.GetBytes(data);
handler.Send(msg);
handler.Shutdown(SocketShutdown.Both);
handler.Close();
}
catch (Exception e)
{
    Console.WriteLine(e.ToString());
}
}

```

TCP Client:

TCP client yaratish uchun, Socket obyektini yaratish va serverga ulanish kerak. Keyin esa serverga ma'lumot yuborish va javobini qabul qilish mumkin.

```

using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

```

```

class TCPClient
{
    public static void Main()
    {
        byte[] bytes = new byte[1024];

        try
        {
            IPHostEntry host = Dns.GetHostEntry("localhost");
            IPAddress ipAddress = host.AddressList[0];
            IPEndPoint remoteEP = new IPEndPoint(ipAddress, 11000);

            Socket sender = new Socket(ipAddress.AddressFamily,
SocketType.Stream, ProtocolType.Tcp);

            try
            {
                sender.Connect(remoteEP);
                Console.WriteLine("Socket connected to {0}",
sender.RemoteEndPoint.ToString());

                byte[] msg = Encoding.ASCII.GetBytes("This is a test<EOF>");
                int bytesSent = sender.Send(msg);

                int bytesRec = sender.Receive(bytes);
                Console.WriteLine("Echoed test = {0}",
Encoding.ASCII.GetString(bytes, 0, bytesRec));

                sender.Shutdown(SocketShutdown.Both);
                sender.Close();
            }
            catch (ArgumentNullException ane)
            {
                Console.WriteLine("ArgumentNullException : {0}", ane.ToString());
            }
            catch (SocketException se)
            {
                Console.WriteLine("SocketException : {0}", se.ToString());
            }
        }
    }
}

```

```
        catch (Exception e)
        {
            Console.WriteLine("Unexpected exception : {0}", e.ToString());
        }
    }
    catch (Exception e)
    {
        Console.WriteLine(e.ToString());
    }
}
```

Yuqoridagi misollar yordamida siz C# tilida Socket klassidan foydalanib, TCP protokoli orqali server va klient dasturlarini yozishingiz mumkin. UDP protokoli uchun ham shunga o‘xshash tarzda Socket obyektidan foydalaniladi, faqatgina transport protokoli sifatida ProtocolType.Udp ko‘rsatiladi va ma’lumot almashinuvi datagramlar asosida amalga oshiriladi.

Nazorat savollari

1. C# dasturlash tilida System.Net namespacedan foydalanib, example.com kabi bir domen nomi uchun barcha IP manzillarni qanday qilib topish mumkinligini tushuntiring.
2. System.Net.NetworkInformation.NetworkInterface klassidan foydalanib, tizimdagi barcha tarmoq interfeyslarini va ularning har birining holatini (masalan, faol yoki faol emas) qanday qilib aniqlash mumkinligini tushuntiring
3. C# da Socket klassidan foydalanib, TCP server va TCP client yaratish jarayonini qadamma-qadam tushuntiring. TCP serverning qanday qilib klientlardan keluvchi ulanishlarni qabul qilishi va ma’lumot almashinishi mumkinligini izohlang.

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**2-Mashg‘ulot: HTTP protokoli.****Reja:**

1. HTTP protokoli bilan tanishish.
2. HTTP status kodlari.
3. HttpClient yaratish.

1. HTTP protokoli bilan tanishish.

Ushbu bobda biz HttpClient sinfidan foydalanib so‘rovlarni yuborishni ko‘rib chiqamiz. Ammo boshida biz Http protokoli nimani anglatishini qisqacha ko‘rib chiqamiz. HTTP (Hypertext Transfer Protocol) veb-serverdan resurslarni so‘rash uchun protokolni taqdim etadi. Veb-brauzerda biron bir veb-saytga kirganimizda, biz HTTP protokolidan foydalanamiz.

.Net-da, HTTP protokoli bilan ishlashning asosiy funktsionalligi System.Net.Http nomlarining soddaligiga qaratilgan bo‘lib, unda asosiy sinflar orasida HttpClient sinfi (vab-resurslarga so‘rov yuborish imkonini beruvchi sinf) va veb-server vazifasini bajaradigan HttpListener sinfi mavjud. Ammo ushbu sinflarni ko‘rib chiqishdan oldin siz HTTP protokolining asosiy fikrlarini qisqacha ko‘rib chiqishingiz kerak.

HTTP metodlari:

Mijoz serverga so‘rov yubormoqchi bo‘lganida, ushbu mijoz serverning so‘rovga javob berish usulini aniqlashi kerak. HTTP usuli so‘rovni qayta ishlashda server harakatlarini tavsiflaydi. Ushbu metodlar asosiylari:

OPTIONS: belgilangan URL manzili uchun server qo‘llab-quvvatlaydigan boshqa HTTP usullarining ro‘yxatini qaytaradi

TRACE: server tomonidan qabul qilingan asl so‘rovni takrorlaydigan xizmat usuli. So‘rov yo‘lda bo‘lganida tarmoqdagi ob‘ektlar tomonidan so‘rovga kiritilgan har qanday o‘zgarishlarni aniqlash uchun foydalidir.

CONNECT: TCP/IP tunnelini asl xost va masofaviy xost o‘rtasida o‘rnatish.

GET: HTTP so‘rovi yuborilgan URL manzilidan resurs nusxasini oladi.

HEAD: GET kabi, manba nusxasini URL orqali chiqaradi, faqat javobsiz faqat sarlavhalarni olishni kutadi

POST: serverda yangi manba sifatida saqlash uchun so‘rov tanasida ma’lumotlarni yuborish uchun mo‘ljallangan

PUT: serverda mavjud bo‘lgan resursni o‘zgartirish uchun so‘rov tanasida ma’lumotlarni yuborish uchun mo‘ljallangan

PATCH: serverdagi mavjud resursni qisman o‘zgartirish uchun so‘rov tanasida ma’lumotlarni yuborish uchun mo‘ljallangan

DELETE: belgilangan URL manzilidagi manbani o‘chirish uchun mo‘ljallangan

.Net-da HTTP metodlarini **SYSTEM.Net.HTTP** nom maydonida ifodalash uchun `Http method` klassi aniqlandi. Muayyan turdagi so‘rovlar ilovasida aniqlash uchun sinf ma’lum bir so‘rov turini ifodalovchi `HttpMethod` ob’ektini qaytaradigan bir qator statik xususiyatlarni taqdim etadi:

- `HttpMethod.Delete`
- `HttpMethod.Get`
- `HttpMethod.Head`
- `HttpMethod.Options`
- `HttpMethod.Patch`
- `HttpMethod.Post`
- `HttpMethod.Put`
- `HttpMethod.Trace`

Agar ushbu xususiyatlar bilan qoplanmagan HTTP usuli ishlatilsa, unda usul nomi berilgan konstruktordan foydalanish mumkin:

```
HttpMethod customMethod = new HttpMethod("CUSTOM");
```

Kelajakda biz masofaviy manbalarga so‘rovlar yuborishda ushbu sinfdan foydalanamiz.

2. HTTP status kodlari.

Javob berilganda, server so‘rovni qayta ishlash holatini ko‘rsatadigan HTTP holat kodini o‘rnatadi. Odatiy bo‘lib, bu 3 raqamli kod bo‘lib, unda birinchi raqam javobning umumiy xususiyatini, ikkinchi va uchinchi raqamlar esa holatni belgilaydi. Status kodlarining beshta guruhi mavjud:

- 1xx: so‘rov qabul qilinganligini va uni qayta ishlash davom etayotganligini ko‘rsatadigan ma’lumot kodlari
- 2xx: so‘rovni muvaffaqiyatli qayta ishlashni ko‘rsatadigan kodlar
- 3XX: yo‘naltirish kodlari
- 4xx: so‘rovda xatolar borligini ko‘rsatadigan kodlar. Ya’ni, xato so‘rovni yuborgan mijoz tomonida paydo bo‘ldi
- 5xx: so‘rovni qayta ishlash jarayonida serverda xatolik yuz berganligini ko‘rsatadigan kodlar. Ya’ni, mijozning so‘rovi to‘g‘ri va muammo server tomonida

So‘rov formati bir qator tarkibiy qismlardan iborat. Birinchidan, bu usul, so‘ralgan manbaga yo‘l va protokolning o‘ziga xos versiyasidan iborat bo‘lgan so‘rov liniyasi (so‘rov liniyasi), masalan:

```
GET /users/id/12 HTTP/1.1
```

Shuningdek, so‘rov bir qator sarlavhalarni o‘z ichiga oladi, masalan:

```
Accept: application/json
```

Ixtiyoriy ravishda, so'rov tarkibiga tarkib sarlavhalaridan (tarkib turi bo'yicha Metama'lumotlarni taqdim etadigan), shuningdek tarkibning o'zi kiritilgan so'rov tanasini o'z ichiga olishi mumkin.

So'rovdagi har bir segment boshqasidan \r\n belgilar bilan ajratilgan - pastki qavs va yangi qatorga tarjima belgisi.

```
var message = "GET / HTTP/1.1\r\n" +
    "Host: www.google.com\r\n" +
    "Connection: Close\r\n\r\n";
```

Ushbu HTTP so'rovining mazmuni GET so'rovi " / " manziliga (ya'ni veb-sayt ildiziga) yuborilganligini ko'rsatadi, HTTP/1.1 protokoli qo'llaniladi. Keyinchalik ikkita sarlavha aniqlandi - "Host" qiymati bilan "www.google.com" va "Close" ma'nosiga ega "Connection".

Server javobi quyidagi komponentlardan iborat:

- Muayyan protokol, holat kodi va holat xabari ko'rsatilgan holat satri:

```
HTTP/1.1 401 Bad Request
```

Bunday holda, javob 401 holat kodini o'z ichiga oladi, ya'ni mijoz tomonidagi muammo-so'rovda noto'g'ri ma'lumotlar yuborilgan

- Javobni qanday tahlil qilishni ko'rsatadigan metadata bilan javob sarlavhalari
- ixtiyoriy javob tanasi

so'rov va javob ma'lumotlari umumiy tushunish uchun ko'proq berilgan, chunki aslida HttpClient va HttpListener sinflaridan foydalanib, biz http so'rovi va javob formatini aniqlashga majbur bo'lmaymiz.

3. HttpClient yaratish.

HttpClient ob'ektini yaratish tabiiy ravishda siz uning dizaynerlaridan birini ishlatishingiz mumkin:

```
public HttpClient (System.Net.Http.HttpMessageHandler handler);
```

Parametr sifatida HTTP protokoli orqali xabarlarni yuborish uchun ishlatiladigan HttpMessageHandler ob'ekti uzatiladi. Ushbu sinf mavhum, shuning uchun odatda merosxo'r sinflardan birining ob'ekti uzatiladi, masalan, HttpClientHandler yoki SocketsHttpHandler. HttpMessageHandler klassi Dispose usulini amalga oshiradi va HttpClient asosiy ob'ekti bilan birga utilizatsiya qilinadi.

```
public HttpClient (System.Net.Http.HttpMessageHandler handler, bool disposeHandler);
```

Bu erda ikkinchi parametr qo'shiladi - disposeHandler. Agar u rost bo'lsa, HttpMessageHandler ob'ekti HttpClient qo'ng'irog'i bilan birga o'chiriladi.Dispose(). Agar biz HttpClient-ni o'chirib tashlaganimizdan so'ng HttpMessageHandler ob'ektidan foydalanishni davom ettirishni istasak, unda bu parametr false qiymatiga o'tkazilishi kerak

```
public HttpClient ();
```

Ushbu qo‘ng‘iroq aslida qo‘ng‘iroqqa tengdir.

```
HttpClient(new HttpClientHandler(), true)
```

HttpClient ob‘ektini yaratishda shuni yodda tutish kerakki, u dasturning butun hayoti davomida qayta ishlatilishi mumkin. Har bir so‘rov uchun alohida ob‘ekt yaratish mavjud rozetkalar sonining tugashiga olib kelishi va SocketException xatolariga olib kelishi mumkin. Muammo nima ekanligini tushunish uchun quyidagi misolni ko‘rib chiqing.

```
Console.WriteLine("Приложение начало работы");
for (int i = 0; i < 10; i++)
{
    using (var client = new HttpClient())
    {
        using var result = await client.GetAsync("https://google.com");
        Console.WriteLine(result.StatusCode);
    }
}
Console.WriteLine("Dastur ishini yakunladi");
```

Bu erda HttpClient ob‘ekti tsiklda 10 marta yaratiladi. Kodda hamma narsa yaxshi ko‘rinadi - HttpClient klassi IDisposable interfeysini amalga oshiradi va using dizaynidan foydalanish HttpClient bilan ishlashni tugatgandan so‘ng, unga tegishli barcha manbalar bo‘shatilishini ta‘minlashi kerak. Using dizaynining o‘zida biz resurs uchun so‘rovni ishga tushiramiz ("https://google.com" GetAsync () usuli yordamida (bundan keyin biz ushbu usulni ko‘rib chiqamiz). Ushbu usul StatusCode xususiyatidan foydalanib, javobning holat kodini olishimiz mumkin bo‘lgan ba‘zi natijalarni qaytaradi. Keyinchalik, biz Internet-resurslar bilan o‘zaro aloqani batafsil ko‘rib chiqamiz, ammo hozircha muammo nima ekanligini ko‘rib chiqamiz. Ushbu kodni ishga tushiring:

```

Microsoft Visual Studio Debug Console
Приложение начало работу
OK
OK
OK
OK
OK
OK
OK
OK
OK
OK
OK
Приложение завершило работу

C:\Users\eugen\source\repos\csharp\Console\NetworkApp\NetworkApp\bin\Debug\net7.0\NetworkApp.exe (process 1716) exited with code 0.
Press any key to close this window . . .
    
```

Konsol chiqishidan ko‘rinib turibdiki, kod normal ishladi, 10 ta http so‘rovi muvaffaqiyatli bajarildi va dastur o‘z ishini yakunladi.

Ammo agar biz netstat konsol yordam dasturini ishga tushirsak, biz bir nechta osilgan ulanishlarni ko‘ramiz:

```

Администратор: Командная
C:\Users\eugen>netstat

Активные подключения

Имя      Локальный адрес      Внешний адрес      Состояние
TCP      127.0.0.1:53329      Eugene:53330      ESTABLISHED
TCP      127.0.0.1:53330      Eugene:53329      ESTABLISHED
TCP      192.168.0.112:55247  hem09s03-in-f14:https  TIME_WAIT
TCP      192.168.0.112:55249  hem09s03-in-f14:https  TIME_WAIT
TCP      192.168.0.112:55251  hem09s03-in-f14:https  TIME_WAIT
TCP      192.168.0.112:55252  hem09s03-in-f4:https   TIME_WAIT
TCP      192.168.0.112:55253  hem09s03-in-f14:https  TIME_WAIT
TCP      192.168.0.112:55254  hem09s03-in-f4:https   TIME_WAIT
TCP      192.168.0.112:55256  hem09s03-in-f4:https   TIME_WAIT
TCP      192.168.0.112:55258  hem09s03-in-f4:https   TIME_WAIT
TCP      192.168.0.112:55259  hem09s03-in-f14:https  TIME_WAIT
TCP      192.168.0.112:55260  hem09s03-in-f4:https   TIME_WAIT
TCP      192.168.0.112:55261  hem09s03-in-f14:https  TIME_WAIT
TCP      192.168.0.112:55262  hem09s03-in-f4:https   TIME_WAIT
TCP      192.168.0.112:55263  hem09s03-in-f14:https  TIME_WAIT
TCP      192.168.0.112:55264  hem09s03-in-f4:https   TIME_WAIT
TCP      192.168.0.112:55265  hem09s03-in-f14:https  TIME_WAIT
TCP      192.168.0.112:55266  hem09s03-in-f4:https   TIME_WAIT

C:\Users\eugen>
    
```

Shunday qilib, biz bir qator ulanishlar `TIME_WAIT` holatiga ega ekanligini ko‘ramiz. Gap shundaki, `HttpClient`-da `Dispose()` usuli chaqirilganda, u bilan birgalikda ishlatiladigan `HttpMessageHandler` ob‘ektida `Dispose` usuli chaqiriladi, bu aslida xabarlarini yuborishni boshqaradi va rozetkalardan foydalanadi. Shu bilan birga, 240 soniya davomida ulanish `TIME_WAIT` holatida ochiq qoladi bundan tashqari, bir vaqtning o‘zida ishlatilishi mumkin bo‘lgan rozetkalar soniga cheklov mavjud. Va Agar `HttpClient` ob‘ekti har bir so‘rov uchun yaratilgan bo‘lsa, u holda og‘ir yuk ostida mavjud bo‘lgan rozetkalar soni tezda tugashi mumkin, bu esa `SocketException` xatolariga olib keladi.

Xususan, bu holda yaratilgan barcha mijozlar uchun `HttpMessageHandler` ishlovchisidan foydalanish mumkin:

```
HttpMessageHandler handler = new HttpClientHandler();

Console.WriteLine("Dastur ishini boshladi");
for (int i = 0; i < 10; i++)
{
    using (var client = new HttpClient(handler, false))
    {
        using var result = await client.GetAsync("https://google.com");
        Console.WriteLine(result.StatusCode);
    }
}
Console.WriteLine("Dastu ishini yakunladi");
```

Ammo, asosan, rozetkaning etishmasligi muammosini oldini olish uchun Microsoft `HttpClient`-ni aniqlash uchun quyidagi yondashuvlardan birini tavsiya qiladi:

`HttpClient`-ning uzoq muddatli nusxalari statik ob‘ektlar yoki Singleton shaklida bo‘lib, ular dasturning butun hayoti davomida mavjud.

`IHttpClientFactory` fabrikasi yordamida yaratilgan qisqa muddatli `HttpClient` nusxalari.

Ushbu yondashuvlarni qisqacha ko‘rib chiqing.

Bunday holda, dasturning butun hayoti davomida mavjud bo‘lgan bitta `HttpClient` ob‘ekti:

```
class Program
{
    static HttpClient client = new HttpClient();
    static async Task Main(string[] args)
    {
        // HttpClient ni qo‘llash
    }
}
```

```
}
}
```

Shuni yodda tutish kerakki, HttpClient faqat ulanishni yaratishda DNS yozuvlarini o'rnatadi. U so'rov yaratish uchun xost manzilini aniqlashga imkon beradigan ma'lum bir dns yozuvi uchun DNS-server tomonidan belgilangan umrni (TTL) kuzatmaydi. Agar DNS yozuvlari muntazam ravishda o'zgarib tursa, bu ba'zi stsenariylarda yuz berishi mumkin bo'lsa, HttpClient mijozi ushbu o'zgarishlarni hisobga olmaydi. Shuning uchun, ushbu muammoni hal qilish uchun Sockets Http Handler sozlamalari bilan xususiyatni o'rnatish tavsiya etiladi. PooledConnectionLifetime, shuning uchun ulanishni almashtirishda DNS qidiruvi amalga oshiriladi. Masalan,:

```
class Program
{
    static HttpClient? httpClient;

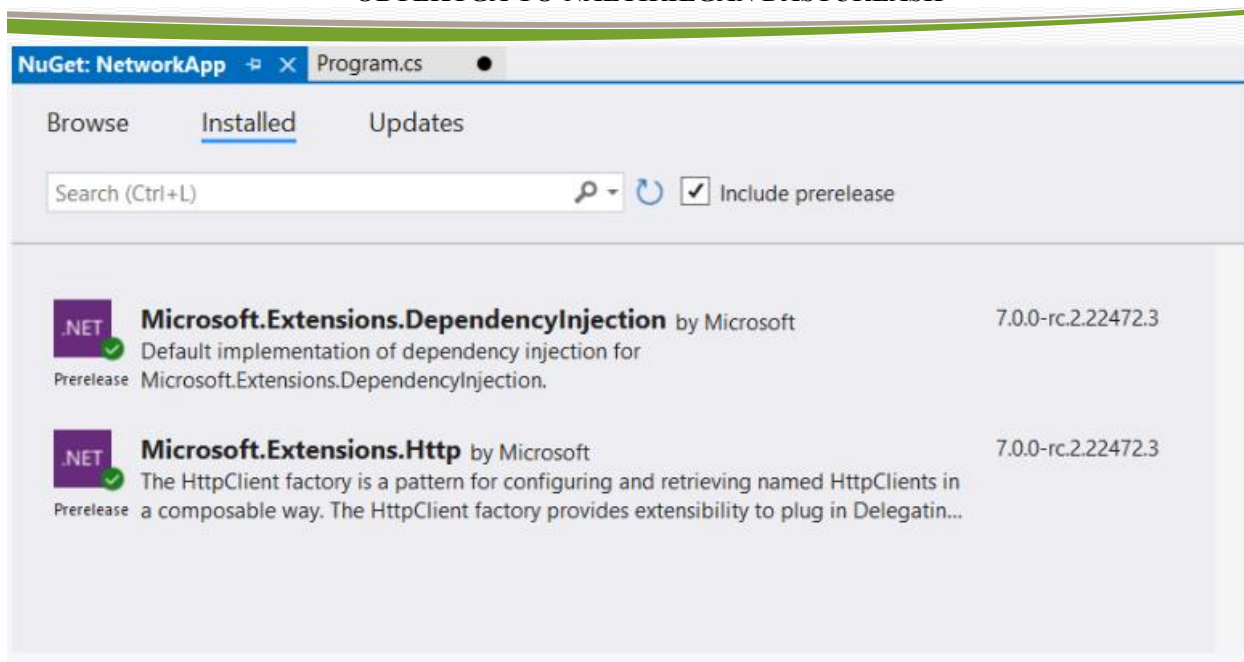
    static async Task Main(string[] args)
    {
        var socketsHandler = new SocketsHttpHandler
        {
            PooledConnectionLifetime = TimeSpan.FromMinutes(2)
        };
        httpClient = new HttpClient(socketsHandler);

        // использование HttpClient
    }
}
```

Bu erda Pooledconnection Lifetime xususiyati uchun belgilangan interval tugagach (bu holda 2 daqiqa), joriy ulanish yopiladi va yangisi yaratiladi.

IHttpClientFactory zavodi HttpResponseMessage ob'ekt hovuzini boshqaradi. Agar ba'zi HttpResponseMessage ob'ekti ushbu hovuzda mavjud bo'lib qolsa, u yangi HttpClient ob'ektini yaratish uchun qayta ishlatilishi mumkin. Bu rozetkalarining charchash ehtimolini kamaytiradi.

HttpClient ob'ektini yaratish uchun IHttpClientFactory fabrikasida Client yaratish usuli chaqiriladi. Masalan, agar bizda oddiy konsol yoki ish stoli ilovasi bo'lsa, unda biz Nuget orqali Microsoft paketlarini qo'shishimiz kerak. Extensions.DependencyInjection va Microsoft.Extensions.Http:



Agar bizda veb-dastur loyihasi bo‘lsa, unda bunday paketlar allaqachon loyihaga sukut bo‘yicha o‘rnatilgan.

Konsol ilovasi uchun HttpClient yaratish misoli:

```
using Microsoft.Extensions.DependencyInjection;

// определяем коллекцию сервисов
var services = new ServiceCollection();
// добавляем сервисы, связанные с HttpClient, в том числе IHttpHttpClientFactory
services.AddHttpClient();
// создаем провайдер сервисов
var serviceProvider = services.BuildServiceProvider();
// получаем сервис IHttpHttpClientFactory
var httpClientFactory = serviceProvider.GetService<IHttpHttpClientFactory>();
// создаем объект HttpClient
var httpClient = httpClientFactory?.CreateClient();
// использование HttpClient
```

Birinchidan, biz dastur xizmatlari to‘plamini olamiz. Keyin o‘rnatilgan Dependency Injection mexanizmidan foydalanib, biz HttpClient bilan bog‘liq xizmatlarni (shu jumladan IHttpHttpClientFactory) dasturga kiritamiz. Buning uchun xizmatlar usuli qo‘llaniladi. Add Http Client()

Shundan so‘ng, biz IHttpHttpClientFactory xizmatini.net-da o‘rnatilgan bog‘liqliklarni olish uchun mavjud bo‘lgan har qanday usul bilan olishimiz mumkin. Bunday holda, qulaylik uchun Service locator naqsh ishlatiladi:

```
var httpClientFactory = serviceProvider.GetService<IHttpHttpClientFactory>();
```

Boshqa holatlarda siz boshqa usullardan foydalanishingiz mumkin, masalan, konstruktor parametri orqali xizmatni olish.

Nazorat savollari:

- 1) HTTP protokoli nima va u qanday ishlaydi? Ushbu protokol qanday maqsadlarda qo'llaniladi va uning asosiy xususiyatlari nimadan iborat?
- 2) HTTP status kodlari qanday toifalarga bo'linadi va ularning har bir toifasi nimani bildiradi? 200, 404, va 500 kodlari qanday holatlarni bildiradi?
- 3) C# dasturida HttpClient sinfini qanday yaratish va undan qanday foydalanish mumkin? Oddiy GET so'rovi yuborish uchun HttpClient'dan qanday foydalanasiz?
- 4) HTTP va HTTPS protokollari orasidagi asosiy farqlar qanday? HTTPS qanday xavfsizlik mexanizmlarini ta'minlaydi?

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**3-Mashg‘ulot: So‘rovlar yuborish.****Reja:**

1. HttpClient bilan so‘rovlarni yuborish.
2. SendAsync metodi orqali so‘rovlar jo‘natish.
3. JSON formatida ma’lumotlarni olish.

1. HttpClient bilan so‘rovlarni yuborish.

So‘rovlarni bajarish uchun HttpClient klassi bir qator usullarni taqdim etadi:

- **Send()** / **SendAsync()**: so‘rovni HttpRequestMessage ob’ekti sifatida yuboradi va Server javobini HttpResponseMessage ob’ekti sifatida oladi
- **GetAsync()**: belgilangan manzilga get so‘rovini yuboradi va server javobini HttpResponseMessage ob’ekti sifatida oladi
- **GetByteArrayAsync()**: belgilangan manzilga get so‘rovini yuboradi va javobni baytlar qatori sifatida qaytaradi (byte[])
- **GetStreamAsync()**: belgilangan manzilga get so‘rovini yuboradi va javobni Stream ob’ekti sifatida qaytaradi
- **GetStringAsync()**: belgilangan manzilga get so‘rovini yuboradi va javobni satr sifatida qaytaradi
- **PostAsync()**: belgilangan manzilga post so‘rov yuboradi va server javobini HttpResponseMessage ob’ekti sifatida oladi
- **PutAsync()**: belgilangan manzilga put so‘rovini yuboradi va server javobini HttpResponseMessage ob’ekti sifatida oladi
- **PatchAsync()**: belgilangan manzilga patch so‘rovini yuboradi va server javobini HttpResponseMessage ob’ekti sifatida oladi
- **DeleteAsync()**: belgilangan manzilga Delete so‘rovini yuboradi va server javobini HttpResponseMessage ob’ekti sifatida oladi

Bir qator xususiyatlar yordamida so‘rovlarni yuborish uchun asosiy sozlamalarni sozlash mumkin:

- **BaseAddress**: so‘rovlarni yuborishda foydalaniladigan URI ob’ekti sifatida asosiy manzilni qaytaradi yoki o‘rnatadi.
- **DefaultProxy**: proksi-server sozlamalarini qaytaradi yoki o‘rnatadi.
- **DefaultRequestHeaders**: har bir so‘rovda serverga yuboriladigan standart sarlavhalar to‘plamini qaytaradi yoki o‘rnatadi
- **DefaultRequestVersion**: so‘rovlarni yuborishda qo‘llaniladigan HTTP protokolining standart versiyasini qaytaradi yoki o‘rnatadi
- **DefaultVersionPolicy**: standart versiya siyosatini qaytaradi yoki o‘rnatadi
- **Maxresponsecontentbuffersize**: javob ma’lumotlarini o‘qiyotganda maksimal bufer hajmini baytlarga qaytaradi yoki o‘rnatadi

- **Timeout:** so‘rovni yuborishdan oldin vaqtni qaytaradi yoki o‘rnatadi.
-

2. *SendAsync metodi orqali so‘rovlar jo‘natish.*

SendAsync () usuli yordamida universal so‘rov yuborishni ko‘rib chiqing. Ushbu usul bir qator versiyalarga ega, ammo hech bo‘lmaganda so‘rov ma‘lumotlarini (HttpRequestMessage ob‘ekti) qabul qiladi va javobni HttpResponseMessage ob‘ekti sifatida qaytaradi:

```
public Task<System.Net.Http.HttpResponseMessage> SendAsync
(System.Net.Http.HttpRequestMessage request);
```

HttpRequestMessage klassi so‘rov ma‘lumotlarini quyidagi xususiyatlar bilan sozlash imkonini beradi:

Content: so‘rov tarkibini qaytaradi yoki o‘rnatadi

Method: so‘rov usulini qaytaradi yoki o‘rnatadi.

RequestUri: so‘rov manzilini Uri ob‘ekti sifatida qaytaradi yoki o‘rnatadi.

Version: HTTP protokoli versiyasini qaytaradi yoki o‘rnatadi.

So‘rov usuli va manzilini sinf konstruktorlaridan biri yordamida ham o‘rnatishingiz mumkin:

```
class Program
{
    static HttpClient httpClient = new HttpClient();

    static async Task Main()
    {
        // определяем данные запроса
        using HttpRequestMessage request = new
        HttpRequestMessage(HttpMethod.Get, "https://www.google.com");
        // выполняем запрос
        await httpClient.SendAsync(request);
    }
}
```

So‘rov bajarilgandan so‘ng, SendAsync usuli javobni HttpResponseMessage ob‘ekti sifatida qaytaradi. Ushbu ob‘ekt quyidagi xususiyatlardan foydalangan holda javob ma‘lumotlarini olish imkonini beradi:

Content: HTTP javobining tarkibini qaytaradi yoki o‘rnatadi. Ushbu xususiyat http Content ob‘ektini ifodalaydi, bu sizga so‘rov ma‘lumotlarini usullardan biri yordamida hisoblash imkonini beradi:

- ReadAsStringAsync (): javobni satr sifatida qaytaradi;
- ReadAsByteArrayAsync (): javobni bayt qatori sifatida qaytaradi;

- **ReadAsStreamAsync ()**: javobni oqim ob'ekti sifatida qaytaradi.

Headers: HTTP javob sarlavhalari to'plamini qaytaradi (httpresponseheaders ob'ekti). Ushbu to'plamdagi har bir sarlavha kalit-qiymat juftligi bilan ifodalanadi. Muayyan sarlavhani olish uchun getvalues () usulidan foydalanishingiz mumkin, unda sarlavha nomi uzatiladi:

```
response.Headers.GetValues("Accept")
```

IsSuccessStatusCode: HTTP so'rovi muvaffaqiyatli bo'lsa, true ni qaytaradi.

ReasonPhrase: holat xabarini qaytaradi.

RequestMessage: ushbu javob bilan bog'liq bo'lgan so'rov ma'lumotlarini qaytaradi.

StatusCode: javobning holat kodini qaytaradi.

TrailingHeaders: HTTP so'roviga kiritilgan qo'shimcha sarlavhalarni qaytaradi.

Versiya: ishlatilgan HTTP protokolining versiyasini qaytaradi.

Masalan, so'rov yuboring "www.google.com" va biz konsolga javob ma'lumotlarini chiqaramiz:

```
class Program
{
    static HttpClient httpClient = new HttpClient();

    static async Task Main()
    {
        // so'rov ma'lumotlari aniqlashtirilmoqda
        using HttpRequestMessage request = new HttpRequestMessage(HttpMethod.Get,
"https://www.google.com");

        // javob qaytib olindi
        using HttpResponseMessage response = await httpClient.SendAsync(request);

        // javob kodini ko'ramiz
        // status
        Console.WriteLine($"Status: {response.StatusCode}\n");
        //Sarlavha
        Console.WriteLine("Headers");
        foreach (var header in response.Headers)
        {
            Console.WriteLine($"{header.Key}:");
            foreach (var headerValue in header.Value)
            {
                Console.WriteLine(headerValue);
            }
        }
        // содержимое ответа
        Console.WriteLine("\nContent");
        string content = await response.Content.ReadAsStringAsync();
        Console.WriteLine(content);
    }
}
```

Bunday holda, server javobining mazmuni aslida html-sahifa bo‘ladi, natijada biz quyidagi turdagi konsol chiqishini olamiz:

```

Выбрать Microsoft Visual Studio Debug Console
Status: OK

Headers
Date:Sun, 16 Oct 2022 11:39:10 GMT
Cache-Control:max-age=0, private
P3P:CP="This is not a P3P policy! See g.co/p3phelp for more info."
Server:gws
X-XSS-Protection:0
X-Frame-Options:SAMEORIGIN
Set-Cookie:1P_JAR=2022-10-16-11; expires=Tue, 15-Nov-2022 11:39:10 GMT; path=/; domain=.google.com; Secure
AEC=AakniG0NCu10JNqYsySA-M7nn0MSZPKdWofZ1f1zn9sRIpdYB_wJdaRH05s; expires=Fri, 14-Apr-2023 11:39:10 GMT; path=/; domain=.google.com; Secure; HttpOnly; SameSite=lax
NID=511=D3wD5N9HJifN1Ra52I92pEizOPAnX3Xi-agV9lXv9qKmsVCjRKLOKw6nD7Y0SxgpbNq3d75FqGU1rDgAeSYDYrEnTix_yXIV2G3UqiyRg_7m4t4ek30t0WnoJqk8eHQAioVqEg_V-7CcgAqthBEyhi0kghoZ5CvusQgtafvbus; expires=Mon, 17-Apr-2023 11:39:10 GMT; path=/; domain=.google.com; HttpOnly
Alt-Svc:h3=":443"; ma=2592000
h3-29=":443"; ma=2592000
h3-Q050=":443"; ma=2592000
h3-Q046=":443"; ma=2592000
h3-Q043=":443"; ma=2592000
quic=":443"; ma=2592000
Accept-Ranges:none
Vary:Accept-Encoding
Transfer-Encoding:chunked

Content
<!doctype html><html itemscope="" itemtype="http://schema.org/WebPage" lang="ru"><head><meta content="&#1055;&#1086;&#1080;&#1089;&#1082; &#1080;&#1085;&#1092;&#1086;&#1088;&#1084;&#1072;&#1094;&#1080;&#1080;&#1074; &#1080;&#1085;&#1090;&#1077;&#1088;&#1085;&#1077;&#1090;&#1077;: &#1074;&#1077;&#1073; &#108

```

GetAsync()

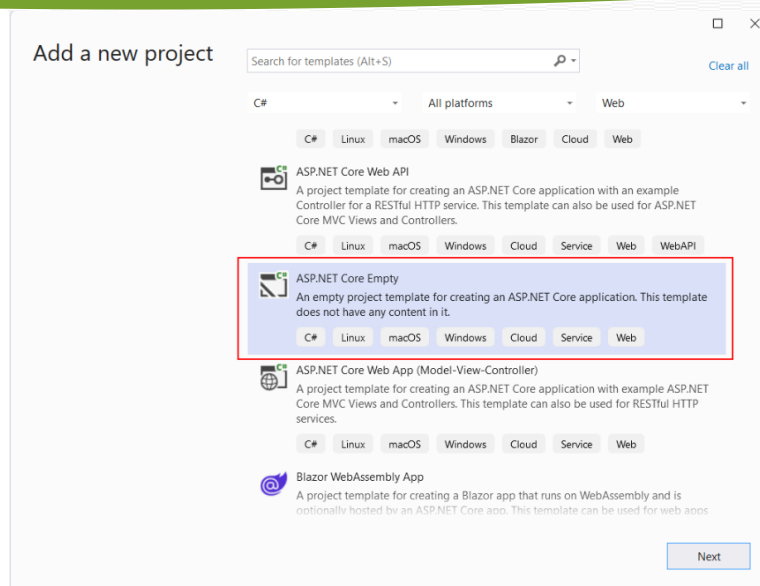
GetAsync()/PostAsync()/PatchAsync()/DeleteAsync() usuli so‘rovni yuborishni biroz osonlashtiradi, chunki ular ma’lum bir so‘rov turiga moslashtirilgan va ularda HttpResponseMessage uzatilishi shart emas. Masalan, GetAsync () usulidan foydalanamiz, u GET so‘rovini yuboradi va parametr sifatida resurs manzilini oladi:

3. JSON formatida ma’lumotlarni olish.

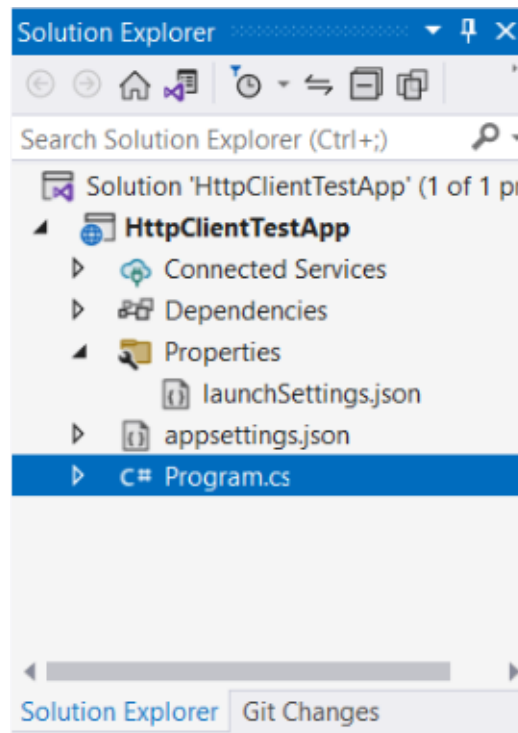
JSON mijozdan serverga va undan ma’lumotlarni uzatishning eng mashhur formatlaridan birini taqdim etadi. Serverdan JSON formatidagi ma’lumotlarni qanday olishni ko‘rib chiqamiz.

Serverni aniqlash

Birinchidan, biz JSON formatida ma’lumotlarni yuboradigan serverni aniqlaymiz. Buning uchun biz loyiha yaratamiz ASP.NET turi bo‘yicha ASP.NET Core Empty:



Masalan, mening holimda loyiha Http Client Test App deb nomlanadi. Ushbu loyihada dastur fayliga ochamiz.cs



va undagi quyidagi kodni aniqlang:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapGet("/", () => new Person("Tom", 38));

app.Run();
record Person(string Name, int Age);
```

Veb-ilovani yaratish uchun ASP.NET Core avval biz WebApplicationBuilder ob'ektini yaratishimiz kerak: Keyin, uning qurish usulidan foydalanib, biz veb-ilova ob'ektini yaratamiz, u aslida dasturni ifodalaydi ASP.NET:

```
var builder = WebApplication.CreateBuilder();
```

Keyin, uning qurish usulidan foydalanib, biz veb-ilova ob'ektini yaratamiz, u aslida dasturni ifodalaydi ASP.NET:

```
var app = builder.Build();
```

App usuli yordamida.Map Get () biz oxirgi nuqtani aniqlaymiz - ma'lum bir marshrut bo'yicha so'rov ishlovchisi:

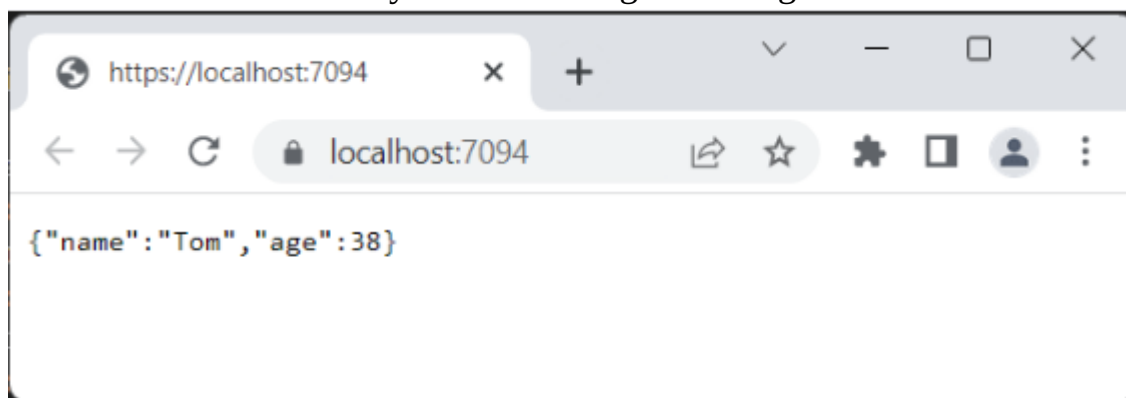
```
app.MapGet("/", () => new Person("Tom", 38));
```

Bunday holda, "/" manziliga (ya'ni veb-ilovaning ildiziga) murojaat qilganda, ishlov beruvchi shaxs ob'ektini qaytaradi("Tom", 38), u yuborilganda avtomatik ravishda JSON formatiga seriyalanadi.

```
app.Run();
```

Loyihani ishga tushiramiz va brauzer bizga yuborilgan ob'ektni JSON formatida ko'rsatadi:

Keyin dasturni ishga tushiring:



Darhol dastur ishlaydigan manzilga e'tibor qaratamiz ASP.NET, keyin unga HttpClient-dan ulanish uchun. Shunday qilib, mening holimda bu "https://localhost:7094/".

Nazorat savollari:

- 1) HttpClient sinfi yordamida qanday turdagi HTTP so'rovlarini yuborish mumkin? Oddiy GET va POST so'rovlari qanday amalga oshiriladi?
- 2) SendAsync metodi nima va u qanday ishlaydi? Ushbu metod yordamida asinxron so'rovlarni qanday yuborasiz va javoblarni qanday qabul qilasiz?
- 3) HttpClient yordamida olingan javobni JSON formatida qanday deserializatsiya qilish mumkin? JSON formatidagi ma'lumotlarni qanday qilib C# obyekt sifatida olish va ulardan foydalanish mumkin?
- 4) HttpClient sinfidan foydalanishda qanday xavfsizlik va samaradorlik jihatlarni e'tiborga olish kerak? HttpClient ob'ektining uzoq muddatli foydalanishdagi kamchiliklari nimada?

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**4-Mashg'ulot: Sarlavhalarni yuborish va qabul qilish.****O'quv savollari:**

1. Sarlavhalarni yuborish va qabul qilish haqida tushuncha.
2. So'rov uchun "sarlavha" o'rnatish.
3. So'rovda ma'lumotlarni yuborish.

1. Sarlavhalarni yuborish va qabul qilish haqida tushuncha.

HttpClient yordamida so'rovlarni yuborishda sarlavhalarni yuborish va qabul qilishning bir qator usullaridan foydalanish mumkin. Ammo har qanday holatda ham biz sarlavhalar to'plamini taqdim etadigan System.Net.Http.Headers turi bilan ishlashimiz kerak. Sarlavhalarni boshqarish uchun ushbu tur bir qator usullarni taqdim etadi:

Add(String headerName, IEnumerable<String> values): belgilangan sarlavha va uning qiymatlarini HttpHeaders to'plamiga qo'shadi

Add(String headerName, String headerValue): belgilangan sarlavha va uning qiymatini HttpHeaders to'plamiga qo'shadi

Clear (): HttpHeaders to'plamidagi barcha sarlavhalarni olib tashlaydi

Contains(String headerName): HttpHeaders to'plamida ma'lum bir sarlavha mavjud bo'lsa, true ni qaytaradi

Remove(String headerName): belgilangan sarlavhani HttpHeaders to'plamidan olib tashlaydi

TryGetValues (String headerName, IEnumerable<String> values): agar httpheaders to'plamida mavjud bo'lsa, ko'rsatilgan sarlavha uchun qiymatlar to'plamini qaytaradi

Ushbu sinf mavhum bo'lib, **HttpContentHeaders**, **HttpRequestHeaders** va **HttpResponseHeaders** sinflari tomonidan amalga oshiriladi.

Sarlavhalarni yuborish

HttpClient uchun global sarlavhalarni o'rnatish

Avvalo, biz **DefaultRequestHeaders** HttpClient sinfidan foydalanib sarlavhalarni global miqyosda sozlashimiz mumkin. Ushbu xususiyat **DefaultRequestHeaders** turini ifodalaydi, ya'ni yuqorida ko'rsatilgan barcha usullarni amalga oshiradi.

Masalan, bizda quyidagi veb-ilova bo'lsin ASP.NET, bu ikkita sarlavhani oladi: standart "User-Agent" va maxsus "SecreteCode":

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapGet("/", (HttpContext context) =>
{
    // пытаемся получить заголовок "SecreteCode"
    context.Request.Headers.TryGetValue("User-Agent", out var userAgent);
```



```
// пытаемся получить заголовок "SecreteCode"
context.Request.Headers.TryGetValue("SecreteCode", out var secreteCode);
// отправляем данные обратно клиенту
return $"User-Agent: {userAgent}    SecreteCode: {secreteCode}";
});

app.Run();
```

HttpClient yordamida konsol dasturidan sarlavhalarni yuboring:

```
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        // адрес сервера
        var serverAddress = "https://localhost:7094/";
        // устанавливаем оба заголовка
        httpClient.DefaultRequestHeaders.Add("User-Agent", "Mozilla Firefox 5.4");
        httpClient.DefaultRequestHeaders.Add("SecreteCode", "hello");

        using var response = await httpClient.GetAsync(serverAddress);
        var responseText = await response.Content.ReadAsStringAsync();
        Console.WriteLine(responseText);
    }
}
```

Shuni ta'kidlash kerakki, alohida sarlavhalarni o'rnatishni osonlashtirish uchun httprequestheaders klassi bir qator maxsus xususiyatlarni taqdim etadi:

- **Accept:** qabul qilish sarlavhasi qiymatini qaytaradi;
- **AcceptCharset:** accept-Charset sarlavhasi qiymatini qaytaradi;
- **AcceptEncoding:** qabul qilish-kodlash sarlavhasi qiymatini qaytaradi;
- **AcceptLanguage:** qabul qilish-til sarlavhasi qiymatini qaytaradi;
- **Authorization:** avtorizatsiya sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **CacheControl:** Keshni boshqarish sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **Connection:** ulanish sarlavhasi qiymatini qaytaradi;
- **Date:** sana sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **Expect:** expect sarlavhasi qiymatini qaytaradi;
- **From:** From sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **Xost:** xost sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **IfMatch:** if-Match sarlavhasi qiymatini qaytaradi;
- **IfModifiedSince:** if-Modified-Since sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **IfNoneMatch:** if-None-Match sarlavhasi qiymatini qaytaradi;
- **IfRange:** if-Range sarlavhasi qiymatini qaytaradi yoki o'rnatadi;

- **IfUnmodifiedSince:** if-Unmodified-Since sarlavhasi qiymatini qaytaradi yoki o‘rnatadi;
- **MaxForwards:** Max-Forwards sarlavhasi qiymatini qaytaradi yoki o‘rnatadi;
- **Pragma:** Pragma sarlavhasi qiymatini qaytaradi;
- **ProxyAuthorization:** proxy-Authorization sarlavhasi qiymatini qaytaradi yoki o‘rnatadi;
- **Range:** Range sarlavhasi qiymatini qaytaradi yoki o‘rnatadi;
- **Referrer:** referer sarlavhasi qiymatini qaytaradi yoki o‘rnatadi;
- **TE:** te sarlavhasi qiymatini qaytaradi;
- **Trailer:** Trailer sarlavhasi qiymatini qaytaradi;
- **TransferEncoding:** transfer-Encoding sarlavhasi qiymatini qaytaradi;
- **Upgrade:** yangilash sarlavhasi qiymatini qaytaradi;
- **UserAgent:** user-Agent sarlavhasi qiymatini qaytaradi;
- **Via:** Via sarlavhasi qiymatini qaytaradi;
- **Warning:** ogohlantirish sarlavhasi qiymatini qaytaradi.

Bunday xususiyatlardan foydalanish sarlavhani nomlashda xatolikka yo‘l qo‘yilmasligini ta‘minlashga imkon beradi. Har bir shunga o‘xshash xususiyat `httpheaderValueCollection` ob‘ektini ifodalaydi, u ma‘lum bir tur bilan yoziladi - bu turdagi ob‘ekt sarlavha qiymatini ifodalaydi. Masalan, "foydalanuvchi-Agent"ni o‘rnatish:

```
httpClient.DefaultRequestHeaders.UserAgent.Add(new ProductInfoHeaderValue("User-Agent", "Mozilla Firefox 5.4"));
```

2. So‘rov uchun “sarlavha” o‘rnatish.

Agar sarlavha faqat ma‘lum bir so‘rov uchun o‘rnatilishi kerak bo‘lsa, unda siz `HttpRequestMessage` sinfining `Headers` xususiyatidan foydalanishingiz mumkin, bu ham `HttpRequestHeaders` turini ifodalaydi:

```
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        // адрес сервера
        var serverAddress = "https://localhost:7094/";
        using var request = new HttpRequestMessage(HttpMethod.Get,
serverAddress);
        // устанавливаем оба заголовка
        request.Headers.Add("User-Agent", "Mozilla Firefox 5.6");
        request.Headers.Add("SecreteCode", "hello");

        using var response = await httpClient.SendAsync(request);
        var responseText = await response.Content.ReadAsStringAsync();
        Console.WriteLine(responseText);
    }
}
```

Server o‘rnatgan sarlavhalarni olish uchun siz HttpResponseMessage sinfining Headers xususiyatidan foydalanishingiz mumkin, bu HttpHeaders turini ifodalaydi. Masalan, biz kirish sanasini ko‘rsatadigan "sana" sarlavhasini olamiz:

```
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        var serverAddress = "https://localhost:7094/";
        using var response = await httpClient.GetAsync(serverAddress);
        var dateValues = response.Headers.GetValues("Date");

        Console.WriteLine(dateValues.FirstOrDefault());
    }
}
```

Shuni ta’kidlash kerakki, agar ko‘rsatilgan nom bilan sarlavha javobda bo‘lmasa, unda dastur istisno yaratadi. Bunday vaziyatni oldini olish uchun sarlavha qiymatini olishdan oldin uning mavjudligini Contains () usuli bilan tekshirishimiz mumkin (agar sarlavha mavjud bo‘lsa, rost qaytadi). Yoki TryGetValue () usulidan foydalaning, u chiqish parametriga sarlavha qiymatlarini muvaffaqiyatli qabul qilganda true ni qaytaradi. Ammo sarlavha bo‘lmasa, chiqish parametri null bo‘ladi.

```
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        var serverAddress = "https://localhost:7094/";
        using var response = await httpClient.GetAsync(serverAddress);
        response.Headers.TryGetValue("date", out var dateValues);

        Console.WriteLine(dateValues?.FirstOrDefault());
    }
}
```

Shuni ham ta’kidlash kerakki, umumiy sarlavhalarni olishni soddalashtirish uchun HttpHeaders klassi bir qator maxsus usullarni taqdim etadi:

- **AcceptRanges:** qabul qilish-chegaralar sarlavhasi qiymatini qaytaradi
- **Age:** Age sarlavhasi qiymatini qaytaradi yoki o‘rnatadi
- **CacheControl:** Keshni boshqarish sarlavhasi qiymatini qaytaradi yoki o‘rnatadi
- **Connection:** ulanish sarlavhasi qiymatini qaytaradi
- **Date:** sana sarlavhasi qiymatini qaytaradi yoki o‘rnatadi
- **ETag:** etag sarlavhasi qiymatini qaytaradi yoki o‘rnatadi
- **Location:** joylashuv sarlavhasi qiymatini qaytaradi yoki o‘rnatadi

- **Pragma:** Pragma sarlavhasi qiymatini qaytaradi
- **ProxyAuthenticate:** proxy-Authenticate sarlavhasi qiymatini qaytaradi
- **RetryAfter:** Retry-after sarlavhasi qiymatini qaytaradi yoki o‘rnatadi
- **Server:** Server sarlavhasi qiymatini qaytaradi
- **Trailer:** Trailer sarlavhasi qiymatini qaytaradi
- **TransferEncoding:** transfer-Encoding sarlavhasi qiymatini qaytaradi
- **Upgrade:** yangilash sarlavhasi qiymatini qaytaradi
- **Vary:** Vary sarlavhasi qiymatini qaytaradi
- **Via:** Via sarlavhasi qiymatini qaytaradi
- **Warning:** ogohlantirish sarlavhasi qiymatini qaytaradi
- **WwwAuthenticate:** HTTP javobi uchun www-Authenticate sarlavha qiymatini qaytaradi.

3. So‘rovda ma’lumotlarni yuborish.

HttpClient ba’zi ma’lumotlarni serverga yuborish imkonini beradi. Yuborilgan ma’lumotlar System.Net.Http.HttpContent ob’ektini ifodalaydi. ammo bu sinf mavhum bo‘lganligi sababli, aslida yuborish uchun eski sinflardan biri ishlatiladi. Xususan, net-da HttpContent-dan meros bo‘lib o‘tgan quyidagi ichki sinflar mavjud:

System.Net.Http. ByteArrayContent: baytlar qatorini yuborish uchun

System. Net. Http. FormUrlEncodedContent: "application/x-www-form-urlencoded" MIME turi bilan kodlangan ma’lumotlarni yuborish uchun. (ByteArrayContent-dan meros qilib olingan)

System.Net. Http. StringContent: matnni yuborish uchun (ByteArrayContent-dan meros qilib olingan)

System.Net.Http.JSON.JsonContent: JSON yuborish uchun

System.Net.Http. MultipartContent: fayllarni yuborish uchun

System.Net.Http. MultipartFormDataContent: fayllarni yuborish uchun (Multipart Content-dan meros qilib olingan)

System.Net. Http. ReadOnlyMemoryContent: yuborilgan tarkib sifatida ReadOnlyMemory<t> ishlatiladi

Vazifalarga va biz o‘zaro aloqada bo‘lgan maxsus serverga qarab, biz ushbu turlardan birini qo‘llashimiz mumkin.

Odatda ma’lumotlar Post/Put/Patch kabi so‘rovlar orqali yuboriladi. HttpClient - da siz ma’lumotlarni yuborish uchun SendAsync () usulidan foydalanishingiz mumkin-parametr sifatida u HttpRequestMessage ob’ektini qabul qiladi, unda siz yuborilgan tarkibni tarkib xususiyati orqali o‘rnatishingiz mumkin.

Biroq, so‘rovlarni yuborishni osonlashtirish uchun HttpClient har bir so‘rov turi uchun usullarni ham taqdim etadi:

- **PostAsync()**

- **PutAsync()**
- **Patch sync()**

Ushbu usullarning barchasi kamida ikkita parametрни oladi: resurs manzili va http Content ob’ekti sifatida yuborilgan ma’lumotlar?:

```
Task<HttpResponseMessage> PostAsync (string? requestUri,
System.Net.Http.HttpContent? content);
Task<HttpResponseMessage> PutAsync (string? requestUri, System.Net.Http.HttpContent?
content);
Task<HttpResponseMessage> PatchAsync (string? requestUri,
System.Net.Http.HttpContent? content);
```

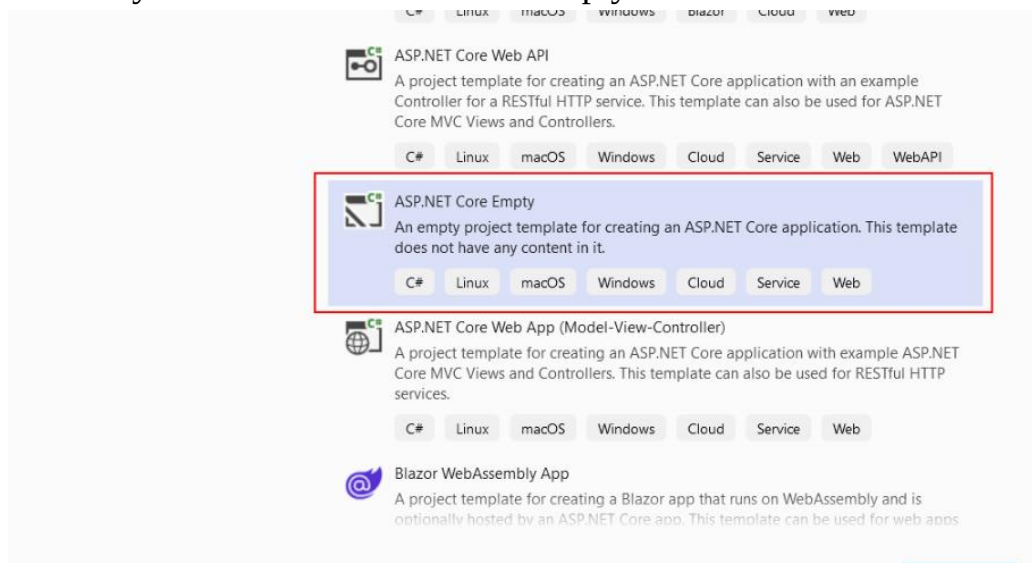
Eng oddiy misol - matnni yuborish orqali ma’lumotlarning odatiy yuborilishini ko‘rib chiqing.

Matnni yuborish

Oddiy matnli tarkibni yuborish uchun stringcontent klassi ishlatiladi, u konstruktor orqali yuborilgan qatorni o‘rnatadi:

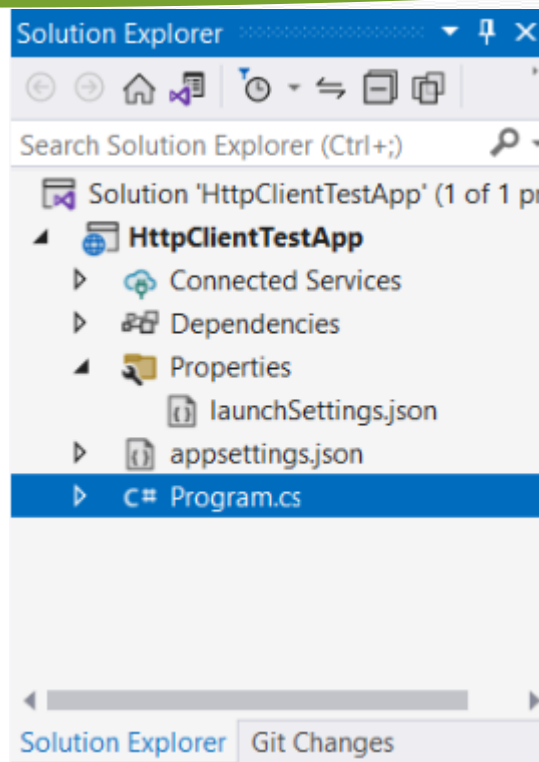
```
StringContent content = new StringContent(отправляемая_строка);
```

Sinov uchun avval server dasturini aniqlaymiz ASP.NET yadro. Buning uchun biz loyiha turini yaratamiz ASP.NET Core Empty:



Masalan, mening holimda loyiha Http Client Test App deb nomlanadi. Ushbu loyihada dastur fayliga ochamiz.cs

Nazorat savollari:



```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapPost("/data", async (HttpContext httpContext) =>
{
    using StreamReader reader = new StreamReader(httpContext.Request.Body);
    string name = await reader.ReadToEndAsync();
    return $"Получены данные: {name}";
});

app.Run();
```

Veb-ilovani yaratish uchun ASP.NET Core avval biz WebApplicationBuilder ob'ektini yaratishimiz kerak:

```
var builder = WebApplication.CreateBuilder();
```

Keyin, uning qurish usulidan foydalanib, biz veb-ilova ob'ektini yaratamiz, u aslida dasturni ifodalaydi ASP.NET:

```
var app = builder.Build();
```

App usuli yordamida.Map Post () "/data"yo'lida post so'rovlarini bajaradigan so'nggi nuqtani aniqlang. Ushbu so'nggi nuqta ishlovchisi so'rov kontekstini parametr sifatida qabul qiladi-httpcontext ob'ekti:

```
app.MapPost("/data", async (HttpContext httpContext) =>
```

Bu holda biz satrni yuborishni sinab ko'rayotganimiz sababli, so'rov ma'lumotlari httpContext.Request.Body - ni satrga-name o'zgaruvchisiga o'qing va keyin mijozga "ma'lumotlar olingan: {name}"qatorini qaytaring.

Keyin dasturni ishga tushiring:

```
app.Run();
```

Endi biz konsol dasturini aniqlaymiz, unda HttpClient veb-ilovaga ba’zi qatorlarni yuboradi. Birinchidan, SendAsync () usulidan foydalaning:

```
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        StringContent content = new StringContent("Tom");
        // определяем данные запроса
        using var request = new HttpRequestMessage(HttpMethod.Post,
"https://localhost:7094/data");
        // установка отправляемого содержимого
        request.Content = content;
        // отправляем запрос
        using var response = await httpClient.SendAsync(request);
        // получаем ответ
        string responseText = await response.Content.ReadAsStringAsync();
        Console.WriteLine(responseText);
    }
}
```

Yuborilgan ma’lumotlarni aniqlash uchun StringContent ob’ekti yaratiladi. Yuborilgan satr uning konstruktoriga uzatiladi.

Keyinchalik, biz HttpRequestMessage ob’ektini aniqlaymiz va uning tarkibini - serverga yuboriladigan tarkibni o’rnatamiz.

Keyinchalik, SendAsync () usuli yordamida ushbu ma’lumotlar veb-ilovaga yuboriladi. Mening holimda veb-ilova quyidagi manzilda ishlaydi https://localhost:7094", shunga ko’ra, post so’rovi quyidagi manzilga yuboriladi https://localhost:7094/data". Serverning javobi satrda o’qiladi va konsolda ko’rsatiladi. Ya’ni, so’rovni bajargandan so’ng, biz konsolda ko’rishimiz kerak:

Nazorat savollari:

1) HTTP sarlavhalari (headers) nima va ular qanday maqsadlarda ishlatiladi? Sarlavhalarni yuborish va qabul qilish jarayonida nimalarga e’tibor berish kerak?

2) C# dasturida HttpClient yordamida HTTP so’roviga sarlavha qanday qo’shiladi? Misol tariqasida User-Agent yoki Authorization sarlavhasini qanday o’rnatish mumkinligini tushuntiring.

3) HTTP POST so’rovida ma’lumotlarni qanday qilib yuborasiz? JSON yoki x-www-form-urlencoded formatlarida ma’lumotlarni yuborishning farqlari qanday?

4) HTTP sarlavhalarining qanday asosiy turlari mavjud va ularning har biri qanday maqsadda ishlatiladi? Masalan, Content-Type, Accept, va Cache-Control sarlavhalari haqida tushuntiring.

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**5-Mashg‘ulot: HttpClient orqali json malumotlarni yuborish.****Reja:**

1. Sarlavhalarni yuborish va qabul qilish haqida tushuncha.
2. So‘rov uchun “sarlavha” o‘rnatish.
3. So‘rovda ma’lumotlarni yuborish.

1. Sarlavhalarni yuborish va qabul qilish haqida tushuncha.

HttpClient yordamida so‘rovlarni yuborishda sarlavhalarni yuborish va qabul qilishning bir qator usullaridan foydalanish mumkin. Ammo har qanday holatda ham biz sarlavhalar to‘plamini taqdim etadigan System.Net.Http.Headers turi bilan ishlashimiz kerak. Sarlavhalarni boshqarish uchun ushbu tur bir qator usullarni taqdim etadi:

Add(String headerName, IEnumerable<String> values): belgilangan sarlavha va uning qiymatlarini HttpHeaders to‘plamiga qo‘shadi

Add(String headerName, String headerValue): belgilangan sarlavha va uning qiymatini HttpHeaders to‘plamiga qo‘shadi

Clear (): HttpHeaders to‘plamidagi barcha sarlavhalarni olib tashlaydi

Contains(String headerName): HttpHeaders to‘plamida ma’lum bir sarlavha mavjud bo‘lsa, true ni qaytaradi

Remove(String headerName): belgilangan sarlavhani HttpHeaders to‘plamidan olib tashlaydi

TryGetValues (String headerName, IEnumerable<String> values): agar httpheaders to‘plamida mavjud bo‘lsa, ko‘rsatilgan sarlavha uchun qiymatlar to‘plamini qaytaradi

Ushbu sinf mavhum bo‘lib, **HttpContentHeaders**, **HttpRequestHeaders** va **HttpResponseHeaders**. sinflari tomonidan amalga oshiriladi.

Sarlavhalarni yuborish

HttpClient uchun global sarlavhalarni o‘rnatish

Avvalo, biz **DefaultRequestHeaders** HttpClient sinfidan foydalanib sarlavhalarni global miqyosda sozlashimiz mumkin. Ushbu xususiyat **DefaultRequestHeaders** turini ifodalaydi, ya’ni yuqorida ko‘rsatilgan barcha usullarni amalga oshiradi.

Masalan, bizda quyidagi veb-ilova bo‘lsin ASP.NET, bu ikkita sarlavhani oladi: standart "User-Agent" va maxsus "SecreteCode":

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapGet("/", (HttpContext context) =>
{
    context.Request.Headers.TryGetValue("User-Agent", out var userAgent);
```



```

context.Request.Headers.TryGetValue("SecreteCode", out var
secreteCode);
return $"User-Agent: {userAgent}    SecreteCode: {secreteCode}";
});

app.Run();

```

HttpClient yordamida konsol dasturidan sarlavhalarni yuboring:

```

class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        var serverAddress = "https://localhost:7094/";
        httpClient.DefaultRequestHeaders.Add("User-Agent", "Mozilla Firefox
5.4");
        httpClient.DefaultRequestHeaders.Add("SecreteCode", "hello");

        using var response = await httpClient.GetAsync(serverAddress);
        var responseText = await response.Content.ReadAsStringAsync();
        Console.WriteLine(responseText);
    }
}

```

Shuni ta'kidlash kerakki, alohida sarlavhalarni o'rnatishni osonlashtirish uchun httprequestheaders klassi bir qator maxsus xususiyatlarni taqdim etadi:

- **Accept:** qabul qilish sarlavhasi qiymatini qaytaradi;
- **AcceptCharset:** accept-Charset sarlavhasi qiymatini qaytaradi;
- **AcceptEncoding:** qabul qilish-kodlash sarlavhasi qiymatini qaytaradi;
- **AcceptLanguage:** qabul qilish-til sarlavhasi qiymatini qaytaradi;
- **Authorization:** avtorizatsiya sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **CacheControl:** Keshni boshqarish sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **Connection:** ulanish sarlavhasi qiymatini qaytaradi;
- **Date:** sana sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **Expect:** expect sarlavhasi qiymatini qaytaradi;
- **From:** From sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **Xost:** xost sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **IfMatch:** if-Match sarlavhasi qiymatini qaytaradi;
- **IfModifiedSince:** if-Modified-Since sarlavhasi qiymatini qaytaradi yoki o'rnatadi;
- **IfNoneMatch:** if-None-Match sarlavhasi qiymatini qaytaradi;
- **IfRange:** if-Range sarlavhasi qiymatini qaytaradi yoki o'rnatadi;

- **IfUnmodifiedSince:** if-Unmodified-Since sarlavhasi qiymatini qaytaradi yoki o‘rnatadi;
- **MaxForwards:** Max-Forwards sarlavhasi qiymatini qaytaradi yoki o‘rnatadi;
- **Pragma:** Pragma sarlavhasi qiymatini qaytaradi;
- **ProxyAuthorization:** proxy-Authorization sarlavhasi qiymatini qaytaradi yoki o‘rnatadi;
- **Range:** Range sarlavhasi qiymatini qaytaradi yoki o‘rnatadi;
- **Referer:** referer sarlavhasi qiymatini qaytaradi yoki o‘rnatadi;
- **TE:** te sarlavhasi qiymatini qaytaradi;
- **Trailer:** Trailer sarlavhasi qiymatini qaytaradi;
- **TransferEncoding:** transfer-Encoding sarlavhasi qiymatini qaytaradi;
- **Upgrade:** yangilash sarlavhasi qiymatini qaytaradi;
- **UserAgent:** user-Agent sarlavhasi qiymatini qaytaradi;
- **Via:** Via sarlavhasi qiymatini qaytaradi;
- **Warning:** ogohlantirish sarlavhasi qiymatini qaytaradi.

Bunday xususiyatlardan foydalanish sarlavhani nomlashda xatolikka yo‘l qo‘yilmasligini ta‘minlashga imkon beradi. Har bir shunga o‘xshash xususiyat `httpheaderValueCollection` ob‘ektini ifodalaydi, u ma‘lum bir tur bilan yoziladi - bu turdagi ob‘ekt sarlavha qiymatini ifodalaydi. Masalan, "foydalanuvchi-Agent"ni o‘rnatish:

```
httpClient.DefaultRequestHeaders.UserAgent.Add(new
ProductInfoHeaderValue("User-Agent", "Mozilla Firefox 5.4");
```

2. So‘rov uchun “sarlavha” o‘rnatish.

Agar sarlavha faqat ma‘lum bir so‘rov uchun o‘rnatilishi kerak bo‘lsa, unda siz `HttpRequestMessage` sinfining `Headers` xususiyatidan foydalanishingiz mumkin, bu ham `HttpRequestHeaders` turini ifodalaydi:

```
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        var serverAddress = "https://localhost:7094/";
        using var request = new HttpRequestMessage(HttpMethod.Get,
serverAddress);
        request.Headers.Add("User-Agent", "Mozilla Firefox 5.6");
        request.Headers.Add("SecreteCode", "hello");

        using var response = await httpClient.SendAsync(request);
        var responseText = await
response.Content.ReadAsStringAsync();
        Console.WriteLine(responseText);
    }
}
```

```
}
```

Server o‘rnatgan sarlavhalarni olish uchun siz HttpResponseMessage sinfining Headers xususiyatidan foydalanishingiz mumkin, bu HttpHeaders turini ifodalaydi. Masalan, biz kirish sanasini ko‘rsatadigan "sana" sarlavhasini olamiz:

```
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        var serverAddress = "https://localhost:7094/";
        using var response = await httpClient.GetAsync(serverAddress);
        var dateValues = response.Headers.GetValues("Date");

        Console.WriteLine(dateValues.FirstOrDefault());
    }
}
```

Shuni ta’kidlash kerakki, agar ko‘rsatilgan nom bilan sarlavha javobda bo‘lmasa, unda dastur istisno yaratadi. Bunday vaziyatni oldini olish uchun sarlavha qiymatini olishdan oldin uning mavjudligini Contains () usuli bilan tekshirishimiz mumkin (agar sarlavha mavjud bo‘lsa, rost qaytadi). Yoki TryGetValue () usulidan foydalaning, u chiqish parametriga sarlavha qiymatlarini muvaffaqiyatli qabul qilganda true ni qaytaradi. Ammo sarlavha bo‘lmasa, chiqish parametri null bo‘ladi.

```
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        var serverAddress = "https://localhost:7094/";
        using var response = await httpClient.GetAsync(serverAddress);
        response.Headers.TryGetValue("date", out var dateValues);

        Console.WriteLine(dateValues?.FirstOrDefault());
    }
}
```

Shuni ham ta’kidlash kerakki, umumiy sarlavhalarni olishni soddalashtirish uchun HttpHeaders klassi bir qator maxsus usullarni taqdim etadi:

- **AcceptRanges:** qabul qilish-chegaralar sarlavhasi qiymatini qaytaradi
- **Age:** Age sarlavhasi qiymatini qaytaradi yoki o‘rnatadi
- **CacheControl:** Keshni boshqarish sarlavhasi qiymatini qaytaradi yoki o‘rnatadi
- **Connection:** ulanish sarlavhasi qiymatini qaytaradi
- **Date:** sana sarlavhasi qiymatini qaytaradi yoki o‘rnatadi
- **ETag:** etag sarlavhasi qiymatini qaytaradi yoki o‘rnatadi

- **Location:** joylashuv sarlavhasi qiymatini qaytaradi yoki o‘rnatadi
- **Pragma:** Pragma sarlavhasi qiymatini qaytaradi
- **ProxyAuthenticate:** proxy-Authenticate sarlavhasi qiymatini qaytaradi
- **RetryAfter:** Retry-after sarlavhasi qiymatini qaytaradi yoki o‘rnatadi
- **Server:** Server sarlavhasi qiymatini qaytaradi
- **Trailer:** Trailer sarlavhasi qiymatini qaytaradi
- **TransferEncoding:** transfer-Encoding sarlavhasi qiymatini qaytaradi
- **Upgrade:** yangilash sarlavhasi qiymatini qaytaradi
- **Vary:** Vary sarlavhasi qiymatini qaytaradi
- **Via:** Via sarlavhasi qiymatini qaytaradi
- **Warning:** ogohlantirish sarlavhasi qiymatini qaytaradi
- **WwwAuthenticate:** HTTP javobi uchun www-Authenticate sarlavha qiymatini qaytaradi.

3. So‘rovda ma’lumotlarni yuborish.

HttpClient ba’zi ma’lumotlarni serverga yuborish imkonini beradi. Yuborilgan ma’lumotlar System.Net.Http.HttpContent ob’ektini ifodalaydi. ammo bu sinf mavhum bo‘lganligi sababli, aslida yuborish uchun eski sinflardan biri ishlatiladi. Xususan,. net-da HttpContent-dan meros bo‘lib o‘tgan quyidagi ichki sinflar mavjud:

System.Net.Http. ByteArrayContent: baytlar qatorini yuborish uchun

System. Net. Http. FormUrlEncodedContent: "application/x-www-form-urlencoded" MIME turi bilan kodlangan ma’lumotlarni yuborish uchun. (ByteArrayContent-dan meros qilib olingan)

System.Net. Http. StringContent: matnni yuborish uchun (ByteArrayContent-dan meros qilib olingan)

System.Net.Http.JSON.JsonContent: JSON yuborish uchun

System.Net.Http. MultipartContent: fayllarni yuborish uchun

System.Net.Http. MultipartFormDataContent: fayllarni yuborish uchun (Multipart Content-dan meros qilib olingan)

System.Net. Http. ReadOnlyMemoryContent: yuborilgan tarkib sifatida ReadOnlyMemory<t>ishlatiladi

Vazifalarga va biz o‘zaro aloqada bo‘lgan maxsus serverga qarab, biz ushbu turlardan birini qo‘llashimiz mumkin.

Odatda ma’lumotlar Post/Put/Patch kabi so‘rovlar orqali yuboriladi. HttpClient - da siz ma’lumotlarni yuborish uchun SendAsync () usulidan foydalanishingiz mumkin-parametr sifatida u HttpRequestMessage ob’ektini qabul qiladi, unda siz yuborilgan tarkibni tarkib xususiyati orqali o‘rnatishingiz mumkin.

Biroq, so‘rovlarni yuborishni osonlashtirish uchun HttpClient har bir so‘rov turi uchun usullarni ham taqdim etadi:

- **PostAsync()**
- **PutAsync()**
- **Patch sync()**

Ushbu usullarning barchasi kamida ikkita parametрни oladi: resurs manzili va http Content ob’ekti sifatida yuborilgan ma’lumotlar?:

```
Task<HttpResponseMessage> PostAsync (string? requestUri,
System.Net.Http.HttpContent? content);
Task<HttpResponseMessage> PutAsync (string? requestUri,
System.Net.Http.HttpContent? content);
Task<HttpResponseMessage> PatchAsync (string? requestUri,
System.Net.Http.HttpContent? content);
```

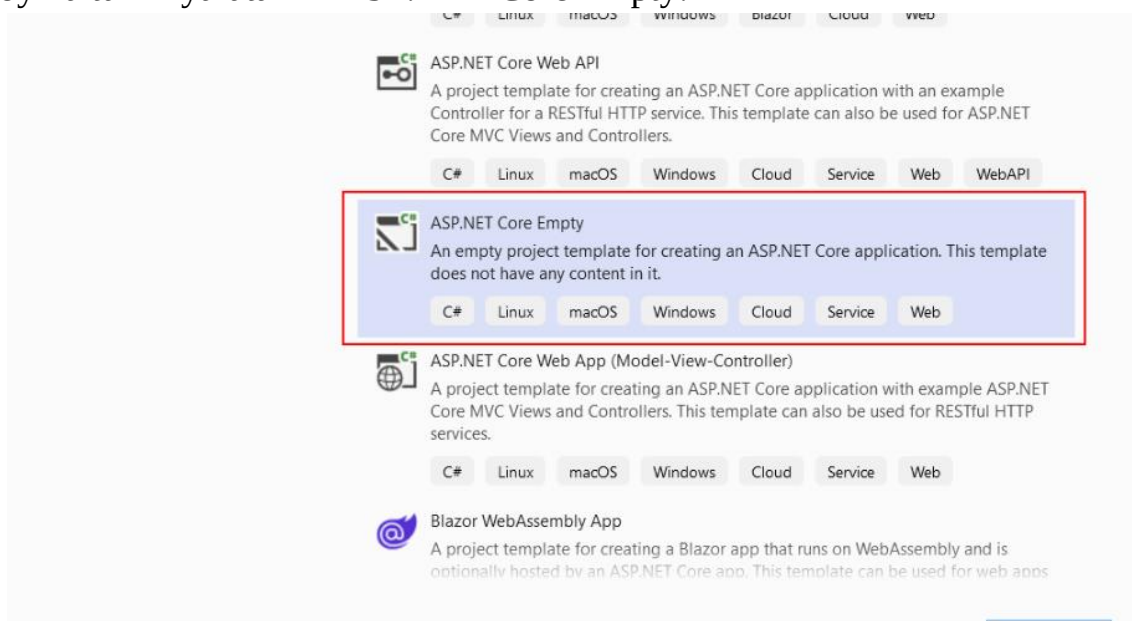
Eng oddiy misol - matnni yuborish orqali ma’lumotlarning odatiy yuborilishini ko‘rib chiqing.

Matnni yuborish

Oddiy matnli tarkibni yuborish uchun stringcontent klassi ishlatiladi, u konstruktor orqali yuborilgan qatorni o‘rnatadi:

```
StringContent content = new StringContent(отправляемая_строка);
```

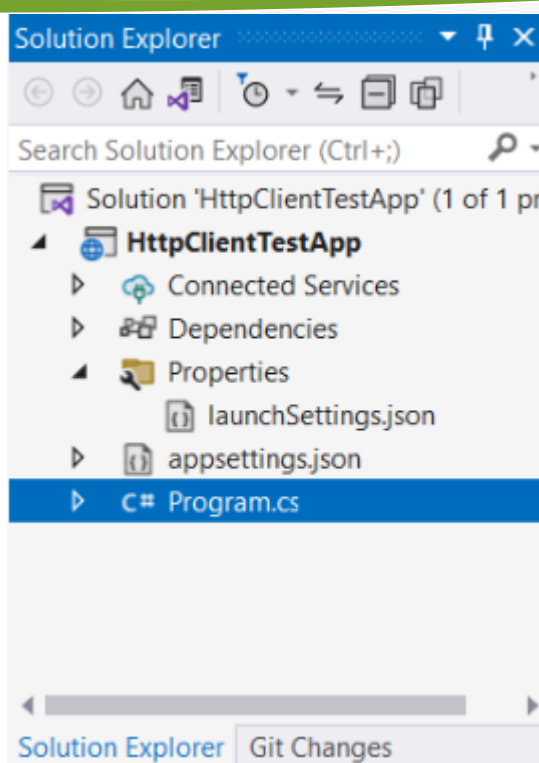
Sinov uchun avval server dasturini aniqlaymiz ASP.NET yadro. Buning uchun biz loyiha turini yaratamiz ASP.NET Core Empty:



8-rasm: ASP.NET Core ni o‘rnatish oynasi.

Masalan, mening holimda loyiha Http Client Test App deb nomlanadi. Ushbu loyihada dastur fayliga ochamiz.cs

Nazorat savollari:



9-rasm: Project Solution oynasi natijalari.

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapPost("/data", async (HttpContext httpContext) =>
{
    using StreamReader reader = new StreamReader(httpContext.Request.Body);
    string name = await reader.ReadToEndAsync();
    return $"Получены данные: {name}";
});

app.Run();
```

Veb-ilovani yaratish uchun ASP.NET Core avval biz WebApplicationBuilder ob'ektini yaratishimiz kerak:

```
var builder = WebApplication.CreateBuilder();
```

Keyin, uning qurish usulidan foydalanib, biz veb-ilova ob'ektini yaratamiz, u aslida dasturni ifodalaydi ASP.NET:

```
var app = builder.Build();
```

App usuli yordamida.Map Post () "/data"yo'lida post so'rovlarini bajaradigan so'nggi nuqtani aniqlang. Ushbu so'nggi nuqta ishlovchisi so'rov kontekstini parametr sifatida qabul qiladi-httpcontext ob'ekti:

```
app.MapPost("/data", async (HttpContext httpContext) =>
```

Bu holda biz satrni yuborishni sinab ko'rayotganimiz sababli, so'rov ma'lumotlari httpContext.Request.Body - ni satrga-name o'zgaruvchisiga o'qing va keyin mijozga "ma'lumotlar olingan: {name}"qatorini qaytaring.

Keyin dasturni ishga tushiring:

```
app.Run();
```

Endi biz konsol dasturini aniqlaymiz, unda HttpClient veb-ilovaga ba'zi qatorlarni yuboradi. Birinchidan, SendAsync () usulidan foydalaning:

Yuborilgan ma'lumotlarni aniqlash uchun StringContent ob'ekti yaratiladi. Yuborilgan satr uning konstruktoriga uzatiladi.

Keyinchalik, biz HttpRequestMessage ob'ektini aniqlaymiz va uning tarkibini - serverga yuboriladigan tarkibni o'rnatamiz.

Keyinchalik, SendAsync () usuli yordamida ushbu ma'lumotlar veb-ilovaga yuboriladi. Mening holimda veb-ilova quyidagi manzilda ishlaydi `https://localhost:7094`", shunga ko'ra, post so'rovi quyidagi manzilga yuboriladi `https://localhost:7094/data`". Serverning javobi satrda o'qiladi va konsolda ko'rsatiladi. Ya'ni, so'rovni bajargandan so'ng, biz konsolda ko'rishimiz kerak:

Nazorat savollari:

1. HTTP protokolida paket sarlavhasi qanday o'rnatiladi.
2. So'rovlarda ma'lumot yuborish usuli haqida ma'lumot bering.
3. Ma'lumot paketlari qanday tuzilmaga ega ?

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**6-Mashg‘ulot:** FormUrlEncodedContent klassi**Reja:**

1. Shakllar va FormUrlEncodedContent sinfini yuborish.
2. Oqimlar va bayt massivini yuborish.
3. Baytlar massivi yuborish(ByteArrayContent).

1. Shakllar va FormUrlEncodedContent sinfini yuborish.

HttpClient yordamida shakl ma'lumotlarini yuborish uchun formurlencodedcontent klassi qo'llaniladi. U parametr sifatida IEnumerable<KeyValuePair<string, string>> ob'ektini qabul qiladigan bitta konstruktorga ega, ya'ni ba'zi kalit-qiymat juftliklari to'plami.

Sinov uchun quyidagi veb-ilovani aniqlang ASP.NET Core:2. Oqimlar va bayt massivini yuborish.

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapPost("/data", async(HttpContext httpContext) =>
{
    var form = httpContext.Request.Form;
    string? name = form["name"];
    string? email = form["email"];
    string? age = form["age"];
    await httpContext.Response.WriteAsync($"Name: {name}    Email:{email}
Age: {age}");
});
app.Run();
```

Bu erda app usuli yordamida.Map Post () "/data"/manzilidagi post so'rovlarini qayta ishlaydigan bitta so'nggi nuqtani belgilaydi. HttpContext xususiyati orqali HttpContext so'rovi kontekstidan foydalangan holda oxirgi nuqta ishlov Beruvchisida.Request.Form biz yuborilgan shaklni olamiz va keyin ma'lum kalitlardan yuborilgan qiymatlarni chiqaramiz. Bunday holda, mijoz "name", "email" va "age" tugmachalari bilan uchta qiymatni yuboradi deb taxmin qilamiz. Va oxirida biz ushbu ma'lumotlarni mijozga bir qatorda qaytarib yuboramiz.

Quyidagi konsol ilovasi mijoz sifatida ishlaydi:

```
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        Dictionary<string, string> data = new Dictionary<string, string>
        {
            ["name"] = "Tom",
            ["email"] = "tom@localhost.com",
```



```

        ["age"] = "38"
    };
    HttpContent contentForm = new FormUrlEncodedContent(data);
    using var response = await
httpClient.PostAsync("https://localhost:7094/data", contentForm);
    string responseText = await response.Content.ReadAsStringAsync();
    Console.WriteLine(responseText);
}
}

```

Yuborilgan ma'lumotlar `IEnumerable<keyvaluepair<string, string>>` turini aks ettirishi kerak va biz standart lug'atdan foydalanishimiz mumkin, bu erda kalitlar va qiymatlar satrlarni ifodalaydi. Va bu holda, shunga o'xshash lug'at aniqlanadi, unga ma'lumotlarni olish uchun serverda ishlatiladigan "ism", "elektron pochta" va "yosh" tugmachalari bilan uchta element qo'shiladi.

Konsolda dasturni bajarish natijasida biz yuborilgan ma'lumotlarni o'z ichiga olgan serverning javobini ko'rib chiqamiz:

Name: Tom Email:tom@localhost.com Age: 38

18-rasm: Console oynasi natijalari.

2. Oqimlar va bayt massivini yuborish.

Oqim tarkibi sinfi `HttpClient` yordamida oqimlarni yuborish uchun ishlatiladi. Uning konstruktori oqim ob'ektini, ya'ni ma'lumotlari serverga yuboriladigan oqimni majburiy parametr sifatida qabul qiladi. Ma'lumotlar oqimida biron bir faylni yuborish misolini ko'rib chiqing .

Sinov uchun quyidagi veb-ilovani aniqlang ASP.NET Core:

```

var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapPost("/data", async(HttpContext httpContext) =>
{
    var uploadPath = $"{Directory.GetCurrentDirectory()}/uploads";
    Directory.CreateDirectory(uploadPath);
    string fileName = Guid.NewGuid().ToString();
    using (var fileStream = new FileStream($"{uploadPath}/{fileName}.jpg",
    FileMode.Create))
    {
        await httpContext.Request.Body.CopyToAsync(fileStream);
    }

    await httpContext.Response.WriteAsync("Данные сохранены");
});
app.Run();

```

Bu erda app usuli yordamida `Map Post () "/data"/manzilidagi` post so'rovlarini qayta ishlaydigan bitta so'nggi nuqtani belgilaydi. Oxirgi nuqta ishlov Beruvchisida

biz yuklab olingan fayllar uchun katalog yaratamiz - u "yuklamalar" deb nomlanadi va loyiha papkasida joylashgan bo'ladi. Shuningdek, GUID yordamida o'zboshimchalik bilan fayl nomini yarating.NewGuid()).

So'rovda yuborilgan barcha ma'lumotlar httpContext xususiyati orqali mavjud.Request.Body-bu Stream ob'ektini, ya'ni aslida streamcontent yordamida mijozga yuboriladigan oqimni anglatadi. Va ushbu oqimdan ma'lumotlarni faylga saqlash uchun biz FileStream ob'ektini aniqlaymiz va CopyToAsync() usuli yordamida httpContext-dan ma'lumotlarni nusxalaymiz.Request.FileStream-dagi tana (asosan faylga)

```
await httpContext.Request.Body.CopyToAsync(fileStream);
```

Bu erda sinov uchun fayl "jpg" kengaytmasiga ega bo'ladi deb taxmin qilamiz. Oxirida biz mijozga javoban ba'zi diagnostika xabarlarini yuboramiz.

Quyidagi konsol ilovasi mijoz sifatida ishlaydi:

```
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        string filePath = "D:\\forest.jpg";
        using var fileStream = File.OpenRead(filePath);
        StreamContent content = new StreamContent(fileStream);
        using var response = await
httpClient.PostAsync("https://localhost:7094/data", content);
        string responseText = await response.Content.ReadAsStringAsync();
        Console.WriteLine(responseText);
    }
}
```

Sinov sifatida bu erda fayldan ma'lumotlar yuboriladi "D:\\forest.jpg". Yuborish uchun avval ushbu faylni ifodalovchi FileStream oqimi yaratiladi:

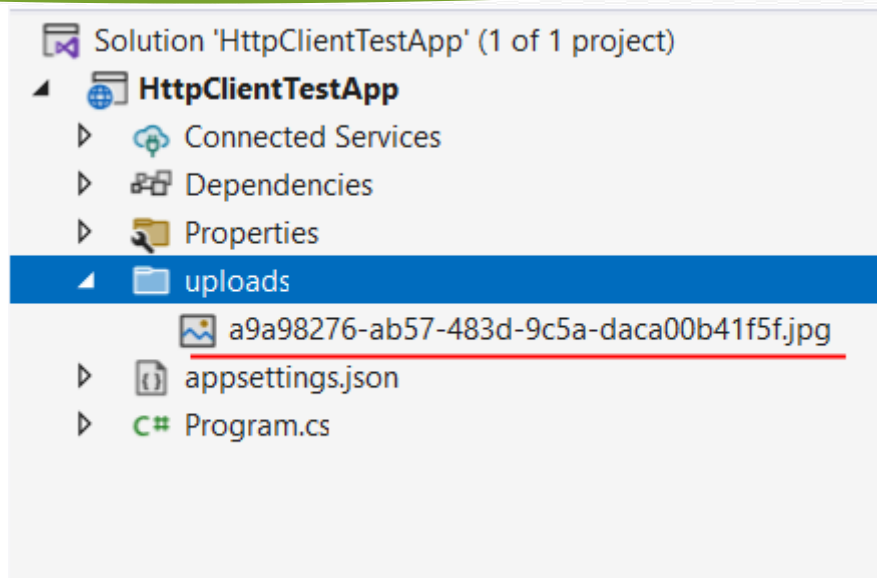
```
using var fileStream = File.OpenRead(filePath);
```

Bundan tashqari, oqim StreamContent-ga uzatiladi:

```
StreamContent content = new StreamContent(fileStream);
```

Keyin stream Content serverga httpClient usulida yuboriladi.PostAsync () (mening holimda veb-ilova ASP.NET "manzilida ishga tushirildi https://localhost:7094/").

Veb-ilova loyihasida dasturni bajarish natijasida ASP.NET "Yuklashlar" papkasi paydo bo'ladi va fayl unga saqlanadi:



19-rasm: Project Solution oynasi tuzilmasi.

3. Baytlar massivi yuborish(ByteArrayContent).

Baytlar qatorini yuborish uchun bytearraycontent klassi ishlatiladi, uning konstruktori baytlar qatorini majburiy parametr sifatida qabul qiladi. Masalan, oddiy misol uchun, qatorni baytlar qatoriga o‘zgartiring va serverga yuboring:

```
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        string message = "Hello METANIT.COM";
        byte[] messageToBytes =
System.Text.Encoding.UTF8.GetBytes(message);
        var content = new ByteArrayContent(messageToBytes);
        using var response = await
httpClient.PostAsync("https://localhost:7094/data", content);
        string responseText = await response.Content.ReadAsStringAsync();
        Console.WriteLine(responseText);
    }
}
```

Bu erda tizim usuli yordamida qatorni baytlar qatoriga aylantiramiz.Text.Encoding.UTF8.GetBytes(). M yuborishni sinab ko‘rish uchun quyidagi veb-ilovani aniqlash mumkin ASP.NET Core:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapPost("/data", async(HttpContext httpContext) =>
{
    using StreamReader streamReader = new
StreamReader(httpContext.Request.Body);
```

```
string message = await streamReader.ReadToEndAsync();  
await httpContext.Response.WriteAsync($"Отправлено сообщение:  
{message}");  
});  
app.Run();
```

Nazorat savollari:

1. FormUrlEncodedContent xususiyatlari haqida ma'lumot bering.
2. Oqim nima va qanday xususiyatlari bor?
3. ByteArrayContent nima ?

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**7-Mashg‘ulot:** MultipartFormDataContent klassi.**Reja:**

1. Fayllarni yuborish va MultipartFormDataContent klassi.
2. Birdaniga bir nechta fayllar yuborish.
3. HttpClient yordamida cookie-fayllarni yuborish va qabul qilish.

1. Fayllarni yuborish va MultipartFormDataContent klassi.

Fayllarni serverga yuborish uchun HttpClient System.Net.Http.Multipart form Data Content sinfidan foydalanadi. Aslida, bu sinf Http Content ob’ektlarining konteyneri sifatida ishlaydi. Va multipart form Data Content-ga elementlarni qo‘shish uchun Add () usuli qo‘llaniladi

```
public void Add(HttpContent content, string name, string fileName);
```

Birinchi parametr - yuborilgan tarkib (bu fayllar yoki boshqa ma’lumotlar bo‘lishi mumkin). Ikkinchi parametr-name serverda faylni olishimiz mumkin bo‘lgan so‘rovdagi ma’lumotlar nomini belgilaydi. Uchinchi parametr-fileName fayl nomini o‘rnatadi.

Bitta faylni yuklash

Faylni yuklab olishni sinab ko‘rish uchun dasturni aniqlang ASP.NET quyidagi kod bilan Core:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapPost("/upload", async (HttpContext context) =>
{
    IFormFileCollection files = context.Request.Form.Files;
    var uploadPath = $"{Directory.GetCurrentDirectory()}/uploads";
    Directory.CreateDirectory(uploadPath);
    foreach (var file in files)
    {
        string fullPath = $"{uploadPath}/{file.FileName}";

        using (var fileStream = new FileStream(fullPath,
        FileMode.Create))
        {
            await file.CopyToAsync(fileStream);
        }
    }
    await context.Response.WriteAsync("Файлы успешно загружены");
});

app.Run();
```

Bu erda app usuli.Map Post () "yuklash"yo‘lidagi so‘rovlarni qayta ishlaydigan so‘nggi nuqtani belgilaydi. Oxirgi nuqta ishlov Beruvchisida parametr sifatida biz httpcontext so‘rovining kontekstini va u orqali context xususiyatidan

foydalanamiz. Request.Form.Files biz yuklab olingan fayllar to'plamini olamiz. Keyinchalik, biz ushbu to'plam bo'ylab yuguramiz va loyihadagi barcha ma'lumotlarni "yuqoriga ko'tarish" papkasida saqlaymiz (agar u bo'lmasa, u yaratiladi).

Endi biz mijozni aniqlaymiz:

```
using System.Net.Http.Headers;
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        var serverAddress = "https://localhost:7094/upload";
        var filePath = @"D:\forest.jpg";
        using var multipartFormContent = new
MultipartFormDataContent();
        var fileStreamContent = new
StreamContent(File.OpenRead(filePath));
        fileStreamContent.Headers.ContentType = new
MediaTypeHeaderValue("image/jpeg");
        multipartFormContent.Add(fileStreamContent, name: "file",
fileName: "forest.jpg");
        using var response = await
httpClient.PostAsync(serverAddress, multipartFormContent);
        var responseText = await
response.Content.ReadAsStringAsync();
        Console.WriteLine(responseText);
    }
}
```

Shunday qilib, mening holimda veb-ilova ""da ishlaydi https://localhost:7094/", shuning uchun fayllarni qabul qiladigan so'nggi nuqtaga kirish uchun men ""manzilidan foydalanaman https://localhost:7094/upload". Bundan tashqari, mening holatimda fayl yuklanadi "D:\forest.jpg".

Yuborish uchun MultipartFormDataContent yarating

```
using var multipartFormContent = new MultipartFormDataContent();
```

Faylni yuborishda ishlatiladigan ob'ekt bilan bog'liq barcha oqimlarni yopishga imkon beradigan using dizayniga e'tibor bering.

Keyin fayl tarkibini fayl oqimi sifatida qabul qiladigan Stream Content ob'ektini yarating:

```
var fileStreamContent = new StreamContent(File.OpenRead(filePath));
```

Yuklab olingan faylga mos keladigan mime turini o'rnatish:

```
fileStreamContent.Headers.ContentType
= new MediaTypeHeaderValue("image/jpeg");
```

Keyinchalik, ushbu ob'ektni MultipartFormDataContent-ga qo'shing:

```
multipartFormContent.Add(fileStreamContent, name: "file", fileName:
"forest.jpg");
```

Oxirida biz faylni yuboramiz va javobni olamiz:

```
using var response = await httpClient.PostAsync(serverAddress,
multipartFormContent);
var responseText = await response.Content.ReadAsStringAsync();
```

Shuni ta’kidlash kerakki, faylni yuborish uchun StreamContent-dan foydalanishimiz shart emas. Masalan, fayl ma’lumotlarini baytlar qatoriga sanash va bytearraycontent sinfidan foydalanib yuborish mumkin:

```
using System.Net.Http.Headers;
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        var serverAddress = "https://localhost:7094/upload";
        var filePath = @"D:\forest.jpg";
        using var multipartFormContent = new
        MultipartFormDataContent();
        byte[] fileToBytes = await File.ReadAllBytesAsync(filePath);
        var content = new ByteArrayContent(fileToBytes);
        content.Headers.ContentType = new
        MediaTypeHeaderValue("image/jpeg");
        multipartFormContent.Add(content, name: "file", fileName:
        "forest5.jpg");
        using var response = await
        httpClient.PostAsync(serverAddress, multipartFormContent);
        var responseText = await
        response.Content.ReadAsStringAsync();
        Console.WriteLine(responseText);
    }
}
```

Server kodi bir xil bo‘lib qoladi.

2. Birdaniga bir nechta fayllar yuborish.

Xuddi shunday, siz ko‘proq fayllarni yuborishingiz mumkin. Masalan,:

```
using System.Net.Http.Headers;
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        var serverAddress = "https://localhost:7094/upload";
        var files = new string[] { "D:\forest.jpg", "D:\cats.jpg" };
        using var multipartFormContent = new
        MultipartFormDataContent();
        foreach (var file in files)
        {
            var fileName = Path.GetFileName(file);
            var fileStreamContent = new
            StreamContent(File.OpenRead(file));
```

```

        fileStreamContent.Headers.ContentType = new
MediaTypeHeaderValue("image/jpeg");
        multipartFormContent.Add(fileStreamContent, name:
"files", fileName: fileName);
    }
    using var response = await
httpClient.PostAsync(serverAddress, multipartFormContent);
    var responseText = await
response.Content.ReadAsStringAsync();
    Console.WriteLine(responseText);
}
}

```

Serverda endi context to‘plamidan.Request.Form biz "foydalanuvchi nomi" va "elektron pochta" kalitlari bilan ma’lumotlarni olamiz (shartli foydalanuvchi nomi va elektron pochta manzili).

Mijozda quyidagi kodni aniqlang:

```

using System.Net.Http.Headers;
class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        using var multipartFormContent = new
MultipartFormDataContent();
        multipartFormContent.Add(new StringContent("Tom"), name:
"username");
        multipartFormContent.Add(new
StringContent("tom@localhost.com"), name: "email");
        var fileStreamContent = new
StreamContent(File.OpenRead("D:\\logo.jpg"));
        fileStreamContent.Headers.ContentType = new
MediaTypeHeaderValue("image/jpeg");
        multipartFormContent.Add(fileStreamContent, name: "files",
fileName: "logo.jpg");
        using var response = await
httpClient.PostAsync("https://localhost:7094/upload",
multipartFormContent);
        var responseText = await
response.Content.ReadAsStringAsync();
        Console.WriteLine(responseText);
    }
}

```

Bunday holda, Multipart form Data Content fayliga qo‘shimcha ravishda, oddiy qatorni ifodalovchi ikkita stringcontent ob’ekti qo‘shiladi. Ikkala ob’ektning kalitlari serverda ma’lumotlarni olish uchun ishlatiladigan narsalarga mos keladi - "foydalanuvchi nomi" va "elektron pochta".

Amalga oshirish natijasida konsol bizni chiqaradi:

Va serverda Yuklashlar papkasida yana bir fayl paydo bo‘ladi - logo.jpg.

3. *HttpClient yordamida cookie-fayllarni yuborish va qabul qilish.*

Cookie-fayllarni olish yoki jo‘natishdan qat’i nazar, biz ikkita sinf bilan shug‘ullanishimiz kerak: Cookie va CookieContainer. Shuning uchun, avval ushbu turlarni qisqacha ko‘rib chiqing.

Cookie

Cookie klassi aslida. NET-da Cookie-fayllarni ifodalaydi. ushbu turdagi to‘rtta konstruktor mavjud:

```
public Cookie ();
public Cookie (string name, string? value);
public Cookie (string name, string? value, string? path);
public Cookie (string name, string? value, string? path, string?
domain);
```

Konstruktorlar parametrlari:

- **name:** Cookie nomi
- **value:** Cookie qiymati
- **path:** yo‘l (standart"/", ya'ni veb-ilovaning ildizi)
- **domain:** Cookie ma'lumotlari yaratilgan domen

Muhim Kukni o‘rnatish yoki olish uchun siz ushbu sinfning bir qator xususiyatlaridan foydalanishingiz mumkin:

- **Comment:** server Cookie-faylga qo‘shishi mumkin bo‘lgan sharhni qaytaradi yoki o‘rnatadi
- **CommentUri:** server Cookie bilan ta'minlashi mumkin bo‘lgan uri uchun sharhni qaytaradi yoki o‘rnatadi
- **Discard:** server tomonidan o‘rnatilgan asl holatini tiklash bayrog‘ini qaytaradi yoki o‘rnatadi
- **Domen:** URI-ni qaytaradi yoki o‘rnatadi, buning uchun Cookie-fayl qabul qilinadi
- **Expired:** Cookie-ning joriy holatini qaytaradi yoki o‘rnatadi
- **Expires:** Cookie-fayllarga sana va tugash vaqtini DateTime sifatida qaytaradi yoki belgilaydi
- **HttpOnly:** Cookie-faylga kirish mumkinligini aniqlaydi sahifa skripti yoki boshqa faol tarkib
- **Name:** Cookie nomini qaytaradi yoki o‘rnatadi
- **Path:** Cookie qo‘llaniladigan URI identifikatorlarini qaytaradi yoki o‘rnatadi
- **Port:** Cookie-fayllar qo‘llaniladigan TCP portlari ro‘yxatini qaytaradi yoki o‘rnatadi
- **Secure:** Cookie xavfsizligi darajasini qaytaradi yoki o‘rnatadi

- **TimeStamp:** cookie-faylning chiqish vaqtini DateTime sifatida qaytaradi
- **Qiymat:** Cookie ob’ekti uchun qiymatni qaytaradi yoki o‘rnatadi
- **Version:** cookie-faylga mos keladigan HTTP holatini eslab qolishni qo‘llab-quvvatlash versiyasini qaytaradi yoki o‘rnatadi

Masalan, Cookie fayllarini yaratish va ularning xususiyatlarini olish:

```
Cookie nameCookie = new Cookie("name", "Tom");
Console.WriteLine(nameCookie.Name);      // name
Console.WriteLine(nameCookie.Value);     // Tom
```

CookieContainer

CookieContainer klassi Cookie konteynerini, ya’ni Cookie-fayllar to‘plami ustidagi ba’zi qo‘shimchalarni ifodalaydi. Konteyner konfiguratsiyasi uchun sinf bir nechta xususiyatlarni aniqlaydi:

- **Capacity:** CookieContainer-da saqlanishi mumkin bo‘lgan Cookie-fayllar sonini oladi yoki o‘rnatadi. Standart qiymat-300
- **Count:** CookieContainer-da mavjud bo‘lgan Cookie-fayllar sonini qaytaradi.
- **MaxCookieSize:** Cookie-fayllarning ruxsat etilgan maksimal uzunligini baytlarda ifodalaydi. Standart qiymat 4096 bayt
- **PerDomainCapacity:** har bir domen uchun CookieContainer-da saqlanishi mumkin bo‘lgan Cookie-fayllar sonini oladi yoki belgilaydi. Standart qiymat-20

CookieContainer ob’ektini yaratish uchun uchta konstruktor ishlatiladi:

```
public CookieContainer (int capacity, int perDomainCapacity, int
maxCookieSize);
public CookieContainer (int capacity);
public CookieContainer ();
```

CookieContainer-ning tegishli xususiyatlarini o‘rnatish uchun konstruktor parametrlari qo‘llaniladi.

Qo‘g‘irchoqlarni boshqarish uchun CookieContainer bir qator usullarni taqdim etadi. Ularning asosiylari:

Add(Cookie): Cookie ob’ektini CookieContainer-ga qo‘shadi. Usul Cookie qo‘shilgan urini aniqlash uchun Cookie ob’ektidan Domen xususiyatidan foydalanadi.

- **Add(Uri, Cookie):** Cookie nusxasini ma’lum bir URI uchun CookieContainer-ga qo‘shadi.
- **GetAllCookies ():** konteynerning barcha Cookie ob’ektlarini o‘z ichiga olgan CookieCollection ob’ektini qaytaradi.
- **GetCookieHeader (Uri):** ma’lum bir URI bilan bog‘liq barcha Cookie ob’ektlarining ma’lumotlarini o‘z ichiga olgan HTTP cookie sarlavhasini qaytaradi.

- **GetCookies (Uri):** belgilangan URI bilan bog‘langan Cookie ob‘ektlari bilan CookieCollection to‘plamini qaytaradi.
- **SetCookies (Uri, String):** ma‘lum bir URI uchun Cookie-fayllarni qo‘shadi.

Masalan, Kuk qo‘shilishi:

```
Uri uri = new Uri("http://kronos.com");
Cookie nameCookie = new Cookie("name", "Tom");
Cookie emailCookie = new Cookie("email", "tom@localhost.com");
CookieContainer cookieContainer = new CookieContainer();
cookieContainer.Add(uri, nameCookie);
cookieContainer.Add(uri, emailCookie);
```

shu bilan bir qatorda, Cookie-fayllarni setcookie-dan foydalanib, Cookie-fayllarning sarlavhasini(ya‘ni "Cookie-fayl nomi:qiymat_cuki"formatidagi qatorni)o‘tkazish orqali o‘rnatish mumkin.:

```
Uri uri = new Uri("http://kronos.com");
CookieContainer cookieContainer = new CookieContainer();
cookieContainer.SetCookies(uri, "name=Bob");
cookieContainer.SetCookies(uri, "email=bob@localhost.com");
```

Konteynerdan kuklarni olish:

```
var allCookies = cookieContainer.GetAllCookies();
foreach (Cookie cookie in allCookies){
    Console.WriteLine($"{cookie.Name} : {cookie.Value}");
}
var uriCookies = cookieContainer.GetCookies(uri);
foreach (Cookie cookie in uriCookies)
{
    Console.WriteLine($"{cookie.Name} : {cookie.Value}");
}
```

Kuk sarlavhasini olish:

```
var cookieHeader = cookieContainer.GetCookieHeader(uri);
Console.WriteLine(cookieHeader); // name=Tom;
email=tom@localhost.com
```

Kukning umumiy sarlavhasi asosan barcha Cookie-fayllar bir-biridan nuqta-vergul bilan ajratilgan qatorni ifodalaydi.

Endi bularning barchasini HttpClient yordamida http so‘rovida Kukni yuborish va qabul qilishdan qanday foydalanishni ko‘rib chiqing.

Cookie-fayllarni olish

HttpClient-dan foydalanib Cookie-fayllarni olishni sinab ko‘rish uchun quyidagi veb-ilovani aniqlang ASP.NET Core:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapGet("/", (HttpContext context) =>
{
    context.Response.Cookies.Append("name", "Tom");
    context.Response.Cookies.Append("email", "tom@localhost.com");
});
```

```
});  
  
app.Run();
```

Bunday holda, context usuli yordamida `Response.Cookies.Append()` mijozga javoban ikkita Cookie qo'shiladi: ism va elektron pochta.

Endi ushbu ma'lumotlarni mijozda qanday olishni ko'rib chiqing.

Kukni so'rovdan `set-Cookie` sarlavhasi orqali olish

Ikkinchi usul - "Cookie-fayllarni o'rnatish" sarlavhasi orqali so'rovdan Cookie-fayllarni olish:

```
using System.Net;  
  
class Program  
{  
    static HttpClient httpClient = new HttpClient();  
    static async Task Main()  
    {  
        Uri uri = new Uri("https://localhost:7094/");  
  
        using var response = await httpClient.GetAsync(uri);  
        CookieContainer cookies = new CookieContainer();  
        foreach (var cookieHeader in response.Headers.GetValues("Set-Cookie"))  
            cookies.SetCookies(uri, cookieHeader);  
        foreach (Cookie cookie in cookies.GetCookies(uri))  
            Console.WriteLine($"{cookie.Name}: {cookie.Value}");  
        Cookie? email = cookies.GetCookies(uri).FirstOrDefault(c => c.Name == "email");  
        Console.WriteLine($"Электронный адрес: {email?.Value}");  
    }  
}
```

Bunday holda, so'rovni bajargandan so'ng, biz "set-Cookie" tugmachasi orqali barcha sarlavhalarni olamiz va ularni `Setcookies()` yordamida `CookieContainer`-ga qo'shamiz:

```
CookieContainer cookies = new CookieContainer();  
foreach (var cookieHeader in response.Headers.GetValues("Set-Cookie"))  
    cookies.SetCookies(uri, cookieHeader);
```

Keyin Cookie-ni chaqirish orqali `GetCookies(uri)` Cookie fayllarini olamiz. Dasturning konsol chiqishi:

`HttpClientHandler` Ilovasi

`HttpClient` ob'ekti ma'lum bir ishlov beruvchi tomonidan ishga tushiriladi. Odatiy bo'lib, bu `HttpClientHandler` tipidagi ob'ekt. Ushbu tur `CookieContainer` xususiyatiga ega, u `CookieContainer` tipidagi ob'ektni qabul qiladi va berilgan `HttpClientHandler` uchun Cookie konteynerini taqdim etadi.

Agar `httpClientHandler` ob’ektida `Cookie`-fayllardan foydalanish xususiyati rostd bo‘lsa (va bu standart qiymat bo‘lsa), u holda `CookieContainer` xususiyati `Cookie`-fayllarni serverdan saqlash yoki serverga yuborish uchun ishlatiladi.

Masalan, veb-ilovadan "name" va "email" `Cookie`-fayllarini olamiz:

```
using System.Net;
class Program
{
    static HttpClient? httpClient;
    static async Task Main()
    {
        Uri uri = new Uri("https://localhost:7094/");
        var cookies = new CookieContainer();
        using var handler = new HttpClientHandler();
        handler.CookieContainer = cookies;
        httpClient = new HttpClient(handler);
        using var response = await httpClient.GetAsync(uri);
        foreach (Cookie cookie in cookies.GetCookies(uri))
            Console.WriteLine($"{cookie.Name}: {cookie.Value}");
        Cookie? email =
cookies.GetCookies(uri).FirstOrDefault(c=>c.Name== "email");
        Console.WriteLine($"Электронный адрес: {email?.Value}");
    }
}
```

Birinchidan, `Cookie` konteyneri - `CookieContainer` va `httpClientHandler` ob’ekti yaratiladi. Keyin `HttpClientHandler` uchun `Cookie` konteynerini o‘rnating va ushbu `HttpClientHandler` ob’ektidan foydalanadigan `HttpClient` yarating.

So‘rov bajarilgandan so‘ng, `CookieContainer` serverdan `Cookie`-fayllarni o‘z ichiga oladi. Qabul qilish jarayonining qolgan qismi oldingi misol bilan bir xil.

Nazorat savollari:

1. `MultipartFormDataContent` qanday holatlarda qo‘llaniladi
2. `MultiSendFile` metodi asosiy vazifasi qanday ?
3. `HttpClient` vazifasi nima ?

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari
8-Mashg‘ulot: Kukilarni yuborish va HttpListener. HTTP serveri.
Reja:

1. Kukilarni yuborish.
2. HttpListener.
3. So‘rov haqida ma’lumotlarni olish.

1. Kukilarni yuborish.

Mijozdan serverga yuborilgan matn uchun quyidagi veb-ilovani aniqlang
 ASP.NET Core:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.MapGet("/", (HttpContext context) =>
{
    context.Request.Cookies.TryGetValue("name", out string? name);
    context.Request.Cookies.TryGetValue("email", out string? email);

    return $"Name: {name}    Email:{email}";
});

app.Run();
```

Bu erda context usuli yordamida.Request.Cookies.TryGetValue() biz ikkita Kuk - name va elektron pochta qiymatlarini olamiz va qiymatdan bitta satr shaklida mijozga qaytarib yuboramiz.

Eng oson yo‘li-so‘rovni yuborishda "cookie" sarlavhasini o‘rnatish. Masalan,:

```
using System.Net;

class Program
{
    static HttpClient httpClient = new HttpClient();
    static async Task Main()
    {
        Uri uri = new Uri("https://localhost:7094/");

        CookieContainer cookies = new CookieContainer();
        cookies.Add(uri, new Cookie("name", "Bob"));
        cookies.Add(uri, new Cookie("email", "bob@localhost.com"));
        httpClient.DefaultRequestHeaders.Add("cookie",
cookies.GetCookieHeader(uri));

        using var response = await httpClient.GetAsync(uri);
        string responseText = await
response.Content.ReadAsStringAsync();
        Console.WriteLine(responseText);
    }
}
```

Bu erda biz CookieContainer konteyneriga ikkita Cookie faylini qo'shamiz: ism va elektron pochta. Va Cookie-fayllar usulidan foydalanish. GetCookieHeader (uri) biz Kukning barcha ma'lumotlarini o'z ichiga olgan sarlavhaning satr ko'rinishini olamiz va uni "cookie"sarlavhasiga o'tkazamiz.

2. HttpListener.

Http ulanishlarini tinglash va http so'rovlariga javob berish uchun HttpListener sinfi nomlar maydonidan mo'ljallangan System.Net. ushbu sinf HTTP kutubxonasi asosida qurilgan.sys, bu yadro rejimi tinglovchisi va Windows uchun barcha HTTP trafagini boshqaradi. (HttpListener sinfining manba kodi)

HttpListener ob'ektini yaratish uchun uning parametrlarsiz yagona konstruktori ishlatiladi:

```
HttpListener server = new HttpListener();
```

HttpListener xususiyatlari va usullari

HttpListener-ning turli jihatlari haqida ma'lumot olish yoki o'rnatish uchun siz sinf xususiyatlaridan foydalanishingiz mumkin, ularning asosiylari:

- **AuthenticationSchemes:** mijozlarni autentifikatsiya qilish sxemasini qaytaradi yoki o'rnatadi
- **IsListening:** HttpListener ob'ekti ishga tushirilganligini bildiradi
- **IsSupported:** joriy operatsion tizimda HttpListener tinglovchisidan foydalanish mumkinligini ko'rsatadi.
- **Prefixes:** ushbu HttpListener ob'ekti tomonidan qayta ishlangan URI prefikslarini qaytaradi.

HttpListener-ni boshqarish uchun sinf usullari qo'llaniladi. Ulardan ba'zilari:

- **Close():** HttpListener-ni tugatadi.
- **GetContext() / GetContextAsync():** kiruvchi so'rovlarni kutmoqda
- **Start ():** kiruvchi so'rovlarni tinglashni boshlaydi
- **Stop ():** yangi kiruvchi so'rovlarni qabul qilishni to'xtatadi va barcha joriy so'rovlarni qayta ishlashni yakunlaydi.

HttpListener-Ni Ishga Tushirish

HttpListener bilan ishlash uchun avval dasturga kirish uchun foydalaniladigan manzillarni belgilashimiz kerak. Manzillar HttpListener sinfining Prefixes xususiyati orqali o'rnatiladi. Bunday holda, manzil chiziq bilan tugashi kerak, masalan:

```
using System.Net;

HttpListener server = new HttpListener();
server.Prefixes.Add("http://127.0.0.1:8888/connection/");
```

Bunday holda, server quyidagi manzildagi ulanishlarni tinglaydi `http://127.0.0.1:8888/connection/`.

Kiruvchi ulanishlarni tinglashni boshlash uchun Start () usulini, serverni to‘xtatish uchun Stop () usulini va HttpListener - ni yopish () usulini chaqirish kerak.

```
using System.Net;

HttpListener server = new HttpListener();
server.Prefixes.Add("http://127.0.0.1:8888/connection/");
server.Start(); // начинаем прослушивать входящие подключения
// .....
server.Stop();
server.Close();
```

Kontekst

Getcontext()/GetContextAsync () usuli yordamida kiruvchi ulanish kontekstini httpListenercontext ob’ekti sifatida olish mumkin. Ushbu ob’ektning xususiyatlaridan foydalanib, kiruvchi so‘rov haqida ma’lumot olish va javobni o‘rnatish mumkin:

- **Request:** so‘rov ma’lumotlarini httpListenerrequest ob’ekti sifatida qaytaradi
- **Response:** server javobini httpListenerresponse ob’ekti sifatida o‘rnatadi
- **User:** autentifikatsiya qilingan foydalanuvchi ma’lumotlarini tizim ob’ekti sifatida taqdim etadi.Security.Principal.IPrincipal?

```
using System.Net;

HttpListener server = new HttpListener();
// установка адресов прослушки
server.Prefixes.Add("http://127.0.0.1:8888/connection/");
server.Start();
var context = await server.GetContextAsync();
var request = context.Request;
var response = context.Response;
var user = context.User;
server.Stop();
```

Javob yuborish

HttpListenercontext ob’ektining javob xususiyati httpListenerresponse ob’ektini ifodalaydi, bu sizga mijozga ba’zi javoblarni sozlash va yuborish imkonini beradi. Javobni sozlash uchun siz httpListenerresponse xususiyatlaridan foydalanishingiz mumkin:

- **ContentEncoding:** Content-Type sarlavhasidan Encoding obyektini sifatida javob kodlashni tizim obyektini sifatida o‘rnatadi.Text.Encoding.
- **ContentLength64:** content-Length sarlavhasini o‘rnatadi (javob hajmi).
- **ContentType:** content-Type sarlavhasi qiymatini o‘rnatadi.
- **Cookies:** mijozga System.Net.CookieCollection ob’ekti sifatida yuboriladigan cookie-fayllarni o‘rnatadi.

- **Headers:** sarlavhalarni webheadercollection to‘plami sifatida o‘rnatadi (sarlavha nomlari kalit bo‘lib xizmat qiladigan lug‘at analoglari).
- **KeepAlive:** doimiy ulanish zarurligini ko‘rsatadigan bool qiymatini o‘rnatadi.
- **OutputStream:** javob oqimini ifodalaydi.
- **Protokolversion:** javobda ishlatiladigan HTTP protokolining versiyasini o‘rnatadi.
- **RedirectLocation:** yo‘naltirish manzilini o‘rnatadi.
- **SendChunked:** javobning qismlarga/ qismlarga bo‘linishini ko‘rsatadigan bool qiymatini ifodalaydi.
- **StatusCode:** javobning holat kodini o‘rnatadi.
- **StatusDescription:** tavsifni holat kodiga (holat satri) o‘rnatadi.

Bundan tashqari, sinf javob yuborishda xatti-harakatlarni sozlash uchun bir qator usullarni taqdim etadi:

void AddHeader (string name, string value): javobga ism nomi va qiymat qiymati bilan sarlavha qo‘shadi

void AppendCookie (System. Net. Cookie): Cookie-fayllarga javob qo‘shadi

void AppendHeader (string name, string value): name nomli sarlavhaga qiymat qiymatini qo‘shadi

void Redirect (string url): url manziliga yo‘naltirishni amalga oshiradi

void SetCookie (System. Net. Cookie): Cookie fayllarini o‘rnatadi

Masalan, biz mijozga javob yuboramiz:

```
using System.Net;
using System.Text;
HttpListener server = new HttpListener();
server.Prefixes.Add("http://127.0.0.1:8888/connection/");
server.Start(); // начинаем прослушивать входящие подключения
var context = await server.GetContextAsync();
var response = context.Response;
string responseText =
    @"<!DOCTYPE html>
    <html>
        <head>
            <meta charset='utf8'>
            <title>METANIT.COM</title>
        </head>
        <body>
            <h2>Hello METANIT.COM</h2>
        </body>
    </html>";
byte[] buffer = Encoding.UTF8.GetBytes(responseText);
response.ContentLength64 = buffer.Length;
using Stream output = response.OutputStream;
await output.WriteAsync(buffer);
```

```
await output.FlushAsync();
Console.WriteLine("Запрос обработан");
server.Stop();
```

Bunday holda, bizning dasturimiz mahalliy manzildagi barcha murojaatlarni tinglaydi `http://localhost:8888/connection/`

Listener usulini chaqirganda `GetContext Async ()` joriy oqim bloklanadi va tinglovchi kiruvchi ulanishlarni kutishni boshlaydi. Ushbu usuldan biz `HttpListenerContext` kontekstini olamiz va u javob ob'ektiga ega - `context.Response`.

Javobni olgandan so'ng, biz javob xususiyati orqali chiqish oqimini olamiz. `OutputStream` va unga baytlar qatorini yozing:

```
await output.WriteAsync(buffer);
```

3. So'rov haqida ma'lumotlarni olish.

So'rov xususiyati so'rov ma'lumotlarini `httpListenerRequest` ob'ekti sifatida qaytaradi. Ushbu ma'lumotlarning barchasi sinf xususiyatlarida saqlanadi, ular asosan so'rov sarlavhalarining qiymatlarini qaytaradi:

- **AcceptTypes:** qabul qilish sarlavhasi qiymatini `string []` qatori sifatida qaytaradi.
- **ClientCertificateError:** mijoz tomonidan taqdim etilgan `x509certificate` sertifikatini bilan bog'liq xato kodini qaytaradi.
- **ContentEncoding:** `Content-Type` sarlavhasidan kodlash qiymatini tizim ob'ekti sifatida qaytaradi. `Text.Encoding`.
- **ContentLength64:** `content-Length` sarlavhasini qaytaradi (agar u o'rnatilmagan bo'lsa, -1 qaytadi).
- **ContentType:** `content-Type` sarlavhasi qiymatini qaytaradi.
- **Cookies:** so'rov bilan birga yuborilgan cookie-fayllarni `System.Net.CookieCollection` ob'ekti sifatida qaytaradi.
- **HasEntityBody:** agar so'rovda sarlavhalardan tashqari ba'zi ma'lumotlar (so'rovning asosiy qismi) uzatilsa, rost qaytadi.
- **Headers:** barcha sarlavhalarni tizim to'plami sifatida qaytaradi. `Collections.Specialized.NameValueCollection` (sarlavha nomlari kalit bo'lib xizmat qiladigan lug'at analoglari).
- **HttpMethod:** mijoz tomonidan so'rov yuborish uchun ishlatiladigan HTTP usulini qaytaradi.
- **InputStream:** so'rov tanasi ma'lumotlarini o'z ichiga olgan oqimni qaytaradi.
- **IsAuthenticated:** ushbu so'rovni yuborgan mijoz autentifikatsiya qilinganligini ko'rsatadigan `bool` qiymatini qaytaradi.
- **IsLocal:** so'rov mahalliy kompyuterdan yuborilganligini ko'rsatadigan `bool` qiymatini qaytaradi.

- **IsSecureConnection:** SSL protokoli so‘rov uchun qo‘llanilishini ko‘rsatadigan bool qiymatini qaytaradi.
- **IsWebSocketRequest:** WebSocket so‘rovida ishlatilganligini ko‘rsatadigan bool qiymatini qaytaradi.
- **KeepAlive:** mijoz doimiy ulanishni talab qiladimi yoki yo‘qligini ko‘rsatadigan bool qiymatini qaytaradi.
- **LocalEndPoint:** serverning IP-manzilini va so‘rov yuboriladigan port raqamini qaytaradi.
- **Protokolversion:** so‘rovda ishlatiladigan HTTP protokolining versiyasini qaytaradi.
- **QueryString:** so‘rovlar qatorini qaytaradi.
- **RawUrl:** so‘rovning URL manzilini qaytaradi (tugun va port yo‘q).
- **RemoteEndPoint:** so‘rovni yuborgan mijozning IP-manzilini qaytaradi.
- **RequestTraceIdentifier:** HTTP so‘rov identifikatorini qaytaradi.
- **Url:** so‘rov manzilini uri ob’ekti sifatida qaytaradi.
- **UrlReferrer:** mijozni serverga yo‘naltiradigan resursning URI-ni qaytaradi.
- **AcceptTypes:** user-Agent sarlavhasini qaytaradi.
- **UserHostAddress:** so‘rov yo‘naltiriladigan IP-manzilni qaytaradi.
- **UserHostName:** mijoz tomonidan belgilangan DNS xostini qaytaradi.
- **UserLanguages:** AcceptLanguage sarlavhasi qiymatini qaytaradi (u yo‘q bo‘lganda null qaytariladi).

Shunday qilib, biz kiruvchi ulanishni qayta ishlaymiz va ushbu ma’lumotni olamiz

```
using System.Net;
using System.Text;

HttpListener server = new HttpListener();
server.Prefixes.Add("http://127.0.0.1:8888/connection/");
server.Start(); Console.WriteLine("Сервер запущен. Ожидание подключений...");
var context = await server.GetContextAsync();
var request = context.Request;
Console.WriteLine($"адрес приложения: {request.LocalEndPoint}");
Console.WriteLine($"адрес клиента: {request.RemoteEndPoint}");
Console.WriteLine(request.RawUrl);
Console.WriteLine($"Запрошен адрес: {request.Url}");
Console.WriteLine("Заголовки запроса:");
foreach (string item in request.Headers.Keys)
{
    Console.WriteLine($"{{item}}:{{request.Headers[item]}}");
}
var response = context.Response;
byte[] buffer = Encoding.UTF8.GetBytes("Hello METANIT");
response.ContentLength64 = buffer.Length;
```

```
using Stream output = response.OutputStream;  
await output.WriteAsync(buffer);  
await output.FlushAsync();  
server.Stop();
```

Nazorat savollari:

1. Cookiening vazifasi nimalardan iborat?
2. HttpListener metodlari haqida ma'lumot bering
3. So'rov orqali ma'lumot yuborish va qabul qilishda nimalarni e'tiborga olish talab qilinadi.

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**9-Mashg‘ulot:** Soketlarda TCP-klient.**Reja:**

1. Soketlarda TCP mijozi.
2. Hostdan ma’lumotlarni olish.
3. Avialable xususiyati.

1. Soketlarda TCP mijozi.

Tarmoqdagi eng keng tarqalgan o‘zaro ta’sir protokollaridan biri bu TCP (Transmission Control Protocol) protokoli. Ushbu protokol xabarlarini etkazib berishni kafolatlaydi va bugungi kunda mavjud bo‘lgan turli xil dasturlarda keng qo‘llaniladi. TCP protokoli bilan ishlash uchun. net TcpClient va TcpListener sinflariga mo‘ljallangan. Ushbu sinflar System.Net.Sockets.Socket. TcpClient va TcpListener sinfining tepasida qurilgan bo‘lib, TCP protokolini amalga oshiradigan mijoz va serverni yaratishni osonlashtiradi. Agar ushbu sinflarning funksional imkoniyatlari etarli bo‘lmasa, unda yanada rivojlangan va murakkab stsenariylar uchun siz bir xil Socket sinfidan foydalanishingiz mumkin. Ushbu bobda biz TcpClient va TcpListener yordamida ham, toza rozetkalar yordamida ham tcp mijozi va tcp serverini qurishda turli xil yondashuvlarni ko‘rib chiqamiz.

Soketlarda TCP mijozi

Xostga ulanish, ma’lumotlarni yuborish va qabul qilish uchun TCP soketlaridan foydalanadigan eng oddiy mijozning ta’rifini ko‘rib chiqing. Avvalo, TCP protokolidan foydalanadigan rozetkani aniqlash uchun rozetkani Tcp protokoli turi sifatida, rozetkani esa oqim turi sifatida ko‘rsatish kerak:

```
Socket tcpSocket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
```

Masofaviy xostga ulanish uchun Connect()/ConnectAsync () usuli qo‘llaniladi. Ushbu ikkala usul ham ko‘plab versiyalarga ega, ammo umuman olganda, masofaviy xostga ulanish uchun bizga ip-manzil yoki domen va port ko‘rinishidagi xost manzili kerak. Men bir nechta ortiqcha yuklarni qayd etaman:

```
public Task ConnectAsync (string host, int port);
public Task ConnectAsync (IPAddress address, int port);
```

Masalan, biz xostga ulanamiz "google.com":

```
using System.Net.Sockets;

var port = 80;
var url = "www.google.com";

using var socket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    await socket.ConnectAsync(url, port);
```

```

        Console.WriteLine($"Подключение к {url} установлено");
    }
    catch (SocketException)
    {
        Console.WriteLine($"Не удалось установить подключение к {url}");
    }

```

Masalan, mening ishimdan konsol chiqishi:

Xostdan uzilish

Agar biz xost bilan o‘zaro aloqani tugatgan bo‘lsak, lekin masofaviy xost ulanishi osilmasligi uchun rozetkadan foydalanishni davom ettirishni rejalashtirsak, biz Disconnect() / DisconnectAsync () usuli yordamida uzishimiz mumkin. Ushbu usul parametr sifatida bool qiymatini oladi-agar u rost bo‘lsa, u holda o‘chirilgandan so‘ng siz yangi ulanishlar uchun rozetkadan qayta foydalanishingiz mumkin:

```

using System.Net.Sockets;

var port = 80;
var url = "www.google.com";

using var socket = new Socket(AddressFamily.InterNetwork,
    SocketType.Stream, ProtocolType.Tcp);
try
{
    await socket.ConnectAsync(url, port);
    await socket.DisconnectAsync(true);
}
catch (SocketException ex)
{
    Console.WriteLine(ex.Message);
}

```

Ulanishni yopishdan oldin barcha ma’lumotlar yuborilishini va qabul qilinishini ta’minlash uchun, Disconnect/DisconnectAsync usulini chaqirishdan oldin, Microsoft Shutdown usulini chaqirishni tavsiya qiladi.

Ma’lumotlarni yuborish

Ma’lumotlarni yuborish uchun Send()/SendAsync () usuli qo‘llaniladi. Ushbu ikkala usul ham turli xil versiyalarga ega. Eng oddiy versiyalarni ko‘rib chiqing:

```

public Task<int> SendAsync (ArraySegment<byte> buffer);
public Task<int> SendAsync (ArraySegment<byte> buffer, SocketFlags
    socketFlags);

```

Parametr sifatida u yuborilgan ma’lumotlarni arraysegment<byte> tuzilishi sifatida oladi - taxminan baytlar qatorining bir qismi. Bundan tashqari, socketflags enum qiymatlari yordamida siz yuborish parametrlarini o‘rnatishingiz mumkin.

Natijada SendAsync () usuli yuborilgan ma’lumotlar sonini qaytaradi.

Masalan, biz yuboramiz google.com ba’zi ma’lumotlar bilan so‘rov:

```

using System.Net.Sockets;
using System.Text;

```

```

var port = 80;
var url = "www.google.com";

using var socket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    await socket.ConnectAsync(url, port);
    var message = $"GET / HTTP/1.1\r\nHost: {url}\r\nConnection:
close\r\n\r\n";
    var messageBytes = Encoding.UTF8.GetBytes(message);
    int bytesSent = await socket.SendAsync(messageBytes);
    Console.WriteLine($"на адрес {url} отправлено {bytesSent}
байт(a)");
}
catch (SocketException ex)
{
    Console.WriteLine(ex.Message);
}

```

Server bilan o‘zaro aloqada bo‘lganingizda, ushbu server qaysi protokolni amalga oshirayotganini, ya’ni server so‘rovlarni qabul qilish qoidalarini tushunishingiz kerak. Shunday qilib, google.com, har qanday standart sayt kabi, HTTP protokoliga mos keladigan xabarlarni qabul qiladi. Taxminan aytganda, masofaviy xost bizni tushunishi uchun biz uning tilida gaplashishimiz kerak. Va bu holda biz HTTP protokoliga mos keladigan xabar yuboramiz:

```

var message = $"GET / HTTP/1.1\r\nHost: {url}\r\nConnection:
close\r\n\r\n";

```

HTTP so‘rov formati birinchi navbatda so‘rov turini, so‘ralgan manbaga yo‘lni va protokolning o‘ziga xos versiyasini o‘z ichiga olgan so‘rovlar qatorini o‘z ichiga oladi. Ya’ni, bu erda xabarda biz "/" (ya’ni saytning ildiziga) yo‘l bo‘ylab get tipidagi so‘rov yuborilganligini ko‘rsatamiz google.com"). bunday holda, http / 1.1 protokoli qo‘llaniladi. So‘rov chizig‘i \r\n satrining ikki tomonlama karek va tarjima belgilar to‘plami bilan yakunlanishi kerak.

Bundan tashqari, HTTP so‘rovida sarlavhalar bo‘lishi mumkin. Shunday qilib, bu holda biz xost manzilini ko‘rsatadigan "xost" sarlavhasini yuboramiz. Bu holda, bu "www.google.com:80". shuningdek, bu holda biz "ulanish" sarlavhasini yuboramiz, u "yopish" qiymatiga ega - bu qiymat serverga ulanishni yopishni buyuradi.

Biz satrlarni emas, balki faqat baytlarni yuborishimiz mumkinligi sababli, biz qatorni baytlar qatoriga o‘tkazamiz:

```

var messageBytes = Encoding.UTF8.GetBytes(message);

```

Va biz m‘lumotlarni yuboramiz:

```

int bytesSent = await socket.SendAsync(messageBytes);

```

Shuni ta’kidlash kerakki, SendAsync usuli ArraySegment ob’ektini qabul qilsa-da, bu erda biz avtomatik ravishda ArraySegment<bayt>tuzilishiga aylantiriladigan ma’lumotlar bilan to‘g‘ridan-to‘g‘ri massivni uzatamiz.

2. Hostdan ma’lumotlarni olish.

Ma’lumotlarni olish uchun Socket sinfi Receive() / ReceiveAsync () usullarini qo‘llaydi. Ikkala usulda ham turli xil variantlar to‘plamiga ega bo‘lgan juda ko‘p Yuklangan versiyalar mavjud, ammo asosiy parametr - bu olingan ma’lumotlar yuklanadigan bufer. Sinxron Receive usuli uchun odatda bayt massivi bufer vazifasini bajaradi, asenkron ReceiveAsync uchun esa ArraySegment<byte>tuzilishi:

```
Task<int> ReceiveAsync (ArraySegment<byte> buffer);
int Receive (byte[] buffer);
```

Ikkala usulning natijasi hisoblangan baytlar sonidir.

Masalan, biz olamiz google.com javob:

```
using System.Net.Sockets;
using System.Text;

var port = 80;
var url = "www.google.com";

using var socket = new Socket(AddressFamily.InterNetwork,
    SocketType.Stream, ProtocolType.Tcp);
try
{
    await socket.ConnectAsync(url, port);
    var message = $"GET / HTTP/1.1\r\nHost: {url}\r\nConnection:
close\r\n\r\n";
    var messageBytes = Encoding.UTF8.GetBytes(message);
    await socket.SendAsync(messageBytes);
    var responseBytes = new byte[512];
    var bytes = await socket.ReceiveAsync(responseBytes);
    string response = Encoding.UTF8.GetString(responseBytes, 0,
bytes);
    Console.WriteLine(response);
}
catch (SocketException ex)
{
    Console.WriteLine(ex.Message);
}
```

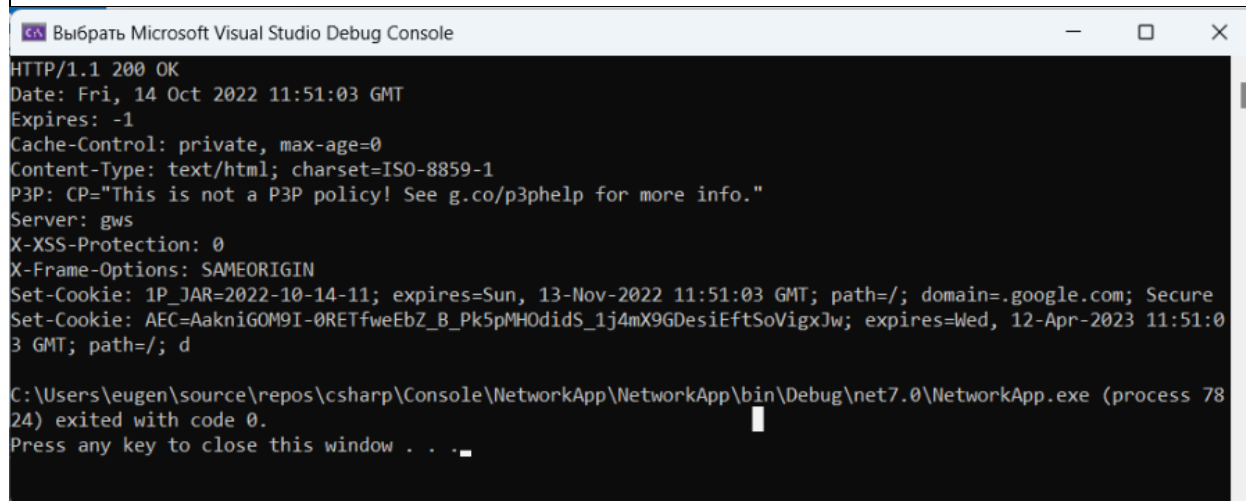
Yuborilgan baytlarning buferi sifatida 512 baytli massivni ifodalovchi responsebytes o‘zgaruvchisi aniqlanadi. The bufer hajmi bu holda muhim emas. Asosiysi, olingan ma’lumotlar qanchalik katta bo‘lishi mumkinligini tasavvur qilish va shunga muvofiq bufer uchun o‘lchamni aniqlash. Va O‘qilgan baytlarning haqiqiy sonini kuzatish uchun (bufer hajmidan kichikroq bo‘lishi mumkin) bytes o‘zgaruvchisi aniqlanadi.

Bundan tashqari, SendAsync usulida bo'lgani kabi, baytlar qatori avtomatik ravishda ArraySegment<bayt>ga aylantiriladigan ReceiveAsync usuliga o'tkaziladi.

```
var bytes = await socket.ReceiveAsync(responseBytes);
```

Chunki bu holda javob google.com aslida, bu satrni ifodalaydi, keyin biz o'zgartirilgan ma'lumotlarni satrga aylantiramiz va uni konsolga chiqaramiz.

```
string response = Encoding.UTF8.GetString(responseBytes, 0, bytes);
Console.WriteLine(response);
```



19-rasm: Serverdan qabul qilingan ma'lumotlar oynasi.

Ko'rib turganimizdek, bu odatiy HTTP javobidir, bu erda birinchi navbatda javob holati, sarlavhalar mavjud. Biroq, javob shuni ko'rsatadiki, u to'liq emas. Masofaviy xost qancha bayt yuborishini aniq bilsak, vaziyat ideal bo'ladi. Va shunga mos ravishda ular tegishli buferni aniqlashlari mumkin. Ammo bu holda biz buni aniq bilmaymiz-ular bufer hajmidan kattaroq yoki kichikroq bo'lishi mumkin. Shubhasiz, biz javobning oxirgi baytini olmaganimizcha, tsikldagi ma'lumotlarni o'qishimiz kerak:

```
using System.Net.Sockets;
using System.Text;

var port = 80;
var url = "www.google.com";

using var socket = new Socket(AddressFamily.InterNetwork,
    SocketType.Stream, ProtocolType.Tcp);
try
{
    await socket.ConnectAsync(url, port);
    var message = $"GET / HTTP/1.1\r\nHost: {url}\r\nConnection:
close\r\n\r\n";
    var messageBytes = Encoding.UTF8.GetBytes(message);
    await socket.SendAsync(messageBytes);
    var responseBytes = new byte[512];
    var builder = new StringBuilder();
    int bytes;
```

```

do
{
    bytes = await socket.ReceiveAsync(responseBytes);
    string responsePart = Encoding.UTF8.GetString(responseBytes,
0, bytes);
    builder.Append(responsePart);
}
while (bytes > 0);
Console.WriteLine(builder);
}
catch (SocketException ex)
{
    Console.WriteLine(ex.Message);
}

```

Endi o‘qish uchun do tsiklidan foydalanamiz..while. ReceiveAsync qancha baytni qaytarishini ko‘rib chiqamiz. Va u 0 baytni qaytarganda, tsiklni takrorlang. Olingan baytlarni mag‘lubiyatga aylantiramiz va StringBuilder-ga qo‘shamiz. Oxirida biz olingan tarkibni StringBuilder-dan konsolga chiqaramiz:

```

Выбрать Microsoft Visual Studio Debug Console
HTTP/1.1 200 OK
Date: Fri, 14 Oct 2022 10:23:47 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=ISO-8859-1
P3P: CP="This is not a P3P policy! See g.co/p3phelp for more info."
Server: gws
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
Set-Cookie: 1P_JAR=2022-10-14-10; expires=Sun, 13-Nov-2022 10:23:47 GMT; path=/; domain=.google.com; Secure
Set-Cookie: AEC=AakniGMYZ5yo8TNKKD2AGw_OhbbXQqzVx6pC0LuPI5NfIXwirJ3gYeuzYNQ; expires=Wed, 12-Apr-2023 10:23:47 G
MT; path=/; domain=.google.com; Secure; HttpOnly; SameSite=lax
Set-Cookie: NID=511=Id_qjoYNrdJ6bhaNH-4ARKvIrs2Qnx7lwIaDPzwpkBAw_UKk1gVi2xx3SeNkR5n6YsKap5EF1oQbE26CNkcWfgoWRbi5
EPs2nR4GHqrNappGHiabAA5o_xv6I8JZthVjZwJrx1Gy94Qkj8r37ihaKty0GrFNHre8qHJV50G0Txw; expires=Sat, 15-Apr-2023 10:23:
47 GMT; path=/; domain=.google.com; HttpOnly
Accept-Ranges: none
Vary: Accept-Encoding
Connection: close
Transfer-Encoding: chunked

5498
<!doctype html><html itemscope="" itemtype="http://schema.org/WebPage" lang="ru"><head><meta content="#1055;&#1
086;&#1080;&#1089;&#1082; &#1080;&#1085;&#1092;&#1086;&#1088;&#1084;&#1072;&#1094;&#1080;&#1080; &#1074; &#1080;
&#1085;&#1090;&#1077;&#1088;&#1085;&#1077;&#1090;&#1077;: &#1074;&#1077;&#1073; &#1089;&#1090;&#1088;&#1072;&#10

```

20-rasm: Console oynasi.

Endi biz barcha javoblarni oldik google.com.ammo shuni yodda tutish kerakki, TCP rozetkalari uchun ReceiveAsync ma’lumotlari bo‘lmasa, u bajarilishini tugatadi va faqat bufer to‘lganida yoki tomonlardan biri ulanishni tugatgandan so‘ng baytlar sonini qaytaradi. TCP xostlar uzoq vaqt davomida ma’lumotlarni yuborishi va qabul qilishi mumkin bo‘lgan ulanishni o‘rnatishni o‘z ichiga olganligi sababli. Yuqoridagi misolda bu oddiy: serverga google.com quyidagi xabar yuboriladi:

```

var message = $"GET / HTTP/1.1\r\nHost: {url}\r\nConnection:
close\r\n\r\n";

```

HTTP usuli, yo'li va protokolidan tashqari, bu erda "ulanish: yopish" sarlavhasi ham yuboriladi, bu joriy operatsiya tugagandan so'ng ulanishni yopishni buyuradi. Buning yordamida biz ijro chiziq ustida osilmaydi

```
bytes = await socket.ReceiveAsync(responseBytes);
```

Ushbu sarlavhani olib tashlang

```
var message = $"GET / HTTP/1.1\r\nHost: {url}\r\n\r\n";
```

Va rozetka kutishni davom ettiradi google.com yangi ma'lumotlar. Chunki bu holda biz oqim bilan shug'ullanmoqdamiz va rozetka ma'lumotlarning yangi qismini cheksiz kutishi mumkin. Ammo bu misol juda vaziyatli, chunki google.com HTTP so'rovlarini qabul qiladi va bu erda HTTP sarlavhalari manipulyatsiyasi mavjud. Ammo TCP socketlaridan foydalanish kengligi HTTP protokoli bilan cheklanmaydi va boshqa holatlarda boshqa imkoniyatlar talab qilinishi mumkin (hatto HTTP protokolining yangi versiyalarida ham ulanish sarlavhasi bekor qilinganligi haqida gapirmaslik kerak). Va bu erda yana hamma narsa masofaviy xostga bog'liq. Mijoz va serverni o'zimiz yaratganimizda, o'zaro ta'sirning har qanday mantig'ini o'zimiz aniqlashimiz mumkin. Agar biz masofaviy xostning ishlashiga ta'sir qila olmasak, rozetkaning ishlashini masofaviy xostning ishlashi bilan moslashtirishimiz kerak. Shunga qaramay, variantlar ham mavjud. Shunday qilib, Shutdown()usuli yordamida rozetkada ma'lumotlarni qabul qilish va/yoki yuborishni o'chirib qo'yishimiz mumkin

```
using System.Net.Sockets;
using System.Text;

var port = 80;
var url = "www.google.com";

using var socket = new Socket(AddressFamily.InterNetwork,
    SocketType.Stream, ProtocolType.Tcp);
try
{
    await socket.ConnectAsync(url, port);

    var message = $"GET / HTTP/1.1\r\nHost: {url}\r\n\r\n";
    var messageBytes = Encoding.UTF8.GetBytes(message);
    await socket.SendAsync(messageBytes);
    socket.Shutdown(SocketShutdown.Send);

    var responseBytes = new byte[512];
    var builder = new StringBuilder();
    int bytes;
    do
    {
        bytes = await socket.ReceiveAsync(responseBytes);
        string responsePart = Encoding.UTF8.GetString(responseBytes,
            0, bytes);
        builder.Append(responsePart);
    }
}
```

```

    }
    while (bytes > 0);
    Console.WriteLine(builder);
}
catch (SocketException ex)
{
    Console.WriteLine(ex.Message);
}

```

Bunday holda, ma'lumotlar yuborilgandan so'ng, socket socket qo'ng'irog'i yordamida ma'lumotlarni keyingi yuborishdan uziladi. `Shutdown(SocketShutdown.Yuborish)` va `ReceiveAsync` usulida ma'lumotlarni olgandan keyin `google.com` shuningdek, jo'natishni yakunlaydi.

3. Available xususiyati.

`Available` xususiyati o'qish mumkin bo'lgan baytlar sonini saqlaydi. Shunga ko'ra, nima uchun ushbu xususiyatlardan ma'lumotlarning mavjudligini kuzatish uchun foydalanmaslik kerakligi haqida savol tug'iladi, masalan:

```

do
{
    bytes = await socket.ReceiveAsync(responseBytes);
    string responsePart = Encoding.UTF8.GetString(responseBytes, 0,
bytes);
    builder.Append(responsePart);
}
while (socket.Available > 0);

```

Ammo mavjud bo'lgan xususiyat nolga teng bo'lmagan qiymatga ega bo'ladi, agar oqim hozirda o'qilishi mumkin bo'lgan ma'lumotlarga ega bo'lsa. Ammo TCP protokolining mohiyati shundaki, katta ma'lumotlar to'plamlari alohida paketlarda yuboriladi. Ba'zi paketlar tezroq kelishi mumkin, ba'zilari kechiktiriladi, ba'zilari yo'qoladi va qayta yuborish kerak bo'ladi. Shuning uchun, server ma'lumotlarni yuborgan, ma'lumotlarning bir qismi kelgan vaziyat yuzaga kelishi mumkin. Bir vaqtning o'zida rozetkada mavjud bo'lgan xususiyat 0 ga teng edi, mos ravishda tsikldan chiqish sodir bo'ldi. Va oxirida biz to'liq bo'lmagan ma'lumotlarni olamiz. Shuning uchun, bu holda `Available` xususiyatidan foydalanish eng yaxshi variant emas.

Odatda, ma'lumotlarni olishda quyidagi strategiyalardan biri qo'llaniladi, bu sizga ma'lumotlarni olishning tugashini aniqlashga imkon beradi:

Qancha ma'lumot yuborilishini aniq bilganimizda, belgilangan uzunlikdagi buferdan foydalanish

Javobda javobning hajmi to'g'risida ma'lumot yuborish, uni qabul qilib, kerakli bayt sonini hisoblash osonroq bo'ladi

Javobni tugatish markeridan foydalanib, biz ma'lumotlarni o'qishni tugatamiz

Muayyan strategiyani tanlash va amalga oshirish butunlay so'rovni qabul qiladigan va javobni yuboradigan serverga bog'liq.

Nazorat savollari:

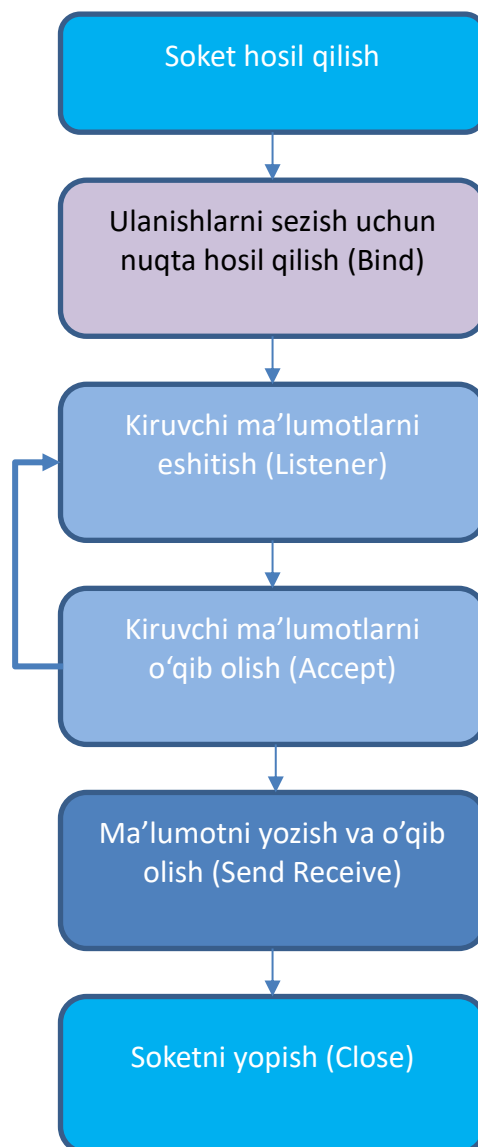
1. Soketlar haqida ma'lumot bering
2. Host haqida ma'lumot bering.
3. Available xususiyati qanday holatlarda qo'llaniladi.

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**10-Mashg‘ulot:** Soketlarda TCP Server.**Reja:**

1. Soketlarda TCP serveri haqida tushuncha.
2. NetworkStream.
3. Ma’lumotlarni o‘qish.

1. Soketlarda TCP serveri haqida tushuncha.

Socket klassi nafaqat tcp mijozini yaratish, balki tcp serverini aniqlash uchun ham qo‘llaniladi. TCP server socketining umumiy ishlash sxemasi quyidagicha bo‘ladi:



21-rasm: TCP protokoli asosida ma’lumot jo‘natish ketma-ketligi.

Oxirgi nuqtaga ulanish. Bind Usuli

Dastlab, server rozetkasi bind usuli yordamida mahalliy nuqta bilan bog‘lanadi. Parametr sifatida ushbu usul soket mijozlardan ulanishlarni qabul qiladigan mahalliy EndPoint-ni qabul qiladi:

```
public void Bind (EndPoint localEP);
```

Agar server qaysi mahalliy manzilda ishlashi muhim bo‘lmasa, siz ipaddress qiymatini manzil sifatida ishlatishingiz mumkin. Any. Keyin serverga eng mos tarmoq manzili tayinlanadi (agar bir nechta tarmoq interfeyslari mavjud bo‘lsa). Bundan tashqari, agar port raqami muhim bo‘lmasa, u holda 0 raqamini port sifatida ko‘rsatish mumkin. Keyin serverga mavjud portlardan biri taqdim etiladi. Ushbu yondashuv yordamida aniq manzil va portni LocalEndpoint xususiyati orqali olish mumkin.

```
using System.Net;
using System.Net.Sockets;

IPEndPoint ipPoint = new IPEndPoint(IPAddress.Any, 8888);
using Socket socket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
socket.Bind(ipPoint); // связываем с локальной точкой ipPoint
Console.WriteLine(socket.LocalEndPoint); // 0.0.0.0:8888
```

Ushbu misolda rozetka har qanday mahalliy manzillarda 8888 port ulanishlarini tinglaydi. Ya’ni, mijoz mahalliy manzilga, masalan, 127.0.0.1 va 8888 portiga ulanishi kerak.

Ulanishlarni tinglash. Tinglash Usuli

Tanlangan mahalliy so‘nggi nuqtada ulanishlarni tinglashni boshlash uchun Listen usuli qo‘llaniladi:

```
public void Listen ();
public void Listen (int backlog);
```

Serverga kirishda kiruvchi ulanishlar keyingi ishlov berish uchun navbatga qo‘yiladi. Odatiy bo‘lib, bu navbat 2147483647 ulanishga imkon beradi. Variant orqali tinglash usulining ikkinchi versiyasi navbat uzunligini bekor qilishga imkon beradi.

```
System.Net;
using System.Net.Sockets;
IPEndPoint ipPoint = new IPEndPoint(IPAddress.Any, 8888);
using Socket socket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
socket.Bind(ipPoint);
socket.Listen(1000);
поместить в очередь, равно 1000
```

Mijozni ulash

Tinglash boshlangandan so‘ng, rozetka ulanishlarni qabul qilishga tayyor. Ulanishlarni qabul qilish uchun Accept()/AcceptAsync () usullari qo‘llaniladi. Ushbu

usullar bir qator haddan tashqari Yuklangan versiyalarga ega. Men ulardan eng oddiyini ta'kidlayman:

```
public Task<Socket> AcceptAsync ();
```

Accept()/AcceptAsync () usullarining barcha versiyalari kiruvchi ulanishni qamrab oladigan Socket ob'ektini qaytaradi, ya'ni aslida ulangan mijozni ifodalaydi.

```
using System.Net;
using System.Net.Sockets;

IPEndPoint ipPoint = new IPEndPoint(IPAddress.Any, 8888);
using Socket socket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
socket.Bind(ipPoint);
socket.Listen();
Console.WriteLine("Сервер запущен. Ожидание подключений...");
using Socket client = await socket.AcceptAsync();
Console.WriteLine($"Адрес подключенного клиента:
{client.RemoteEndPoint}");
```

Socket xususiyatlari orqali siz mijozning ulanishi haqida ma'lumot olishingiz mumkin, xususan, RemoteEndPoint xususiyati ulangan mijozning manzilini olish imkonini beradi.

Sinov uchun bunday oddiy server uchun biz mijozni aniqlaymiz. C# - da yangi konsol ilovasi loyahasini oling va undagi quyidagi kodni aniqlang:

```
using System.Net.Sockets;

using var socket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    await socket.ConnectAsync("127.0.0.1", 8888);
    Console.WriteLine($"Подключение к {socket.RemoteEndPoint}
установлено");
}
catch (SocketException)
{
    Console.WriteLine($"Не удалось установить подключение с
{socket.RemoteEndPoint}");
}
```

Bu erda biz Serverning so'nggi nuqtasi ma'lumotlarini ConnectAsync usuliga o'tkazamiz va muvaffaqiyatli ulanishda xabarni ko'rsatamiz.

2. NetworkStream.

Ma'lumotlarni.net-ga qabul qilish va yuborish uchun bir qator sinflar SYSTEM.Net. Sockets nom maydonidan networkstream oqimlari sinfidan foydalanishlari mumkin.u asosiy Stream sinfidan meros bo'lib o'tadi. Shu bilan birga, u boshqa oqim sinflaridan farq qiladi, chunki u buferlanmagan va Seek usuli yordamida o'zboshimchalik holatiga o'tishni qo'llab-quvvatlamaydi. Bundan tashqari, oqimga

yoziyotganda, barcha ma'lumotlarni oqimga qaytarish uchun Flush usulidan foydalanish shart emas. (NetworkStream sinfining manba kodi)

NetworkStream ob'ektini yaratish uchun uning konstruktoriga kamida bitta parametrni o'tkazish kerak-ishlatilgan soket-Socket ob'ekti:

```
public NetworkStream (System.Net.Sockets.Socket socket);
```

Bunday holda, rozetka oqim bo'lishi kerak, ya'ni SocketType turiga ega bo'lishi kerak.Stream. Va bunday rozetkalar hozirda faqat TCP protokoli uchun ishlatilganligi sababli, networkstream klassi faqat TCP protokoli orqali ma'lumotlarni qabul qilish/yuborishda qo'llaniladi, shunga ko'ra biz NetworkStream-ni faqat TCP protokoli kontekstida qo'llash haqida gaplashamiz.

Shuni ta'kidlash kerakki, biz tarmoq oqimini faqat rozetkani serverga ulaganimizdan so'ng yaratishimiz mumkin:

```
using System.Net.Sockets;

using var mySocket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
var server = "www.google.com";
mySocket.Connect(server, 80);
using var stream = new NetworkStream(mySocket);
Console.WriteLine($"Локальный адрес: {stream.Socket.LocalEndPoint}");
Console.WriteLine($"Адрес сервера: {stream.Socket.RemoteEndPoint}");
```

Socket xususiyati orqali siz ishlatilgan rozetkaga kirishingiz mumkin. Shunday qilib, yuqoridagi misolda biz manzilga ulanamiz "google.com", biz NetworkStream-ni yaratamiz va rozetkaga murojaat qilib, mijozning mahalliy manzili va server manzilini olamiz.

NetworkStream bilan ishlashni tugatgandan so'ng, uni har qanday ip kabi yopish kerak. Buning uchun Close () usulidan foydalanishingiz mumkin:

```
var stream = new NetworkStream(mySocket);
stream.Close();
```

Oldingi misolda bo'lgani kabi using dizaynidan ham foydalanish mumkin.

NetworkStream metodlari:

NetworkStream usullari yordamida ma'lumotlarni olish yoki yuborish mumkin:

- **Read () / ReadAsync ():** oqimdan baytlar qatoriga ma'lumotlarni o'qiydi va O'qilgan baytlar sonini qaytaradi
- **ReadByte ():** oqimdan bir baytni o'qiydi va uni qaytaradi
- **Read At Least () / ReadAtLeastAsync ():** oqimdan kamida ma'lum miqdordagi baytlarni (yoki undan ko'p) o'qiydi va O'qilgan baytlar sonini qaytaradi
- **ReadExactly () / ReadExactlyAsync ():** oqimdan baytlarning aniq sonini o'qiydi

- **Write () / WriteAsync ():** ma'lumotlarni bayt qatori sifatida oqimga yuboradi
- **WriteByte():** bir baytni oqimga yuboradi

Ba'zi usullardan foydalanishni ko'rib chiqing.

Ma'lumotlarni yuborish

NetworkStream-ga ma'lumotlarni yuborish uchun Write()/WriteAsync() usuli qo'llaniladi:

```
public ValueTask WriteAsync (ReadOnlyMemory<byte> buffer,
Cancellation token cancellationToken = default);
public Task WriteAsync (byte[] buffer, int offset, int count);
```

Yuborilgan ma'lumotlar baytlar qatori yoki readonlymemory<bayt> ob'ekti sifatida uzatiladi, u yana baytlar qatorini qamrab oladi.

```
using System.Net.Sockets;
using System.Text;
using var mySocket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
var server = "www.google.com";
mySocket.Connect(server, 80);
using var stream = new NetworkStream(mySocket);
var message = $"GET / HTTP/1.1\r\nHost: {server}\r\nConnection:
Close\r\n\r\n";
var data = Encoding.UTF8.GetBytes(message);
await stream.WriteAsync(data);
Console.WriteLine($"Данные отправлены на сервер {server}");
```

Qoida tariqasida, mijoz va serverning o'zaro ta'siri mijoz va server amal qiladigan ba'zi protokollarga muvofiq amalga oshiriladi. So'rovlarni ko'rib chiqadigan server "www.google.com" http serverini ifodalaydi va http protokoli bo'yicha so'rovlarni qayta ishlaydi. Shuning uchun bizning mijozimiz http protokoliga mos keladigan xabarni yuboradi. Bunday holda, serverga xabar yuboriladi

```
var message = $"GET / HTTP/1.1\r\nHost: {server}\r\nConnection:
Close\r\n\r\n";
```

Aslida biz satrlarni emas, balki faqat baytlarni yuborishimiz mumkinligi sababli, biz xabarni baytlar qatoriga aylantiramiz va uni WriteAsync usuliga o'tkazamiz:

```
var data = Encoding.UTF8.GetBytes(message);
await stream.WriteAsync(data);
```

Garchi bu erda biz readonlymemory<byte> ob'ektini qabul qiladigan ortiqcha yukdan foydalanamiz. Ammo. net avtomatik ravishda bayt qatorini shunga o'xshash ob'ektga aylantirishi mumkin.

3. Ma'lumotlarni o'qish.

NetworkStream-ga ma'lumotlarni olish uchun umumiy holda Read()/ReadAsync () usullari qo'llaniladi, ular bir qator ortiqcha yuklarga ega. Masalan, ReadAsync usulining haddan tashqari yuklanishi:

```
public ValueTask<int> ReadAsync (Memory<byte> buffer,
CancellationTokentoken = default);
public Task<int> ReadAsync (byte[] buffer, int offset, int count);
public Task<int> ReadAsync (byte[] buffer, int offset, int count,
CancellationTokentoken);
```

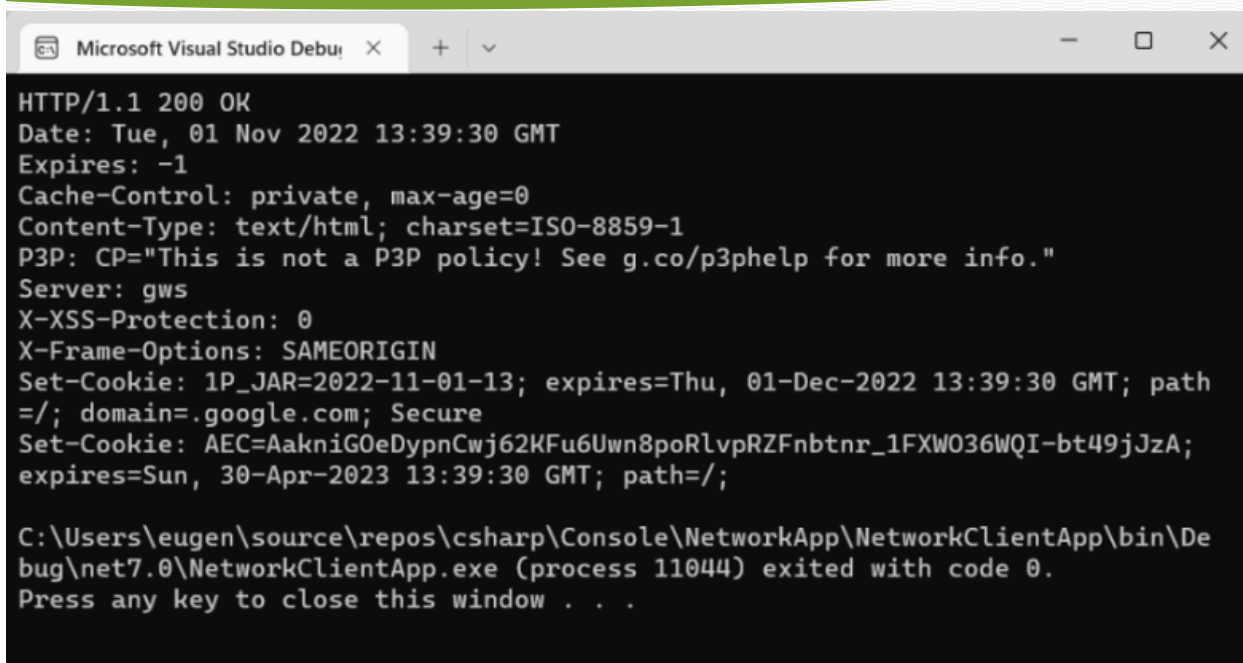
Birinchi parametr sifatida usul baytlar qatori yoki oqimdan ma'lumotlar o'qiladigan Memory<byte> ob'ekti sifatida o'qish buferini oladi. Bundan tashqari, siz massivda ofsetni va O'qilgan baytlar sonini o'rnatishingiz mumkin.

Natijada, usul o'qish buferining uzunligidan kam bo'lishi mumkin bo'lgan haqiqiy O'qilgan baytlar sonini bilvosita qaytaradi.

Shunday qilib, yuqoridagi misolda biz serverga http so'rovini yubordik "www.google.com". endi biz undan javob olamiz:

```
using System.Net.Sockets;
using System.Text;
using var mySocket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
var server = "www.google.com";
mySocket.Connect(server, 80);
using var stream = new NetworkStream(mySocket);
var message = $"GET / HTTP/1.1\r\nHost: {server}\r\nConnection:
Close\r\n\r\n";
var data = Encoding.UTF8.GetBytes(message);
await stream.WriteAsync(data);
var responseData = new byte[512];
var bytes = await stream.ReadAsync(responseData);
string response = Encoding.UTF8.GetString(responseData, 0, bytes);
Console.WriteLine(response);
```

Yuborilgan baytlarning buferi sifatida baytlar qatorini ifodalovchi response Data o'zgaruvchisi aniqlanadi. Odatiy bo'lib, bufer hajmi 512 baytni tashkil qiladi. Ammo bu muhim emas. Asosiysi, olingan ma'lumotlar qanchalik katta bo'lishi mumkinligini tasavvur qilish va shunga muvofiq bufer uchun o'lchamni aniqlash. Va O'qilgan baytlarning haqiqiy sonini kuzatish uchun (bufer hajmidan kichikroq bo'lishi mumkin) bytes o'zgaruvchisi aniqlanadi. Chunki bu holda javob google.com aslida, bu satrni ifodalaydi, keyin biz o'zgartirilgan ma'lumotlarni qatorga aylantiramiz. Oxirida biz konsolga javob satrini ko'rsatamiz.



```

HTTP/1.1 200 OK
Date: Tue, 01 Nov 2022 13:39:30 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=ISO-8859-1
P3P: CP="This is not a P3P policy! See g.co/p3phelp for more info."
Server: gws
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
Set-Cookie: 1P_JAR=2022-11-01-13; expires=Thu, 01-Dec-2022 13:39:30 GMT; path
=;/; domain=.google.com; Secure
Set-Cookie: AEC=AakniGOeDypnCwj62KFu6Uwn8poRlvpRZFnbtr_1FXW036WQI-bt49jJzA;
expires=Sun, 30-Apr-2023 13:39:30 GMT; path=/;

C:\Users\eugen\source\repos\csharp\Console\NetworkApp\NetworkClientApp\bin\De
bug\net7.0\NetworkClientApp.exe (process 11044) exited with code 0.
Press any key to close this window . . .

```

23-rasm: Kod natijasi oynasi.

Ko‘rib turganimizdek, bu odatiy HTTP javobidir, bu erda javob holati, sarlavhalar boshida keladi. Aslida, bu brauzer murojaat qilganda oladigan ma’lumotlar "www.google.com". biroq, javob shuni ko‘rsatadiki, u to‘liq emas. Masofaviy xost qancha bayt yuborishini aniq bilsak, vaziyat ideal bo‘ladi. Va shunga mos ravishda ular tegishli buferni aniqlashlari mumkin. Ammo bu holda biz buni aniq bilmaymiz-ular bufer hajmidan kattaroq yoki kichikroq bo‘lishi mumkin. Shubhasiz, biz oxirgi baytni olmagunimizcha ma’lumotlarni tsiklda o‘qishimiz kerak.

Endi o‘qish uchun do tsiklidan foydalanamiz..while. ReadAsync qancha baytni qaytarishini ko‘rib chiqamiz. Va u 0 baytni qaytarganda, tsiklni takrorlang. Olingan baytlarni mag‘lubiyatga aylantiramiz va StringBuilder-ga qo‘shamiz. Oxirida biz olingan tarkibni StringBuilder-dan konsolga chiqaramiz:

ReadAtLeastAsync

Read At Least()/ReadAtLeastAsync () usullari siz o‘qigan baytlarning minimal sonini belgilashga imkon beradi:

Ma’lumotlarni o‘qish uchun bufer Xotira<bayt> turidagi birinchi parametr orqali uzatiladi va ikkinchi parametr - minimal baytlar oqimdan hisoblanishi kerak bo‘lgan baytlarning minimal sonini ko‘rsatadi.

Nazorat savollari:

1. Soket va TCP orasidaqi farq nimalardan iborat ?
2. NetworkStream vazifasi nima?
3. TCP serverdan ma’lumotlarni o‘qish qanday amalga oshiriladi.

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari

11-Mashg‘ulot: TCP protokoli.

Reja:

1. TCP mijozi. TcpClient klassi.
2. Ma’lumot yuborish va qabul qilish.
3. TCP serveri. TcpListener sinfi.

1. TCP mijozi. TcpClient klassi.

Tcp mijozini yaratish uchun. net platformasi sinfni taqdim etadi TcpClient klassi, rozetkalar ustiga qurilgan va yana ma’lumotlarni yuborish va qabul qilish uchun rozetkadan foydalanadi, shu bilan birga ba’zi narsalarni yozishni osonlashtiradi. (TcpClient manba kodi)

TcpClient Yaratish

TcpClient yaratish uchun sinf konstruktorlaridan biri qo‘llaniladi:

```
public TcpClient ();
public TcpClient (System.Net.Sockets.AddressFamily family);
public TcpClient (string hostname, int port);
public TcpClient (System.Net.IPEndPoint localEP);
```

Oxirgi TcpClient ob’ektidan tashqari barcha konstruktorlardan foydalanganda avtomatik ravishda eng mos mahalliy IP-manzil va port tayinlanadi. Oxirgi konstruktor orqali siz mahalliy so‘nggi nuqtani - TcpClient biriktiriladigan ipendpoint ob’ektini qo‘lda o‘rnatishingiz mumkin.

Ikkinchi konstruktordan foydalanganda addressfamily qiymati unga uzatiladi. InterNetwork (IPv4 manzillari uchun) yoki AddressFamily.InterNetworkV6 (IPv6 manzillari uchun).

Mijoz ulanmoqchi bo‘lgan masofaviy tugunning manzili va porti uchinchi konstruktorga uzatiladi.

TcpClient Xususiyatlari

TcpClient xususiyatlari sizga ob’ektning holatini sozlash yoki u haqida ma’lumot olish imkonini beradi. Xususiyatlar orasida quyidagilarni ta’kidlaymiz:

- **Available:** tarmoqdan olingan va o‘qilishi mumkin bo‘lgan ma’lumotlar baytlari sonini qaytaradi.
- **Client:** TcpClient ob’ekti tomonidan ishlatiladigan Socket ob’ektini qaytaradi yoki o‘rnatadi.
- **Connected:** TcpClient uzoq tugunga ulangan bo‘lsa, true ni qaytaradi.
- **LingerState:** port faqat bitta mijoz uchun mavjudligini qaytaradi yoki o‘rnatadi.
- **NoDelay:** yuborish va qabul qilish buferlari to‘ldirilmaganda kechikish amal qiladimi yoki yo‘qligini ko‘rsatadi. Agar u noto‘g‘ri bo‘lsa, TcpClient faqat etarli ma’lumot to‘planganda paketlarni tarmoq orqali yuboradi. Bu amalga oshiriladi, chunki kichik ma’lumotlarni yuborish samarasiz bo‘lishi mumkin va

ortiqcha yukga olib kelishi mumkin. Shu bilan birga, kichik hajmdagi ma'lumotlarni yuborish kerak bo'lgan yoki tezroq javob olish kerak bo'lgan holatlar bo'lishi mumkin.

- **ReceiveBufferSize:** qabul qilish buferining hajmini qaytaradi yoki o'rnatadi (standart 65536).
- **ReceiveTimeout:** TcpClient ob'ekti o'qish jarayoni boshlangandan so'ng ma'lumotlarni olishni kutadigan vaqt oralig'ini qaytaradi yoki belgilaydi (standart 0).
- **SendBufferSize:** yuborish buferining hajmini qaytaradi yoki o'rnatadi (sukut bo'yicha 65536).
- **SendTimeout:** TcpClient ob'ekti ma'lumotlarni yuborishning muvaffaqiyatli yakunlanishini kutadigan vaqt oralig'ini qaytaradi yoki belgilaydi (sukut bo'yicha 0).

Serverga ulanish

TCP serverga ulanish uchun ushbu sinfda masofaviy xost manzili uzatiladigan Connect()/ConnectAsync () usuli aniqlanadi. Asosiy versiyalar:

```
public Task ConnectAsync (System.Net.IPEndPoint remoteEP);
public Task ConnectAsync (string host, int port);
public Task ConnectAsync (System.Net.IPAddress address, int port);
```

Masalan, xostga ulanish "www.google.com":

Muvaffaqiyatsiz ulanish urinishida Connect/ConnectAsync usuli SocketException turini istisno qiladi.

TcpClient yopilishi

TcpClient bilan ishlashni tugatgandan so'ng, uni yopish () usuli bilan yopish kerak:

```
using System.Net.Sockets;
TcpClient tcpClient = new TcpClient();
await tcpClient.ConnectAsync("www.google.com", 80);
Console.WriteLine("Подключение установлено");
tcpClient.Close(); // закрываем подключение
Console.WriteLine(tcpClient.Connected); // False - подключение
закрыто
```

Yoki oldingi misolda bo'lgani kabi using dizaynidan foydalanishingiz mumkin.

2. Ma'lumot yuborish va qabul qilish.

Ma'lumotlarni yuborish va qabul qilish uchun TcpClient odatda o'tgan mavzuda ko'rib chiqilgan networkstream sinfidan foydalanadi. Serverga ulangandan so'ng Networkstream ob'ektini olish uchun TcpClient getstream () usulini chaqirishi mumkin:

```
using System.Net.Sockets;
using TcpClient tcpClient = new TcpClient();
await tcpClient.ConnectAsync("www.google.com", 80);
```

```
NetworkStream stream = tcpClient.GetStream();
```

Shunga ko'ra, TcpClient uchun ma'lumotlarni yuborish va qabul qilish NetworkStream haqidagi so'nggi maqolada muhokama qilinganidek amalga oshiriladi.

Shuni ta'kidlash kerakki, aslida NetworkStream ma'lumotlarni yuborish va qabul qilish uchun TcpClient bilan bir xil Socket ob'ektidan foydalanadi. Bundan tashqari, Agar TcpClient obyekti yopilsa (yuqoridagi misolda using konstruksiyasi qo'llaniladi), u bilan birga networkstream obyekti ham avtomatik ravishda yopiladi (shuningdek, ular bilan bog'liq socket ham chiqariladi).

Ma'lumotlarni yuborish

NetworkStream-ga ma'lumotlarni yuborish uchun Write()/WriteAsync() usuli qo'llaniladi:

```
public ValueTask WriteAsync (ReadOnlyMemory<byte> buffer,
CancellationToken cancellationToken = default);
public Task WriteAsync (byte[] buffer, int offset, int count);
```

Yuborilgan ma'lumotlar baytlar qatori yoki readonlymemory<bayt> ob'ekti sifatida uzatiladi, u yana baytlar qatorini qamrab oladi.

```
using System.Net.Sockets;
using System.Text;
using TcpClient tcpClient = new TcpClient();
var server = "www.google.com";
await tcpClient.ConnectAsync(server, 80);
NetworkStream stream = tcpClient.GetStream();
var requestMessage = $"GET / HTTP/1.1\r\nHost:
{server}\r\nConnection: Close\r\n\r\n";
var requestData = Encoding.UTF8.GetBytes(requestMessage);
await stream.WriteAsync(requestData);
```

So'rovlarni ko'rib chiqadigan server "www.google.com" http serverini ifodalaydi va http protokoli bo'yicha so'rovlarni qayta ishlaydi. Shuning uchun bizning mijozimiz http protokoliga mos keladigan xabarni yuboradi.

Ma'lumotlarni olish

NetworkStream-ga ma'lumotlarni olish uchun Read()/ReadAsync () usullari qo'llaniladi. Shunday qilib, yuqoridagi misolda biz serverga http so'rovini yubordik "www.google.com". endi biz undan javob olamiz:

```
using System.Net.Sockets;
using System.Text;
using TcpClient tcpClient = new TcpClient();
var server = "www.google.com";
await tcpClient.ConnectAsync(server, 80);
var stream = tcpClient.GetStream();
var requestMessage = $"GET / HTTP/1.1\r\nHost:
{server}\r\nConnection: Close\r\n\r\n";
var requestData = Encoding.UTF8.GetBytes(requestMessage);
await stream.WriteAsync(requestData);
var responseData = new byte[512];
```



```
var response = new StringBuilder();
int bytes; // количество полученных байтов
do
{
    bytes = await stream.ReadAsync(responseData);
    response.Append(Encoding.UTF8.GetString(responseData, 0, bytes));
}
while (bytes>0);
Console.WriteLine(response);
```

endi o'qish uchun do tsiklidan foydalanamiz..while. ReadAsync qancha baytni qaytarishini ko'rib chiqamiz. Va u 0 baytni qaytarganda, tsiklni takrorlang. Olingan baytlarni mag'lubiyatga aylantiramiz va StringBuilder-ga qo'shamiz. Oxirida biz olingan tarkibni StringBuilder-dan konsolga chiqaramiz:

O'qish TcpClient ma'lumotlari va xususiyatlari.Available va NetworkStream.DataAvailable

Oqimdan ma'lumotlarni o'qiyotganda, tez-tez uchraydigan vazifalardan biri ma'lumotlarning oxirini aniqlashdir, ayniqsa biz ma'lumotlarni cheksiz tsiklda o'qiyotgan bo'lsak. Yuqoridagi misolda bu juda oddiy edi - ular haqiqatan ham O'qilgan baytlar soni nolga yetguncha o'qishdi, ya'ni o'qish uchun mavjud baytlar qolmaydi:

```
do
{
    bytes = await stream.ReadAsync(responseData);
    response.Append(Encoding.UTF8.GetString(responseData, 0, bytes));
}
while (bytes>0); // пока данные есть в потоке
```

Ammo TcpClient sinfida o'qish mumkin bo'lgan baytlar sonini qaytaradigan Available xususiyati mavjud. Bundan tashqari, networkstream klassi shunga o'xshash xususiyatga ega - Data Available, agar oqim o'qilishi mumkin bo'lgan ma'lumotlarga ega bo'lsa, true-ni qaytaradi. Shunga ko'ra, savol tug'iladi, nega bu xususiyatlardan foydalanmaslik kerak? Masalan, Available xususiyati:

```
do
{
    bytes = await stream.ReadAsync(responseData);
    response.Append(Encoding.UTF8.GetString(responseData, 0, bytes));
}
while (tcpClient.Available > 0); // пока данные есть в потоке
```

Yoki dataavailable xususiyati

```
do
{
    bytes = await stream.ReadAsync(responseData);
    response.Append(Encoding.UTF8.GetString(responseData, 0, bytes));
}
while (stream.DataAvailable); // пока данные есть в потоке
```


Aslida, bu ikkala xususiyat ham o‘qish mumkin bo‘lgan baytlar sonini qaytaradigan ishlatilgan Socket ob’ekting Available xususiyati qiymatiga qaraydi. Va shunga ko‘ra, bu erda biz Socket klassi bilan ishlashda bir xil muammoga duch kelamiz - Agar mavjud ma’lumotlar mavjud bo‘lsa, Available xususiyati nolga teng bo‘lmaydi. Ammo TCP protokolining mohiyati shundaki, katta ma’lumotlar to‘plamlari alohida paketlarda yuboriladi. Ba’zi paketlar tezroq kelishi mumkin, ba’zilar kechiktiriladi, ba’zilar yo‘qoladi va qayta yuborish kerak bo‘ladi. Shuning uchun, server ma’lumotlarni yuborgan, ma’lumotlarning bir qismi kelgan vaziyat yuzaga kelishi mumkin. Bir vaqtning o‘zida socketdagi Available xususiyati 0 ga qaytdi, mos ravishda tsikldan chiqish sodir bo‘ldi.

3. TCP serveri. *TcpListener* sinfi.

Tcplistener klassi server vazifasini bajaradi. U ma’lum bir port orqali kiruvchi ulanishlarni tinglaydi. Shuni ta’kidlash kerakki, TcpListener klassi Socket socket klassi asosida qurilgan, ya’ni aslida Socket klassi ustidagi o‘rashni ifodalaydi va toza rozetkalardan foydalanish bilan solishtirganda ba’zi narsalarni yozishni osonlashtiradi.

***TcpListener* Yaratish**

Tcplistener ob’ekti server ishlaydigan manzil va port bilan tavsiflanadi. Tcplistener ob’ektini ishga tushirish uchun uning dizaynerlaridan biri ishlatiladi:

```
public TcpListener (System.Net.IPAddress localAddr, int port);
public TcpListener (System.Net.IPEndPoint localEP);
```

Birinchi konstruktor serverning manzili va tinglash portini belgilaydi:

```
IPAddress localAddr = IPAddress.Parse("127.0.0.1");
TcpListener server = new TcpListener(localAddr, 8888);
```

Bunday holda, TCP serveri 8888 portidagi "127.0.0.1" manzilida ishlaydi.

Ikkinchi konstruktor IPEndPoint ob’ektini qabul qiladi

```
IPAddress localAddr = IPAddress.Parse("127.0.0.1");
IPEndPoint ipLocalEndPoint = new IPEndPoint(localAddr, 8888);
TcpListener server = new TcpListener(ipLocalEndPoint);
```

Agar server qaysi mahalliy manzilda ishlashi muhim bo‘lmasa, siz ipaddress qiymatini manzil sifatida ishlatishingiz mumkin. Any. Keyin serverga eng mos tarmoq manzili tayinlanadi (agar bir nechta tarmoq interfeyslari mavjud bo‘lsa). Bundan tashqari, agar port raqami muhim bo‘lmasa, u holda 0 raqamini port sifatida ko‘rsatish mumkin. Keyin serverga mavjud portlardan biri taqdim etiladi. Ushbu yondashuv yordamida aniq manzil va portni LocalEndpoint xususiyati orqali olish mumkin.

```
TcpListener server = new TcpListener(IPAddress.Any, 0);
Console.WriteLine(server.LocalEndpoint);
```

So‘rovlarni tinglash

Kiruvchi so‘rovlarni qabul qilishni tashkil qilish uchun TcpListener bir qator usullardan foydalanadi. Ularning asosiylari:

- **AcceptSocket () / accept Socket Async ():** kutilayotgan ulanish so‘rovini qabul qiladi. Mijoz bilan bog‘lanish uchun ishlatiladigan Socket ob‘ektini qaytaradi
- **AcceptTcpClient () / AcceptTcpClientAsync ():** kutilayotgan ulanish so‘rovini qabul qiladi. Mijoz bilan bog‘lanish uchun ishlatiladigan TcpClient ob‘ektini qaytaradi
- **Pending ():** kutilayotgan ulanish so‘rovlari mavjudligini aniqlaydi. Agar kutilayotgan ulanishlar mavjud bo‘lsa, true — ni qaytaradi; aks holda noto‘g‘ri..
- **Start():** serverni ishga tushiradi.
- **Start(Int32)** usulini parametr sifatida haddan tashqari o‘chirish kutilayotgan ulanishlarning maksimal navbat uzunligini oladi.
- **Stop ():** serverni to‘xtatadi.

Tcplistenerning umumiy ishlash jarayoni quyidagicha:

Serverni ishga tushirish uchun Start () usuli chaqiriladi.

```
TcpListener server = new TcpListener(IPAddress.Any, 8888);
server.Start();
```

Start usuli ishlatilgan Socket ob‘ektini ishga tushiradi, uni mahalliy so‘nggi nuqta bilan bog‘laydi va kiruvchi so‘rovlarni tinglashni boshlaydi. Kiruvchi so‘rovni qabul qilganda, u navbatga qo‘yiladi. Agar navbatda yangi ulanish uchun joy bo‘lmasa, unda SocketException istisnosi yaratiladi. Odatiy bo‘lib, kutish navbatini o‘z ichiga olishi mumkin bo‘lgan maksimal miqdor 2147483647.

Mijoz serverga kirganda, biz ikkita usuldan birini ishlatishimiz mumkin AcceptSocket()/AcceptSocket Async() yoki AcceptTcpClient()/AcceptTcpClientAsync() navbati bilan Socket yoki TcpClient ob‘ekti sifatida ulanish uchun.

```
TcpClient client = await server.AcceptTcpClientAsync();
Socket socket = await server.AcceptSocketAsync();
```

Olingan ob‘ektlar TcpClient va Socket masofaviy tugunning IP-manzili va port raqami bilan ishga tushiriladi. Bundan tashqari, ushbu sinflarning usullari orqali siz ulangan mijoz bilan o‘zaro aloqada bo‘lishingiz mumkin: undan ma’lumotlarni oling yoki aksincha, unga yuboring.

Oxirida, server tugagandan so‘ng, siz Stop () usulini chaqirishingiz kerak. Bunday holda, navbatda qolgan barcha ulanishlar yo‘qoladi.

Tcp mijozini tcp serveriga ulash

Quyidagi server kodini aniqlang:

```
using System.Net;
using System.Net.Sockets;
```

```
var tcpListener = new TcpListener(IPAddress.Any, 8888);
try
{
    tcpListener.Start();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");

    while (true)
    {
        using var tcpClient = await
tcpListener.AcceptTcpClientAsync();
        Console.WriteLine($"Входящее подключение:
{tcpClient.Client.RemoteEndPoint}");
    }
}
finally
{
    tcpListener.Stop();
}
```

Serverni cheksiz tsiklda ishga tushirgandan so‘ng, kiruvchi ulanishlarni tinglang va qabul qilinganda konsolga ulanish manzilini ko‘rsating.

Ushbu serverga ulanish uchun boshqa loyihada quyidagi mijoz kodini aniqlaymiz:

```
using System.Net.Sockets;
using TcpClient tcpClient = new TcpClient();
Console.WriteLine("Клиент запущен");
await tcpClient.ConnectAsync("127.0.0.1", 8888);
if (tcpClient.Connected)
    Console.WriteLine($"Подключение с
{tcpClient.Client.RemoteEndPoint} установлено");
else
    Console.WriteLine("Не удалось подключиться");
```

Mijoz yuqoridagi ma’lum bir serverga ulanadi. Server mahalliy kompyuterda ishlayotganligi va har qanday mahalliy manzil va 8888 portida mavjud bo‘lganligi sababli, unga ulanish uchun biz "127.0.0.1" mahalliy manzili va 8888 portidan foydalanamiz.

Muvaffaqiyatli ulanishdan so‘ng biz konsolga diagnostika xabarini ko‘rsatamiz.

Serverni ishga tushiring va keyin mijozni ishga tushiring. Natijada, mijoz server konsolidagi serverga ulangandan so‘ng, biz shunga o‘xshash narsani ko‘ramiz:

Nazorat savollari:

1. TCPClass Fieldlari haqida ma’lumot bering.
2. TCPListener ishlash mexanizmi haqida ma’lumot bering
3. TCPListenerha hosil bo‘lishi mumkin bo‘lgan xatoliklar haqida ma’lumot bering

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**12-Mashg‘ulot:** Ma’lumotlarni yuborish va qabul qilish.**Reja:**

1. Ma’lumotlarni yuborish va qabul qilish. Soketlar orasidagi bir tomonlama aloqa.
2. Ma’lumotlarni olishning usullari.
3. Ma’lumotlarni yuborish va qabul qilish. TcpListener va TcpClient o‘rtasida bir tomonlama aloqa.

1. Ma’lumotlarni yuborish va qabul qilish. Soketlar orasidagi bir tomonlama aloqa.

Mijoz rozetkasi va server rozetkasi o‘rtasidagi bir tomonlama aloqani ko‘rib chiqing, bu erda server ma’lumotlarni yuboradi va mijoz qabul qiladi yoki aksincha, mijoz ma’lumotlarni yuboradi va server qabul qiladi.

Mijozga ma’lumotlarni yuborish

Birinchidan, server ma’lumotlarni yuboradigan va mijoz faqat ularni qabul qiladigan vaziyatni ko‘rib chiqing. Shunday qilib, server uchun quyidagi kodni aniqlaymiz:

```
using System.Net;
using System.Net.Sockets;
using System.Text;

IPEndPoint ipPoint = new IPEndPoint(IPAddress.Any, 8888);
using Socket tcpListener = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    tcpListener.Bind(ipPoint);
    tcpListener.Listen();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");
    while (true)
    {
        // получаем входящее подключение
        using var tcpClient = await tcpListener.AcceptAsync();
        // определяем данные для отправки - текущее время
        byte[] data =
Encoding.UTF8.GetBytes(DateTime.Now.ToLongTimeString());
        // отправляем данные
        await tcpClient.SendAsync(data);
        Console.WriteLine($"Клиенту {tcpClient.RemoteEndPoint}
отправлены данные");
    }
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

Misol tariqasida biz mijozga joriy vaqtni hh:mm:ss formatida yuboramiz. Buning uchun ulanishni olgandan so‘ng, qatorni baytlar qatoriga o‘zgartiring va

tcpClient usuli yordamida yuboring.SendAsync(). Va biz diagnostika xabarini server konsolida ko‘rsatamiz.

Mijoz tomonida biz quyidagi kodni aniqlaymiz:

```
using System.Net.Sockets;
using System.Text;

using var tcpClient = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    await tcpClient.ConnectAsync("127.0.0.1", 8888);
    // буфер для считывания данных
    byte[] data = new byte[512];

    int bytes = await tcpClient.ReceiveAsync(data);
    string time = Encoding.UTF8.GetString(data, 0, bytes);
    Console.WriteLine($"Текущее время: {time}");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

Mijoz tomonida biz server sanani satr sifatida yuborishini bilamiz va uni o‘qish uchun 512 baytdan iborat bufer qatorini aniqlaymiz. TcpClient usuli bilan.ReceiveAsync () biz oqimdan ma’lumotlarni o‘qiymiz, baytlarni mag‘lubiyatga aylantiramiz va uni konsolga chiqaramiz.

Server va mijozni ishga tushiring. Serverga kirishda mijoz joriy vaqtni oladi:

Mijozdan ma’lumotlarni olish

Endi mijoz va serverlar o‘rtasidagi bir tomonlama aloqaning boshqa turini ko‘rib chiqing, aksincha, mijoz ma’lumotlarni yuboradi va server shunchaki qabul qiladi. Shunday qilib, server uchun quyidagi kodni aniqlaymiz:

```
using System;
using System.Net.Sockets;
using System.Text;

IPEndPoint ipPoint = new IPEndPoint(IPAddress.Any, 8888);
using Socket tcpListener = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    tcpListener.Bind(ipPoint);
    tcpListener.Listen();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");
    while (true)
    {
        using var tcpClient = await tcpListener.AcceptAsync();
```

```

        byte[] responseData = new byte[512];
        int bytes = 0;          var response = new StringBuilder(); //
для склеивания данных в строку
        do
        {
            bytes = await tcpClient.ReceiveAsync(responseData);
            response.Append(Encoding.UTF8.GetString(responseData, 0,
bytes));
        }
        while (bytes > 0);
        Console.WriteLine(response);
    }
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}

```

Bu erda tcpClient usuli yordamida.ReceiveAsync () biz oqimdan baytlar qatoriga ma’lumotlarni o’qiymiz. Bunday holda, biz ma’lumotlar qatorni ifodalaydi deb taxmin qilamiz. Va baytlardan satr olgandan so‘ng, biz uni konsolga ko‘rsatamiz.

Mijozda biz ba’zi satrlarni yuborish uchun eng oddiy kodni aniqlaymiz:

```

using System.Net.Sockets;
using System.Text;
using var tcpClient = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    await tcpClient.ConnectAsync("127.0.0.1", 8888);
    var message = "Hello METANIT.COM";
    byte[] requestData = Encoding.UTF8.GetBytes(message);
    await tcpClient.SendAsync(requestData);
    Console.WriteLine("Сообщение отправлено");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}

```

2. Ma’lumotlarni olishning usullari.

Ma’lumotlarni olish strategiyalari

TCP-da mijoz-server o‘zaro ta’sirining eng qiyin qismlaridan biri bu ma’lumotlarni yuborish va qabul qilishdir. Va hamma narsani iloji boricha maqbul qilish uchun aniq echim yo‘q, ammo bir qator strategiyalar mavjud:

Qancha ma’lumot yuborilishini aniq bilganimizda, belgilangan uzunlikdagi buferdan foydalanish

Javobda javobning hajmi to‘g‘risida ma’lumot yuborish, uni qabul qilib, kerakli bayt sonini hisoblash osonroq bo‘ladi

Javobni tugatish markeridan foydalanib, biz ma'lumotlarni o'qishni tugatamiz

Ushbu strategiyalarning har qandayidan foydalanish ibtidoiy bo'lsa ham, mijoz va server o'rtasidagi o'zaro ta'sir protokolini aniqlashni o'z ichiga oladi, bunda mijoz va server yagona qoidalarga muvofiq ma'lumotlarni shakllantiradi, yuboradi va oqimdan chiqaradi.

Ruxsat etilgan bufer versiyasi juda aniq, shuning uchun qolgan ikkita strategiyani ko'rib chiqing.

Javobni tugatish markeridan foydalanish

Quyidagi server kodini aniqlang:

```
using System.Net;
using System.Net.Sockets;
using System.Text;

using Socket tcpListener = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);

try
{
    tcpListener.Bind(new IPEndPoint(IPAddress.Any, 8888));
    tcpListener.Listen();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");

    while (true)
    {
        using var tcpClient = await tcpListener.AcceptAsync();
        var buffer = new List<byte>();
        var bytesRead = new byte[1];
        while (true)
        {
            var count = await tcpClient.ReceiveAsync(bytesRead);
            if (count == 0 || bytesRead[0] == '\n') break;
            buffer.Add(bytesRead[0]);
        }
        var message = Encoding.UTF8.GetString(buffer.ToArray());
        Console.WriteLine($"Получено сообщение: {message}");
    }
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

Aytaylik, bu erda \n belgisi yoki 10 qiymatini ifodalovchi chiziq tarjimasini xabarning oxiri belgisi sifatida ishlaydi. Yakuniy belgini kuzatish uchun biz belgini o'qiymiz. O'qish uchun biz bir baytli massivni aniqlaymiz. Biz unga baytni o'qiymiz va uning qiymatini tekshiramiz. Va agar chiziq tarjimasini duch kelsa, biz ma'lumotlarni o'qishni tugatamiz va olingan ma'lumotlarni qatarga aylantiramiz.

Bunday holda, mijoz xabarning oxiriga \n belgisini qo'shishi kerak:

```

using System.Net.Sockets;
using System.Text;
using var tcpClient = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    await tcpClient.ConnectAsync("127.0.0.1", 8888);
    var message = "Hello METANIT.COM\n";
    byte[] requestData = Encoding.UTF8.GetBytes(message);
    await tcpClient.SendAsync(requestData);
    Console.WriteLine("Сообщение отправлено");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}

```

Garchi bu ma'lum darajada biroz ibtidoiy soddalashtirish bo'lsa-da, chunki bu holda biz bir qatorli matnni yuborish bilan cheklanamiz. Ammo, aslida, cheklangan markerni aniqlash mantig'i yanada murakkablashishi mumkin, ayniqsa marker bitta bayt/belgini emas, balki bir nechta, ammo umumiy prinip bir xil bo'lsa.

Bunday holda, bitta baytni o'qishni Socket klassi uchun alohida kengaytirish usuliga o'tkazish orqali dasturni optimallashtirish mumkin:

```

using System.Net;
using System.Net.Sockets;
using System.Text;
using Socket tcpListener = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    tcpListener.Bind(new IPEndPoint(IPAddress.Any, 8888));
    tcpListener.Listen();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");
    while (true)
    {
        using var tcpClient = await tcpListener.AcceptAsync();
        var buffer = new List<byte>();
        int bytesRead = '\n';
        while (true)
        {
            bytesRead = tcpClient.ReadByte();
            if(bytesRead== '\n' || bytesRead==0) break;
            buffer.Add((byte)bytesRead);
        }
        var message = Encoding.UTF8.GetString(buffer.ToArray());
        Console.WriteLine($"Получено сообщение: {message}");
    }
}
catch(Exception ex)

```



```

{
    Console.WriteLine(ex.Message);
}

public static class SocketExtension
{
    public static int ReadByte(this Socket socket)
    {
        byte b = 0;
        var buffer = new Span<byte>(ref b);
        var count= socket.Receive(buffer);
        return count== 0 ? -1 : b;
    }
}

```

Xabar hajmini sozlash

Quyidagi serverni aniqlang:

```

using System.Net;
using System.Net.Sockets;
using System.Text;

using Socket tcpListener = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    tcpListener.Bind(new IPEndPoint(IPAddress.Any, 8888));
    tcpListener.Listen();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");
    while (true)
    {
        using var tcpClient = await tcpListener.AcceptAsync();
        byte[] sizeBuffer = new byte[4];
        await tcpClient.ReceiveAsync(sizeBuffer);
        int size = BitConverter.ToInt32(sizeBuffer, 0);
        byte[] data = new byte[size];
        int bytes = await tcpClient.ReceiveAsync(data);
        var message = Encoding.UTF8.GetString(data, 0, bytes);
        Console.WriteLine($"Размер сообщения: {size} байтов");
        Console.WriteLine($"Сообщение: {message}");
    }
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}

```

Bunday holda, biz o'lcham int turining qiymatini, ya'ni 4 bayt qiymatini ifodalaydi deb taxmin qilamiz. Shunga ko'ra, o'lchamni o'qish uchun biz 4 baytdan bufer yaratamiz va undagi ma'lumotlarning birinchi qismini o'qiymiz. O'lchamni o'qib bo'lgach, biz uni bitconverter statik usuli yordamida int raqamiga aylantirishimiz

mumkin.ToInt32 (), tegishli uzunlikdagi buferni aniqlang va undagi ma’lumotlarni hisoblang.

```
byte[] sizeBuffer = new byte[4];
await tcpClient.ReceiveAsync(sizeBuffer);
int size = BitConverter.ToInt32(sizeBuffer, 0);
byte[] data = new byte[size];
int bytes = await tcpClient.ReceiveAsync(data);
```

Mijoz tomonida biz quyidagi kodni aniqlaymiz:

```
using System.Net.Sockets;
using System.Text;
using var tcpClient = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    await tcpClient.ConnectAsync("127.0.0.1", 8888);
    var message = "Hello METANIT.COM";
    byte[] data = Encoding.UTF8.GetBytes(message);
    byte[] size = BitConverter.GetBytes(data.Length);
    await tcpClient.SendAsync(size);
    await tcpClient.SendAsync(data);
    Console.WriteLine("Сообщение отправлено");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

Bu erda teskari jarayon sodir bo‘ladi. Birinchidan, biz bitconverter usuli yordamida baytlar qatoriga xabar hajmini olamiz.GetBytes():

```
byte[] size = BitConverter.GetBytes(data.Length);
```

Biz o‘lchamni to‘rt bayt shaklida yuboramiz va keyin ma’lumotlarni o‘zimiz yuboramiz

```
await tcpClient.SendAsync(size);
await tcpClient.SendAsync(data);
```

Bir nechta yuborish va qabul qilish

Yuqorida mijoz yuborgan va server faqat bitta xabar olgan. Ammo server alohida qabul qilishi kerak bo‘lgan ko‘plab xabarlar yuborilsa nima bo‘ladi. Bunday holda, biz yana biron bir protokolni aniqlashimiz kerak, unga ko‘ra server bitta xabar qachon tugashini va mijoz umuman shovqinni qachon tugatishini aniqlay oladi. Kichik bir misolni ko‘rib chiqing. Server quyidagi kodga ega bo‘lsin:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
using Socket tcpListener = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
```

```

tcpListener.Bind(new IPEndPoint(IPAddress.Any, 8888));
tcpListener.Listen();
Console.WriteLine("Сервер запущен. Ожидание подключений... ");
while (true)
{
    using var tcpClient = await tcpListener.AcceptAsync();

    var buffer = new List<byte>();
    var bytesRead = new byte[1];
    while (true)
    {
        while (true)
        {
            var count = await tcpClient.ReceiveAsync(bytesRead);
            if (count == 0 || bytesRead[0] == '\n') break;
            buffer.Add(bytesRead[0]);
        }
        var message = Encoding.UTF8.GetString(buffer.ToArray());
        if (message == "END") break;
        Console.WriteLine($"Получено сообщение: {message}");
        buffer.Clear();
    }
}
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}

```

Shunday qilib, bu erda bizning mijoz-server o‘zaro aloqasi protokoli ikkita qoidadan iborat. Birinchidan, har bir alohida xabar \n satrining tarjimai bilan tugashi kerak. ikkinchidan, mijoz o‘zaro aloqani tugatishni va serverdan uzishni xohlasa, u "END" buyrug‘ini yuboradi. Shuning uchun, cheksiz tsiklda biz mijozning barcha xabarlarini qayta ishlaymiz va ichki tsiklda baytni o‘qiymiz. Ushbu server bilan ishlash uchun quyidagi mijozni aniqlang:

```

using System.Net.Sockets;
using System.Text;
using var tcpClient = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    await tcpClient.ConnectAsync("127.0.0.1", 8888);
    var messages = new string[] { "Hello METANIT.COM\n", "Hello
TcpListener\n", "Bye METANIT.COM\n", "END\n" };
    foreach (var message in messages)
    {
        byte[] data = Encoding.UTF8.GetBytes(message);
        await tcpClient.SendAsync(data);
    }
}

```

```

        Console.WriteLine("Все сообщения отправлены");
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }

```

Bu erda yuborilgan har bir xabar \n terminal belgisi bilan tugaydi, bundan tashqari, oxirgi xabar "END" o‘zaro ta’sirini tugatish buyrug‘ini anglatadi. Natijada, ushbu mijoz ulanganda, server konsolda "end" buyrug‘idan tashqari barcha yuborilgan xabarlarni ko‘rsatadi, bu mijoz va serverning o‘zaro ta’sirini yakunlaydi:

3. Ma’lumotlarni yuborish va qabul qilish. TcpListener va TcpClient o‘rtasida bir tomonlama aloqa.

TcpClient mijoz va TcpListener serveri o‘rtasidagi bir tomonlama aloqani ko‘rib chiqing, bu erda server ma’lumotlarni yuboradi va mijoz oladi yoki aksincha, mijoz ma’lumotlarni yuboradi va server oladi. Aslida, bu erda faqat TcpListener va TcpClient-dan foydalangan holda, so‘nggi maqoladagi soketlarda bo‘lgani kabi bir xil misollar ko‘rib chiqiladi.

Mijozga ma’lumotlarni yuborish

Birinchidan, server ma’lumotlarni yuboradigan va mijoz faqat ularni qabul qiladigan vaziyatni ko‘rib chiqing. Shunday qilib, server uchun quyidagi kodni aniqlaymiz:

```

using System.Net;
using System.Net.Sockets;
using System.Text;
var tcpListener = new TcpListener(IPAddress.Any, 8888);
try
{
    tcpListener.Start();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");
    while (true)
    {
        using var tcpClient = await
tcpListener.AcceptTcpClientAsync();
        var stream = tcpClient.GetStream();
        byte[] data =
Encoding.UTF8.GetBytes(DateTime.Now.ToLongTimeString());
        await stream.WriteAsync(data);
        Console.WriteLine($"Клиенту {tcpClient.Client.RemoteEndPoint}
отправлены данные");
    }
}
finally
{
    tcpListener.Stop();
}

```

Misol tariqasida biz mijozga joriy vaqtni hh:mm:ss formatida yuboramiz. Buning uchun qatorni baytlar qatoriga o‘zgartiring va ularni stream usuli yordamida yuboring. WriteAsync(). Va biz diagnostika xabarini server konsolida ko‘rsatamiz.

Mijoz tomonida biz quyidagi kodni aniqlaymiz:

```
using System.Net.Sockets;
using System.Text;
using TcpClient tcpClient = new TcpClient();
await tcpClient.ConnectAsync("127.0.0.1", 8888);
byte[] data = new byte[512];
var stream = tcpClient.GetStream();
int bytes = await stream.ReadAsync(data);
string time = Encoding.UTF8.GetString(data, 0, bytes);
Console.WriteLine($"Текущее время: {time}");
```

Mijoz tomonida biz server sanani satr sifatida yuborishini bilamiz va uni o‘qish uchun 512 baytdan iborat bufer qatorini aniqlaymiz. Stream usuli bilan ReadAsync() biz oqimdan ma’lumotlarni o‘qiymiz, baytlarni mag‘lubiyatga aylantiramiz va uni konsolga chiqaramiz.

Server va mijozni ishga tushiring. Serverga kirishda mijoz joriy vaqtni oladi:

Va server konsolida mijozning ip-manzili ko‘rsatiladi:

Mijozdan ma’lumotlarni olish

Endi mijoz va serverlar o‘rtasidagi bir tomonlama aloqaning boshqa turini ko‘rib chiqing, aksincha, mijoz ma’lumotlarni yuboradi va server shunchaki qabul qiladi. Shunday qilib, server uchun quyidagi kodni aniqlaymiz:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
var tcpListener = new TcpListener(IPAddress.Any, 8888);
try
{
    tcpListener.Start(); // запускаем сервер
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");

    while (true)
    {
        using var tcpClient = await
tcpListener.AcceptTcpClientAsync();
        var stream = tcpClient.GetStream();
        byte[] responseData = new byte[1024];
        int bytes = 0; // количество считанных байтов
        var response = new StringBuilder(); // для склеивания данных
в строку
        do
        {
            bytes = await stream.ReadAsync(responseData);
            response.Append(Encoding.UTF8.GetString(responseData, 0,
bytes));
```

```

    }
    while(bytes > 0);
    Console.WriteLine(response);
}
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    tcpListener.Stop();
}

```

Bu erda biz TcpClient-dan networkstream oqimini olamiz va undan mijozdan ma'lumotlarni o'qish uchun foydalanamiz. Xususan, stream usuli bilan.ReadAsync() oqimdan baytlar qatoriga ma'lumotlarni o'qiydi. Bunday holda, biz ma'lumotlar qatorni ifodalaydi deb taxmin qilamiz. Va baytlardan satr olgandan so'ng, biz uni konsolga ko'rsatamiz.

Mijozda biz ba'zi satrlarni yuborish uchun eng oddiy kodni aniqlaymiz:

```

using System.Net.Sockets;
using System.Text;
using TcpClient tcpClient = new TcpClient();
await tcpClient.ConnectAsync("127.0.0.1", 8888);
var message = "Hello METANIT.COM";
byte[] requestData = Encoding.UTF8.GetBytes(message);
var stream = tcpClient.GetStream();
await stream.WriteAsync(requestData);
Console.WriteLine("Сообщение отправлено");

```

Bunday holda, stream usuli yordamida.WriteAsync() serverga ma'lumotlarni yuboradi. Oddiy satr ma'lumotlar sifatida ishlaydi.

Server va mijozni ishga tushiring. Muvaffaqiyatli ulangandan va mijoz konsoliga ma'lumotlarni yuborganimizdan so'ng, biz tegishli xabarni ko'ramiz:

Va server mijozdan xabar oladi va uni konsolda ko'rsatadi:

Ma'lumotlarni olish strategiyalari

TCP-da mijoz-server o'zaro ta'sirining eng qiyin qismlaridan biri bu ma'lumotlarni olishdir. Va bu erda aniq echim yo'q, lekin bir qator strategiyalar mavjud:

- Qancha ma'lumot yuborilishini aniq bilganimizda, belgilangan uzunlikdagi buferdan foydalanish
- Javobda javobning hajmi to'g'risida ma'lumot yuborish, uni qabul qilib, kerakli bayt sonini hisoblash osonroq bo'ladi
- Javobni tugatish markeridan foydalanib, biz ma'lumotlarni o'qishni tugatamiz

Ushbu strategiyalarning har qandayidan foydalanish ibtidoiy bo‘lsa ham, mijoz va server o‘rtasidagi o‘zaro ta’sir protokolini aniqlashni o‘z ichiga oladi, bunda mijoz va server yagona qoidalarga muvofiq ma’lumotlarni shakllantiradi, yuboradi va oqimdan chiqaradi.

Ruxsat etilgan bufer versiyasi juda aniq, shuning uchun qolgan ikkita strategiyani ko‘rib chiqing.

Javobni tugatish markeridan foydalanish

Quyidagi server kodini aniqlang:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
var tcpListener = new TcpListener(IPAddress.Any, 8888);
try
{
    tcpListener.Start();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");
    while (true)
    {
        using var tcpClient = await
tcpListener.AcceptTcpClientAsync();
        var stream = tcpClient.GetStream();
        var buffer = new List<byte>();
        int bytesRead = '\n';
        while((bytesRead = stream.ReadByte())!='\n')
        {
            buffer.Add((byte)bytesRead);
        }
        var message = Encoding.UTF8.GetString(buffer.ToArray());
        Console.WriteLine($"Получено сообщение: {message}");
    }
}
finally
{
    tcpListener.Stop();
}
```

Bu yerda stream usuli yordamida.ReadByte() har bir baytni oqimdan o‘qiydi. Usulning natijasi baytni int sifatida ifodalashdir. Aytaylik, bu erda \n belgisi yoki 10 qiymatini ifodalovchi chiziq tarjimasini xabarning oxiri belgisi sifatida ishlaydi. Va agar chiziq tarjimasini duch kelsa, biz ma’lumotlarni o‘qishni tugatamiz va olingan ma’lumotlarni qatorga aylantiramiz.

Bunday holda, mijoz xabarning oxiriga \n belgisini qo‘shishi kerak:

```
using System.Net.Sockets;
using System.Text;
using TcpClient tcpClient = new TcpClient();
await tcpClient.ConnectAsync("127.0.0.1", 8888);
var message = "Hello METANIT.COM\n";
```

```
var stream = tcpClient.GetStream();
byte[] requestData = Encoding.UTF8.GetBytes(message);
await stream.WriteAsync(requestData);
Console.WriteLine("Сообщение отправлено");
```

Garchi bu ma’lum darajada biroz ibtidoiy soddalashtirish bo‘lsa-da, chunki bu holda biz bir qatorli matnni yuborish bilan cheklanamiz. Ammo, aslida, cheklangan markerni aniqlash mantig‘i yanada murakkablashishi mumkin, ayniqsa marker bitta bayt/belgini emas, balki bir nechta, ammo umumiy prinip bir xil bo‘lsa.

Xabar hajmini sozlash

Quyidagi serverni aniqlang:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
var tcpListener = new TcpListener(IPAddress.Any, 8888);
try
{
    tcpListener.Start();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");
    while (true)
    {
        using var tcpClient = await
tcpListener.AcceptTcpClientAsync();
        var stream = tcpClient.GetStream();
        byte[] sizeBuffer = new byte[4];
        await stream.ReadExactlyAsync(sizeBuffer, 0,
sizeBuffer.Length);
        int size = BitConverter.ToInt32(sizeBuffer, 0);
        byte[] data = new byte[size];
        int bytes = await stream.ReadAsync(data);
        var message = Encoding.UTF8.GetString(data, 0, bytes);
        Console.WriteLine($"Размер сообщения: {size} байтов");
        Console.WriteLine($"Сообщение: {message}");
    }
}
finally
{
    tcpListener.Stop();
}
```

Bunday holda, biz o‘lcham int turining qiymatini, ya’ni 4 bayt qiymatini ifodalaydi deb taxmin qilamiz. Shunga ko‘ra, o‘lchamni o‘qish uchun biz 4 baytdan bufer yaratamiz va uni stream usuli yordamida o‘qiyamiz. Oqimdan ma’lum miqdordagi baytlarni o‘qiydigan ReadExactlyAsync (bu holda 4 bayt):

```
await stream.ReadExactlyAsync(sizeBuffer, 0, sizeBuffer.Length);
```

O‘lchamni o‘qib bo‘lgach, biz uni bitconverter statik usuli yordamida int raqamiga aylantirishimiz mumkin. ToInt32 (), tegishli uzunlikdagi buferni aniqlang va undagi ma’lumotlarni hisoblang.

Mijoz tomonida biz quyidagi kodni aniqlaymiz:

```
using System.Net.Sockets;
using System.Text;

using TcpClient tcpClient = new TcpClient();
await tcpClient.ConnectAsync("127.0.0.1", 8888);

var message = "Hello METANIT.COM";
var stream = tcpClient.GetStream();
byte[] data = Encoding.UTF8.GetBytes(message);
byte[] size = BitConverter.GetBytes(data.Length);
await stream.WriteAsync(size);
await stream.WriteAsync(data);
Console.WriteLine("Сообщение отправлено");
```

Bu erda teskari jarayon sodir bo‘ladi. Birinchidan, biz bitconverter usuli yordamida baytlar qatoriga xabar hajmini olamiz. `GetBytes()`:

```
byte[] size = BitConverter.GetBytes(data.Length);
```

Biz o‘lchamni to‘rt bayt shaklida yuboramiz va keyin ma’lumotlarni o‘zimiz yuboramiz:

```
await stream.WriteAsync(size);
await stream.WriteAsync(data);
```

Natijada, mijoz ma’lumotlarni yuborgandan so‘ng, server konsolida ma’lumotlar hajmi va ma’lumotlarning o‘zi ko‘rsatiladi:

Bir nechta yuborish va qabul qilish

Yuqorida mijoz yuborgan va server faqat bitta xabar olgan. Ammo server alohida qabul qilishi kerak bo‘lgan ko‘plab xabarlar yuborilsa nima bo‘ladi. Bunday holda, biz yana biron bir protokolni aniqlashimiz kerak, unga ko‘ra server bitta xabar qachon tugashini va mijoz umuman shovqinni qachon tugatishini aniqlay oladi. Kichik bir misolni ko‘rib chiqing. Server quyidagi kodga ega bo‘lsin:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
var tcpListener = new TcpListener(IPAddress.Any, 8888);
try
{
    tcpListener.Start();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");

    while (true)
    {
        using var tcpClient = await
tcpListener.AcceptTcpClientAsync();
        var stream = tcpClient.GetStream();
        var buffer = new List<byte> ();
        int bytesRead = 10;
        while (true)
```

```

        {
            while ((bytesRead = stream.ReadByte()) != '\n')
            {
                buffer.Add((byte)bytesRead);
            }
            var message = Encoding.UTF8.GetString(buffer.ToArray());
            if (message == "END") break;
            Console.WriteLine($"Получено сообщение: {message}");
            buffer.Clear();
        }
    }
}
finally
{
    tcpListener.Stop();
}

```

Shunday qilib, bu erda bizning mijoz-server o‘zaro aloqasi protokoli ikkita qoidadan iborat. Birinchidan, har bir alohida xabar \n satrining tarjimai bilan tugashi kerak. ikkinchidan, mijoz o‘zaro aloqani tugatishni va serverdan uzishni xohlasa, u "END"buyrug‘ini yuboradi. Shuning uchun, cheksiz tsiklda biz mijozning barcha xabarlarini qayta ishlaymiz va ichki tsiklda baytni o‘qiymiz:

Ushbu server bilan ishlash uchun quyidagi mijozni aniqlang:

```

using System.Net.Sockets;
using System.Text;
using TcpClient tcpClient = new TcpClient();
await tcpClient.ConnectAsync("127.0.0.1", 8888);
var messages = new string[] { "Hello METANIT.COM\n", "Hello
TcpListener\n", "Bye METANIT.COM\n", "END\n" };
var stream = tcpClient.GetStream();
foreach (var message in messages)
{
    byte[] data = Encoding.UTF8.GetBytes(message);
    await stream.WriteAsync(data);
}
Console.WriteLine("Все сообщения отправлены");

```

Bu erda yuborilgan har bir xabar \n terminal belgisi bilan tugaydi, bundan tashqari, oxirgi xabar "END"o‘zaro ta’sirini tugatish buyrug‘ini anglatadi. Natijada, ushbu mijoz ulanganda, server konsolda "end" buyrug‘idan tashqari barcha yuborilgan xabarlarni ko‘rsatadi, bu mijoz va serverning o‘zaro ta’sirini yakunlaydi:

Nazorat savollari:

1. Soketlarni ma’lumotlarda o‘ib olishdagi metodlari qaysilar.
- 2.Tarmoqdan ma’lumotlarni o‘qib olishning usullari haqida ma’lumot bering
3. TcpListener va TcpClient farqli tomonlari nimada ?

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**13-Mashg'ulot:** HTTP protokoli.**Reja:**

1. Ma'lumotlarni yuborish va qabul qilish. Ikki tomonlama aloqa.
2. Ko'p tarmoqli TCP mijoz-server haqida tushuncha.
3. Ko'p tarmoqli TCP mijoz-server ilovasi.

1. Ma'lumotlarni yuborish va qabul qilish. Ikki tomonlama aloqa.

O'tgan mavzu mijoz va server o'rtasidagi bir tomonlama aloqani ko'rib chiqdi: server ma'lumotlarni yuborganda va mijoz qabul qilganda yoki aksincha, mijoz yuboradi va server qabul qiladi. Mijoz ham, server ham bir vaqtning o'zida ma'lumotlarni yuboradigan va qabul qiladigan yanada murakkab misolni ko'rib chiqing.

TcpListener va TcpClient bilan misol

Serverni aniqlash

Server kodi quyidagicha bo'lsin:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
var tcpListener = new TcpListener(IPAddress.Any, 8888);
var words = new Dictionary<string, string>()
{
    {"red", "красный" },
    {"blue", "синий" },
    {"green", "зеленый" }
};
try
{
    tcpListener.Start();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");
    while (true)
    {
        using var tcpClient = await
tcpListener.AcceptTcpClientAsync();
        var stream = tcpClient.GetStream();
        var response = new List<byte> ();
        int bytesRead = 10;
        while(true)
        {
            while ((bytesRead = stream.ReadByte()) != '\n')
            {
                response.Add((byte)bytesRead);
            }
            var word = Encoding.UTF8.GetString(response.ToArray());
            if (word == "END") break;
            Console.WriteLine($"Запрошен перевод слова {word}");
```

```

        if (!words.TryGetValue(word, out var translation))
translation = "не найдено в словаре";
        translation += '\n';
        await
stream.WriteAsync(Encoding.UTF8.GetBytes(translation));
        response.Clear();
    }
}
finally
{
    tcpListener.Stop();
}

```

Bunday holda, biz server qandaydir lug‘at vazifasini bajaradi - mijozdan so‘zni qabul qilish va uning tarjimasini qaytarib yuborish. Ma’lumotlarni saqlash uchun words test lug‘ati aniqlandi:

```

var words = new Dictionary<string, string>()
{
    {"red", "красный" },
    {"blue", "синий" },
    {"green", "зеленый" }
};

```

So‘rovlarni qabul qilish va qayta ishlashda server ikkita qoidadan iborat oddiy protokolni qo‘llaydi: mijoz va serverga har bir alohida xabar \n belgisi bilan tugashi kerak va ulanish tugash belgisi "END"qatorini ifodalashi kerak. Ushbu qoidalarga muvofiq, server avval server so‘rovini o‘qiydi va tarjima qilinishi kerak bo‘lgan so‘zni oladi:

```

while ((bytesRead = stream.ReadByte()) != '\n')
{
    response.Add((byte)bytesRead);
}
var word = Encoding.UTF8.GetString(response.ToArray());

```

Agar "END" qatori yuborilgan bo‘lsa, biz ko‘chadan chiqamiz va shu bilan joriy mijoz bilan ishlashni to‘xtatamiz:

```

if (word == "END") break;

```

Aks holda, biz lug‘atda so‘zning tarjimasini topamiz (agar mavjud bo‘lsa) va mijozga yuboramiz:

```

if (!words.TryGetValue(word, out var translation)) translation = "не
найден в словаре";
translation += '\n';
await stream.WriteAsync(Encoding.UTF8.GetBytes(translation));

```

Bunday holda, yana, shartli protokolga muvofiq, yuborishda biz xabarga \n belgisini qo‘shamiz, shunda mijoz bu so‘zning oxiri ekanligini bilib oladi.

Mijozning ta’rifi

Endi yuqoridagi server uchun test mijozini yarating:

```
using System.Net.Sockets;
using System.Text;
using TcpClient tcpClient = new TcpClient();
await tcpClient.ConnectAsync("127.0.0.1", 8888);
var words = new string[] { "red", "yellow", "blue" };
var stream = tcpClient.GetStream();
var response = new List<byte>();
int bytesRead = 10;
foreach (var word in words)
{
    byte[] data = Encoding.UTF8.GetBytes(word + '\n');
    await stream.WriteAsync(data);
    while ((bytesRead = stream.ReadByte()) != '\n')
    {
        response.Add((byte)bytesRead);
    }
    var translation = Encoding.UTF8.GetString(response.ToArray());
    Console.WriteLine($"Слово {word}: {translation}");
    response.Clear();
}
await stream.WriteAsync(Encoding.UTF8.GetBytes("END\n"));
Console.WriteLine("Все сообщения отправлены");
```

Bu erda test uchun biz tarjima qilmoqchi bo‘lgan uchta so‘zning qatorini aniqlaymiz:

```
var words = new string[] { "red", "yellow", "blue" };
```

Biz ushbu qator bo‘ylab yuguramiz va har bir so‘zni serverga yuboramiz:

```
foreach (var word in words)
{
    byte[] data = Encoding.UTF8.GetBytes(word + '\n');
    await stream.WriteAsync(data);
}
```

Shunga qaramay, biz qabul qilgan protokolga muvofiq, har bir so‘z \n belgisi bilan yakunlanadi.

Keyin biz serverning javobini o‘qiymiz va so‘zning tarjimasini olamiz:

```
while ((bytesRead = stream.ReadByte()) != '\n')
{
    response.Add((byte)bytesRead);
}
var translation = Encoding.UTF8.GetString(response.ToArray());
```

Tugatish uchun biz serverni ulanishni tugatish markerini yuboramiz:

```
await stream.WriteAsync(Encoding.UTF8.GetBytes("END\n"));
```

Server va mijozni ishga tushiring. So‘rovlarni qabul qilishda server konsolida so‘ralgan so‘zlarni tarjima qilish uchun ko‘rsatiladi:

Soketlardagi misol

Xuddi shu misol faqat Socket sinfidan foydalangan holda toza rozetkalarda.

Server kodi:

```
using System.Net;
```

```

using System.Net.Sockets;
using System.Text;

using Socket tcpListener = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
var words = new Dictionary<string, string>()
{
    { "red", "красный" },
    { "blue", "синий" },
    { "green", "зеленый" },
};
try
{
    tcpListener.Bind(new IPEndPoint(IPAddress.Any, 8888));
    tcpListener.Listen();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");

    while (true)
    {
        using var tcpClient = await tcpListener.AcceptAsync();
        var response = new List<byte>();
        var bytesRead = new byte[1];
        while (true)
        {
            while (true)
            {
                var count = tcpClient.Receive(bytesRead);
                if (count == 0 || bytesRead[0] == '\n') break;
                response.Add(bytesRead[0]);
            }
            var word = Encoding.UTF8.GetString(response.ToArray());
            if (word == "END") break;
            Console.WriteLine($"Запрошен перевод слова {word}");
            if (!words.TryGetValue(word, out var translation))
translation = "не найдено в словаре";
            translation += '\n';
            await
tcpClient.SendAsync(Encoding.UTF8.GetBytes(translation));
            response.Clear();
        }
    }
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}

```

Mijoz kodi:

```

using System.Net.Sockets;
using System.Text;
var words = new string[] { "red", "yellow", "blue" };

```

```
using var tcpClient = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    await tcpClient.ConnectAsync("127.0.0.1", 8888);
    var response = new List<byte>();
    foreach (var word in words)
    {
        byte[] data = Encoding.UTF8.GetBytes(word + '\n');
        await tcpClient.SendAsync(data);
        var bytesRead = new byte[1];
        while (true)
        {
            var count = tcpClient.Receive(bytesRead);
            if (count == 0 || bytesRead[0] == '\n') break;
            response.Add(bytesRead[0]);
        }
        var translation =
Encoding.UTF8.GetString(response.ToArray());
        Console.WriteLine($"Слово {word}: {translation}");
        response.Clear();
    }
    await tcpClient.SendAsync(Encoding.UTF8.GetBytes("END\n"));
    Console.WriteLine("Все сообщения отправлены");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

2. Ko'p tarmoqli TCP mijoz-server haqida tushuncha.

Ko'p tarmoqli mijoz-server dasturini qanday yaratishni ko'rib chiqing. Aslida, bu bitta ipdan faqat mijozning so'rovini qayta ishlash alohida oqimga kiritilishi bilan farq qiladi. Biz oxirgi maqoladagi loyihani asos qilib olamiz.

TcpListener va TcpClient bilan misol

Server tomonida biz quyidagi kodni aniqlaymiz:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
var tcpListener = new TcpListener(IPAddress.Any, 8888);
try
{
    tcpListener.Start(); // запускаем сервер
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");
    while (true)
    {
        var tcpClient = await tcpListener.AcceptTcpClientAsync();
        Task.Run(async ()=>await ProcessClientAsync(tcpClient));
        ProcessClientAsync(tcpClient).Start();
    }
}
```

```

    }
}
finally
{
    tcpListener.Stop();
}
async Task ProcessClientAsync(TcpClient tcpClient)
{
    var words = new Dictionary<string, string>()
    {
        {"red", "красный" },
        {"blue", "синий" },
        {"green", "зеленый" },
    };
    var stream = tcpClient.GetStream();
    // буфер для входящих данных
    var response = new List<byte>();
    int bytesRead = 10;
    while (true)
    {
        while ((bytesRead = stream.ReadByte()) != '\n')
        {
            response.Add((byte)bytesRead);
        }
        var word = Encoding.UTF8.GetString(response.ToArray());
        if (word == "END") break;
        Console.WriteLine($"Клиент {tcpClient.Client.RemoteEndPoint}
запросил перевод слова {word}");
        if (!words.TryGetValue(word, out var translation)) translation
= "не найдено в словаре";
        translation += '\n';
        await stream.WriteAsync(Encoding.UTF8.GetBytes(translation));
        response.Clear();
    }
    tcpClient.Close();
}

```

Bu erda mijozni qayta ishlash alohida Process Client Async usulida amalga oshiriladi. Qayta ishlash uchun biz yangi vazifani boshlaymiz, u erda biz ushbu usulni chaqiramiz va natijada olingan TcpClient ob'ektini unga o'tkazamiz:

```
Task.Run(async ()=>await ProcessClientAsync(tcpClient));
```

Shu bilan bir qatorda, yangi Thread ipini ishga tushirish mumkin:

```
new Thread(async ()=>await ProcessClientAsync(tcpClient)).Start();
```

Bunday holda, biz server qandaydir lug‘at vazifasini bajaradi - mijozdan so‘zni qabul qilish va uning tarjimasini qaytarib yuborish. Ma’lumotlarni saqlash uchun Process Client Async usuli words test lug‘atini aniqladi:

```

var words = new Dictionary<string, string>()
{
    {"red", "красный" },
    {"blue", "синий" },
}

```



```
{ "green", "зеленый" },
};
```

So'rovlarni qabul qilish va qayta ishlashda server ikkita qoidadan iborat oddiy protokolni qo'llaydi: mijoz va serverga har bir alohida xabar \n belgisi bilan tugashi kerak va ulanish tugash belgisi "END" qatorini ifodalashi kerak. Ushbu qoidalarga muvofiq, server avval server so'rovini o'qiydi va tarjima qilinishi kerak bo'lgan so'zni oladi:

```
while ((bytesRead = stream.ReadByte()) != '\n')
{
    response.Add((byte)bytesRead);
}
var word = Encoding.UTF8.GetString(response.ToArray());
```

Agar "END" qatori yuborilgan bo'lsa, biz ko'chadan chiqamiz va shu bilan joriy mijoz bilan ishlashni to'xtatamiz:

```
if (word == "END") break;
```

Aks holda, biz lug'atda so'zning tarjimasini topamiz (agar mavjud bo'lsa) va mijozga yuboramiz:

```
if (!words.TryGetValue(word, out var translation)) translation = "не  
найдено в словаре";
translation += '\n';
await stream.WriteAsync(Encoding.UTF8.GetBytes(translation));
```

Bunday holda, yana, shartli protokolga muvofiq, yuborishda biz xabarga \n belgisini qo'shamiz, shunda mijoz bu so'zning oxiri ekanligini bilib oladi.

Oxirida biz ulanishni yopamiz:

```
tcpClient.Close();
```

Mijoz yaratish

Yuqoridagi server uchun biz quyidagi TCP sinov mijozini yaratamiz:

```
using System.Net.Sockets;
using System.Text;
var words = new string[] { "red", "yellow", "blue", "green" };
using TcpClient tcpClient = new TcpClient();
await tcpClient.ConnectAsync("127.0.0.1", 8888);
var stream = tcpClient.GetStream();
var response = new List<byte>();
int bytesRead = 10;
foreach (var word in words)
{
    byte[] data = Encoding.UTF8.GetBytes(word + '\n');
    await stream.WriteAsync(data);
    while ((bytesRead = stream.ReadByte()) != '\n')
    {
        response.Add((byte)bytesRead);
    }
    var translation = Encoding.UTF8.GetString(response.ToArray());
    Console.WriteLine($"Слово {word}: {translation}");
    response.Clear();
    await Task.Delay(2000);
}
```

```
}
await stream.WriteAsync(Encoding.UTF8.GetBytes("END\n"));
```

Bu erda test uchun biz tarjima qilmoqchi bo‘lgan to‘rtta so‘zning qatorini aniqlaymiz:

```
var words = new string[] { "red", "yellow", "blue", "green" };
```

Biz ushbu qator bo‘ylab yuguramiz va har bir so‘zni serverga yuboramiz:

```
foreach (var word in words)
{
    // при отправке добавляем маркер завершения сообщения - \n
    byte[] data = Encoding.UTF8.GetBytes(word + '\n');
    await stream.WriteAsync(data);
}
```

Shunga qaramay, biz qabul qilgan protokolga muvofiq, har bir so‘z \n belgisi bilan yakunlanadi.

Keyin biz serverning javobini o‘qiymiz va so‘zning tarjimasini olamiz:

```
while ((bytesRead = stream.ReadByte()) != '\n')
{
    response.Add((byte)bytesRead);
}
var translation = Encoding.UTF8.GetString(response.ToArray());
```

Tugatish uchun biz serverni ulanishni tugatish markerini yuboramiz:

```
await stream.WriteAsync(Encoding.UTF8.GetBytes("END\n"));
```

Biz serverni va bir nechta mijozlarni ishga tushiramiz, shunda ular bir vaqtning o‘zida serverga kirishadi. Server konsoli bizga mijozlarning manzillarini va ular so‘ragan so‘zlarni ko‘rsatadi:

3. Ko‘p tarmoqli TCP mijoz-server ilovasi.

Keling, toza rozetkalarda shunga o‘xshash misol yarataylik. Quyidagi server kodini aniqlang:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
using Socket tcpListener = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    tcpListener.Bind(new IPEndPoint(IPAddress.Any, 8888));
    tcpListener.Listen();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");
    while (true)
    {
        var tcpClient = await tcpListener.AcceptAsync();
        Task.Run(async () => await ProcessClientAsync(tcpClient));
        ProcessClientAsync(tcpClient).Start();
    }
}
catch (Exception ex)
{
}
```

```

        Console.WriteLine(ex.Message);
    }

    async Task ProcessClientAsync(Socket tcpClient)
    {
        var words = new Dictionary<string, string>()
        {
            { "red", "красный" },
            { "blue", "синий" },
            { "green", "зеленый" },
        };
        var response = new List<byte>();
        var bytesRead = new byte[1];
        while (true)
        {
            while (true)
            {
                var count = await tcpClient.ReceiveAsync(bytesRead);
                if (count == 0 || bytesRead[0] == '\n') break;
                response.Add(bytesRead[0]);
            }
            var word = Encoding.UTF8.GetString(response.ToArray());
            if (word == "END") break;

            Console.WriteLine($"Запрошен перевод слова {word}");
            if (!words.TryGetValue(word, out var translation)) translation
= "не найдено в словаре";
            translation += '\n';
            await
tcpClient.SendAsync(Encoding.UTF8.GetBytes(translation));
            response.Clear();
        }
        tcpClient.Shutdown(SocketShutdown.Both);
        tcpClient.Close();
    }
}

```

Quyidagi mijoz kodini aniqlang:

```

using System.Net.Sockets;
using System.Text;
// слова для отправки для получения перевода
var words = new string[] { "red", "yellow", "blue" };
using var tcpClient = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
try
{
    await tcpClient.ConnectAsync("127.0.0.1", 8888);
    var response = new List<byte>();
    foreach (var word in words)
    {
        byte[] data = Encoding.UTF8.GetBytes(word + '\n');
    }
}

```

```

        await tcpClient.SendAsync(data);

        var bytesRead = new byte[1];
        while (true)
        {
            var count = tcpClient.Receive(bytesRead);
            if (count == 0 || bytesRead[0] == '\n') break;
            response.Add(bytesRead[0]);
        }
        var translation =
Encoding.UTF8.GetString(response.ToArray());
        Console.WriteLine($"Слово {word}: {translation}");
        response.Clear();
        await Task.Delay(2000);
    }
    await tcpClient.SendAsync(Encoding.UTF8.GetBytes("END\n"));
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}

```

Nazorat savollari:

1. TCPListener orqali qabul qilingan bitlar o'qib olinishi uchun qanday bosqichlar amalga oshishi kerak.
2. Grafik ma'lumotlarni uzatish usullari haqida ma'lumot bering
3. Uzatilayotgan matnli ma'lumotlarni kodirovka turlari haqida ma'lumot bering

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**14-Mashg‘ulot:** NetworkStream klassi.**Reja:**

1. NetworkStream va matn oqimlari.
2. NetworkStream va ikkilik oqimlar.
3. Konsol TCP suhbat.

1. NetworkStream va matn oqimlari.

NetworkStream bilan ishlashni osonlashtirish uchun biz uni boshqa iplarga o‘rashimiz mumkin.

Masalan, ko‘pincha NetworkStream faqat matnli ma‘lumotlarni yuboradi va oladi - oddiy satrlar, bu holda StreamWriter va StreamReader matn oqimlari sinflaridan foydalanish mantiqan to‘g‘ri keladi. Keling, so‘rovni yuborish misolini ko‘rib chiqaylik www.google.com va undan javob olish:

```
using System.Net.Sockets;
using TcpClient tcpClient = new TcpClient();
var server = "www.google.com";
await tcpClient.ConnectAsync(server, 80);
var stream = tcpClient.GetStream();
var message = $"GET / HTTP/1.1\r\nHost: {server}\r\nConnection:
Close\r\n\r\n";
using var writer = new StreamWriter(stream);
await writer.WriteAsync(message);
await writer.FlushAsync();
using var reader = new StreamReader(stream);
var response = await reader.ReadLineAsync();
Console.WriteLine(response);
```

Matnli ma‘lumotlarni oqimga yuborish uchun StreamWriter-dagi networkstream ob‘ektini o‘rab oling. StreamWriter sinfida matnni oqimga yozish uchun bir qator usullar mavjud, xususan: Write()/WriteAsync() (tasodifiy matnni yozish uchun) va WriteLine()/WriteLineAsync() (matnning bir qatorini yozish uchun). bu erda biz butun matnni oqimga yozamiz.

```
await writer.WriteAsync(message);
```

Matn ma‘lumotlarini oqimdan hisoblash uchun biz Networkstream-ni StreamReader ob‘ektiga o‘rab olamiz. Ma‘lumotlarni o‘qish uchun bir qator usullar mavjud: ReadLine()/ReadLineAsync() (bitta satrni o‘qish uchun), Read () / ReadAsync () (belgilar buferidagi ma‘lumotlar uchun), ReadBlock () / ReadBlockAsync () (ma‘lumotlarning bir qismini belgilar qatoriga o‘qish uchun) va ReadToEnd () / ReadToEndAsync() (barcha ma‘lumotlarni oqimdan satrga o‘qish uchun). Yuqoridagi misolda biz bitta qatorni o‘qiymiz

```
var response = await reader.ReadLineAsync();
```

Natijada, StreamReader barcha belgilarni oqimdan satrning birinchi tarjimasiga o‘qiyotganda va biz asosan http javobining birinchi qatorini olamiz www.google.com:

```
HTTP/1.1 200 OK
```

Nazariy va amaliy jihatdan, bu holda biz ReadToEnd()/ReadToEndAsync() usuli yordamida butun javobni aniq hisoblashimiz mumkin:

```
using System.Net.Sockets;
using TcpClient tcpClient = new TcpClient();
var server = "www.google.com";
await tcpClient.ConnectAsync(server, 80);
var stream = tcpClient.GetStream();
var message = $"GET / HTTP/1.1\r\nHost: {server}\r\nConnection:
Close\r\n\r\n";
using var writer = new StreamWriter(stream);
await writer.WriteAsync(message);
await writer.FlushAsync();
using var reader = new StreamReader(stream);
var response = await reader.ReadToEndAsync();
Console.WriteLine(response);
```

Biroq, bu usullardan ehtiyotkorlik bilan foydalanish kerak. Biz server bilan aloqa qiladigan protokolni aniq tushunishimiz kerak. Masalan, yuqoridagi misolda readtoendasync () usuli javobni oxirigacha o‘qiydi. So‘rovga ulanish: yopish sarlavhasi yuborilganligi sababli, HTTP1.1 standartiga muvofiq ulanishni yopishni buyuradi. Shunday qilib, server javob yuboradi va ulanishni yopadi, kiruvchi oqim tugaydi va panoshd oxirigacha o‘qiydi. Biroq, bu shaxsiy holat, chunki umuman olganda, TCP-bu vaqt davomida ulanishni davom ettirishga va u orqali xabar almashishga imkon beradigan protokol. Shu munosabat bilan, oqimning oxiri yo‘qligi ayon bo‘lishi mumkin. Bunday vaziyatda ReadToEndAsync usulini qo‘llash mantiqiy emas.

```
using System.Net;
using System.Net.Sockets;
var tcpListener = new TcpListener(IPAddress.Any, 8888);
var words = new Dictionary<string, string>()
{
    { "red", "красный" },
    { "blue", "синий" },
    { "green", "зеленый" }
};
try
{
    tcpListener.Start();
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");
    while (true)
    {
        using var tcpClient = await
tcpListener.AcceptTcpClientAsync();
```

```
// получаем объект NetworkStream для взаимодействия с клиентом
var stream = tcpClient.GetStream();

// создаем StreamReader для чтения данных
using var streamReader = new StreamReader(stream);
// создаем StreamWriter для отправки данных
using var streamWriter = new StreamWriter(stream);

while (true)
{
    // считываем запрошенное слово
    var word = await streamReader.ReadLineAsync();

    // если получен маркер окончания подключения - END,
    завершаем соединение с клиентом
    if (word == "END") break;

    Console.WriteLine($"Запрошен перевод слова {word}");
    // находим слово в словаре и отправляем обратно клиенту
    if (word is null || !words.TryGetValue(word, out var
translation))
        translation = "не найдено в словаре";
    // отправляем перевод слова из словаря
    await streamWriter.WriteLineAsync(translation);
    await streamWriter.FlushAsync();
}
}
finally
{
    tcpListener.Stop();
}
</string,>
```

Bunday holda, biz server qandaydir lug‘at vazifasini bajaradi - mijozdan so‘zni qabul qilish va uning tarjimasini qaytarib yuborish. Ma’lumotlarni saqlash uchun words test lug‘ati aniqlandi:

```
var words = new Dictionary<string, string>()
{
    {"red", "красный" },
    {"blue", "синий" },
    {"green", "зеленый" }
};
```

So‘rovlarni qabul qilish va qayta ishlashda server ikkita qoidadan iborat oddiy prokotoldan foydalanadi: mijoz va serverga har bir alohida xabar satrni (\n satrni tarjima qilish bilan tugaydigan belgilar to‘plami) va ulanish tugash belgisi "END"qatorini aks ettirishi kerak. Ushbu qoidalarga muvofiq, server avval server so‘rovini o‘qiydi va tarjima qilinishi kerak bo‘lgan so‘zni oladi:

```
var word = await streamReader.ReadLineAsync();
```

Agar "END" qatori yuborilgan bo'lsa, biz ko'chadan chiqamiz va shu bilan joriy mijoz bilan ishlashni to'xtatamiz:

```
if (word == "END") break;
```

Aks holda, biz lug'atda so'zning tarjimasini topamiz (agar mavjud bo'lsa) va mijozga yuboramiz:

```
if (word is null || !words.TryGetValue(word, out var translation))
    translation = "не найдено в словаре";
await streamWriter.WriteLineAsync(translation);
await streamWriter.FlushAsync();
```

Bunday holda, yana, shartli protokolga muvofiq, biz alohida qatorni yuboramiz, buning natijasida mijoz bu so'zning oxiri ekanligini bilib oladi.

Ushbu server uchun biz quyidagi sinov mijozini aniqlaymiz:

```
using System.Net.Sockets;

// слова для отправки для получения перевода
var words = new string[] { "red", "yellow", "blue" };

using TcpClient tcpClient = new TcpClient();
await tcpClient.ConnectAsync("127.0.0.1", 8888);
// получаем NetworkStream для взаимодействия с сервером
var stream = tcpClient.GetStream();
// создаем StreamReader для чтения данных
using var streamReader = new StreamReader(stream);
// создаем StreamWriter для отправки данных
using var streamWriter = new StreamWriter(stream);

foreach (var word in words)
{
    // отправляем слово на сервер для перевода
    await streamWriter.WriteLineAsync(word);
    await streamWriter.FlushAsync();

    // получаем перевод от сервера
    var translation = await streamReader.ReadLineAsync();
    Console.WriteLine($"{word} - {translation}");
}
// посылаем маркер окончания подключения
await streamWriter.WriteLineAsync("END");
await streamWriter.FlushAsync();
```

Bu erda test uchun biz tarjima qilmoqchi bo'lgan uchta so'zning qatorini aniqlaymiz:

```
var words = new string[] { "red", "yellow", "blue" };
```

Biz ushbu qator bo'ylab yuguramiz va har bir so'zni serverga yuboramiz:

```
foreach (var word in words)
{
    await streamWriter.WriteLineAsync(word);
    await streamWriter.FlushAsync();
}
```


Shunga qaramay, biz qabul qilgan protokolga muvofiq, har bir so‘z alohida satr sifatida yuboriladi, Shuning uchun WriteLineAsync usuli qo‘llaniladi. Ya‘ni, server satrni kutadi va shunga mos ravishda mijoz unga satr yuboradi..

Keyin biz serverning javobini o‘qiymiz va so‘zning tarjimasini olamiz:

```
var translation = await streamReader.ReadLineAsync();
Console.WriteLine($"{word} - {translation}");
}
```

Tugatish uchun biz serverni ulanishni tugatish markerini yuboramiz:

```
await streamWriter.WriteLineAsync("END");
```

Server va mijozni ishga tushiring. So‘rovlarni qabul qilishda server konsolida so‘ralgan so‘zlarni tarjima qilish uchun ko‘rsatiladi:

2. *NetworkStream* va *ikkilik oqimlar*.

Ibtidoiy turlarni ifodalovchi ikkilik ma‘lumotlarni o‘qish va yozish uchun.net binaryreader va BinaryWriter sinflaridan foydalanadi. Ular yordamida faylga ma‘lumotlarni yozish yoki fayldan o‘qish mumkin bo‘lganidek, ikkilik ma‘lumotlarni ham NetworkStream tarmoq oqimiga o‘qish va yozish mumkin.

Ikkilik ma‘lumotlarni oqimga yuborish uchun networkstream ob’ekti binarywriter sinf ob’ektiga o‘ralgan bo‘lib, ma‘lumotlarni yuborish uchun Write () usulini taqdim etadi.

Ikkilik ma‘lumotlarni oqimdan o‘qish uchun networkstream ob’ekti binaryreader sinf ob’ektiga o‘ralgan bo‘lib, u ma‘lumotlarni o‘qish uchun bir qator usullarni taqdim etadi:

```
ReadBoolean()
ReadByte()
ReadBytes()
ReadChar()
ReadChars()
ReadDecimal()
ReadDouble()
ReadHalf()
ReadInt16()
ReadInt32()
ReadInt64()
ReadSByte()
ReadSingle()
ReadString()
ReadUInt16()
ReadUInt32()
ReadUInt64()
```

Aytaylik, mijoz mahsulot nomi, ishlab chiqaruvchisi, miqdori va narxi kabi mahsulot ma'lumotlarini yuboradi. Va server ushbu ma'lumotlarni oladi, ulardan mahsulot yaratadi va mijozga javoban yaratilgan mahsulot identifikatorini yuboradi.

Birinchidan, server kodini aniqlang:

```
using System.Net;
using System.Net.Sockets;

var tcpListener = new TcpListener(IPAddress.Any, 8888);
try
{
    tcpListener.Start();    // запускаем сервер
    Console.WriteLine("Сервер запущен. Ожидание подключений... ");

    while (true)
    {
        // получаем подключение в виде TcpClient
        using var tcpClient = await
tcpListener.AcceptTcpClientAsync();
        // получаем объект NetworkStream для взаимодействия с клиентом
        var stream = tcpClient.GetStream();

        // создаем BinaryReader для чтения данных
        using var binaryReader = new BinaryReader(stream);
        // создаем BinaryWriter для отправки данных
        using var binaryWriter = new BinaryWriter(stream);

        var name = binaryReader.ReadString();
        var company = binaryReader.ReadString();
        var count = binaryReader.ReadInt32();
        var price = binaryReader.ReadDecimal();

        string id = Guid.NewGuid().ToString();
        var newProduct = new Product(id, name, company, count, price);
        Console.WriteLine($"Добавлен новый товар: {newProduct}");

        // отправляем клиенту сгенерированный id
        binaryWriter.Write(newProduct.Id);
        binaryWriter.Flush();
    }
}
finally
{
    tcpListener.Stop();
}

record Product(string Id, string Name, string Company, int Count,
decimal Price);
```

Ma'lumotlarni o'qish uchun biz Networkstream ob'ektini BinaryReader - ga, ma'lumotlarni yozish uchun esa binarywriter ob'ektiga o'rab olamiz:

```
using var binaryReader = new BinaryReader(stream);
using var binaryWriter = new BinaryWriter(stream);
```

Keyinchalik, binarywriter usullaridan foydalanib, biz ma'lum bir turdagi yuborilgan ma'lumotlarni o'qiymiz:

```
var name = binaryReader.ReadString();
var company = binaryReader.ReadString();
var count = binaryReader.ReadInt32();
var price = binaryReader.ReadDecimal();
```

Keyin Guid funksiyasidan foydalanib Id yaratamiz.NewGuid(), mahsulot ob'ektini yarating, uning ma'lumotlarini konsolga chiqaring va mahsulot identifikatorini javob sifatida yuboring:

```
string id = Guid.NewGuid().ToString();
var newProduct = new Product(id, name, company, count, price);
Console.WriteLine($"Добавлен новый товар: {newProduct}");

binaryWriter.Write(newProduct.Id);
binaryWriter.Flush();
```

Mijoz

Endi ushbu server uchun sinov mijozini aniqlaymiz:

```
using System.Net.Sockets;

// данные для отправки
string name = "iPhone 14";
string company = "Apple";
int count = 2;
decimal price = 145890.34m;

using TcpClient tcpClient = new TcpClient();
await tcpClient.ConnectAsync("127.0.0.1", 8888);
// получаем NetworkStream для взаимодействия с сервером
var stream = tcpClient.GetStream();
// создаем BinaryReader для чтения данных
using var binaryReader = new BinaryReader(stream);
// создаем BinaryWriter для отправки данных
using var binaryWriter = new BinaryWriter(stream);

// отправляем данные товара
binaryWriter.Write(name);
binaryWriter.Write(company);
binaryWriter.Write(count);
binaryWriter.Write(price);
binaryWriter.Flush();

// считываем сгенерированный на сервере id нового товара
var id = binaryReader.ReadString();
Console.WriteLine($"Id нового товара: {id}");
```

Bu erda biz avval jo'natish uchun mahsulot ma'lumotlarini aniqlaymiz:

```
string name = "iPhone 14";
string company = "Apple";
int count = 2;
decimal price = 145890.34m;
```

Server kodida bo‘lgani kabi, biz binaryreader va BinaryWriter ob’ektlari shaklida networkstream o‘ramlarini yaratamiz:

```
using var binaryReader = new BinaryReader(stream);
using var binaryWriter = new BinaryWriter(stream);
```

Keyin binarywriter ob’ektini yozish usuli yordamida ma’lumotlarni yuboramiz:

```
binaryWriter.Write(name);
binaryWriter.Write(company);
binaryWriter.Write(count);
binaryWriter.Write(price);
binaryWriter.Flush();
```

Yuborish ketma - ketligiga e’tibor bering-u serverdagi qabul qilish ketma-ketligiga mos kelishi kerak. Ya’ni, server avval qatorni, keyin qatorni, keyin int, keyin deimalni qabul qiladi. Shunga ko‘ra, mijozda biz avval string, keyin string, keyin int va decimal oxirida yuboramiz.

Ma’lumotlarni yuborgandan so‘ng, server tomonidan yuborilgan mahsulot identifikatorini o‘qing:

```
var id = binaryReader.ReadString();
Console.WriteLine($"Id нового товара: {id}");
```

3. Konsol TCP suhbat.

O‘tgan mavzularda tcp mijozlari xabarlarni buyurtma asosida qabul qildilar va yubordilar: so‘rovlarni yubordilar - javob oldilar va ushbu tsiklni takrorladilar. Biroq, xabarlarni qabul qilish va yuborish bir-biri bilan bog‘liq bo‘lmagan holatlar tez-tez uchraydi. Oddiy misol-bu suhbat bo‘lib, u erda odam ularga javob oladimi yoki yo‘qmi degan savolga ko‘plab xabarlarni yozishi mumkin. Va aksincha, so‘rov yubormasdan ko‘plab xabarlarni oling. Bunday o‘zaro ta’sirning misolini ko‘rib chiqish uchun biz kichik dastur yozamiz - konsol tcp suhbat.

Serverni aniqlash

Birinchidan, biz eng oddiy konsol server loyihasini yaratamiz. Unda quyidagi kodni aniqlaymiz:

```
using System.Net;
using System.Net.Sockets;

ServerObject server = new ServerObject(); // создаем сервер
await server.ListenAsync(); // запускаем сервер

class ServerObject
{
    TcpListener tcpListener = new TcpListener(IPAddress.Any, 8888); //
    сервер для прослушивания
```

```

        List<ClientObject> clients = new List<ClientObject>(); // все
подключения
        protected internal void RemoveConnection(string id)
        {
            // получаем по id закрытое подключение
            ClientObject? client = clients.FirstOrDefault(c => c.Id ==
id);
            // и удаляем его из списка подключений
            if (client != null) clients.Remove(client);
            client?.Close();
        }
        // прослушивание входящих подключений
        protected internal async Task ListenAsync()
        {
            try
            {
                tcpListener.Start();
                Console.WriteLine("Сервер запущен. Ожидание
подключений...");

                while (true)
                {
                    TcpClient tcpClient = await
tcpListener.AcceptTcpClientAsync();

                    ClientObject clientObject = new
ClientObject(tcpClient, this);
                    clients.Add(clientObject);
                    Task.Run(clientObject.ProcessAsync);
                }
            }
            catch (Exception ex)
            {
                Console.WriteLine(ex.Message);
            }
            finally
            {
                Disconnect();
            }
        }

        // трансляция сообщения подключенным клиентам
        protected internal async Task BroadcastMessageAsync(string
message, string id)
        {
            foreach (var client in clients)
            {
                if (client.Id != id) // если id клиента не равно id
отправителя
                {

```

```

        await client.Writer.WriteLineAsync(message);
//передача данных
        await client.Writer.FlushAsync();
    }
}
// отключение всех клиентов
protected internal void Disconnect()
{
    foreach (var client in clients)
    {
        client.Close(); //отключение клиента
    }
    tcpListener.Stop(); //остановка сервера
}
}
class ClientObject
{
    protected internal string Id { get; } = Guid.NewGuid().ToString();
    protected internal StreamWriter Writer { get; }
    protected internal StreamReader Reader { get; }

    TcpClient client;
    ServerObject server; // объект сервера

    public ClientObject(TcpClient tcpClient, ServerObject
serverObject)
    {
        client = tcpClient;
        server = serverObject;
        // получаем NetworkStream для взаимодействия с сервером
        var stream = client.GetStream();
        // создаем StreamReader для чтения данных
        Reader = new StreamReader(stream);
        // создаем StreamWriter для отправки данных
        Writer = new StreamWriter(stream);
    }

    public async Task ProcessAsync()
    {
        try
        {
            // получаем имя пользователя
            string? userName = await Reader.ReadLineAsync();
            string? message = $"{userName} вошел в чат";
            // посылаем сообщение о входе в чат всем подключенным
пользователям
            await server.BroadcastMessageAsync(message, Id);
            Console.WriteLine(message);
            // в бесконечном цикле получаем сообщения от клиента

```

```

        while (true)
        {
            try
            {
                message = await Reader.ReadLineAsync();
                if (message == null) continue;
                message = $"{userName}: {message}";
                Console.WriteLine(message);
                await server.BroadcastMessageAsync(message, Id);
            }
            catch
            {
                message = $"{userName} покинул чат";
                Console.WriteLine(message);
                await server.BroadcastMessageAsync(message, Id);
                break;
            }
        }
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
    }
    finally
    {
        // в случае выхода из цикла закрываем ресурсы
        server.RemoveConnection(Id);
    }
}
// закрытие подключения
protected internal void Close()
{
    Writer.Close();
    Reader.Close();
    client.Close();
}
}

```

Dasturning barcha kodlari aslida ikkita sinfga bo‘linadi: Server Object sinfi serverni, ClientObject sinfi esa alohida mijozning ulanishini ifodalaydi. Birinchidan, ClientObject kodini ko‘rib chiqing.

ClientObject ob‘ektini yaratish uchun sinf maydonlari va xususiyatlari o‘rnatiladigan konstruktor chaqiriladi:

```

protected internal string Id { get; } = Guid.NewGuid().ToString();
protected internal StreamWriter Writer { get; }
protected internal StreamReader Reader { get; }

TcpClient client;
ServerObject server; // объект сервера

```

```
public ClientObject(TcpClient tcpClient, ServerObject serverObject)
{
    client = tcpClient;
    server = serverObject;
    var stream = client.GetStream();
    Reader = new StreamReader(stream);
    Writer = new StreamWriter(stream);
}
```

Har bir ulanish Guid qiymatini kichik satrda saqlaydigan Id xususiyati bilan noyob tarzda aniqlanadi. Konstruktor orqali biz ulangan mijoz va ota-ona ServerObject bilan o‘zaro aloqada bo‘lish uchun TcpClient ob’ektini olamiz. Xabarlarini yuborish va qabul qilish uchun oddiylik uchun Writer va Reader xususiyatlari qo‘llaniladi, ular mos ravishda StreamWriter va StreamReader sinflarini ifodalaydi.

Asosiy harakatlar Process () usulida amalga oshiriladi, unda mijoz bilan xabar almashish uchun eng oddiy protokol amalga oshiriladi. Shunday qilib, boshida biz ulangan foydalanuvchi nomini olamiz, so‘ngra tsiklda biz boshqa barcha xabarlarini olamiz. Ushbu xabarlarini boshqa barcha mijozlarga etkazish uchun ServerObject sinfidagi BroadcastMessageAsync() usuli qo‘llaniladi.

Serverobject sinfi serverni ifodalaydi. U ikkita o‘zgaruvchini aniqlaydi: tcpListener o‘zgaruvchisi kiruvchi ulanishlarni tinglash uchun TcpListener ob’ektini saqlaydi va clients o‘zgaruvchisi barcha ulangan mijozlar qo‘shiladigan ro‘yxatni taqdim etadi.

```
TcpListener tcpListener = new TcpListener(IPAddress.Any, 8888); //
сервер для прослушивания
List<ClientObject> clients = new List<ClientObject>(); // все
подключения
```

Asosiy usul ListenAsync () bo‘lib, unda barcha kiruvchi ulanishlar tinglanadi:

```
protected internal async Task ListenAsync()
{
    try
    {
        tcpListener.Start();
        Console.WriteLine("Сервер запущен. Ожидание подключений...");

        while (true)
        {
            TcpClient tcpClient = await
tcpListener.AcceptTcpClientAsync();

            ClientObject clientObject = new ClientObject(tcpClient,
this);

            clients.Add(clientObject);
            Task.Run(clientObject.ProcessAsync);
        }
    }
}
```


Ulanishni olgandan so‘ng, biz unga Client Object ob‘ektini yaratamiz, uni mijozlar ro‘yxatiga qo‘shamiz va ClientObject ob‘ektining Process usuli bajariladigan yangi vazifani ishga tushiramiz.

BroadcastMessageAsync () usuli yuboruvchidan tashqari barcha mijozlarga xabarlarni uzatish uchun mo‘ljallangan:

```
protected internal async Task BroadcastMessageAsync(string message,
string id)
{
    foreach (var client in clients)
    {
        if (client.Id != id) // если id клиента не равно id отправителя
        {
            await client.Writer.WriteLineAsync(message); //передача
данных
            await client.Writer.FlushAsync();
        }
    }
}
```

Shunday qilib, ulangan mijozning mohiyati va serverning mohiyati ajratiladi.

Va serverobject va ClientObject sinflarini aniqlagandan so‘ng, tinglashni boshlashingiz kerak:

```
ServerObject server = new ServerObject(); // создаем сервер
await server.ListenAsync(); // запускаем сервер
```

Bu erda faqat Server ob‘ektining ListenAsync() usuliga kiradigan yangi oqim boshlanadi.

Bu mukammal server loyihasi emas, lekin kodni tashkil qilish va mijoz bilan o‘zaro munosabatlarni asosiy namoyish qilish uchun etarli.

Mijoz yaratish

Endi biz mijoz uchun yuqoridagi ma’lum bir serverga ulanadigan yangi konsol loyihasini yaratamiz. Mijoz uchun quyidagi kodni aniqlaymiz:

```
using System.Net.Sockets;

string host = "127.0.0.1";
int port = 8888;
using TcpClient client = new TcpClient();
Console.WriteLine("Введите свое имя: ");
string? userName = Console.ReadLine();
Console.WriteLine($"Добро пожаловать, {userName}");
StreamReader? Reader = null;
StreamWriter? Writer = null;

try
{
    client.Connect(host, port); //подключение клиента
    Reader = new StreamReader(client.GetStream());
```

```

        Writer = new StreamWriter(client.GetStream());
        if (Writer is null || Reader is null) return;
        // запускаем новый поток для получения данных
        Task.Run(() => ReceiveMessageAsync(Reader));
        // запускаем ввод сообщений
        await SendMessageAsync(Writer);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
    Writer?.Close();
    Reader?.Close();

    // отправка сообщений
    async Task SendMessageAsync(StreamWriter writer)
    {
        // сначала отправляем имя
        await writer.WriteLineAsync(userName);
        await writer.FlushAsync();
        Console.WriteLine("Для отправки сообщений введите сообщение и нажмите Enter");

        while (true)
        {
            string? message = Console.ReadLine();
            await writer.WriteLineAsync(message);
            await writer.FlushAsync();
        }
    }

    // получение сообщений
    async Task ReceiveMessageAsync(StreamReader reader)
    {
        while (true)
        {
            try
            {
                // считываем ответ в виде строки
                string? message = await reader.ReadLineAsync();
                // если пустой ответ, ничего не выводим на консоль
                if (string.IsNullOrEmpty(message)) continue;
                Print(message); // вывод сообщения
            }
            catch
            {
                break;
            }
        }
    }

    // чтобы полученное сообщение не накладывалось на ввод нового сообщения

```

```

void Print(string message)
{
    if (OperatingSystem.IsWindows())    // если ОС Windows
    {
        var position = Console.GetCursorPosition(); // получаем
текущую позицию курсора
        int left = position.Left;    // смещение в символах относительно
левого края
        int top = position.Top;    // смещение в строках относительно
верха
        // копируем ранее введенные символы в строке на следующую
строку
        Console.MoveBufferArea(0, top, left, 1, 0, top + 1);
        // устанавливаем курсор в начало текущей строки
        Console.SetCursorPosition(0, top);
        // в текущей строке выводит полученное сообщение
        Console.WriteLine(message);
        // переносим курсор на следующую строку
        // и пользователь продолжает ввод уже на следующей строке
        Console.SetCursorPosition(left, top + 1);
    }
    else Console.WriteLine(message);
}

```

Mijoz kodi, shuningdek, aslida ikkita funktsional qismdan iborat: ma'lumotlarni yuborish uchun Async Message yuborish usuli va ma'lumotlarni olish uchun ReceiveMessageAsync usuli.

SendMessageasync usuli StreamWriter ob'ektini parametr sifatida oladi, u mijozning NetworkStream-dan foydalanib, serverga satr yuboradi. Va ReceiveMessageAsync usuli StreamReader ob'ektini oladi, u orqali networkstream-dan yuborilgan xabarni satr sifatida o'qiydi.

Nazorat savollari:

1. NetworkStream xususiyat va metodlari haqida ma'lumot bering
2. NetworkStream orqali binar ma'lumot almashinish imkoniyati haqida ma'lumot bering.
3. TCP protokolida Listener xususiyati vazifasi qanday ?

8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari**15-Mashg‘ulot:** UDP protokoli.**Reja:**

1. UDP bilan ishlash uchun rozetkalardan foydalanish.
2. UdpClient.
3. Konsol UDP suhbat.
4. Translyatsiya.

1. UDP bilan ishlash uchun rozetkalardan foydalanish.

UDP (User Datagram Protocol) ma’lumotlarni uzoq tugunga etkazish imkonini beruvchi tarmoq protokolini taqdim etadi. UDP protokoli orqali xabarlarni uzatish uchun serverdan foydalanishning hojati yo‘q, ma’lumotlar to‘g‘ridan-to‘g‘ri bir tugundan boshqasiga uzatiladi. TCP bilan taqqoslaganda, uzatishda qo‘shimcha xarajatlar kamayadi, ma’lumotlarning o‘zi tezroq uzatiladi. UDP protokoli orqali yuborilgan barcha xabarlar datagramlar deb ataladi. Shuningdek, UDP orqali siz translyatsiya xabarlarini uzatishingiz mumkin pastki tarmoqdagi manzillar to‘plami uchun.

Shu bilan birga, UDP TCP bilan taqqoslaganda kamchiliklarga ega. Xususan, UDP datagramni oxirgi manzilga etkazib berishni kafolatlamaydi. Ushbu protokol tasvirlarni, multimedia fayllarini, audio, video va boshqalarni uzatish uchun ko‘proq mos keladi, agar uzatish paytida yo‘qolgan ma’lumotlarning oz miqdori etishmasligi uzatilgan ma’lumotlarning sifatiga katta ta’sir ko‘rsatmasa. Lekin nafaqat. Shunday qilib, HTTP - HTTP 3 protokolining yangi versiyasi UDP ustida ishlaydi (HTTP - 1.0, 1.1, 2.0 ning oldingi versiyalari TCP ustida ishlagan).

UDP bilan ishlash uchun. net platformasining bir qismi sifatida biz standart System.Net.Sockets.Socket sinfidan yoki System.Net.Sockets.UdpClient sinfidan foydalanishimiz mumkin, bu asosan UDP rozetkasi ustidagi qo‘shimcha hisoblanadi.

UDP bilan ishlash uchun rozetkalardan foydalanish

UDP protokoli orqali ma’lumotlarni yuborish va qabul qilishning asosiy klassi standart Socket sinfidir. Ammo rozetkaning UDP protokoli bilan ishlashi uchun uni Socket Type turi sifatida o‘rnatish kerak.Dgram va protokol sifatida ProtocolType.Udp:

```
using System.Net.Sockets;
using Socket udpSocket = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
```

Kiruvchi xabarlarni tinglash

UDP rozetkalari ma’lumotlarni qabul qilishi va/yoki yuborishi mumkin. Agar rozetka xabarlarni qabul qilishi kerak bo‘lsa, uni tinglanadigan so‘nggi nuqta uzatiladigan Bind () usuli yordamida mahalliy manzilga va portlardan biriga ulashingiz kerak:

```
using System.Net;
using System.Net.Sockets;
using var udpSocket = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
var localIP = new IPEndPoint(IPAddress.Parse("127.0.0.1"), 5555);
udpSocket.Bind(localIP);
```

Bunday holda, soket ma'lumotlarni qabul qilish va yuborish uchun 127.0.0.1:5555 mahalliy manzilidan foydalanadi. Shundan so'ng siz xabarlarini yuborishingiz va qabul qilishingiz mumkin.

Ma'lumotlarni olish

Xabarlarini qabul qilish uchun udp-soket `ReceiveFrom()`/`ReceiveFromAsync()` usulidan foydalanadi. Ular bir qator versiyalarga ega, shuning uchun men eng oddiylarini ta'kidlayman:

```
public Task<SocketReceiveFromResult> ReceiveFromAsync
(ArraySegment<byte> buffer, EndPoint remoteEndPoint);
public Task<SocketReceiveFromResult> ReceiveFromAsync
(ArraySegment<byte> buffer, SocketFlags socketFlags, EndPoint
remoteEndPoint);
```

Birinchi parametr orqali birinchi versiya - `arraysegment` ob'ekti `<byte>` ma'lumotlarni baytlar to'plami sifatida oladi va ikkinchi parametr orqali - `EndPoint` ma'lumotlar yuborgan mijozning manzilini oladi.

Ikkinchi versiya, shuningdek, ma'lumotlarni qabul qilish parametrlarini sozlash imkonini beruvchi `SocketFlags` ob'ektini qabul qiladi.

Usulning natijasi `socketreceivefromresult` tuzilmasi bo'lib, u operatsiya ma'lumotlarini ikkita xususiyat yordamida olish imkonini beradi: `ReceivedBytes` - olingan baytlar soni (agar operatsiya bajarilmasa, 0 qaytariladi) va `RemoteEndPoint` - `Endpoint` ob'ekti yoki ma'lumotlarni yuborgan mijozning manzili.

Shuni ta'kidlash kerakki, asenkron versiyalar .net 6 dan boshlab qo'llaniladi (va ba'zi versiyalar .net 7 dan boshlab qo'shilgan). Ko'proq erta versiyalar uchun sinxron versiyalardan foydalanish mumkin:

```
public int ReceiveFrom (byte[] buffer, ref EndPoint remoteEP);
```

Ushbu versiya yuborish uchun baytlar qatorini va `EndPoint` ob'ektini oladi, unga masofaviy so'nggi nuqta ma'lumotlari uzatiladi. Usulning natijasi yuborilgan baytlar soni.

Namuna olish:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
using var udpSocket = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
var localIP = new IPEndPoint(IPAddress.Parse("127.0.0.1"), 5555);
udpSocket.Bind(localIP);
Console.WriteLine("UDP-сервер запущен...");
```

```
byte[] data = new byte[256];
EndPoint remoteIp = new IPEndPoint(IPAddress.Any, 0);
var result = await udpSocket.ReceiveFromAsync(data, remoteIp);
var message = Encoding.UTF8.GetString(data, 0, result.ReceivedBytes);
Console.WriteLine($"Получено {result.ReceivedBytes} байт");
Console.WriteLine($"Удаленный адрес: {result.RemoteEndPoint}");
Console.WriteLine(message); // выводим полученное сообщение
```

Keling, ushbu dasturni shartli ravishda udp-server deb ataymiz. Bu erda udp rozetkasi 127.0.0.1:5555 terminali bilan bog‘langan. Ma’lumotlar ushbu manzilga yuboriladi. Ularni o‘qish uchun 256 baytdan iborat massiv aniqlanadi (yuborilgan ma’lumotlar 256 baytdan oshmaydi deb taxmin qiling). UdpSocket usuli yordamida.ReceiveFromAsync() ma’lumotlar qatoriga ma’lumotlarni, Remote Ip o‘zgaruvchisiga esa ma’lumotlar kelgan manzilni olamiz.

```
var result = await udpSocket.ReceiveFromAsync(data, remoteIp);
```

Bundan tashqari, biz remoteIp-da yangi qiymatga ega bo‘lsak-da, usulga o‘tishdan oldin ushbu ob’ektning o‘zi hali ham ishga tushirilishi kerak. Bunday holda, sukut bo‘yicha, ushbu so‘nggi nuqta o‘zboshimchalik bilan mahalliy manzil va port 0 ni ifodalaydi, ammo bu aniq emas.

```
EndPoint remoteIp = new IPEndPoint(IPAddress.Any, 0);
```

Ma’lumotlarni olgandan so‘ng, natijani konsolga ko‘rsatamiz.

Ma’lumotlarni yuborish

Ma’lumotlarni yuborish uchun SendTo()/SendToAsync () usullari qo‘llaniladi. Ushbu usullarning ba’zi versiyalariga e’tibor beraman:

```
public Task<int> SendToAsync (ArraySegment<byte> buffer, EndPoint remoteEP);
public Task<int> SendToAsync (ArraySegment<byte> buffer, SocketFlags socketFlags, EndPoint remoteEP);
```

Birinchi holda, usul yuborilgan ma’lumotlarni arraysegment<byte> ob’ekti va Endpoint so‘nggi nuqtasi sifatida yuborish manzili sifatida oladi. Ikkinchi holda, yuborish parametrlarini sozlash uchun SocketFlags qiymati ham uzatiladi. Usullarning natijasi yuborilgan baytlar soni.

Shuni yodda tutish kerakki, asenkron versiyalar.net 6 va 7-ga qo‘shilgan. Ammo.net-ning oldingi versiyalari uchun mavjud bo‘lgan sinxron usullar o‘xshash bo‘ladi. Masalan,:

```
public int SendTo (byte[] buffer, EndPoint remoteEP);
```

Bu erda birinchi parametr yuborilgan baytlar qatorini, ikkinchisi esa yuborish manzilini anglatadi. Usulning natijasi yuborilgan baytlar soni.

Shuni ta’kidlash kerakki, yuborish uchun Sendto usulidan oldin Bind usulini chaqirish shart emas.

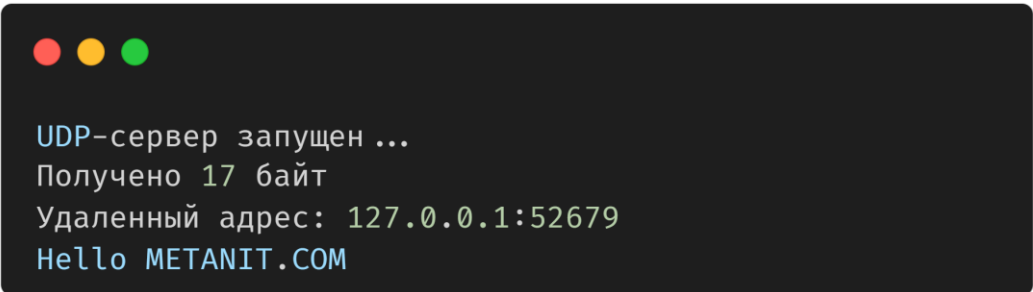
Masalan, biz yuqorida ko‘rsatilgan udp serveri uchun ma’lumotlarni yuboradigan udp mijoz dasturini aniqlaymiz:

```
using System.Net;
```

```
using System.Net.Sockets;
using System.Text;
using var udpSocket = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
string message = "Hello METANIT.COM";
byte[] data = Encoding.UTF8.GetBytes(message);
EndPoint remotePoint = new IPEndPoint(IPAddress.Parse("127.0.0.1"),
5555);
int bytes = await udpSocket.SendToAsync(data, remotePoint);
Console.WriteLine($"Отправлено {bytes} байт");
```

Bunday holda, "Salom" qatori METANIT.COM " baytlar qatoriga aylantiriladi va 127.0.0.1:5555 manziliga yuboriladi.

UDP serverini, so‘ngra udp mijozini ishga tushiring. Natijada, udp mijozi ma’lumotlarni serverga yuboradi va udp server konsoli olingan ma’lumotlarni ko‘rsatadi:



```
UDP-сервер запущен ...
Получено 17 байт
Удаленный адрес: 127.0.0.1:52679
Hello METANIT.COM
```

Ma’lumotlarni olishda bufer hajmi

Udp rozetkasi orqali ma’lumotlarni olishda keng tarqalgan muammolardan biri bu ishlatiladigan bufer hajmining Datagram o‘lchamiga mos kelmasligi. Shunday qilib, agar bufer hajmi Datagram hajmidan kichik bo‘lsa, unda dasturni bajarish muvaffaqiyatsiz bo‘ladi.

Ushbu muammoning yagona mumkin bo‘lgan echimi tegishli bufer hajmini o‘rnatishdir. Masalan, yuqoridagi misolda bufer 256 baytli massivni ifodalaydi. Sug‘urta sifatida siz 65535 baytli buferni o‘rnatishingiz mumkin-bu udp datagramining mumkin bo‘lgan maksimal hajmi.

2. UdpClient.

.Net-da UDP protokoli bilan ishlashni soddalashtirish uchun siz Socket sinfiga asoslangan UdpClient sinfidan foydalanishingiz mumkin. (Udpclient sinfining manba kodi)

UdpClient yaratish uchun bir qator konstruktorlar qo‘llaniladi:

```
public UdpClient ();
public UdpClient (AddressFamily family);
public UdpClient (int port);
public UdpClient (IPEndPoint localEP);
public UdpClient (int port, AddressFamily family);
public UdpClient (string hostname, int port);
```

Birinchi va ikkinchi konstruktorlar UdpClient-ni yaratadilar va shu bilan tarmoq orqali ma'lumotlarni qabul qilish/yuborish uchun foydalanadigan rozetkani yaratadilar. Uchinchi, to'rtinchi va beshinchi konstruktorlar sizga kiruvchi xabarlarni tinglash uchun port yoki so'nggi nuqtani o'rnatishga imkon beradi. Oxirgi konstruktor ruxsat beradi xost manzili va ma'lumotlar yuboriladigan uning porti. Amaldagi rozetkaning o'zi Client xususiyati yordamida olinishi mumkin:

Ma'lumotlarni olish

UdpClient ma'lumotlarini olish uchun Receive()/ReceiveAsync() usullari qo'llaniladi.:

```
public byte[] Receive (ref System.Net.IPEndPoint? remoteEP);
public Task<System.Net.Sockets.UdpReceiveResult> ReceiveAsync ();
```

Sinxron versiya ma'lumotni yuborgan mijozning manzili joylashtirilgan ipendpoint ob'ektini ref parametri sifatida qabul qiladi. Usulning natijasi-olingan baytlar qatori.

Asenkron versiya udpreceiveresult ob'ektini qaytaradi, unda bufer xususiyatidan foydalanib siz yuborilgan baytlar qatorini va Remoteendpoint xususiyatidan foydalanib, jo'natuvchining manzilini ipendpoint ob'ekti sifatida olishingiz mumkin.

Masalan, ma'lumotlarni qabul qiladigan mini UDP serverini aniqlaymiz:

```
using System.Net.Sockets;
using System.Text;
using var udpServer = new UdpClient(5555);
Console.WriteLine("UDP-сервер запущен...");
var result = await udpServer.ReceiveAsync();
var message = Encoding.UTF8.GetString(result.Buffer);
Console.WriteLine($"Получено {result.Buffer.Length} байт");
Console.WriteLine($"Удаленный адрес: {result.RemoteEndPoint}");
Console.WriteLine(message);
```

Bunday holda, UdpClient 5555 portini tinglaydi.

Ma'lumotlarni yuborish

UdpClient-dan ma'lumotlarni yuborish uchun Send()/SendAsync () usullari qo'llaniladi. Ushbu usullar bir nechta ortiqcha yuklarga ega, ba'zilarini ko'rib chiqing:

```
public int Send (ReadOnlySpan<byte> datagram, IPEndPoint? endPoint);
public ValueTask<int> SendAsync (ReadOnlyMemory<byte> datagram,
System.Net.IPEndPoint? endPoint, CancellationToken cancellationToken
= default);
```

Parametr sifatida ikkala versiya ham readonlyspan<byte> ob'ekti sifatida yuborilgan baytlar to'plamini va ipendpoint ob'ekti sifatida yuborilgan manzilni oladi. Usullarning natijasi yuborilgan baytlar soni.

Send () usuli bir qator haddan tashqari Yuklangan versiyalarga ega, shuning uchun biz yuborilganda xost manzili va portni aniq belgilashimiz mumkin:

Masalan, biz yuqorida ko‘rsatilgan udp serveri uchun ma’lumotlarni yuboradigan udp mijoz dasturini aniqlaymiz:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
using var udpClient = new UdpClient();
string message = "Hello METANIT.COM";
byte[] data = Encoding.UTF8.GetBytes(message);
IPEndPoint remotePoint = new IPEndPoint(IPAddress.Parse("127.0.0.1"),
5555);
int bytes = await udpClient.SendAsync(data, remotePoint);
Console.WriteLine($"Отправлено {bytes} байт");
```

Ulanish Usuli

UdpClient-dan ma’lumotlarni yuborish uchun manzilni o‘rnatish uchun siz Connect () usulini chaqirishingiz mumkin. Bundan tashqari, agar biz konstruktorda yoki ulanish usulida ulanish ma’lumotlarini (xost nomi, port raqami) ko‘rsatgan bo‘lsak, u holda qabul qilish usuli ma’lumotlarni faqat ko‘rsatilgan uzoq nuqtadan oladi, qolgan ulanishlar e’tiborga olinmaydi. Bundan tashqari, bu holda Send()/SendAsync()usulida ma’lumotlarni qabul qiluvchining manzilini ko‘rsatish shart emas:

```
using System.Net.Sockets;
using System.Text;
using var udpClient = new UdpClient();
udpClient.Connect("127.0.0.1", 5555);
string message = "Hello METANIT.COM";
byte[] data = Encoding.UTF8.GetBytes(message);
int bytes = await udpClient.SendAsync(data);
Console.WriteLine($"Отправлено {bytes} байт");
```

Ulanish uchun manzil va port ulanish usuliga o‘tkaziladi, ya’ni biz ulanmoqchi bo‘lgan tashqi manzilni sozlash. Mahalliy manzil bo‘lsa, bitta port ko‘rsatilishi mumkin. Shu bilan birga, Connect masofaviy xost bilan doimiy aloqani o‘rnatmaydi. U faqat konstruktor yordamida o‘rnatilishi mumkin bo‘lgan ulanish parametrlarini o‘rnatadi:

```
using var udpClient = new UdpClient("127.0.0.1", 5555);
```

3. Konsol UDP suhbat.

O‘tgan mavzular UDP protokoli yordamida tarmoq orqali o‘zaro aloqani ko‘rib chiqdi. Endi udp mijoz bir vaqtning o‘zida ma’lumotlarni yuboradigan va yuboradigan biroz murakkabroq misolni ko‘rib chiqing. Buning uchun eng oddiy udp suhbatini aniqlang. Konsol suhbatlarini yaratish eng qulay ish bo‘lmasa - da, xabarlarni kiritish maydoni xabarlarni chiqarish maydoni bilan birlashtirilganligini hisobga olsak. Ammo bu holda biz ma’lum bir grafik dastur texnologiyasidan mavhum bo‘lib, UDP protokoli yordamida o‘zaro bog‘liqlikka e’tibor qaratamiz.

UDP rozetkalarida konsol suhbat

Birinchidan, biz toza udp rozetkalarini ishlatamiz va buning uchun quyidagi dastur kodini aniqlaymiz:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
IPAddress localAddress = IPAddress.Parse("127.0.0.1");
Console.Write("Введите свое имя: ");
string? username = Console.ReadLine();
Console.Write("Введите порт для приема сообщений: ");
if (!int.TryParse(Console.ReadLine(), out var localPort)) return;
Console.Write("Введите порт для отправки сообщений: ");
if (!int.TryParse(Console.ReadLine(), out var remotePort)) return;
Console.WriteLine();
Task.Run(ReceiveMessageAsync);
await SendMessageAsync();
async Task SendMessageAsync()
{
    using Socket sender = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
    Console.WriteLine("Для отправки сообщений введите сообщение и
нажмите Enter");
    while (true)
    {
        var message = Console.ReadLine();
        if (string.IsNullOrEmpty(message)) break;
        message = $"{username}: {message}";
        byte[] data = Encoding.UTF8.GetBytes(message);
        await sender.SendToAsync(data, new IPEndPoint(localAddress,
remotePort));
    }
}

async Task ReceiveMessageAsync()
{
    byte[] data = new byte[65535];

    using Socket receiver = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);

    receiver.Bind(new IPEndPoint(localAddress, localPort));
    while (true)
    {
        var result = await receiver.ReceiveFromAsync(data, new
IPEndPoint(IPAddress.Any, 0));
        var message = Encoding.UTF8.GetString(data, 0,
result.ReceivedBytes);

        Console.WriteLine(message);
    }
}
```

```
}
```

Birinchidan, dastur ishlaydigan manzilni aniqlang

```
IPAddress localAddress = IPAddress.Parse("127.0.0.1");
```

Dastlab, foydalanuvchi o‘z nomini, shuningdek ma’lumotlarni qabul qilish va yuborish uchun portlarni kiritadi. Bir-biri bilan o‘zaro aloqada bo‘ladigan ikkita mijoz ilovasi "127.0.0.1" manzilida bitta mahalliy mashinada ishlaydi deb taxmin qilinadi. Agar mijozlarning manzillari turlicha bo‘lsa, unda siz ma’lumotlarni yuborish uchun manzilni kiritishni ham ta’minlashingiz mumkin.

Portlar kiritilgandan so‘ng, kirish qutisini tinglash vazifasi va ma’lumotlarni yuborish usuli boshlanadi:

```
Task.Run(ReceiveMessageAsync);  
await SendMessageAsync();
```

Qabul qiluvchi xabar Async usulida ma’lumotlarni olish uchun rozetka yarating va bind usuli yordamida tinglashni boshlang:

```
using Socket receiver = new Socket(AddressFamily.InterNetwork,  
SocketType.Dgram, ProtocolType.Udp);  
receiver.Bind(new IPEndPoint(localAddress, localPort));
```

Keyinchalik, cheksiz tsiklda biz ma’lumotlarni ma’lumotlar qatoriga olamiz, ularni qatorga aylantiramiz va konsolga chiqaramiz:

```
var result = await receiver.ReceiveFromAsync(data, new  
IPEndPoint(IPAddress.Any, 0));  
var message = Encoding.UTF8.GetString(data, 0, result.ReceivedBytes);  
Print(message);
```

Xabarlarini yuborish Async () usulida xabarlarini yuborish uchun alohida rozetka yaratiladi:

```
using Socket sender = new Socket(AddressFamily.InterNetwork,  
SocketType.Dgram, ProtocolType.Udp);
```

Keyin biz kiritilgan xabarni o‘qiymiz, unga foydalanuvchi nomini qo‘shamiz, xabarni baytlar qatoriga aylantiramiz va remotePort portiga yuboramiz:

```
var message = Console.ReadLine();  
if (string.IsNullOrEmpty(message)) break;  
message = $"{username}: {message}";  
byte[] data = Encoding.UTF8.GetBytes(message);  
await sender.SendToAsync(data, new IPEndPoint(localAddress,  
remotePort));
```

Agar bo‘sh satr kiritilgan bo‘lsa, biz ko‘chadan va shunga mos ravishda usuldan chiqib, dasturni tugatamiz.

Endi biz dasturning ikkita nusxasini ishga tushiramiz va portlar uchun turli xil ma’lumotlarni kiritamiz.

Bu erda birinchi mijoz 4004 portiga xabarlarini oladi va 4005 portiga yuboradi, ikkinchi mijoz esa, aksincha, 4005 portiga xabarlarini oladi va 4004 portiga yuboradi. Shunday qilib, bitta mashinada ishlaydigan ikkala mijoz ham bir-biri bilan o‘zaro aloqada bo‘ladi.

Shuni ta’kidlash kerakki, bu erda xabarlarini yuborish va qabul qilish uchun turli xil rozetkalar yaratilgan bo‘lsa-da, bu ixtiyoriy. Asosan, biz bir xil rozetkadan foydalanishimiz mumkin.

Udpclient yordamida konsol suhbat

UdpClient-dan foydalanish dasturni biroz soddalashtirishga imkon beradi.

Xuddi shu suhbat, faqat UdpClient bilan:

```
using System.Net;
using System.Net.Sockets;
using System.Text;
IPAddress localAddress = IPAddress.Parse("127.0.0.1");
Console.Write("Введите свое имя: ");
string? username = Console.ReadLine();
Console.Write("Введите порт для приема сообщений: ");
if (!int.TryParse(Console.ReadLine(), out var localPort)) return;
Console.Write("Введите порт для отправки сообщений: ");
if (!int.TryParse(Console.ReadLine(), out var remotePort)) return;
Console.WriteLine();
Task.Run(ReceiveMessageAsync);
await SendMessageAsync();
async Task SendMessageAsync()
{
    using UdpClient sender = new UdpClient();
    Console.WriteLine("Для отправки сообщений введите сообщение и нажмите Enter");
    while (true)
    {
        var message = Console.ReadLine();
        if (string.IsNullOrEmpty(message)) break;
        message = $"{username}: {message}";
        byte[] data = Encoding.UTF8.GetBytes(message);
        await sender.SendAsync(data, new IPEndPoint(localAddress, remotePort));
    }
}
async Task ReceiveMessageAsync()
{
    using UdpClient receiver = new UdpClient(localPort);
    while (true)
    {
        var result = await receiver.ReceiveAsync();
        var message = Encoding.UTF8.GetString(result.Buffer);
        Console.WriteLine(message);
    }
}
```

Qabul qilingan xabarni xabar kiritishda qoplash

Yuqoridagi konsol suhbatiga misollar juda oddiy, xuddi shu kompyuterda ishlaydi. Va biz ketma-ket xabarni avval bitta ilovada, keyin boshqasida yuboramiz.

Ammo mijozlar turli xil mashinalarda bo‘lsa va bitta mijoz to‘g‘ridan-to‘g‘ri boshqa mijoz o‘z xabarini kiritayotgan paytda xabar yuborsa-chi? Keyin aniq qabul qilingan xabar mijoz o‘z xabarini kiritadigan konsolning bir qatorida ko‘rsatiladi. Agar Windows operatsion tizimi bo‘lsa, unda biz bunga yo‘l qo‘ymaslik uchun dasturga kichik Hack qo‘shishimiz mumkin - allaqachon kiritilgan belgilarni hali yuborilmagan xabarni keyingi qatorga nusxalash, lekin qabul qilingan xabarni joriy satrda aks ettiradi. Buning uchun `ReceiveMessageAsync` usulini quyidagicha o‘zgartiring:

```
async Task ReceiveMessageAsync()
{
    using UdpClient receiver = new UdpClient(localPort);
    while (true)
    {
        var result = await receiver.ReceiveAsync();
        var message = Encoding.UTF8.GetString(result.Buffer);
        Print(message);
    }
}
void Print(string message)
{
    if (OperatingSystem.IsWindows())
    {
        var position = Console.GetCursorPosition();          int left =
position.Left;
        int top = position.Top;
        Console.MoveBufferArea(0, top, left, 1, 0, top + 1);
        Console.SetCursorPosition(0, top);
        Console.WriteLine(message);
        Console.SetCursorPosition(left, top + 1);
    }
    else Console.WriteLine(message);
}
```

4. Translyatsiya.

UDP protokoli sizga xabarlarni guruhli translyatsiya orqali yuborish imkonini beradi. Bunday pochta orqali mijozga bitta xabar yuborish kifoya qiladi va uni guruhga ulangan barcha boshqa mijozlar oladi. Va har bir mijozga bitta xabar yuborishning hojati yo‘q.

Eshittirishdan foydalanganda, uzatish mahalliy tarmoqlardan nariga o‘tmasligini yodda tutish kerak, chunki marshrutizatorlar bunday xabarlarni o‘tkazib yubormaydilar.

Guruhga qo‘shilish. `JoinMulticastGroup` va `DropMulticastGroup`

Broadcast Group axborotnomasi faqat guruh uchun mavjud. `UdpClient` mijozini guruhga qo‘shish uchun unga `JoinMulticastGroup()` usuli chaqiriladi. Uning versiyalaridan biri:

```
public void JoinMulticastGroup (System.Net.IPAddress multicastAddr);
```

```
public void JoinMulticastGroup (System.Net.IPAddress multicastAddr,
int timeToLive);
```

Parametr sifatida guruh manzili usulga uzatiladi. Manzil sifatida 224.0.0.0 dan 239.255.255.255 gacha bo‘lgan manzillardan biri ishlatilishi mumkin. Agar boshqa manzil uzatilsa yoki yo‘riqnoma guruh xabarlarini qo‘llab-quvvatlamasa, dastur SocketException istisnosini yaratadi.

Ikkinchi parametr-timeToLive marshrutizatorlar orqali o‘tish sonini ifodalaydi. Ya’ni, xabar qabul qiluvchiga etib borguncha, uni kerakli yo‘lga yo‘naltiradigan bir qator marshrutizatorlardan o‘tishi mumkin. Masalan, 50 raqami jo‘natuvchi kompyuterdan qabul qiluvchi kompyuterga o‘tish uchun Datagram 50 ta yo‘riqchini chetlab o‘tishi mumkinligini anglatadi.

Ushbu usul chaqirilgandan so‘ng, Socket ob’ekti guruhga qo‘shilish uchun yo‘riqchiga IGMP (Internet Group Management Protocol) formatidagi ma’lumotlar paketini yuboradi. Bundan tashqari, udpclient ob’ekti udpclient ob’ektlarining butun guruhga mo‘ljallangan datagramlarni olish imkoniyatiga ega bo‘ladi.

Shuni ta’kidlash kerakki, guruhga qo‘shilish uchun UdpClient ma’lum bir portga ulanishi kerak, u orqali uning rozetkasi kiruvchi xabarlarni tinglaydi. Buning uchun port raqamini UdpClient konstruktoriga o‘tkazish mumkin. Bundan tashqari, guruhga qo‘shilish faqat guruhga mo‘ljallangan xabarlarni olish uchun kerak. Va guruhga xabarlarni yuborish uchun guruhga tegishli bo‘lish shart emas.

Guruhdan olib tashlash uchun Udpclient dropmulticastgroup()usulini chaqiradi:

```
public void DropMulticastGroup (System.Net.IPAddress multicastAddr);
```

The usul JoinMulticastGroup-ga uzatilgan guruhning bir xil manzili uzatiladi:

```
using var udpClient = new UdpClient();
var broadcastAddress = IPAddress.Parse("235.5.5.11"); ; // хост для
отправки данных

udpClient.JoinMulticastGroup(broadcastAddress);
// .... рассылка и получение сообщений
udpClient.DropMulticastGroup(broadcastAddress);
```

Keling, kichik bir misol bilan ko‘rib chiqaylik. Birinchidan, eshittirish xabarlarini qabul qiladigan Udp mijozini aniqlaymiz:

```
using System.Net.Sockets;
using System.Net;
using System.Text;

using var udpClient = new UdpClient(8001);
var broadcastAddress = IPAddress.Parse("235.5.5.11"); ; // хост для
отправки данных
// присоединяемся к группе
udpClient.JoinMulticastGroup(broadcastAddress);
Console.WriteLine("Начало прослушивания сообщений");
```

```
while (true)
{
    var result = await udpClient.ReceiveAsync();
    string message = Encoding.UTF8.GetString(result.Buffer);
    if (message == "END") break;
    Console.WriteLine(message);
}
// отсоединяемся от группы
udpClient.DropMulticastGroup(broadcastAddress);
Console.WriteLine("Udp-клиент завершил свою работу");
```

Bu erda mijoz 8001 portidagi ulanishlarni tinglaydi. Bundan tashqari, u "235.5.5.11" manzilidagi guruhga ulangan.

Tsiklda mijoz guruhga yuborilgan xabarlarini oladi. Va agar "END" buyrug‘i yuborilgan bo‘lsa, u tsikldan chiqib, ishini tugatadi.

Shunday qilib, ushbu mijoz "235.5.5.11:8001" guruhidan xabarlarini oladi. Sinov uchun biz ushbu guruhga ma’lumotlarni yuboradigan yuboruvchi dasturni aniqlaymiz:

```
using System.Net;
using System.Net.Sockets;
using System.Text;

var messages = new string[] { "Hello World!", "Hello METANIT.COM",
    "Hello work", "END" };
var broadcastAddress = IPAddress.Parse("235.5.5.11"); ; // хост для
отправки данных
using var udpSender = new UdpClient();
Console.WriteLine("Начало отправки сообщений...");
// отправляем сообщения
foreach (var message in messages)
{
    Console.WriteLine($"Отправляется сообщение: {message}");
    byte[] data = Encoding.UTF8.GetBytes(message);
    await udpSender.SendAsync(data, new IPEndPoint(broadcastAddress,
        8001));
    await Task.Delay(1000);
}
```

Bu erda oddiy udpclient mijozni xabarlar qatoridan barcha xabarlarini yuboradi. Aytish joizki, UdpClient hech qanday guruhga qo‘shilmagan. Ammo yuborish manziliga e’tibor bering - 235.5.5.11:8001-yuqoridagi ma’lum bir mijoz guruhdan xabarlarini qabul qiladigan manzil.

Birinchidan, guruhdan xabarlarini qabul qiladigan Udp mijoz dasturini ishga tushiramiz. Va keyin biz yuboruvchi dasturni ishga tushiramiz. Yuboruvchi konsolida xabarlarini yuborish jarayoni ko‘rsatiladi:

Bunday holda, soddaligi uchun siz ikkala dasturni bir xil kompyuterda ishga tushirishingiz mumkin. Ammo biz mahalliy tarmoqdagi turli xil kompyuterlarda

xabarlarni qabul qiluvchi dasturni ishga tushirsak ham, ularning barchasi bir vaqtning o‘zida xabarlarni qabul qilishadi, chunki ular bir xil guruhga qo‘shilgan.

Bitta ilovada xabarlarni qabul qilish va yuborish

Guruhga kiritilgan bir xil dastur xabarlarni yuborishi va qabul qilishi mumkin. Ammo bu erda ba’zi xususiyatlar mavjud. Shunday qilib, sinov uchun biz translyatsiya yordamida konsol suhbatini yaratamiz:

```
using System.Net;
using System.Net.Sockets;
using System.Text;

int localPort = 8001;
IPAddress broadcastAddress = IPAddress.Parse("235.5.5.11");
Console.Write("Введите свое имя: ");
string? username = Console.ReadLine();

Task.Run(ReceiveMessageAsync);
await SendMessageAsync();

// отправка сообщений в группу
async Task SendMessageAsync()
{
    using var sender = new UdpClient(); // создаем UdpClient для отправки
    // отправляем сообщения
    while (true)
    {
        string? message = Console.ReadLine(); // сообщение для отправки
        // если введена пустая строка, выходим из цикла и завершаем ввод сообщений
        if (string.IsNullOrEmpty(message)) break;
        // иначе добавляем к сообщению имя пользователя
        message = $"{username}: {message}";
        byte[] data = Encoding.UTF8.GetBytes(message);
        // и отправляем в группу
        await sender.SendAsync(data, new IPEndPoint(broadcastAddress, localPort));
    }
}

// получение сообщений из группы
async Task ReceiveMessageAsync()
{
    using var receiver = new UdpClient(localPort); // UdpClient для получения данных
    receiver.JoinMulticastGroup(broadcastAddress);
    receiver.MulticastLoopback = false; // отключаем получение своих же сообщений
    while (true)
```



```
{
    var result = await receiver.ReceiveAsync();
    string message = Encoding.UTF8.GetString(result.Buffer);
    Console.WriteLine(message);
}
```

Endi mahalliy tarmoqdagi guruhli pochta uchun "235.5.5.11" manzili, port sifatida - 8001 ishlatiladi.

Shuningdek, asosiy oqimda asinxron Send Message Async () usuli ishga tushiriladi. Unda cheksiz tsiklda biz konsoldan kiritilgan xabarni o‘qiymiz va foydalanuvchi nomini xabarga qo‘shib guruhga yuboramiz. Agar foydalanuvchi bo‘sh qatorni kiritgan bo‘lsa, unda biz ko‘chadan chiqamiz va shu bilan usul va butun dasturning bajarilishini yakunlaymiz.

Loyihani qurgandan so‘ng, biz mahalliy tarmoqdagi turli xil mashinalarda kompilyatsiya qilingan dasturni ishga tushirishimiz mumkin.

Nazorat savollari:

1. UDPClient vazifalari haqida ma’lumot bering.
2. UDPda Listener xususiyatlari
3. Translation jarayoni nima va qanday majburiyatlarni bajaradi?

**O‘ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN’IY INTELLEKT”
KAFEDRASI**



“OBYEKTGA YO‘NALTIRILGAN DASTURLASH”

FANI BO‘YICHA MASHG‘ULOTLAR O‘TISH REJALARI

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH REJASI
1-MASHG‘ULOT. 1-MA’RUZA.**

Mavzu: C# va .NET da tarmoq bilan ishlash asoslari.

Mashg‘ulotning maqsadi: C# va .NET da tarmoq bilan ishlash asoslari haqida bilim va ko‘nikmaga ega bo‘lish

O‘quv guruhi: -AT-4/4-20.

Vaqt: 80 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Ma’ruza.

O‘quv-moddiy ta’minot: Taqdimot va elektron ma’lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Программируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOBI

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e’lon qilish; – Mavzu materialini bayon etishga kirishish.	10	
2.	II. ASOSIY QISM O‘quv savollari: 1. Tarmoq haqida tushuncha va tarmoq protokollari. 2. .NET da adreslar haqida tushuncha(IP adres, URI).	60 15 15	komputer, elektron taqdimot,

	3. Addreslash sxemasi. 4. URI adresslar haqida tushuncha.	15 15	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	10	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH REJASI
2-MASHG‘ULOT. 2-MA’RUZA.**

Mavzu: : Tarmoq konfiguratsiyasi va Soket klassi.

Mashg‘ulotning maqsadi: : Tarmoq konfiguratsiyasi va Soket klassi haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20.

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Ma’ruza.

O‘quv-moddiy ta’minot: Taqdimot va elektron ma’lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Программируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOBI

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e’lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari: 1. DNS.	120 40 40	komputer, elektron taqdimot,

	2. Tarmoq konfiguratsiyasi va tarmoq trafigi haqida ma'lumot olish. 3. Soket klassi.	40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	20	

**“OBJEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH REJASI
3-MASHG‘ULOT. 1-AMALIY.**

Mavzu: HTTP protokoli.

Mashg‘ulotning maqsadi: HTTP protokoli haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektron ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Программируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOBI

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari: 1. HTTP protokoli bilan tanishish. 2. HTTP status kodlari.	120 40 40	komputer, elektron taqdimot,

	3. HttpClient yaratish.	40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	20	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

4-MASHG‘ULOT. 2-AMALIY.

Mavzu: So‘rovlar yuborish..

Mashg‘ulotning maqsadi So‘rovlar yuborish haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektron ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Программируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOBI

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari: 1. HttpClient bilan so‘rovlarni yuborish.	120 40	komputer, elektron taqdimot,

	2. SendAsync metodi orqali so‘rovlar jo‘natish. 3. JSON formatida ma’lumotlarni olish.	40 40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e’lon qilish.	20	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

5-MASHG‘ULOT. 3-AMALIY.

Mavzu: Sarlavhalarni yuborish va qabul qilish.

Mashg‘ulotning maqsadi: Sarlavhalarni yuborish va qabul qilish haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektronik ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Программируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOB

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari:	120 40	komputer, elektron taqdimot,

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

	1. Sarlavhalarni yuborish va qabul qilish haqida tushuncha. 2. So‘rov uchun “sarlavha” o‘rnatish. 3. So‘rovda ma’lumotlarni yuborish.	40 40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e’lon qilish.	20	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

6-MASHG‘ULOT. 4-AMALIY.

Mavzu: HttpClient orqali JSON malumotlarni yuborish.

Mashg‘ulotning maqsadi: HttpClient orqali JSON malumotlarni yuborish haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektron ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Програмируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOB

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari: 1. HttpClient bilan json yuborish.	120 40	komputer, elektron taqdimot,

	2. HttpClient-ning Web API bilan o‘zaro ta’siri. 3. Malumotlar bilan ishlash.	40 40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e’lon qilish.	20	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

7-MASHG‘ULOT. 5-AMALIY.

Mavzu: FormUrlEncodedContent klassi.

Mashg‘ulotning maqsadi: FormUrlEncodedContent klassi haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektron ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Програмируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOBI

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari:	120	komputer, elektron taqdimot,

	1. Shakllar va FormUrlEncodedContent sinfini yuborish. 2. Oqimlar va bayt massivini yuborish. 3. Baytlar massivi yuborish(ByteArrayContent.	40 40 40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e’lon qilish.	20	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

8-MASHG‘ULOT. 6-AMALIY.

Mavzu: MultipartFormDataContent klassi.

Mashg‘ulotning maqsadi: MultipartFormDataContent klassi haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektron ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Програмируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOB

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari:	120 40	komputer, elektron taqdimot,

	1. Fayllarni yuborish va MultipartFormDataContent klassi. 2. Birdaniga bir nechta fayllar yuborish. 3. HttpClient yordamida cookie-fayllarni yuborish va qabul qilish.	40 40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e’lon qilish.	20	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

9-MASHG‘ULOT. 7-AMALIY.

Mavzu: Kukilarni yuborish va HttpListener. HTTP serveri.

Mashg‘ulotning maqsadi: Kukilarni yuborish va HttpListener. HTTP serveri haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektronik ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Програмируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOB

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari: 1. Kukilarni yuborish.	120 40	komputer, elektron taqdimot,

	2. HttpListener. 3. So‘rov haqida ma’lumotlarni olish.	40 40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e’lon qilish.	20	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

10-MASHG‘ULOT. 8-AMALIY.

Mavzu: Soketlarda TCP-klient.

Mashg‘ulotning maqsadi: Soketlarda TCP-klient haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektronik ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Програмируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOB

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari: 1. Soketlarda TCP mijosi.	120 40	komputer, elektron taqdimot,

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

	2. Hostdan ma’lumotlarni olish. 3. Avialable xususiyati.	40 40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e’lon qilish.	20	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

11-MASHG‘ULOT. 9-AMALIY.

Mavzu: Soketlarda TCP Server.

Mashg‘ulotning maqsadi: Soketlarda TCP Server haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektron ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Программируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOB

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari: 1. Soketlarda TCP serveri haqida tushuncha.	120 40	komputer, elektron taqdimot,

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

	2. NetworkStream. 3. Ma’lumotlarni o‘qish.	40 40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e’lon qilish.	20	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

12-MASHG‘ULOT. 10-AMALIY.

Mavzu: TCP protokoli.

Mashg‘ulotning maqsadi: TCP protokoli haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektron ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Програмируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiriziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiriziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOB

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari: 1. TCP mijosi. TcpClient klassi.	120 40	komputer, elektron taqdimot,

	2. Ma'lumot yuborish va qabul qilish. 3. TCP serveri. TcpListener sinfi.	40 40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	20	

**“OBJEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

13-MASHG‘ULOT. 11-AMALIY.

Mavzu: Ma’lumotlarni yuborish va qabul qilish.

Mashg‘ulotning maqsadi: Ma’lumotlarni yuborish va qabul qilish haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta’minot: Taqdimot va elektronik ma’lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Програмируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiroziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O'QUV SAVOLLARI VA VAQT HISOBI

№	O'quvsavollari	Vaqt, min	Ko'rgazmali qo'llanma va O'TV
	I. KIRISH QISMI		
1.	– Dalolat qabul qilish, kursantlar davomatini, tashqi ko'rinishini, mashg'ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg'ulot maqsadini e'lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
	II. ASOSIY QISM	120	
	O'quv savollari:		
2.	1. Ma'lumotlarni yuborish va qabul qilish. Soketlar orasidagi bir tomonlama aloqa.	40	komputer, elektron taqdimot,
	2. Ma'lumotlarni olishning usullari.	40	
	3. Ma'lumotlarni yuborish va qabul qilish. TcpListener va TcpClient o'rtasida bir tomonlama aloqa.	40	
	III. YAKUNIY QISM		
3.	–Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	20	

	1. Ma’lumotlarni yuborish va qabul qilish. Ikki tomonlama aloqa. 2. Ko‘p tarmoqli TCP mijoz-server haqida tushuncha. 3. Ko‘p tarmoqli TCP mijoz-server ilovasi.	20 20	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e’lon qilish.	20	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

15-MASHG‘ULOT. 13-AMALIY.

Mavzu: NetworkStream klassi.

Mashg‘ulotning maqsadi: NetworkStream klassi haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektronik ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Программируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiriziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiriziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOB

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari: 1. NetworkStream va matn oqimlari.	120 40	komputer, elektron taqdimot,

	2. NetworkStream va ikkilik oqimlar. 3. Konsol TCP suhbatl.	40 40	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo‘yilgan maqsadga mashg‘ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg‘ulot mavzusi va o‘tish joyini e’lon qilish.	20	

**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN
MASHG‘ULOT O‘TISH
REJASI**

16-MASHG‘ULOT. 14-AMALIY.

Mavzu: UDP protokoli.

Mashg‘ulotning maqsadi: UDP protokoli haqida bilim va ko‘nikmaga ega bo‘lish.

O‘quv guruhi: - AT 4/4-20;

Vaqt: 160 min.

O‘tish joyi: 437–o‘quv sinfi.

Mashg‘ulot turi: Amaliy.

O‘quv-moddiy ta‘minot: Taqdimot va elektronik ma‘lumotlar.

O‘quv texnik vositalar: Videoproektor, 65 dyumli monoblok, kompyuter.

Kursantlar mustaqil tayyorgarligi uchun mavzu bo‘yicha adabiyotlar:

1. C# 7.0 Карманный справочник: Учебное пособие / Джозеф Албахари и Бен Албахари, Перевод с англ. под ред. Ю.Н. Артеменко 2017.174 с
2. Программируем на C# 8.0: Разработка приложений – СПб: Питер, 2021.944с
3. Программирование на C# для начинающих. Основные сведения / Алексей Васильев. – Москва : Эксмо, 2018. 592 с.
4. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiriziqov. “Dasturlash texnologiyalari” fanidan amaliy mashg‘ulotlarni bajarish bo‘yicha uslubiy ko‘rsatma. AKT va AHI. 74 bet, Toshkent, 2020.
5. A.A.Raximov. B.K.Yusupov, O.Sh.Abdiriziqov. “Dasturlash texnologiyalari” fanidan o‘quv qo‘llanma. AKT va AHI. 83 bet, Toshkent, 2020.

O‘QUV SAVOLLARI VA VAQT HISOB

№	O‘quvsavollari	Vaqt, min	Ko‘rgazmali qo‘llanma va O‘TV
1.	I. KIRISH QISMI – Dalolat qabul qilish, kursantlar davomatini, tashqi ko‘rinishini, mashg‘ulotga tayyorgarligini tekshirish; – Tezkor axborotni etkazish; – Mavzu va mashg‘ulot maqsadini e‘lon qilish; – Mavzu materialini bayon etishga kirishish.	20	
2.	II. ASOSIY QISM O‘quv savollari: 1. UDP bilan ishlash uchun rozetkalardan foydalanish.	120 30	komputer, elektron taqdimot,

	2. UdpClient. 3. Konsol UDP suhbat. 4. Translyatsiya.	30 30 30	
3.	III. YAKUNIY QISM –Mavzu yuzasidan qo'yilgan maqsadga mashg'ulot jarayonida erishilganligini tekshirish. –Savollarga javob berish. –Mustaqil tayyorgarlikka topshiriq berish. –Keyingi mashg'ulot mavzusi va o'tish joyini e'lon qilish.	20	

**O‘ZBEKISTON RESPUBLIKASI HARBIIY XAVFSIZLIK VA
MUDOFAA UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIIY
ALOQA INSTITUTI**

**AXBOROT TEXNOLOGIYALARI VA SUN’IY INTELLEKT
KAFEDRASI**



**«OBJEKTGA YO‘NALTIRILGAN DASTURLASH»
FANIDAN**

GLOSSARIY

GLOSSARIY

Termin va uning o‘zbek tilidagi sharhi	Termin va uning rus tilidagi sharhi	Termin va uning ingliz tilidagi sharhi
<i>C# - zamonaviy OBYEKTga yo‘naltirilgan dasturlash tili.</i>	<i>C# (произносится как "си шарп") — современный объектно-ориентированный и типобезопасный язык программирования.</i>	<i>C# (pronounced "C Sharp") is a modern object-oriented and type-safe programming language.</i>
WinForm (Windows form .Net) – bu UI framework bo‘lib kompyuter dasturlarini yaratishda foydalaniladi.	WinForm (Windows form .Net) - это инфраструктура пользовательского интерфейса, используемая для создания компьютерных программ.	WinForm (Windows form .Net) is a UI framework used to create computer programs.
Toolbox – Instrumentlar paneli.	Toolbox - (Панель инструментов).	Toolbox – is a (Toolbar).
Visial studio Microsoft kompaniyasi mahsulotlaridan biri bo‘lib, integratsiyalashgan dasturiy ta‘minot ishlab chiqish muhiti va bir qator boshqa vositalarni o‘z ichiga oladi.	Visial studio - линейка продуктов компании Microsoft, включающих интегрированную среду разработки программного обеспечения и ряд других инструментов.	Visial studio is a line of Microsoft products that includes an integrated software development environment and a number of other tools.
Axborot texnologiyasi – axborotni ishlash usullari va texnik vositalari tizimi.	Информационная технология - система технических средств и способов обработки информации.	Information technology - system of technical means and ways of information processing.
Tugma - ilova, dasturiy ta‘minot yoki veb-sahifaning muhim qismidir.	Кнопка — это неотъемлемая часть приложения, программного обеспечения или веб-страницы.	A Button is an essential part of an application, or software, or webpage.
CheckBox - fizik muhit, u orqali axborot bir qurilmadan ikkinchisiga uzatiladi.	CheckBox чтобы предоставить пользователю возможность, например true, false или yes/no. Элемент CheckBox управления может отображать изображение, текст или оба элемента управления.	CheckBox Use a CheckBox to give the user an option, such as true/false or yes/no. The CheckBox control can display an image or text or both.
CheckedList – 1. Axborotni uzatuvchi fizik muhit, apparat va ba’zi kanallarda dasturiy vositalar. 2. Signallarni manbadan qabul qiluvchiga uzatishni ta‘minlovchi texnik vositalar (peredatchik, aloqa liniyasi, priemnik, apparat va dasturiy vositalar) majmui.	CheckedList Этот элемент управления представляет список элементов, которые пользователь может перемещать с помощью клавиатуры или полосы прокрутки справа от элемента управления.	CheckedList This control presents a list of items that the user can navigate by using the keyboard or the scrollbar on the right side of the control.
ColorDialog – hisoblash tarmog‘ining ishgga	ColorDialog Для создания этого общего диалогового окна	ColorDialog The inherited member ShowDialog must

layoqatligini tiqlash uchun bajariladigan harakatlar majmui.	<i>необходимо вызвать унаследованный элемент ShowDialog .</i>	<i>be invoked to create this specific common dialog box.</i>
FontDialog – berilgan kollektiv foydalanuvchi tizimda ustuvor nomerli foydalanuvchi.	FontDialog Этот код требует, чтобы уже Form был создан с помощью кнопки и кнопки, помещенной на нее TextBox .	FontDialog The inherited member ShowDialog must be invoked to create this specific common dialog box. HookProc can be overridden to implement specific dialog box hook functionality.
GroupBox Windows Forms GroupBox boshqaruv elementlari boshqa boshqaruv elementlari uchun identifikatsiya qilinadigan guruhlashni ta’minlash uchun ishlatiladi.	GroupBox - Элементы управления GroupBox в Windows Forms используются для идентифицируемой группировки для других элементов управления.	GroupBox – Windows Forms GroupBox controls are used to provide an identifiable grouping for other controls.
ImageList – Windows Forms ImageList komponenti tasvirlarni saqlash uchun ishlatiladi, ular keyinchalik boshqaruv elementlari tomonidan ko‘rsatilishi mumkin. Tasvirlar ro‘yxati yagona, izchil tasvirlar katalogi uchun kod yozish imkonini beradi.	ImageList - Компонент ImageList в Windows Forms используется для хранения изображений, которые затем будут отображаться элементами управления. Список изображений позволяет написать код для создания единого согласованного каталога изображений.	ImageList The Windows Forms ImageList component is used to store images, which can then be displayed by controls. An image list allows you to write code for a single, consistent catalog of images.
Label – Windows Forms Label boshqaruv elementlari foydalanuvchi tomonidan tahrir qila olmaydigan matn yoki rasmlarni ko‘rsatish uchun ishlatiladi.	Label - Элементы управления Label Windows Forms используются для вывода текста, который недоступен для изменения пользователем.	Label - Windows Forms Label controls are used to display text or images that cannot be edited by the user.
LinkLabel – Windows Forms LinkLabel boshqaruvi Windows Forms ilovalariga veb uslubidagi havolalarni qo‘shish imkonini beradi. YorliqLinkLabel boshqaruvidan foydalanishingiz mumkin bo‘lgan hamma narsa uchun boshqaruvdan foydalanishingiz mumkin	LinkLabel - Элемент управления Windows Forms LinkLabel позволяет добавлять веб-ссылки в приложения Windows Forms. Элемент управления LinkLabel можно использовать для всего, для чего можно использовать элемент управления Label..	LinkLabel - The Windows Forms LinkLabel control enables you to add Web-style links to Windows Forms applications. You can use the LinkLabel control for everything that you can use the Label control for
HScrollBar Windows Forms aylantirish paneli boshqaruv elementlari dastur yoki boshqaruv ichida gorizontal yoki vertikal aylantirish orqali elementlarning uzun ro‘yxati yoki katta hajmdagi	HScrollBar Элементы управления Windows Forms в виде полос прокрутки используются для обеспечения удобного просмотра длинных списков элементов или большого объема данных с	HScrollBar Windows Forms scroll bar controls are used to provide easy navigation through a long list of items or a large amount of information by scrolling either

ma'lumotlar bo'ylab oson navigatsiyani ta'minlash uchun ishlatiladi.	помощью горизонтальной или вертикальной прокрутки окна приложения либо элемента управления.	horizontally or vertically within an application or control.
ListBox — Windows Forms ListBox boshqaruvi foydalanuvchi bir yoki bir nechtasini tanlashi mumkin bo'lgan elementlar ro'yxatini ko'rsatadi.	ListBox — Элемент управления ListBox в Windows Forms отображает список элементов, из которого пользователь может выбрать один элемент или несколько.	ListBox - A Windows Forms ListBox control displays a list of items from which the user can select one or more.
ComboBox ComboBox ListBox bilan birlashtirilgan matn maydonini ko'rsatadi, bu foydalanuvchiga ro'yxatdan elementlarni tanlash yoki yangi qiymat kiritish imkonini beradi.	ComboBox - Отображает ComboBox текстовое поле в сочетании с элементом ListBox, которое позволяет пользователю выбирать элементы из списка или вводить новое значение.	ComboBox - A ComboBox displays a text box combined with a ListBox, which enables the user to select items from the list or enter a new value.
ContextMenuStrip - ContextMenuStrip ContextMenu o'rnini egallaydi. ContextMenuStrip -ni istalgan boshqaruv bilan bog'lashingiz mumkin va sichqonchani o'ng tugmasi avtomatik ravishda yorliq menyusini ko'rsatadi.	ContextMenuStrip - ContextMenuStrip заменяет ContextMenu. Вы можете привязать ContextMenuStrip к любому элементу управления, и правая кнопка мыши автоматически отобразит контекстное меню.	ContextMenuStrip - ContextMenuStrip replaces ContextMenu. You can bind ContextMenuStrip to any control and the right mouse button will automatically show the shortcut menu
DataGridView – Boshqaruv DataGridView ma'lumotlarni jadval formatida ko'rsatishning kuchli va moslashuvchan usulini ta'minlaydi. Siz DataGridView kichik hajmdagi ma'lumotlarning faqat o'qish uchun mo'ljallangan ko'rinishlarini ko'rsatish uchun boshqaruvdan foydalanishingiz mumkin yoki uni juda katta hajmdagi ma'lumotlar to'plamining tahrirlanadigan ko'rinishlarini ko'rsatish uchun masshtablashingiz mumkin.	DataGridView : Элемент управления DataGridView предоставляет мощный и гибкий способ отображения данных в табличном формате. Элемент управления DataGridView можно использовать для отображения представлений небольшого объема данных только для чтения, либо можно масштабировать его для отображения редактируемого представления очень больших наборов данных.	DataGridView - The DataGridView control provides a powerful and flexible way to display data in a tabular format. You can use the DataGridView control to show read-only views of a small amount of data, or you can scale it to show editable views of very large sets of data.
FlowLayoutPanel Boshqaruv FlowLayoutPanelning tarkibini gorizontal yoki vertikal oqim yo'nalishida tartibga soladi. Uning mazmuni bir qatordan ikkinchisiga yoki bir ustundan ikkinchisiga o'ralishi mumkin..	FlowLayoutPanel - Элемент управления FlowLayoutPanel упорядочивает свое содержимое в горизонтальном или вертикальном направлении. Его содержимое может быть перенесено из одной строки в следующую или из одного столбца в следующий	FlowLayoutPanel - he FlowLayoutPanel control arranges its contents in a horizontal or vertical flow direction. Its contents can be wrapped from one row to the next, or from one column to the next.
DateTimePicker – Windows Forms	DateTimePicker – Элемент управления Windows Forms	DateTimePicker – The Windows Forms

DateTimePicker boshqaruvi foydalanuvchiga sanalar yoki vaqtlar ro‘yxatidan bitta elementni tanlash imkonini beradi. Sanani ifodalash uchun foydalanilganda, u ikki qismdan iborat bo‘ladi: matnda sanasi ko‘rsatilgan ochiladigan ro‘yxat va ro‘yxat yonidagi pastga o‘qni bosganingizda paydo bo‘ladigan panjara.	DateTimePicker позволяет пользователю выбрать один элемент из списка дат или времени. Если он используется для представления даты, она отображается в двух частях: раскрывающийся список с датой, представленной в тексте, и сетка, которая отображается при щелчке стрелки вниз рядом с списком.	DateTimePicker control allows the user to select a single item from a list of dates or times. When used to represent a date, it appears in two parts: a drop-down list with a date represented in text, and a grid that appears when you click on the down-arrow next to the list.
Listview – Windows Forms ListView boshqaruvi piktogramma bilan elementlar ro‘yxatini ko‘rsatadi. Windows Explorer-ning o‘ng paneli kabi foydalanuvchi interfeysini yaratish uchun ro‘yxat ko‘rinishidan foydalanishingiz mumkin.	Listview - Элемент управления ListView Windows Forms отображает список элементов со значками. Представление списка можно использовать для создания пользовательского интерфейса, аналогичного правой области окна проводника.	Listview - The Windows Forms ListView control displays a list of items with icons. You can use a list view to create a user interface like the right pane of Windows Explorer.
MenuStrip – ContextMenuStrip boshqaruvi boshqaruv elementi bilan bog‘langan yorliq menyusini taqdim etadi .	MenuStrip - Элемент управления ContextMenuStrip предоставляет контекстное меню, связываемое с каким-либо элементом управления.	MenuStrip - The ContextMenuStrip control provides a shortcut menu that you associate with a control.
NotifyIcon – Windows Forms NotifyIcon komponenti fonda ishlaydigan va foydalanuvchi interfeyslariga ega bo‘lmagan jarayonlar uchun vazifalar panelining holat bildirishnomasi sohasida piktogrammalarni ko‘rsatadi.	NotifyIcon - Компонент Windows Forms NotifyIcon отображает значки в области уведомлений о состоянии на панели задач для процессов, которые выполняются в фоновом режиме и не имеют пользовательских интерфейсов.	NotifyIcon - he Windows Forms NotifyIcon component displays icons in the status notification area of the taskbar for processes that run in the background and would not otherwise have user interfaces.
Panel – Windows Forms Panel boshqaruv elementlari boshqa boshqaruv elementlari uchun identifikatsiya qilinadigan guruhlashni ta‘minlash uchun ishlatiladi.	Panel — Элементы Panel в Windows Forms управления используются для идентифицируемой группировки для других элементов управления.	Panel - Windows Forms Panel controls are used to provide an identifiable grouping for other controls.
PictureBox - Windows Forms PictureBox boshqaruvi grafiklarni bitmap, GIF, JPEG, meta fayl yoki piktogramma formatida ko‘rsatish uchun ishlatiladi.	PictureBox - Элемент управления Windows Forms PictureBox используется для отображения графических объектов следующих форматов: растровые изображения, GIF, JPEG, метафайл или значок.	PictureBox - The Windows Forms PictureBox control is used to display graphics in bitmap, GIF, JPEG, metafile, or icon format.
ProgressBar – Windows Forms ProgressBar boshqaruvi gorizontal chiziqda joylashtirilgan to‘rtburchaklarning tegishli	ProgressBar - Элемент управления ProgressBar в Windows Forms указывает ход выполнения действия, отображая соответствующее	ProgressBar - The Windows Forms ProgressBar control indicates the progress of an action by displaying an

sonini ko‘rsatish orqali harakatning borishini ko‘rsatadi. Harakat tugagach, bar to‘ldiriladi.	количество прямоугольников, расположенных в виде горизонтальной полосы. После завершения действия полоса будет заполнена.	appropriate number of rectangles arranged in a horizontal bar. When the action is complete, the bar is filled.
Splitter – monitorlarni dublikat qilish uchun foydalaniladi	Splitter используется для дублирования мониторов.	Splitter - used to duplicate monitors
StatusStrip – Windows Forms StatusStrip boshqaruvi shakllarda odatda oynaning pastki qismida ko‘rsatiladigan maydon sifatida ishlatiladi, unda dastur har xil turdagi holat ma’lumotlarini ko‘rsatishi mumkin..	StatusStrip Элемент управления Windows Forms StatusStrip используется в формах в качестве области, обычно отображаемой в нижней части окна, в которой выводятся различные сведения о состоянии приложения..	StatusStrip The Windows Forms StatusStrip control is used on forms as an area, usually displayed at the bottom of a window, in which an application can display various kinds of status information..
TabControl -Windows Forms TabControl bir nechta yorliqlarni ko‘rsatadi, masalan, daftardagi ajratgichlar yoki fayl kabinetidagi papkalar to‘plamidagi teglar. Yorliqlarda rasmlar va boshqa boshqaruv elementlari bo‘lishi mumkin. TabControl Mulk sahifalarini yaratish uchun foydalaning .	TabControl - Элемент управления TabControl Windows Forms используется для отображения нескольких вкладок, аналогичных разделителям в записной книжке или меткам в наборе папок в картотеке. Вкладки могут содержать изображения и другие элементы управления. Используйте элемент управления TabControl для создания страниц свойств.	TabControl - The Windows Forms TabControl displays multiple tabs, like dividers in a notebook or labels in a set of folders in a filing cabinet. The tabs can contain pictures and other controls. Use the TabControl to create property pages.
TableLayoutPanel – TableLayoutPanel boshqaruvi o‘z tarkibini to‘rga joylashtiradi. Tartib ham loyihalash vaqtida, ham ish vaqtida bajarilganligi sababli, dastur muhiti o‘zgarishi bilan u dinamik ravishda o‘zgarishi mumkin.	TableLayoutPanel - Элемент управления TableLayoutPanel упорядочивает содержимое в сетке. Так как макет строится как во время разработки, так и во время выполнения, его можно изменять динамически по мере изменения среды приложения.	TableLayoutPanel - The TableLayoutPanel control arranges its contents in a grid. Because the layout is performed both at design time and run time, it can change dynamically as the application environment changes.
TextBox – Windows Forms matn maydonlari foydalanuvchidan ma’lumot olish yoki matnni ko‘rsatish uchun ishlatiladi. Boshqaruv TextBox odatda tahrirlanadigan matn uchun ishlatiladi, lekin uni faqat o‘qish uchun ham qilish mumkin.	TextBox - Текстовые поля Windows Forms используются для получения входных данных от пользователя или для отображения текста. Элемент управления TextBox обычно используется для редактируемого текста, хотя для него также можно сделать настройки разрешения только для чтения.	TextBox - Windows Forms text boxes are used to get input from the user or to display text. The TextBox control is generally used for editable text, although it can also be made read-only.
Timer - Windows Forms Timer - bu hodisani muntazam ravishda ko‘taradigan komponent. Ushbu komponent	Timer - Компонент Windows Forms Timer вызывает событие через определенные интервалы времени. Этот компонент	Timer - The Windows Forms Timer is a component that raises an event at regular intervals.

Windows Forms muhiti uchun mo‘ljallangan.	предназначен для работы в среде Windows Forms.	This component is designed for a Windows Forms environment.
ToolStrip - ToolStrip boshqaruvlari - bu Windows Forms ilovalarida menyular, boshqaruv elementlari va foydalanuvchi boshqaruvlarini joylashtirishi mumkin bo‘lgan asboblarning paneli.	ToolStrip - Элементы управления ToolStrip — это панели инструментов, на которых можно разместить меню, элементы управления и пользовательские элементы управления в приложениях Windows Forms.	ToolStrip - ToolStrip controls are toolbars that can host menus, controls, and user controls in your Windows Forms applications.
ToolStripContainer - ToolStrip boshqaruvlari ToolStripContainer yordamida o‘rnatilgan raftingni (o‘rnatilganda asbob maydoni ichida gorizontaal yoki vertikal bo‘shliqni taqsimlash) xususiyatiga ega .	ToolStripContainer — Элементы управления ToolStrip обладают возможностью встроенного нависания (совместного использования горизонтального или вертикального пространства в области инструментов при прикреплении) благодаря использованию ToolStripContainer.	ToolStripContainer - ToolStrip controls feature built-in rafting (sharing of horizontal or vertical space within the tool area when docked) by using the ToolStripContainer.
ToolTip — Windows Forms ToolTip komponenti foydalanuvchi boshqaruv elementlariga ishora qilganda matnni ko‘rsatadi. Asboblarning maslahati har qanday boshqaruv bilan bog‘lanishi mumkin.	ToolTip - Компонент ToolTip в Windows Forms отображает текст при наведении указателя мыши на элементы управления. Компонент ToolTip можно связать с любым элементом управления.	ToolTip The Windows Forms ToolTip component displays text when the user points at controls.
TrackBar - Windows Forms TrackBar boshqaruvi (ba‘zan "slayder" boshqaruvi deb ham ataladi) katta hajmdagi ma‘lumotlar bo‘ylab harakatlanish yoki raqamli sozlamalarni vizual sozlash uchun ishlatiladi.	TrackBar — Элемент управления TrackBar в Windows Forms (иногда называемый элементом управления «ползунок») используется для перемещения по большому объему информации или для визуальной настройки числовых параметров.	TrackBar — The Windows Forms TrackBar control (also sometimes called a "slider" control) is used for navigating through a large amount of information or for visually adjusting a numeric setting.
TreeView – Windows Forms TreeView boshqaruvi Windows operatsion tizimlarida Windows Explorer funksiyasining chap panelida fayllar va papkalarni ko‘rsatish usuli kabi tugunlar ierarxiasini ko‘rsatadi.	TreeView - Элемент управления TreeView в Windows Forms выводит на экран иерархию узлов аналогично отображению файлов и папок на левой панели проводника в операционных системах Windows.	TreeView The Windows Forms TreeView control displays a hierarchy of nodes, like the way files and folders are displayed in the left pane of the Windows Explorer feature in Windows operating systems.
OpenFileDialog Windows Forms OpenFileDialog komponenti oldindan tuzilgan	OpenFileDialog - Компонент Windows Forms OpenFileDialog является стандартным	OpenFileDialog - The Windows Forms OpenFileDialog

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

muloqot oynasidir. Bu Windows operatsion tizimi tomonidan ochilgan bir xil Ochiq Fayl dialog oynasi. U CommonDialog sinfidan meros .	диалоговым окном. Он аналогичен диалоговому окну Открыть файл операционной системы Windows. Он наследуется от класса CommonDialog.	component is a pre-configured dialog box. It is the same Open File dialog box exposed by the Windows operating system. It inherits from the CommonDialog class.
---	---	--

**O‘ZBEKISTON RESPUBLIKASI HARBIIY XAVFSIZLI VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIIY
ALOQA INSTITUTI**

**AXBOROT TEXNOLOGIYALARI VA SUN‘IY INTELLEKT
KAFEDRASI**



**“OBJEKTGA YO‘NALTIRILGAN DASTURLASH”
FANIDAN**

**TARQATMA MATERIALLAR
TO‘PLAMI (1-ILOVA)**

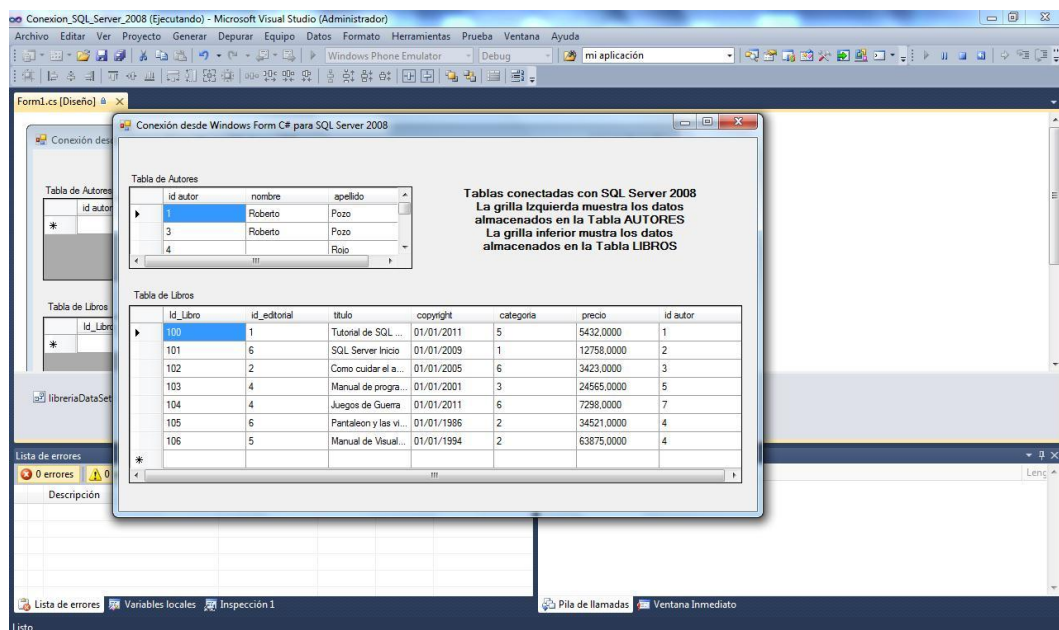
Toshkent – 2025

OBJEKTGA YO‘NALTIRILGAN DASTURLASH

```
Form1.cs [Design]
HelloWorld
HelloWorld.Form1

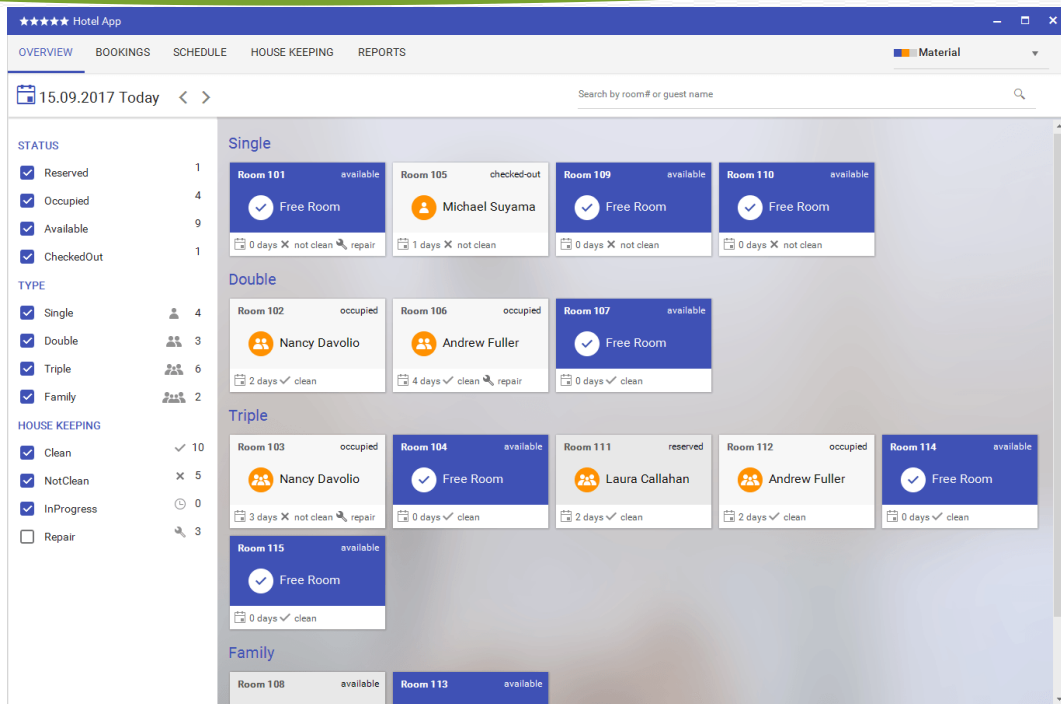
1 using System;
2 using System.Windows.Forms;
3
4 namespace HelloWorld
5 {
6     public partial class Form1 : Form
7     {
8         public Form1()
9         {
10             InitializeComponent();
11         }
12
13         private void btnClickThis_Click(object sender, EventArgs e)
14         {
15             lblHelloWorld.Text = "Hello World!";
16         }
17     }
18 }
19
```

Winformda kod qismini o‘zgartirish

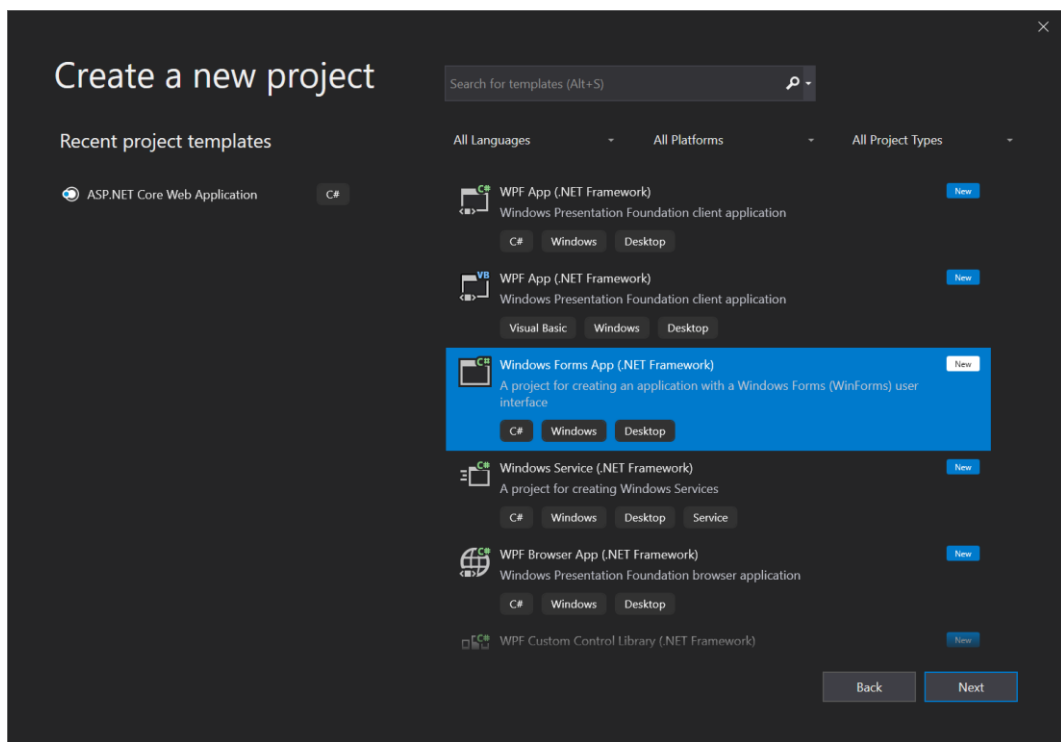


Winfromda quyidagi formani yarating

OBYEKTGA YO‘NALTIRILGAN DASTURLASH



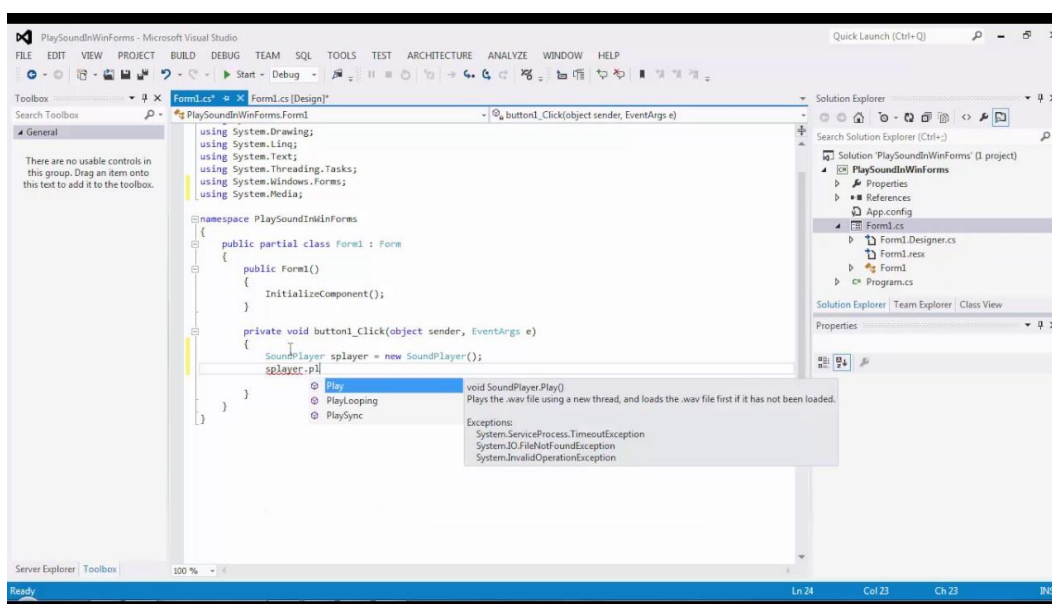
Admin oynasini yaratish



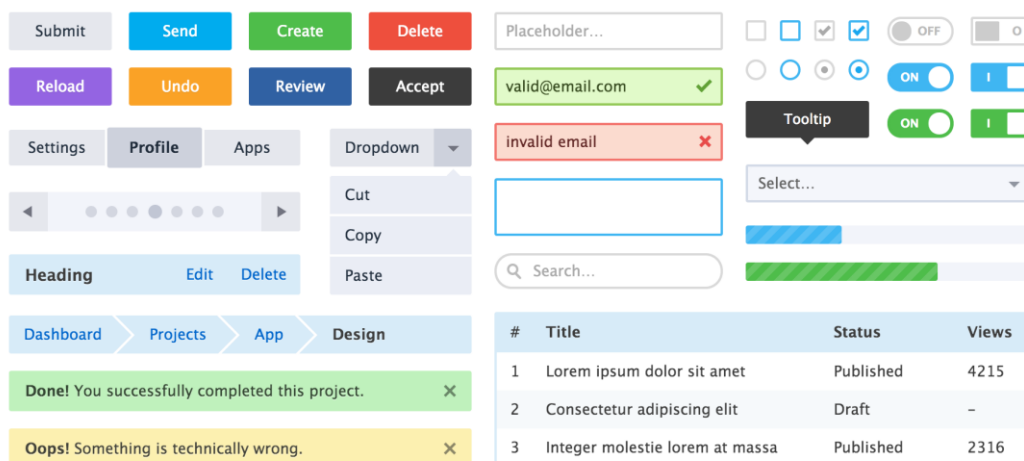
Winformda yangi loyiha yaratish



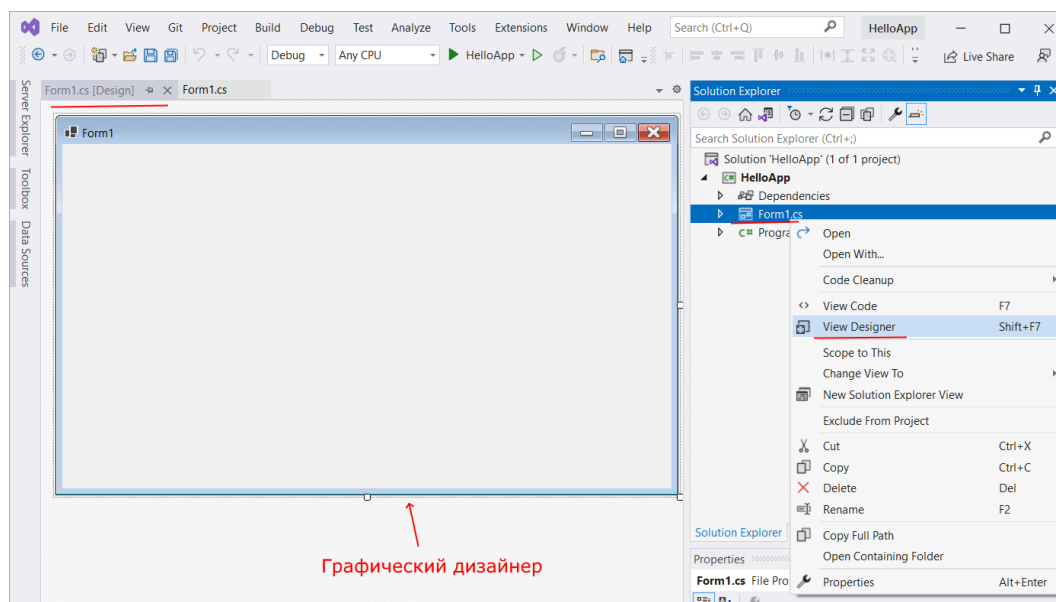
Admin oynasini yaratish



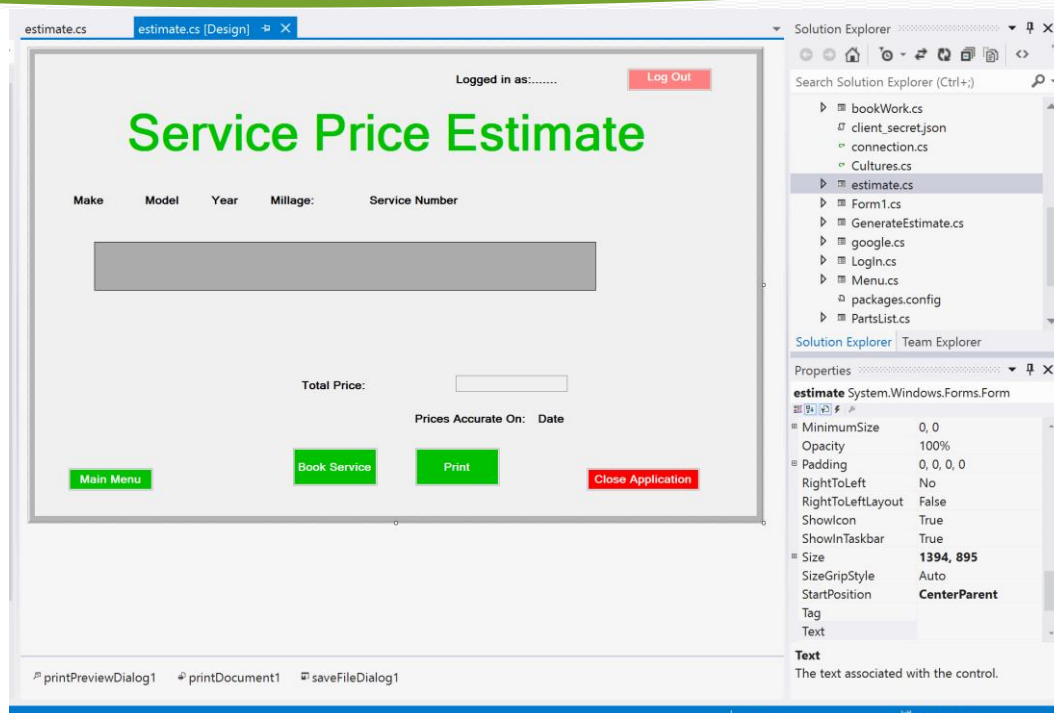
Musika qo‘yish usuli



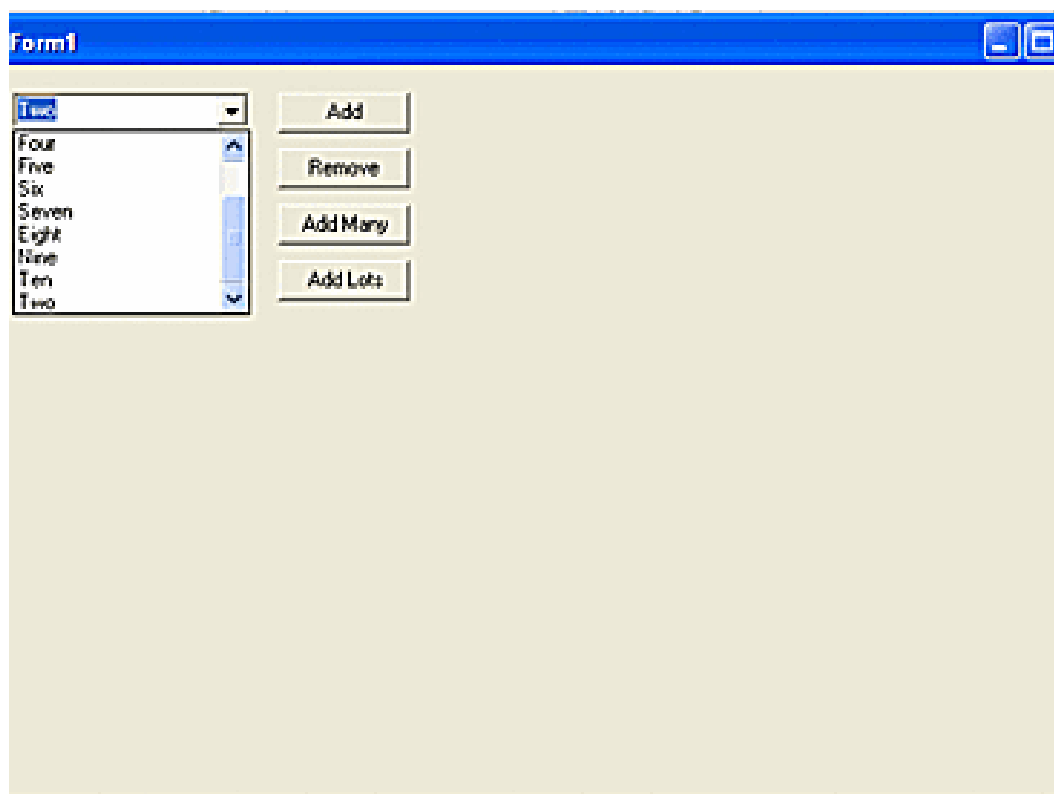
Componentalar turlari va ularning dizaynlari



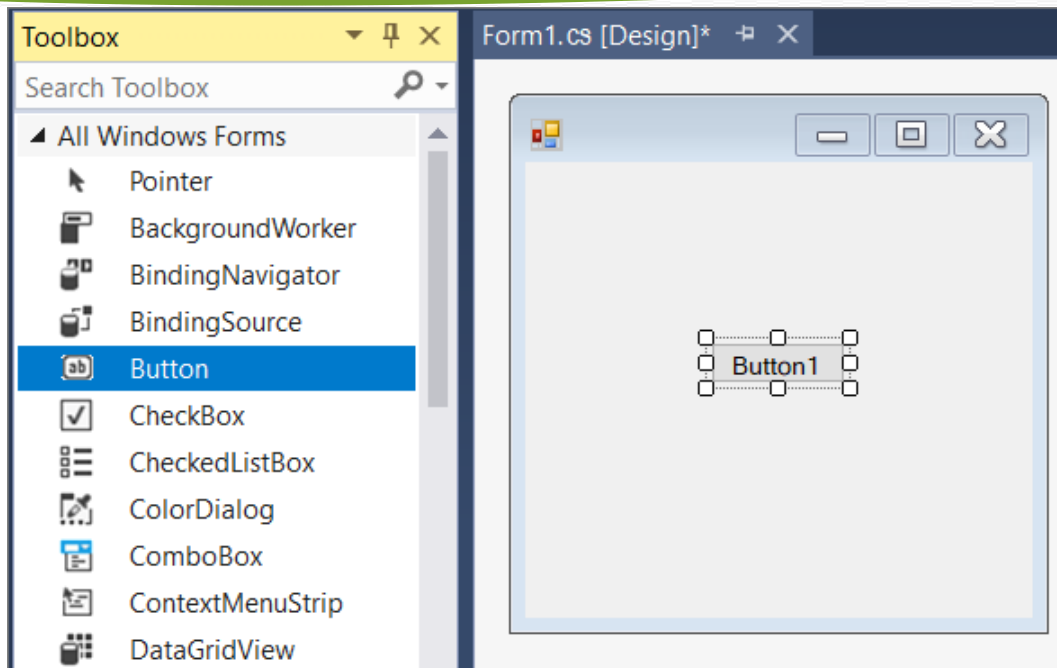
Winformda 1- loyiha yaratish



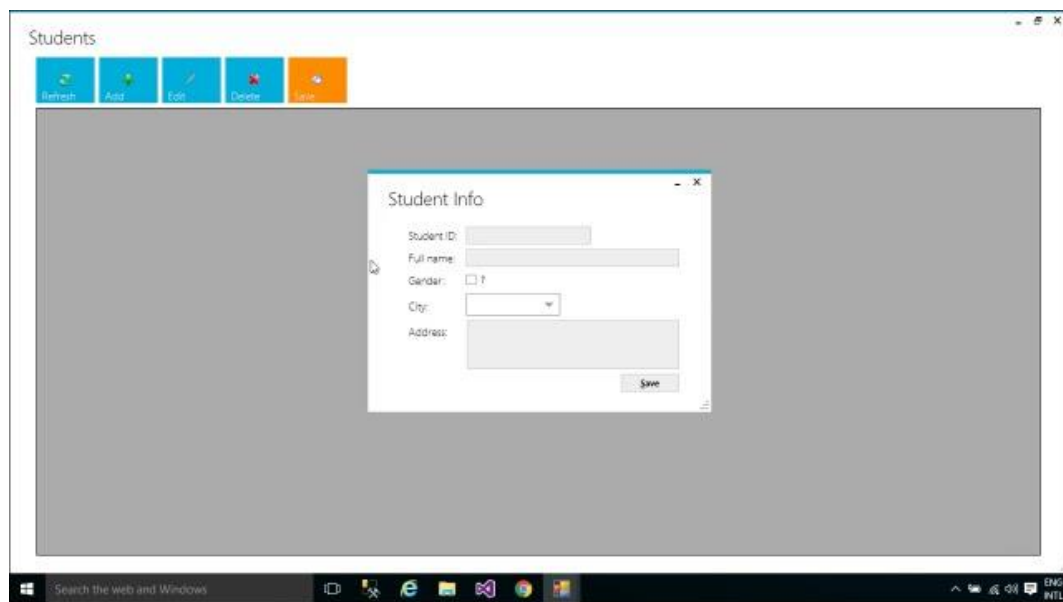
Shakllar o‘lchamlari



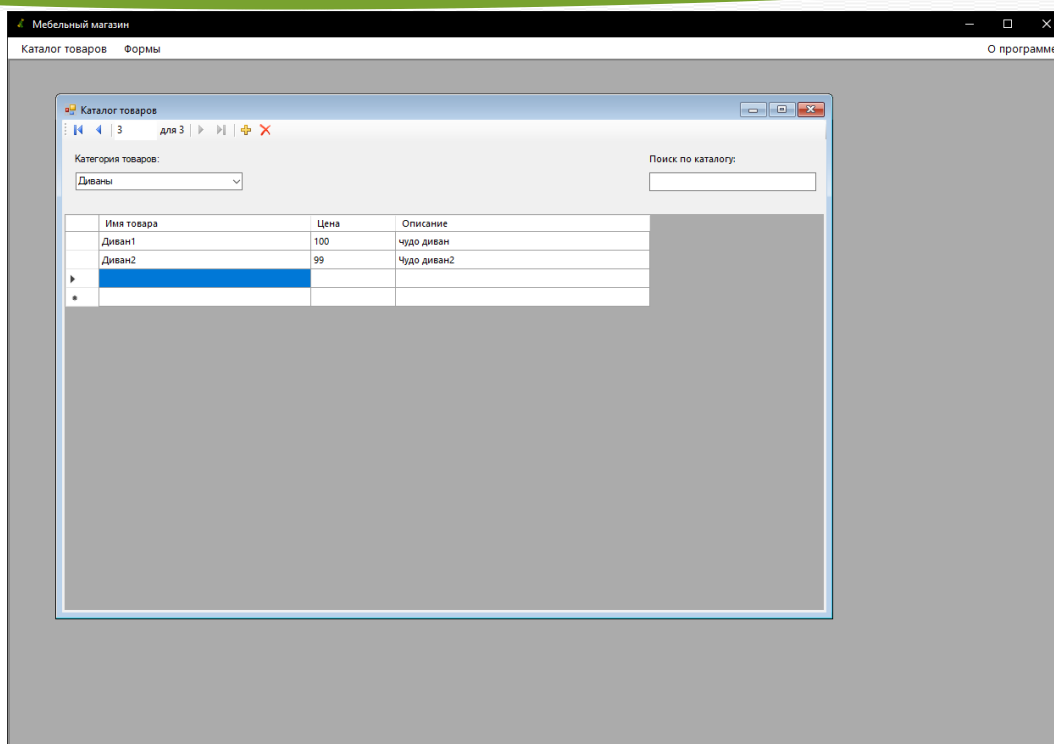
TreeView elementi



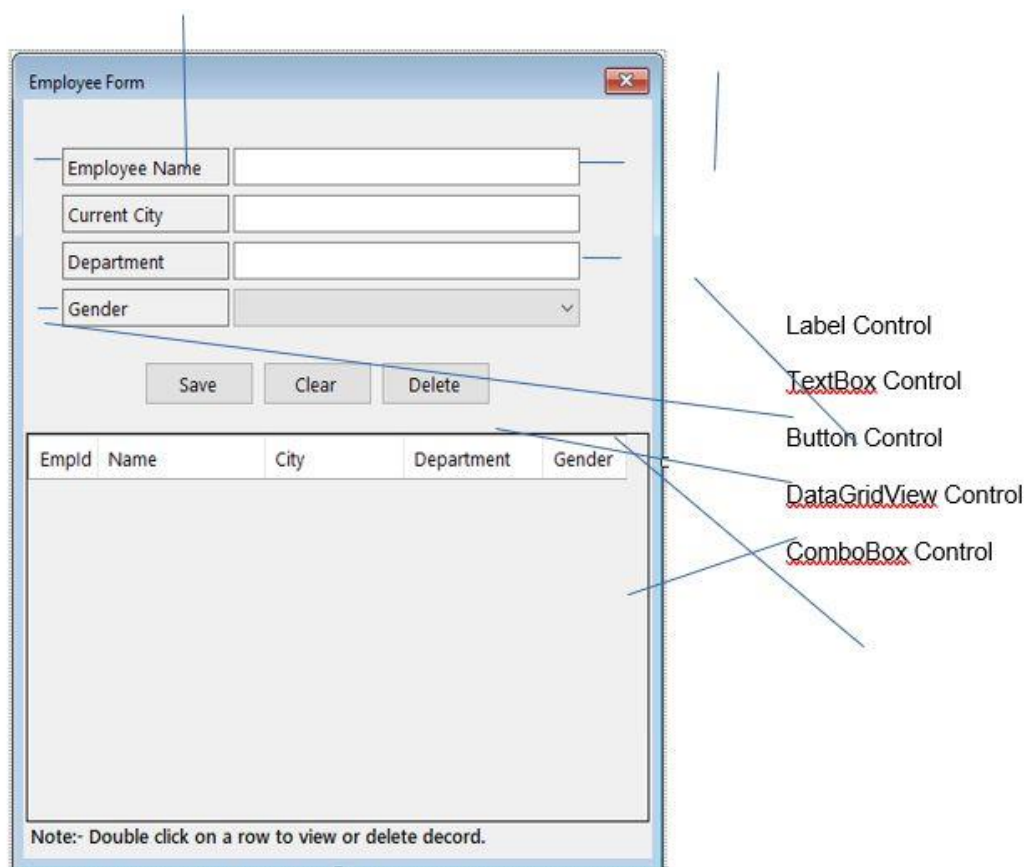
Button komponentasi



Winformda UI Metro kutubxonasi



DataGridView orqali dizayn yaratish



Forma yaratish uchun shablon

CRUD Web Service Call Function In Windows Applications

Full name:

Amount:

Note:

	Id	FullName	Amount	Note	PaymentDate
▶	1	Lucy	12.50	debt	9/18/2020 2:17 ...
	2	Tom	3.50	debt	9/18/2020 2:18 ...
	3	John	2.50	debt	9/18/2020 2:20 ...

Crud amalini bajarish

Form1

STUDENT INFO

Student ID:

Name:

Surname:

Sex: ☐ Female ☐ Male

Date of Birth: 09 Ekim 2013 Çarşamba

Phone Number:

E-mail:

Class:

Year of Join:

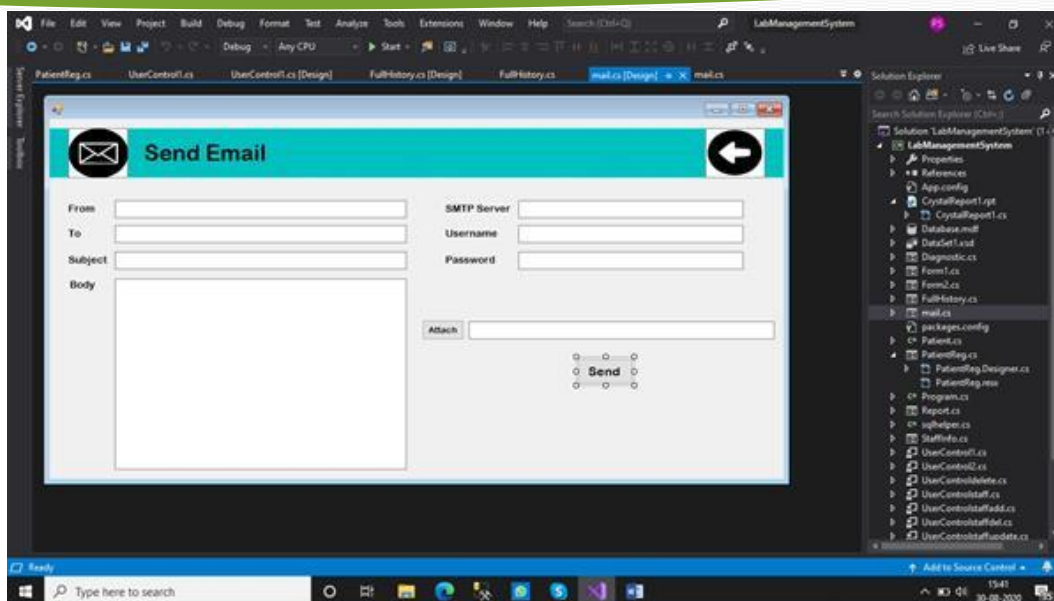
Advisor:

Attendance Status:

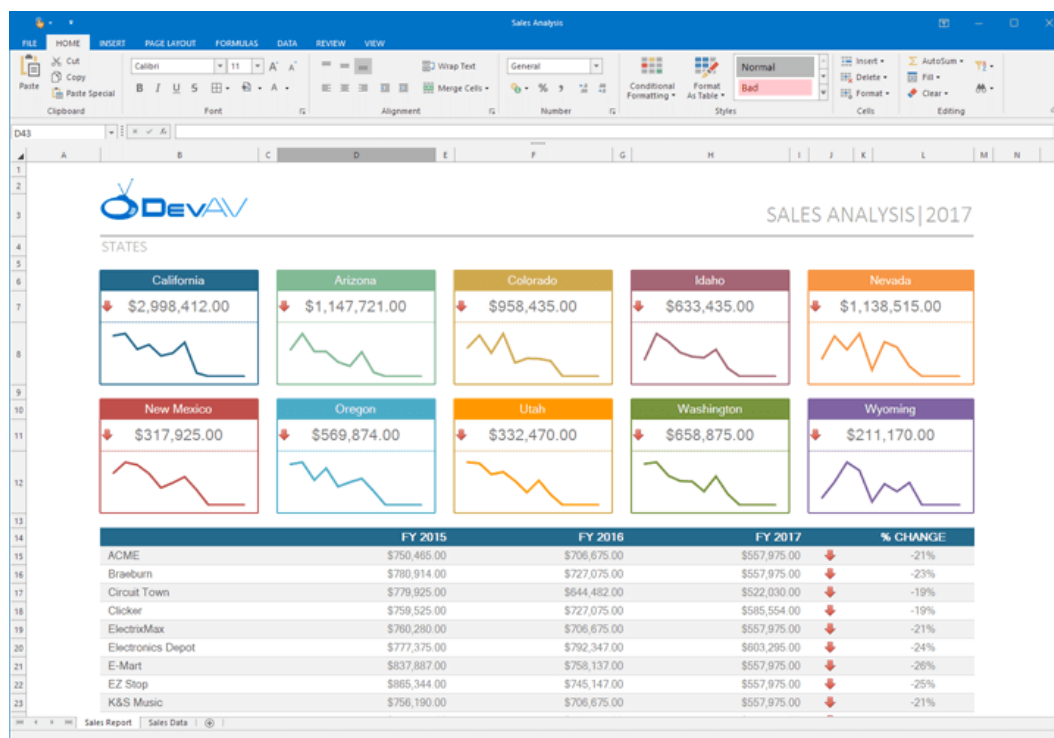
Division:

Sub Division:

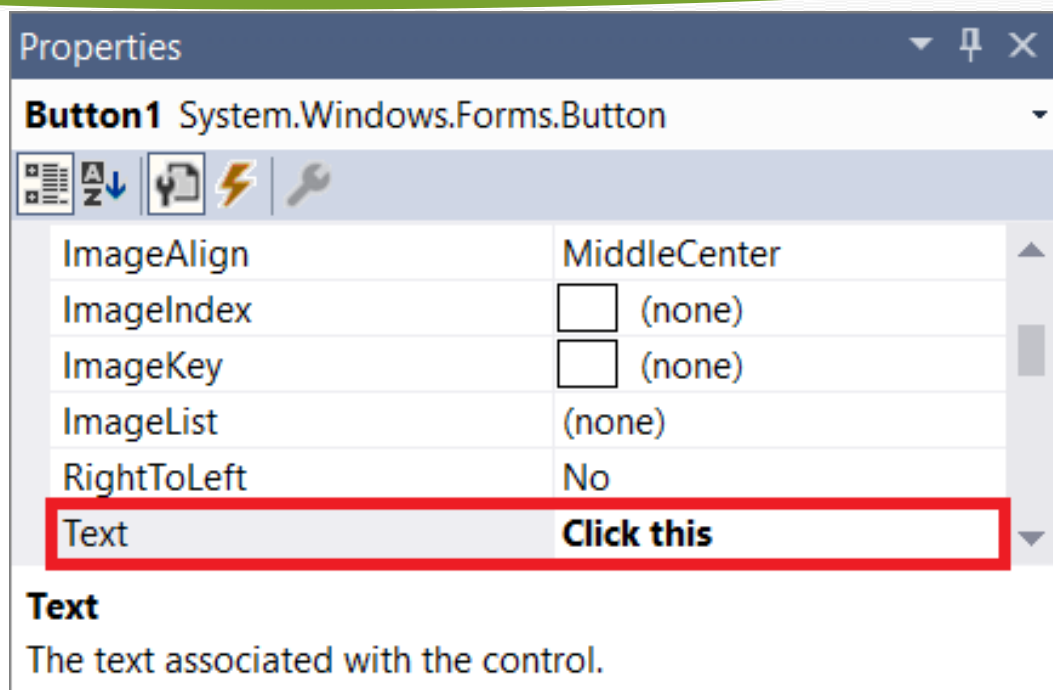
Student haqidagi malumotlarni yaratish



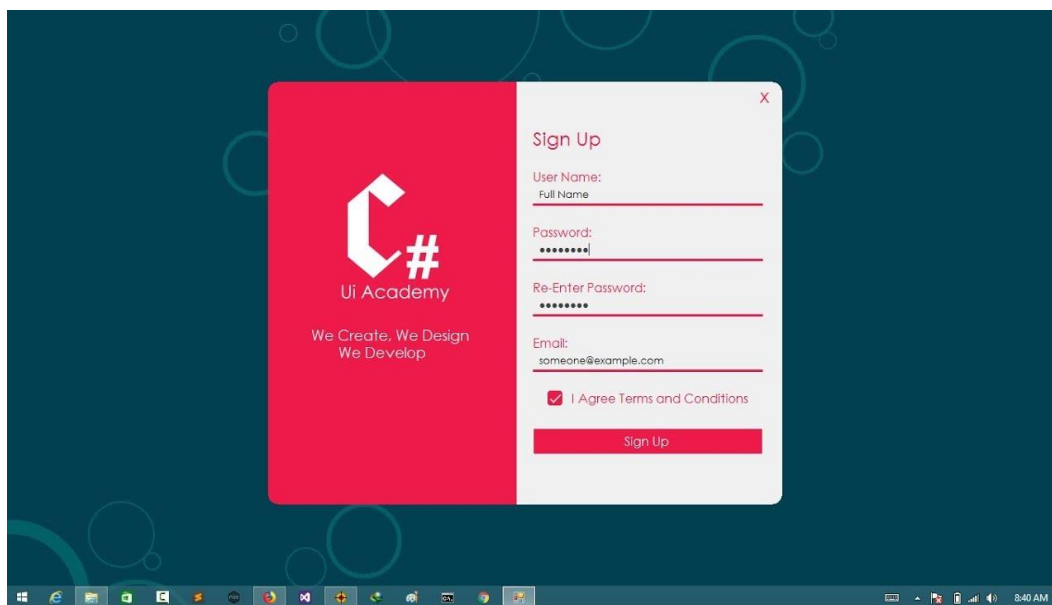
Emailga xat yuborish uchun yaratilgan shakl shabloni



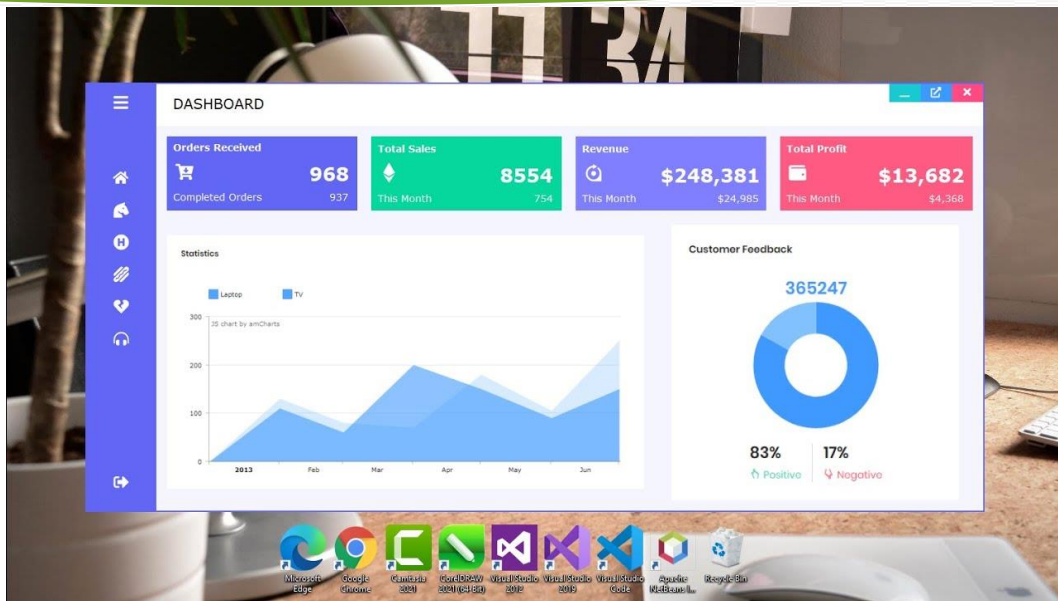
.NET 6 da UI Elementlardan foydalanish



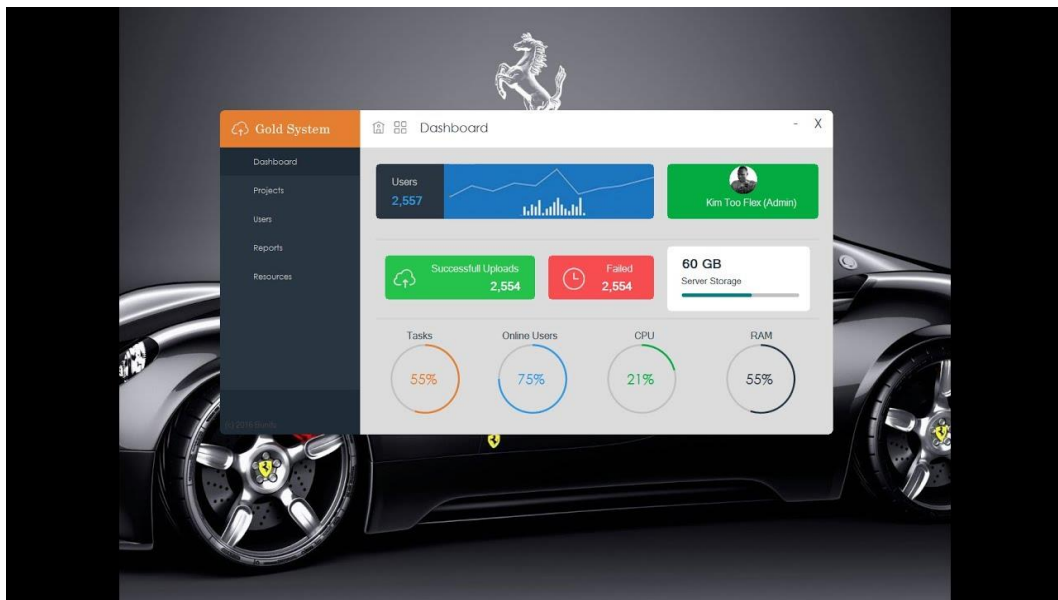
Knopkaning text xususiyati



Kalendar formasini yaratish

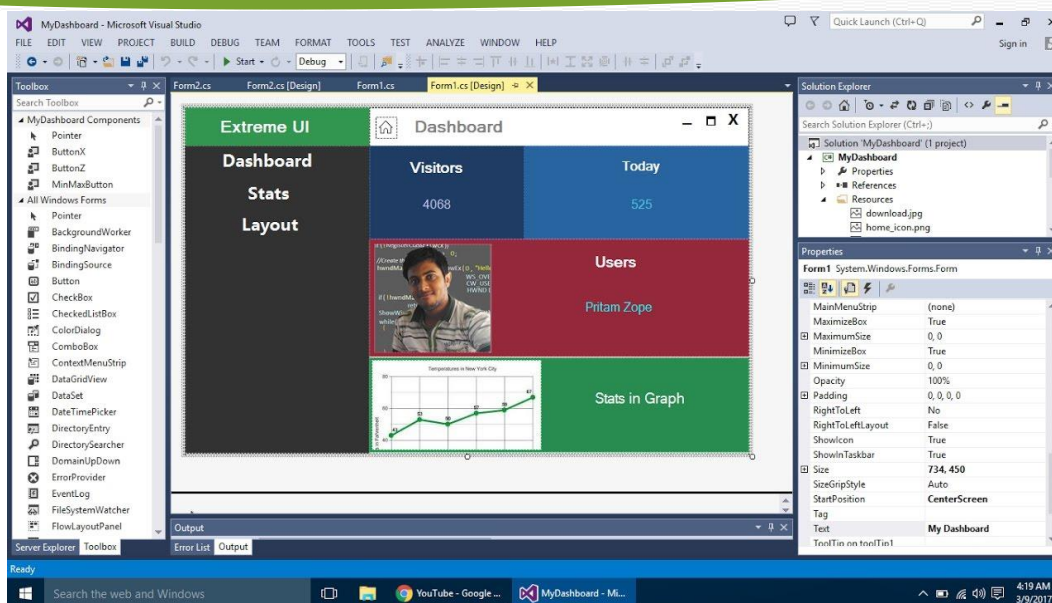


Formalarni yaratish uchun shablon

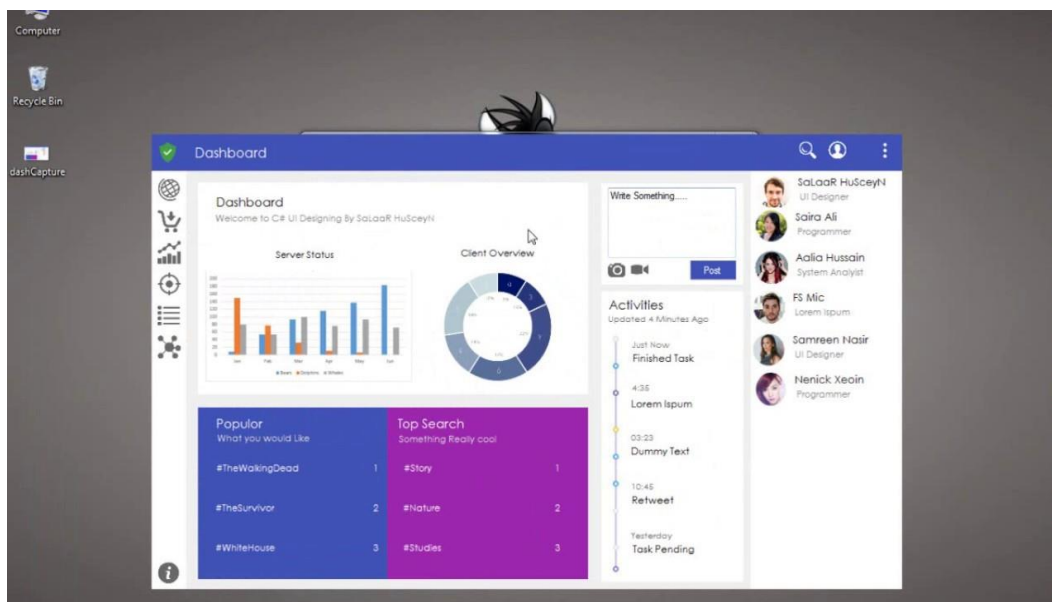


Formalarni yaratish uchun shablon

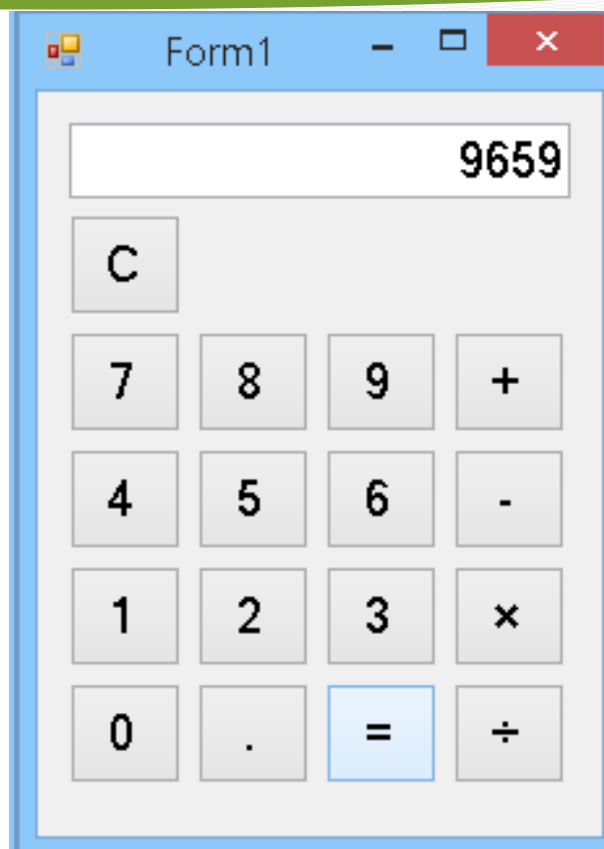
OBJEKTGA YO‘NALTIRILGAN DASTURLASH



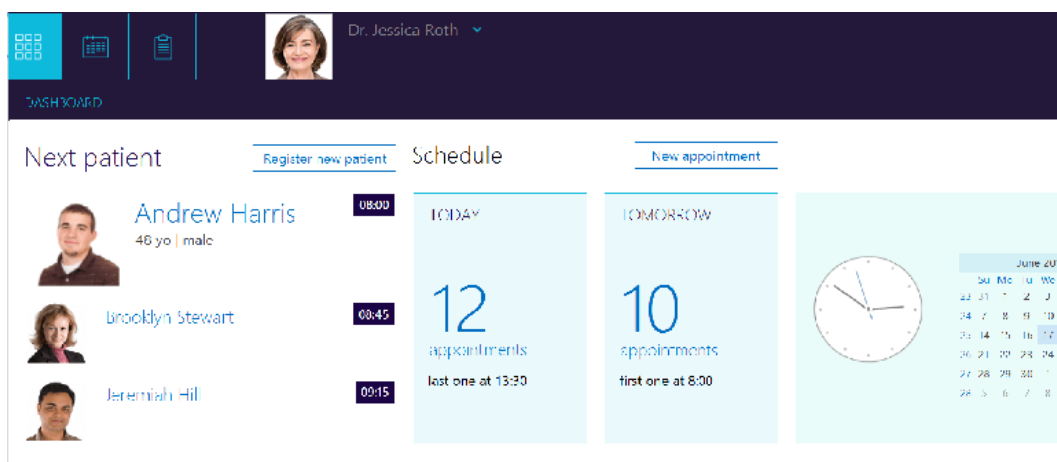
Formalarni yaratish uchun shablon



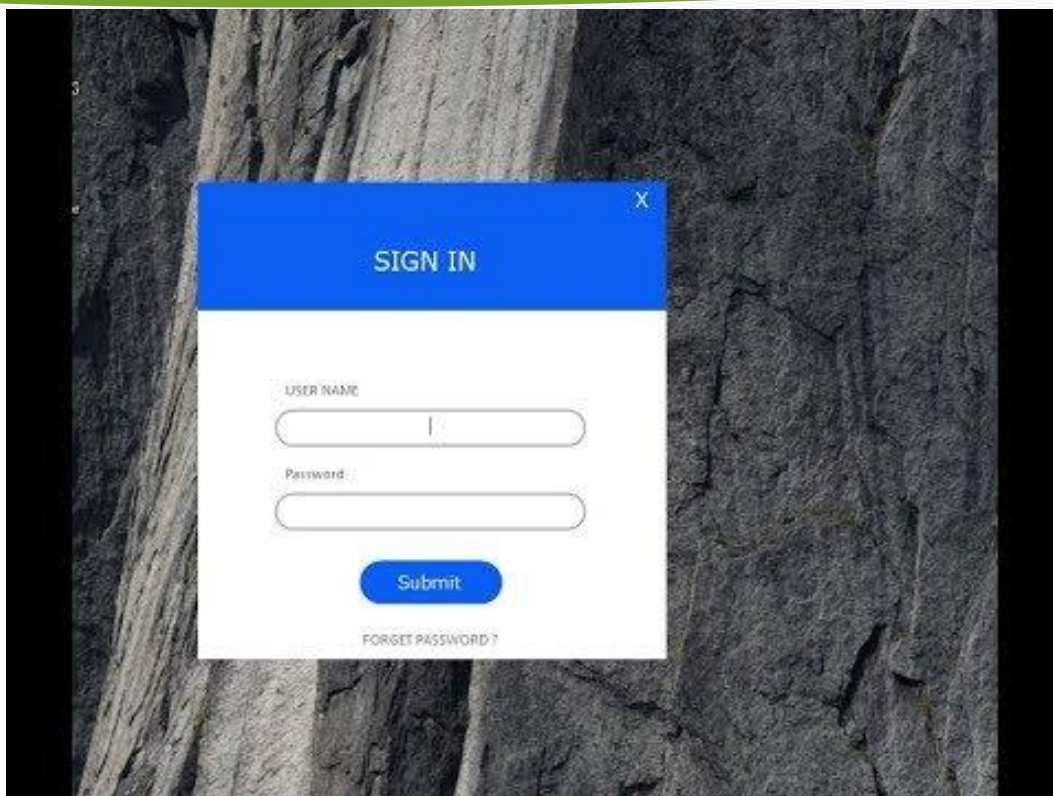
Formalarni yaratish uchun shablon



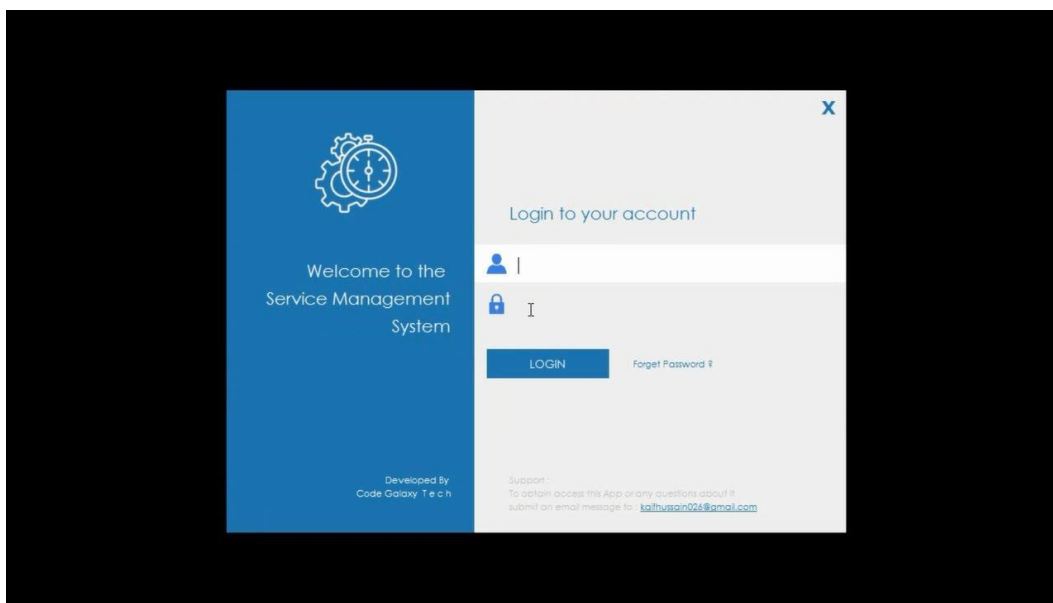
Kalkulator yasash



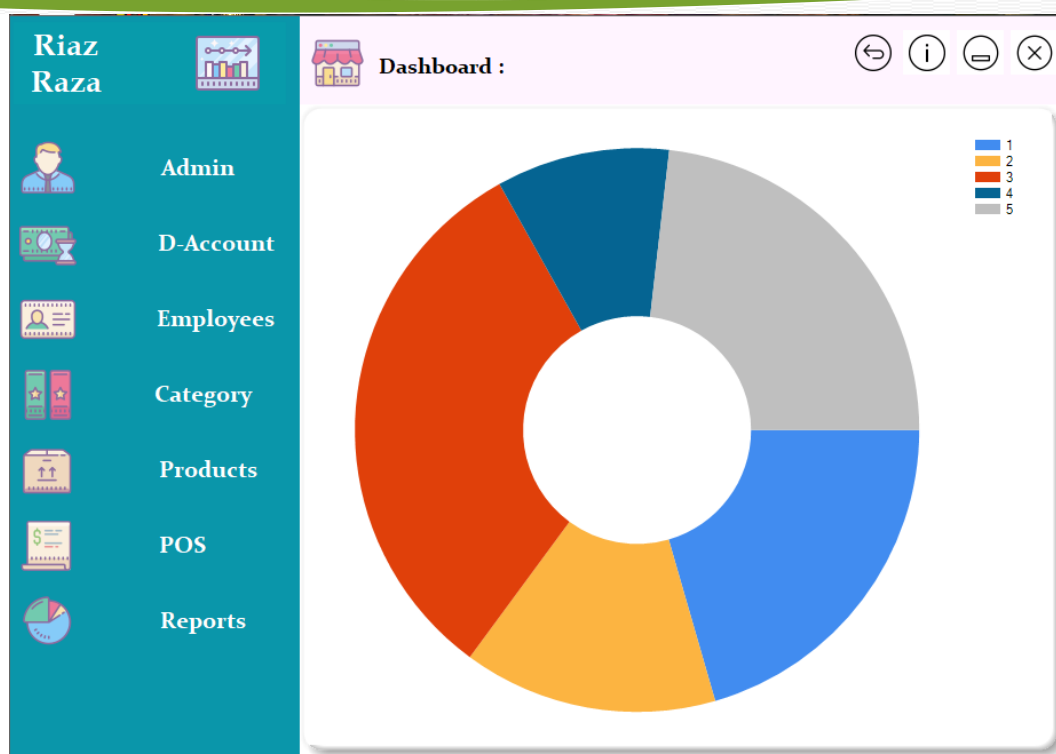
Formalarni yaratish uchun shablon



Login formasini yaratish uchun shablon



Login formasini yaratish uchun shablon



Admin formasini yaratish uchun shablon

**O‘ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEKNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEKNOLOGIYALARI VA SUN’IY INTELLEKT”
KAFEDRASI**



“OBYEKTGA YO‘NALTIRILGAN DASTURLASH” FANIDAN

**FAN BO‘YICHA YAKUNIY NAZORAT
QABUL QILISH UCHUN USLUBIY
ISHLANMA (2-ILOVA)**

Toshkent 2025 y.

“AXBOROT TEXNOLOGIYALARI” KAFEDRASI

“Obyektga yo‘naltirilgan dasturlash” fanidan yakuniy nazorat qabul qilish uchun

USLUBIY ISHLANMA

O‘quv-tarbiyaviy maqsadi: Kursantlarning “OBYEKTga yo‘naltirilgan dasturlash” fanidan o‘quv dasturlariga ko‘ra o‘tilgan mavzular va mashg‘ulotlar bo‘yicha olgan nazariy bilim va amaliy ko‘nikmalarini tekshirish, OBYEKTga yo‘naltirilgan dasturlashni o‘rganganligi bo‘yicha amaliy bilimlarini mustahkamlash.

O‘tkazish joyi: 138, 437 -o‘quv sinfi

O‘tkazish usuli: og‘zaki-amaliy

Vaqt: 6 o‘quv soati (240 daqiqa)

O‘quv guruhleri	O‘tkaziladigan sana	O‘tkaziladigan vaqti

“OBYEKTga yo‘naltirilgan dasturlash” o‘quv fanidan mavzular rejasiga asosan 73 ta savolni tashkil etadi

1. Windows Forms haqida tushuncha
2. Grafik dastur yaratish
3. Dasturni ishga tushirish
4. Shakl asoslari va shakllarning asosiy xususiyatlari
5. Dasturiy xususiyatlarni sozlash
6. Shaklning o‘lchamlarini sozlash
7. Shaklning boshlang‘ich pozitsiyasi
8. Shaklning rang va fonni
9. Windows formda hodisalar. Shakl hodisalari
10. To‘rtburchaklar bo‘lmagan shakllarni yaratish. Shaklni yopish
11. Elementlarni dinamik qo‘shish
12. GroupBox, Panel va FlowLayoutPanel elementlari
13. TableLayoutPanel elementi
14. Joylashuv
15. O‘lchamlarni sozlash
16. Anchor xususiyatlari
17. Dock xususiyati
18. TabControl

- 19.Koddagi yorliqlarni boshqarish
- 20.SplitContainer
- 21.Tugmani bezatish
- 22.Tugma ustidagi rasm
- 23.Standart tugmalar
- 24.Label
- 25.LinkLabel
- 26.LinkClicked
- 27.Avtomatik to‘ldirilgan matnli maydon
- 28.So‘zlar bo‘yicha ko‘chirish
- 29.Parol kiritish, TextChanged hodisasi
- 30.ToolStrip asboblari paneli.
- 31.MaskedTextBox elementi
- 32.RadioButton elementi
- 33.CheckBox elementi
- 34.ListBox -dagi dasturiy elementlar
- 35.ListBoxda SelectedIndexChanged hodisasi
- 36.ComboBox tashqi ko‘rinishini sozlash
- 37.ComboBoxda SelectedIndexChanged hodisasi
- 38.LinkLabel
- 39.LinkClicked
- 40.DataSource ma’lumotlar manbai
- 41.DisplayMember: ob’ektning xususiyati
- 42.ValueMember: ob’ektning xususiyati
- 43.Items xususiyatlari
- 44.SetItemChecked va SetItemCheckState
- 45.NumericUpDown hodisasi
- 46.DomainUpDown hodisasi
- 47.ImageList elementi
- 48.ListViewItem hodisasi
- 49.Element tasvirlari
- 50.Ko‘rsatish turlari
- 51.ListView. Amaliyot
- 52.Dasturiy tugunlarni boshqarish
- 53.Tugunlarni yashirish va ochish
- 54.CheckBox va tasvirni qo‘shish
- 55.TrackBar elementi
- 56.Timer elementi
- 57.ProgressBar elementidagi indikator hodisasi

- 58.DateTimePicker elementi
- 59.MonthCalendar elementi
- 60.ImageLocation, IntialImage xususiyatlari
- 61.Rasm o‘lchami
- 62.WebBrowser
- 63.NotifyIcon
- 64.MessageBox xabarlar oynasi
- 65.OpenFileDialog funksiyasi bilan ishlash
- 66.SaveFileDialog funksiyasi bilan ishlash
- 67.FontDialog va ColorDialog funksiya bilan ishlash
- 68.BlinkRate: xato belgisi
- 69.BlinkStyle: xato belgisi
- 70.ToolStrip asboblari paneli
- 71.MenuStrip yaratish
- 72.StatusStrip qator holati
- 73.ContextMenuStrip kontekst menyusi

O‘quv-moddiy ta‘minoti: ruxsat etilgan stendlar, OBYEKTga yo‘naltirilgan dasturlash vositalari va adabiyotlar; doska, bo‘r, ko‘rsatkich (*rus. ukazka*); yakuniy nazorat biletlari, o‘quv bo‘limidan ro‘yxatdan o‘tkazilgan toza varaqlar, baholash vedomosti.

VAQT TAQSIMOTI VA USLUBIY TIZIMI

№ t/r	Yakuniy nazoratning olib borilishi	Vaqt (daq)	O‘qituvchining harakati
1.	I. KIRISH QISMI - guruhni tavsiya qiluvchidan bildiruv qabul qilish, salomlashish; - guruhning davomatini va kursantlarning yakuniy nazoratga tayyorligini tekshirish (tashqi ko‘rinishi, daftarlari, sinov daftarchalari, guruh jurnali, yozuv-chizuv anjomlari va h.k.); - yakuniy nazorat maqsadini va qabul qilish tartibini etkazish.	5	Guruh ikki sherengaga saflanadi. Aniqlangan kamchiliklar bartaraf etiladi. Yakuniy nazoratdan ozod qilingan kursantlar e‘lon qilinadi va taqdirlanadi. Yakuniy nazoratga kirishga ruxsat berilmagan kursantlar e‘lon qilinadi. Javob berishga tayyorlanish uchun vaqt 30 daqiqa ekanligi e‘lon qilinadi, dastlabki topshiruvchi 5 ta kursant bittadan o‘quv sinfiga

			kiritiladi. Qolgan kursantlar kafedraning bo‘sh o‘quv sinfiga kirib tayyorlanib, kutishadi.
2.	II. ASOSIY QISM Yakuniy nazoratning borishi. Yakuniy nazorat fanning o‘quv dasturlariga ko‘ra o‘tilgan mavzular va mashg‘ulotlar bo‘yicha kursantlarning olgan nazariy bilim va amaliy ko‘nikmalarini tekshirib ko‘rish va baholash maqsadida amalga oshiriladi. Chaqirilgan kursant yakuniy nazorat topshirishga kelgani to‘g‘risida yakuniy nazorat qabul qiluvchiga bildiruv beradi, masalan: <i>“O‘rtoq katta leytenant! Kursant G‘ulomov “OBYEKTga yo‘naltirilgan dasturlash” fanidan yakuniy nazorat topshirish uchun etib keldi!”</i>, sinov daftarchasini o‘qituvchiga taqdim qiladi, so‘ngra bilet oladi, uning raqamini aytadi, bilet savollari bilan tanishib chiqadi va savollarni tushungani yoki tushunmagani to‘g‘risida bildiruv beradi, zarur bo‘lganda savollarni aniqlashtiradi, javoblarni yozish uchun o‘quv bo‘limidan ro‘yxatdan o‘tgan toza varaq oladi, so‘ngra ko‘rsatilgan joyga borib o‘tiradi va javob berishga tayyorgarlik ko‘radi, zarur bo‘lganda, o‘qituvchining ruxsati bilan kompyuterda mashq qilib tayyorlanadi. Javob berishga tayyorlanayotgan kursant reja tuzib olishi yoki javobni yozishi mumkin, zarur hollarda sinf doskasiga yoki varaqqa chizmalarni, sxemalarni, hisoblarni va shunga o‘xshashlarni tushirishi va bunda ruxsat etilgan materiallardan, o‘quv plakatlaridan, sxemalardan foydalanishi mumkin. Javob berishga tayyor yoki belgilangan	220	O‘qituvchi yakuniy nazorat topshirgan kursantni baholashi tartibi: 50-45 ball “a’lo” - agarda kursant o‘quv dasturi bo‘yicha teran va puxta bilimini ko‘rsatsa, uni aniq va to‘g‘ri ifoda eta olsa, tezda to‘g‘ri qaror qabul qila olsa, javoblarda jiddiy xatolarga yo‘l qo‘ymasa, kompyuter va maxsus amaliy dasturlarda to‘g‘ri ishlay olsa; 44-35 ball “yaxshi” - agarda kursant o‘quv dasturi bo‘yicha puxta bilimini ko‘rsatsa, uni aniq va to‘g‘ri ifoda eta olsa, to‘g‘ri qaror qabul qila olsa, javoblarda jiddiy xatolarga yo‘l qo‘ymasa, kompyuter va maxsus amaliy dasturlarda to‘g‘ri ishlay olsa; 34-30 ball “qoniqarli” - agarda kursant o‘quv dasturini faqat asosiy qisimi bo‘yicha bilimini ko‘rsatsa, mashg‘ulotning har bir qismini to‘liq o‘zlashtirmagan bo‘lsa, javoblarda jiddiy xatolarga yo‘l qo‘ymasa, ayrim savollarga to‘g‘ri javob uchun yunaltiruvchi savollar berishni talab qilsa, kompyuterda amaliy dasturlarda ishlay olsa; 29-1 ball “qoniqarsiz” – agarda kursant javob berishda qo‘pol xatolarga yo‘l qo‘ysa, olgan bilimlarini kompyuterda va

	vaqti tugagan kursant o‘qituvchining ruxsati bilan yoki uning chaqiruvi bo‘yicha etib kelib, bildiruv beradi, masalan: <i>“O‘rtoq katta leytenant! Kursant G‘ulomov 5-bilet savollariga javob berish uchun etib keldi!”</i> va biletga berilgan savollarga javob bera boshlaydi. Bilet savoliga javobni tugatgan kursant yakuniy nazorat qabul qiluvchiga bildiruv beradi, masalan: <i>«O‘rtoq katta leytenant! Kursant Abdullaev 1-savolga javobni tugatdi!»</i>. Yakuniy nazoratda har bir topshiruvchiga faqat bitta bilet olishga ruxsat beriladi. Yakuniy nazorat topshiruvchi bilet savollariga javob bera olmasligi to‘g‘risida bildiruv bergan hollarda u yakuniy nazoratdan o‘tmagan hisoblanadi. Yakuniy nazorat topshirish vaqtida ruxsat etilmagan har xil materiallardan foydalayotgan yoki yakuniy nazoratda belgilangan tartib qoidani buzayotgan kursantlar intizomiy javobgarlikka tortiladilar. Yakuniy nazorat qabul qiluvchi qaroriga ko‘ra, bunday kursantlar biletsiz, butun o‘quv fani bo‘yicha yakuniy nazorat sinov topshirishi mumkin. Javob berishga tayyorlanish uchun 30 daqiqagacha, javob berish uchun esa o‘quv me‘yorini bajarish vaqtidan kelib chiqib, vaqt ajratiladi.		dasturlarda amaliy ishlata olmasa. Topshiruvchi bilet savoliga javobni tugatgach, yakuniy nazorat qabul qiluvchi unga qo‘shimcha va aniqlashtiruvchi savollar berishi mumkin. Yakuniy nazorat natijasi bilet bo‘yicha va berilgan qo‘shimcha savollarga javoblarni tugatgach, kursantga e‘lon qilinadi. Yakuniy nazoratda o‘quv me‘yorini bajarish nazariy (o‘quv me‘yori nomi, bajariladigan ishlar hajmi va baholanish tartibini aytadi) va amaliy qismlardan iborat. O‘quv me‘yorini bajarishda umumiy baho (ball) O‘R MV 2022-yil 15-sentabrdagi 686-sonli buyrug‘i talablariga asosan baholanadi. Yakuniy nazorat topshirish natijasi yakuniy nazorat qabul qilgan o‘qituvchi tomonidan topshiruvchiga e‘lon qilinadi, yakuniy nazorat vedomostiga va kursantning sinov daftarchasiga qo‘yiladi, yakuniy nazoratdan o‘tolmagan kursantning sinov daftarchasiga baho qo‘yilmaydi. Har bir belgi yakuniy nazorat qabul qiluvchi imzosi bilan tasdiqlanadi.
3.	III.YAKUNIY QISM: - yakuniy nazorat natijasi e‘lon qilinadi; - yakuniy nazorat natijalari tahlil qilinadi, a‘lo va yomon tayyorgarlik ko‘rgan kursantlar ta’kidlab o‘tiladi, umumiy kamchiliklar ko‘rsatiladi va ularni bartaraf qilish bo‘yicha ko‘rsatmalar	15	Guruhning barcha shaxsiy tarkibi ikki sherengaga saflanadi. Yakuniy nazorat natijalari, umumiy hatolar aytiladi, kelgusida ushbu hatolarni bartaraf etish bo‘yicha ko‘rsatmalar beriladi.

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

	beriladi; - tug‘ilgan savollarga javob beriladi; - yakuniy nazorat tugaganligi e‘lon qilinadi.		Ish joylari, kompyuter va boshqa jihozlar tartibga keltiriladi.
--	--	--	---

**O'ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT”
KAFEDRASI**



“OBJEKTGA YO'NALTIRILGAN DASTURLASH” FANIDAN

**FANNI O'QITISHDA
FOYDALANILADIGAN
INTERFAOL TA'LIM
METODLARI (3-ILOVA)**

“AXBOROT TEXNOLOGIYALARI” KAFEDRASI
“OBJEKTga yo'naltirilgan dasturlash” fanidan yakuniy nazorat
SAVOLLARI

1. Windows Forms haqida tushuncha
2. Grafik dastur yaratish
3. Dasturni ishga tushirish
4. Shakl asoslari va shakllarning asosiy xususiyatlari
5. Dasturiy xususiyatlarni sozlash
6. Shaklning o'lchamlarini sozlash
7. Shaklning boshlang'ich pozitsiyasi
8. Shaklning rang va fonni
9. Windows formda hodisalar. Shakl hodisalari
10. To'rtburchaklar bo'lmagan shakllarni yaratish. Shaklni yopish
11. Elementlarni dinamik qo'shish
12. GroupBox, Panel va FlowLayoutPanel elementlari
13. TableLayoutPanel elementi
14. Joylashuv
15. O'lchamlarni sozlash
16. Anchor xususiyatlari
17. Dock xususiyati
18. TabControl
19. Koddagi yorliqlarni boshqarish
20. SplitContainer
21. Tugmani bezatish
22. Tugma ustidagi rasm
23. Standart tugmalar
24. Label
25. LinkLabel
26. LinkClicked
27. Avtomatik to'ldirilgan matnli maydon
28. So'zlar bo'yicha ko'chirish
29. Parol kiritish, TextChanged hodisasi
30. ToolStrip asboblari paneli.
31. MaskedTextBox elementi
32. RadioButton elementi
33. CheckBox elementi
34. ListBox -dagi dasturiy elementlar
35. ListBoxda SelectedIndexChanged hodisasi
36. ComboBox tashqi ko'rinishini sozlash
37. ComboBoxda SelectedIndexChanged hodisasi
38. LinkLabel
39. LinkClicked
40. DataSource ma'lumotlar manbai
41. DisplayMember: ob'ektning xususiyati

42. ValueMember: ob'ektning xususiyati
43. Items xususiyatlari
44. SetItemChecked va SetItemCheckState
45. NumericUpDown hodisasi
46. DomainUpDown hodisasi
47. ImageList elementi
48. ListViewItem hodisasi
49. Element tasvirlari
50. Ko'rsatish turlari
51. ListView. Amaliyot
52. Dasturiy tugunlarni boshqarish
53. Tugunlarni yashirish va ochish
54. CheckBox va tasvirni qo'shish
55. TrackBar elementi
56. Timer elementi
57. ProgressBar elementidagi indikator hodisasi
58. DateTimePicker elementi
59. MonthCalendar elementi
60. ImageLocation, IntialImage xususiyatlari
61. Rasm o'lchami
62. WebBrowser
63. NotifyIcon
64. MessageBox xabarlar oynasi
65. OpenFileDialog funksiyasi bilan ishlash
66. SaveFileDialog funksiyasi bilan ishlash
67. FontDialog va ColorDialog funksiya bilan ishlash
68. BlinkRate: xato belgisi
69. BlinkStyle: xato belgisi
70. ToolStrip asboblari paneli
71. MenuStrip yaratish
72. StatusStrip qator holati
73. ContextMenuStrip kontekst menyusi

**O‘ZBEKISTON RESPUBLIKASI HARBIIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN’IY INTELLEKT”
KAFEDRASI**



“OBYEKTGA YO‘NALTIRILGAN DASTURLASH”

FANNING O‘QUV DASTURI

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

1. O‘quv fanining dolzarbligi va oliy kasbiy ta’lim dasturidagi o‘rni

Obyektga yo‘naltirilgan dasturlash (OYD) fanining asosiy maqsadi dasturlashda mustahkam nazariy bilim va amaliy ko‘nikmalarga ega dasturchilarni tayyorlashdan iboratdir. Kursantlar zamonaviy OYD metodologiyalari, ularning asosiy prinsiplari, OYD ga asoslangan dasturiy ta’minotning tuzilishi va ularni loyihalash haqida to‘liq ma’lumotga ega bo‘ladilar. Shuningdek, kursantlar OYD asosida dasturlarni ishlab chiqish, sinflar va obyektlarni yaratish, meros olish, polimorfizm va inkapsulyatsiya kabi OYD tamoyillarini qo‘llash orqali murakkab dasturiy tizimlarni loyihalash va ularga texnik xizmat ko‘rsatish qobiliyatiga ega bo‘ladilar.

2. O‘quv fanining maqsadi va vazifasi

Obyektga yo‘naltirilgan dasturlash (OYD) fanining asosiy maqsadi dasturchilarni OYD tamoyillari va metodologiyalari bo‘yicha mustahkam nazariy bilim va amaliy ko‘nikmalarga ega qilishdir. Bu fan kursantlarga zamonaviy dasturlash amaliyotida keng qo‘llaniladigan OYD usullarini o‘rgatish, ularga sinflar, obyektlar, inkapsulyatsiya, meros olish, polimorfizm, va abstraksiya kabi tamoyillarni tushunishga va qo‘llashga yordam berish uchun mo‘ljallangan.

Fanning asosiy vazifasi kursantlarni OYD tamoyillari va metodologiyalariga asoslangan yangi dasturiy yechimlarni loyihalash usullari to‘g‘risida ma’lumotga ega qilishdan iborat. Kursantlar turli xil dasturiy vositalar va texnologiyalar yordamida OYD tamoyillarini qo‘llab dasturlar yaratish bo‘yicha amaliy ko‘nikmalarga ega bo‘ladilar. Shuningdek, ular lokal va korporativ dasturiy tizimlarni yaratish usullarini bilib, natijada, O‘zbekiston Respublikasi bo‘ylab xavfsiz va barqaror dasturiy tizimlarni yaratish va ularga texnik xizmat ko‘rsatish imkoniyatiga ega bo‘ladilar.

Obyektga yo‘naltirilgan dasturlash (OOP) fanidan dars olingan kursantlar bir qator muhim ko‘nikmalarga ega bo‘lishadi. Birinchidan, ular OOP tamoyillarini, jumladan, inkapsulyatsiya, meros olish, polimorfizm va abstraksiyani mukammal tushunib oladilar. Bu tamoyillarni tushunish va qo‘llay olish orqali kursantlar murakkab dasturiy tizimlarni samarali ravishda ishlab chiqish, saqlash va kengaytirish imkoniyatiga ega bo‘ladilar.

Ikkinchidan, kursantlar real dunyo muammolarini obyektlar va sinflar yordamida model qila olish ko‘nikmasiga ega bo‘ladilar. Bu ularga harbiy va texnik vazifalarni yechishda qo‘llaniladigan dasturlarni yaratishda katta yordam beradi. Ular kodni qayta foydalanish, modullarni mustaqil ishlab chiqish va tizimning boshqa qismlari bilan integratsiyalash qobiliyatlarini rivojlantiradilar.

Shuningdek, kursantlar zamonaviy dasturlash tillaridan, masalan, C++ va Java dan foydalanishni o‘rganadilar, bu esa ularning dasturlash bo‘yicha umumiy bilimlarini kengaytiradi. OOP ko‘nikmalariga ega bo‘lgan kursantlar harbiy texnologiyalarni rivojlantirish va boshqarish, dasturiy ta’minot xavfsizligini ta’minlash va turli tizimlar orasida samarali integratsiyani ta’minlashda katta rol o‘ynaydilar. Bu ko‘nikmalar ularga kelajakdagi harbiy xizmatlarida yuqori darajadagi texnik masalalarni hal qilish imkonini beradi.

3. O‘quv fanining mazmuni**4-kurs 7-semestr****1-Mavzu: Obyektga yo‘naltirilgan dasturlashda C#.**

OOP tamoyillarini (inkapsulyatsiya, meros olish, polimorfizm, abstraktsiya) chuqur tushunish va ularni C# dasturlash tilida qanday qo‘llanilishini o‘rganish. Sinflar va obyektlar bilan ishlash, ular orasidagi munosabatlarni tahlil qilish.

2-Mavzu: Algoritm va uni asosiy tamoyillari.

Algoritmlar, ularning tuzilishi va turlari haqida bilim olish. Algoritmlarni tahlil qilish va samaradorligini baholash usullarini o‘rganish.

3-Mavzu: C# dasturlash tiliga kirish.

C# dasturlash tilining asosiy sintaksisi, o‘zgaruvchilar, turlar va dasturiy muhit bilan tanishish. O‘zgaruvchilarni e‘lon qilish va ularga qiymat berish, ma‘lumot turlarini kasting qilish.

4-Mavzu: Ma‘lumotlarni saqlashda dasturiy imkoniyatlar.

Ma‘lumot turlari, massivlar va kolleksiyalar bilan ishlash. Ma‘lumotlarni saqlash va ularga murojaat qilish usullarini o‘rganish.

5-Mavzu: Amallar va ularning turlari.

Arifmetik, mantiqiy va taqqoslash amallari, ularning dasturiy qo‘llanilishi. Amallarni kombinatsiyalash va dasturda qo‘llash.

6-Mavzu: Tarmoqlash amallari.

If-else va switch-case konstruktsiyalari orqali shartli tarmoqlash amallari. Murakkab shartlarni dasturlash va ularni boshqarish.

7-Mavzu: Takrorlash operatorlari.

For, while, do-while sikllari va ularning dasturiy qo‘llanilishi. Sikl operatorlarini optimallashtirish va samarali foydalanish.

8-Mavzu: Massiv va unga murojat.

Bir o‘lchovli massivlarni yaratish va ularga murojaat qilish usullari. Massivlarni dasturda qo‘llash va ulardan samarali foydalanish.

9-Mavzu: Ko‘p o‘lchovli massivlar.

Ikki va undan ko‘p o‘lchovli massivlar bilan ishlash. Ko‘p o‘lchovli massivlarni tahlil qilish va dasturda qo‘llash.

10-Mavzu: Saralash algoritmlari.

Bubble sort, selection sort va insertion sort kabi asosiy saralash algoritmlari. Saralash algoritmlarini dasturiy realizatsiya qilish va ularning samaradorligini baholash.

11-Mavzu: Matnli ma‘lumotlarni taxrirlash.

String operatsiyalari va matnli ma‘lumotlarni qayta ishlash usullari. Matnli ma‘lumotlarni tahrirlash va ularni formatlash.

12-Mavzu: Foydalanuvchi gibri tiplarini hosil qilish.

Struct va enum turlari, ularning yaratilishi va qo‘llanilishi. Foydalanuvchi tomonidan aniqlangan turlarni yaratish va dasturda ishlatish.

13-Mavzu: Funktsiyalar.

Funktsiyalarni e‘lon qilish, chaqirish, parametrlar va qaytariladigan qiymatlar bilan ishlash. Funktsiyalarni overloading qilish va recursion tamoyillarini o‘rganish.

4-kurs 8-semestr**14-Mavzu: Obyektga yo‘naltirilgan dasturlash asoslari.**

OOP prinsiplarini chuqurroq o‘rganish, sinflar, obyektlar va interfeyslar bilan ishlash. OOP asosida dasturiy loyihalar yaratish.

15-Mavzu: Dasturiy xatoliklarni aniqlash usuli.

Exception handling, try-catch-finally bloklari va xatolarni boshqarish. Xatolarni aniqlash va ularni bartaraf etish usullari.

16-Mavzu: Ma’lumotlar to‘plamini saqlash va boshqarish.

Ma’lumotlar bazalari, ADO.NET va Entity Framework bilan tanishish. CRUD operatsiyalarini amalga oshirish.

17-Mavzu: Generic collection`lar bilan ishlash.

Generic kolektsiyalar (List, Dictionary, Queue, Stack) bilan ishlash. Generic turlarni yaratish va ulardan foydalanish.

18-Mavzu: Maxsus collection`lar.

Maxsus kolleksiyalar va ularning dasturiy qo‘llanilishi. HashSet, LinkedList va boshqa maxsus kolleksiyalar bilan ishlash.

19-Mavzu: WPF haqida boshlang‘ich

Windows Presentation Foundation (WPF) asoslari, XAML va WPF arxitekturasi. WPF loyihalari yaratish.

20-Mavzu: WPF – XAML tahlili.

XAML sintaksisi va uni WPF da qo‘llash. XAML yordamida foydalanuvchi interfeyslarini yaratish.

21-Mavzu: Kompanovka elementlari.

WPF da layout (kompanovka) elementlari bilan ishlash. Layout elementlarini to‘g‘ri joylashtirish va ularni dasturiy boshqarish.

22-Mavzu: Grid hamda GridSplitter kompanovkasi.

Grid va GridSplitter elementlari bilan ishlash, ularni dasturiy loyihalash. Grid va GridSplitter elementlarini to‘g‘ri joylashtirish va ulardan foydalanish.

23-Mavzu: Stack hamda DockPanel kompanovkasi.

StackPanel va DockPanel elementlari bilan ishlash. StackPanel va DockPanel elementlarini dasturiy boshqarish.

24-Mavzu: WrapPanel hamda Canvas kompanovkasi.

WrapPanel va Canvas elementlari bilan ishlash. WrapPanel va Canvas elementlarini dasturiy boshqarish.

25-Mavzu: Button va Border komponentlari.

Button va Border komponentlarini yaratish va dasturda ishlatish. Button va Border komponentlarining xususiyatlari va ularni sozlash.

26-Mavzu: Resources va Styles.

WPF da resurslar va stillarni boshqarish. Resurslar va stillarni yaratish va ulardan foydalanish.

5-kurs 9-semestr

27-Mavzu: Label, TextBlock va TextBox komponentlari.

Label, TextBlock va TextBox komponentlari bilan ishlash. Ushbu komponentlarning xususiyatlari va ulardan foydalanish.

28-Mavzu: MVVM (Model, view va view-model) pattern`i.

MVVM patterini tushunish va uni WPF loyihalarida qo‘llash. MVVM orqali dasturiy komponentlarni ajratish va boshqarish.

29-Mavzu: Murakkab foydalanuvchi interfeysini hosil qilish.

Murakkab foydalanuvchi interfeyslarini loyihalash va dasturlash. Foydalanuvchi interfeysini interaktiv va intuitiv qilish.

30-Mavzu: CheckBox, Radiobutton va ToggleButton komponentlari

CheckBox, RadioButton va ToggleButton komponentlari bilan ishlash. Ushbu komponentlarning xususiyatlari va ulardan foydalanish.

31-Mavzu: PasswordBox, Image va ToolTip komponentlari

PasswordBox, Image va ToolTip komponentlari bilan ishlash. Ushbu komponentlarning xususiyatlari va ulardan foydalanish.

32-Mavzu: DatePicker, Dialogbox va ProgressBar komponentlari.

DatePicker, DialogBox va ProgressBar komponentlari bilan ishlash. Ushbu komponentlarning xususiyatlari va ulardan foydalanish.

33-Mavzu: ListBox va ComboBox komponentlari.

ListBox va ComboBox komponentlari bilan ishlash. Ushbu komponentlarning xususiyatlari va ulardan foydalanish.

34-Mavzu: Menu va ContextMenu komponentlari.

Menu va ContextMenu komponentlari bilan ishlash. Ushbu komponentlarning xususiyatlari va ulardan foydalanish.

35-Mavzu: Navigation va page management.

WPF da navigatsiya va sahifalarni boshqarish. Navigatsiya tizimini yaratish va sahifalarni almashish.

36-Mavzu: ListView komponenti.

ListView komponenti bilan ishlash va ma'lumotlarni ko'rsatish. ListView komponentining xususiyatlari va ulardan foydalanish.

37-Mavzu: GridView komponenti. GridView komponenti bilan ishlash. GridView komponentining xususiyatlari va ulardan foydalanish.

38-Mavzu: DataGrid komponenti.

DataGrid komponenti bilan ishlash, ma'lumotlarni ko'rsatish, tahrirlash va boshqarish. DataGrid komponentining xususiyatlari va ulardan foydalanish.

39-Mavzu: Fayllar va kataloglar.

Fayllar va kataloglar bilan ishlash, fayllarni o'qish va yozish. Fayllar va kataloglarni dasturiy boshqarish.

40-Mavzu: Fayllarga yozish va o'qish class'lari.

FileStream, StreamWriter, StreamReader va boshqa fayl operatsiyalari. Fayllar bilan ishlash uchun kerakli sinflarni o'rganish.

5-kurs 10-semestr

41-Mavzu: Ma'lumotlar ombori.

Ma'lumotlar bazalari bilan ishlash, SQL asoslari va CRUD operatsiyalari. Ma'lumotlar omborlarini boshqarish va ularga murojaat qilish.

42-Mavzu: Ko'p oqimli dasturlashning asoslari.

Multithreading asoslari, Thread klassi va asinxron dasturlash. Ko'p oqimli dasturlash tamoyillarini o'rganish va qo'llash.

43-Mavzu: Ma'lumotlarni xavfsizligini ta'minlash asoslari.

Ma'lumotlarni xavfsizligini ta'minlash usullari va kriptografiya. Ma'lumotlarni himoya qilish va xavfsiz saqlash usullarini o'rganish.

44-Mavzu: Tarmoqda ma'lumot almashinish asoslari.

Tarmoq dasturlash, TCP/IP, HTTP va boshqa tarmoq protokollari. Tarmoqda ma'lumot almashish usullarini o'rganish.

45-Mavzu: Tarmoqda fayl yuborish .

Tarmoq orqali fayl almashish, FTP, SFTP va HTTP protokollari orqali fayllarni yuborish va qabul qilish. Tarmoqda fayllarni xavfsiz yuborish va qabul qilish usullarini o'rganish.

O'qituvchilar amaliy mashg'ulotlarni o'tkazishda, tinglovchilarning yakka tartibdagi sifatlariga ko'proq javob beradigan va o'quv materiallarini ular tomonidan yuqori darajada

o‘zlashtirishni ta‘minlaydigan, shuningdek, mustaqil va ijodiy fikrlashni rivojlantiradigan o‘qitish usul va vositalarini tanlaydilar.

4. Fanni o‘qitish bo‘yicha tashkiliy – uslubiy ko‘rsatmalar

“Obyektga yo‘naltirilgan dasturlash” fanini o‘qitish davomida kursantlarni mustaqil va erkin fikr yuritishga, mantiqiy va algoritmik fikrlashlarini hamda, nutq mahoratini oshirishga, u yoki bu muammoga nisbatan o‘z nuqtai nazarini aniq va ravshan ifoda etishga chorlaydigan innovatsion pedagogik texnologiyalardan hamda “Bumerang”, “Zinama-zina”, “Aqliy hujum” “Charxpalak”, “3 x 4”, “Muammo”, “Labirint”, “Blis-so‘rov”, “Interfaol suhbat”, “T-sxema”, “Klasster”, “FSMU”, “VEN-diagramma”, SWOT-tahlil” va boshqa interfaol ta‘lim metodlardan foydalaniladi.

Ma‘ruza materiallari bayoni mustaqil va tugallangan hususiyatga ega bo‘lib, avval bayon qilingan materiallarga mantiqiy bog‘langan hamda boshqa fanlarda, hamda amaliyotda qo‘llanishga yo‘naltirilgan bo‘lishi kerak. Amaliy mashg‘ulotlarda kursantlar olgan nazariy bilimlarini qo‘llay olishni o‘rganishlari kerak.

Har bir ma‘ruza o‘z ichiga kirish, asosiy va yakuniy qismni oladi.

Kirish qismida: mavzuning nomi, ma‘ruza mavzusining asosiy g‘oyasi va muhimligi; o‘quv maqsadlar; ma‘ruzaning o‘quv savollari; oldingi va keyingi mashg‘ulotlar bilan bog‘liqligi; OHTMda ofitserlarni tayyorlash jarayonidagi ma‘ruzaning tutgan o‘rni bayon qilinadi.

Ma‘ruzaning asosiy qismida o‘quv savollarining mazmuni yetkaziladi. Ma‘ruzaning har bir nazariy jihati eng maqsadga muvofiq usullarni qo‘llagan holda asoslangan va isbotlangan bo‘lishi kerak. Ma‘ruzaning asosiy qismini bayon qilishda ta‘lim oluvchilarga ilmiy g‘oyalarni rivojlanishi, jamlanishi, mavhumlikdan aniqlikka o‘tishining mantiqini yoritib berishga imkon beruvchi dalillarga tayanish ma‘ruzaga bo‘lgan majburiy talab hisoblanadi. Har bir ma‘ruzaning asosiy qismining mazmuni fundamental bo‘lishi kerak.

Amaliy maqsadlarga yo‘naltirilgan ma‘ruzalarda kasbga oid va o‘quv vazifalarni hal etish bo‘yicha amaliy tavsiyalarni ko‘zda tutish maqsadga muvofiq bo‘ladi.

Har bir o‘quv savoli, uni, keyingi o‘quv savoliga mantiqiy olib keluvchi, rivojlanish istiqbollarinin nazariyasi va amaliyoti hamda qisqacha xulosasini yoritish bilan tugatilishi kerak.

Ma‘ruzaning yakuniy qismida, nazariya va amaliyotni qo‘llash soha va chegaralarini ko‘rsatgan holda, asosiy qism mazmuni umumshtiriladi va qisqacha xulosa qilinadi, mustaqil o‘rganish hamda kelgusi amaliy va boshqa turdagi mashg‘ulotlarda muhokama qilish uchun savollar va vazifalar belgilanadi.

Ma‘ruzani o‘qishda kino va videofilmlar, chizmalar, plakatlar, modellar, asboblari va maketlarni namoyish qilgan holda o‘quv materiallarining og‘zaki yetkazilishi o‘qitishning yetakchi uslubi hisoblanadi.

Materialni yetkazish tempini tanlashda, o‘qituvchi, ta‘lim oluvchilar (tinglovchilar, kursantlar) toifasini, ushbu mavzu (yo‘nalish) bo‘yicha o‘quv, ilmiy, uslubiy adabiyotlar mavjudligi va boshqa omillarni albatta hisobga olishi kerak.

Individual va kollektiv yondashish yo‘li bilan o‘qituvchi suhbat orqali ma‘ruzaning o‘z ichiga olgan muammoli savollarning yechimini topadi.

O‘rganilayotgan o‘quv materiallarini faollashtirish uchun “nima uchun bunday qilingan”, “qanchalik bu qulay (ma‘qullik, maqsadga muvofiq)”, bunda o‘rganuvchilar orasida amaliy mashg‘ulot xususiyatga ega bo‘lgan fikrlarni almashuv va metodik usullarni kiritish foydalidir.

Amaliy mashg'ulot o'tkazish maqsadida kursantlar zamonaviy kompyuterlarda zamonaviy dasturlash tillarida dastur yaratishadi va dasturlarni tahlilini o'rganishadi.

Amaliy mashg'ulotlar zamonaviy kompyuterlar va multimedia vositalari bilan jihozlangan maxsus o'quv sinf xonalarida o'tkaziladi. Nazariy tajribani va amaliyotni o'tash mobaynida o'z qobiliyatini hamda ko'nikmalarini takomillashtiradi.

Mashg'ulotlarni individuallashtirish va o'qitishni sifatini oshirish maqsadida vositalarning soniga qarab guruhlar bir qancha guruhlarga bo'linadi va ular o'quv joylariga taqsimlanadi.

Amaliy mashg'ulotlarda kursantlar me'yorlarni bajarishda ishtirok etishi maqsadida bellashuv, musobaqa va sog'lom raqobat elementlarini kiritish lozim.

O'quv-tarbiyaviy jarayonini jadallashtirishga qo'yilgan talablar oshishini inobatga olib mashg'ulotlarni tashkil etish va o'tkazish uslubiyatini doimo takomillashtirish lozim.

Mustaqil ta'lim jarayonida kursantlar tavsiya etilgan adabiyotlarni o'rganib, konspektlarini to'ldirib, olgan bilimlarini mustahkamlaydi.

5. Mustaqil ta'lim va mustaqil ishlar

Mustaqil o'zlashtiriladigan mavzular bo'yicha belgilangan vaqt davomida fan bo'yicha o'tkazilgan mavzular va zarur ko'nikma va malakani shakllantirishga undaydigan qo'shimcha mavzular hamda materiallar ustida kursantlar o'zi mustaqil o'rganishadi. Mustaqil ta'lim davomida kursantlar zarur adabiyotlar va elektron manbaalar bilan ta'minlanadi. Kursantlarning mustaqil ta'lim olishi fanni va mutaxassislik ko'nikmalarini yanada mustahkamroq egallashini ta'minlaydi. Mustaqil ta'lim va mustaqil ish topshiriqlarini kursantlar tomonidan bajarilishi majburiydir va u fanning joriy nazorat bahosining bir qismini tashkil etadi. Mustaqil ta'lim topshiriqlari fan o'qituvchisi tomonidan har bir kursant uchun umumiy bir mavzuda va har biriga individual yo'nalish va shart asosida semestr davomida berib boriladi.

Mustaqil o'zlashtiriladigan mavzular bo'yicha kursantlar tomonidan mustaqil ish AKT vositasi yordamida amaliy ish ko'rinishida tayyorlanadi va uni taqdimoti tashkil qilinadi.

5.1. Mustaqil ta'lim olish uchun tavsiya etiladigan mavzular:

T/r	Mustaqil ta'lim mavzusi	Yakuniy ish shakli
1	Shart operatorlariga doir muammoli masalalar yechimi	Amaliy bajaradi hisobot tayyorlaydi.
2	Sikl operatorlariga doir muammoli masalalar yechimi	Amaliy bajaradi hisobot tayyorlaydi.
3	Ko'p o'lchamli massivlarga doir muammoli masalalar yechimi	Amaliy bajaradi hisobot tayyorlaydi.
4	Satrlarga doir muammoli masalalar yechimi	Amaliy bajaradi hisobot tayyorlaydi.
5	Sinf va obyekt yaratishni real loyihadagi realizatiyasi	Amaliy bajaradi hisobot tayyorlaydi.
6	Polimorfizm va incapsulation`ni dasturiy imkoniyatlarini amaliy qo'llash	Amaliy bajaradi hisobot tayyorlaydi.
7	Fayl va kataloglar bilan amaliy ishlash	Amaliy bajaradi hisobot tayyorlaydi.
8	Xatoliklar bilan ishlash	Amaliy bajaradi hisobot tayyorlaydi.
9	Foydalanuvchilarni ro'yxatga olish dasturiy ta'minotini ishlab chiqish	Amaliy bajaradi hisobot tayyorlaydi.

10	Elektron kutubxona dasturiy ta’minotini ishlab chiqish	Amaliy bajaradi hisobot tayyorlaydi.
11	Fayllarni kriptografik ximoyalash va foydalanish dasturiy ta’minotini ishlab chiqish	Amaliy bajaradi hisobot tayyorlaydi.
12	Nazorat o‘tash punktida xodimlarni kirish-chiqish ro‘yxatga olish dasturiy ta’minotini ishlab chiqish	Amaliy bajaradi hisobot tayyorlaydi.
13	Ishlab chiqilgan foydalanuvchilarni ro‘yxatga olish hamda elektron kutubxona dasturiy ta’minotlarni ma’lumotlar omboria ulash hamda imkoniyatlarini boyitish	Amaliy bajaradi hisobot tayyorlaydi.
14	Ishlab chiqilgan fayllarni kriptografik ximoyalash va foydalanish hamda nazorat o‘tash punktida xodimlarni kirish-chiqish ro‘yxatga olish dasturiy ta’minotlarni ma’lumotlar omboria ulash hamda imkoniyatlarini boyitish	Amaliy bajaradi hisobot tayyorlaydi.
15	Lokal tarmoq orqali fayl almashinish imkonini beruvchi dasturiy ta’minot ishlab chiqish	Amaliy bajaradi hisobot tayyorlaydi.
16	Lokal tarmoq imkoniyatlaridan foydalangan holda br nechta foydalanuvchilar o‘rtasida ma’lumot almashinish imkonini beruvchi messenger dasturini ishlab chiqish	Amaliy bajaradi hisobot tayyorlaydi.

6. Asosiy va qo‘shimcha o‘quv adabiyotlar hamda axborot manbaalari

Asosiy adabiyotlar:

1. O‘zbekiston Respublikasining Mudofaa doktrinasi. - T., 2018 y. B. 54-86.
2. B.K. Yusupov, B.O. Muhamadov. Videokonferensiya aloqa tizimlari fanidan darslik: // Toshkent: Aloqachi 2023. -185 b.
3. Дворкович В.П., Дворкович А.В. Цифровые видеoinформационные системы (теория и практика): Книга // Москва: Техносфера, 2012. – 1008 с. ISBN 978-5-94836-336-3.
4. N.T.Boboyev. C# dasturlash tilining fundamental asoslari. Obyektga yo‘naltirilgan dasturlash fanidan o‘quv qo‘llanma. Toshkent 2024. Aloqachi nashriyoti. -256 b.

Tavsiya qilinadigan qo‘shimcha adabiyotlar:

1. O‘zbekiston Respublikasi Konstitutsiyasi 30.04.2023 yil. 50-51 modda. 63 b. [Elektron manba]. <https://lex.uz/docs/-6445145>
2. Медведева Е.В. Цифровая обработка изображений в видеoinформационных системах: Учебное пособие // Киров: ВятГУ, 2015. – 107 с.

Tavsiya qilinadigan Internet saytlar

1. <http://ziyonet.uz/uzc>
2. <https://W3Schools.com>
3. <https://Codecademy.com>
4. <https://Pluralsight.com>
5. <https://Udemy.com>
6. <https://Coursera.com>
7. <https://SoloLearn.com>
8. <https://TutorialsPoint.com>

**O'ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN'IY INTELLEKT”
KAFEDRASI**



“OBJEKTGA YO'NALTIRILGAN DASTURLASH”

FANNING ISHCHI O'QUV DASTURI

Toshkent 2025 y.

3 OYD 5-kurs

O'ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING

AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY ALOQA INSTITUTI



"TASDIQLAYMAN"

O'RHX va MUX AXBOROT-KOMMUNIKATSIYA
TEXNOLOGIYALARI VA HARBIY ALOQA INSTITUTI
BOSHLIQ INING BIRINCHI O'RINBOSARI
(O'QUV VA ILMIY ISHI BOSHQARUVCHISI)
kapitan

2025-yil

B. To'rayev

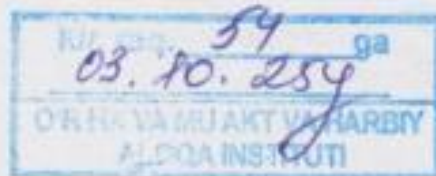
Axborot texnologiyalari va sun'iy intellekt kafedrası

OBYEKTGA YO'NALTIRILGAN DASTURLASH fanining

ISHCHI O'QUV DASTURI

Bilim sohasi:	1 000 000	– Xizmatlar
Ta'lim sohasi:	1 030 000	– Xavfsizlik xizmati
Ta'lim yo'nalishlari:	6 1030 700	– Qo'shinlarning taktik qo'mondonlik muhandisligi (Axborot tizimlari va texnologiyalari)

Toshkent – 2025 y.



Fanning ishchi o'quv dasturi O'zbekiston Respublikasi Mudofaa vazirligi Harbiy kadrlarni tayyorlash boshqarmasi boshlig'i tomonidan 2024-yil 17-iyul kuni tasdiqlangan o'quv dasturi asosida tayyorlangan.

Ishchi o'quv dasturi Axborot-kommunikatsiya texnologiyalari va aloqa harbiy instituti ilmiy-uslubiy kengashining 2024-yil 28-iyun kunidagi 11-sonli bayoni bilan tasdiqlangan.

Ishchi o'quv dasturi Axborot-kommunikatsiya texnologiyalari va aloqa harbiy instituti boshlig'ining 2024-yil 2-avgust kunidagi 289-son buyrug'i bilan ta'lim jarayoniga joriy etilgan.

Tuzuvchilar:

B.K. Yusupov	Axborot kommunikatsiya texnologiyalari va harbiy aloqa instituti "Axborot texnologiyalari va sun'iy intellekt" kafedrası boshlig'i, t.f.f.d., professor.
A.A. Komilov	Axborot kommunikatsiya texnologiyalari va harbiy aloqa instituti "Axborot texnologiyalari va sun'iy intellekt" kafedrası "Axborot texnologiyalari" sikli boshlig'i, PhD
A.X. Fayzullayev	Axborot kommunikatsiya texnologiyalari va harbiy aloqa instituti "Axborot texnologiyalari va sun'iy intellekt" kafedrası "Axborot texnologiyalari" sikli katta o'qituvchisi

Taqrizchilar:

podpolkovnik A. Mulladjanov	O'R QK Bosh shtabi AAT va AH BB Axborot-kommunikatsiya texnologiyalarini rivojlantirish boshqarmasi boshlig'i;
podpolkovnik O. Abdiroziqov	O'R HX va MU "Istiqbolli harbiy texnologiyalar" kafedrası boshlig'i, PhD, dotsent.

O'R HX va MU AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI
VA HARBIY ALOQA INSTITUTI O'QUV BO'LIMI BOSHIG'I
mayor

N. Kuzibekov

AXBOROT TEXNOLOGIYALARI VA SUN'IY
INTELLEKT KAFEDRASİ BOSHIG'I

kapitan

B. Yusupov

I. O‘QUV VAQTINI SEMESTRLAR VA O‘QUV MASHG‘ULOT TURLARI BO‘YICHA TAQSIMLANISHI

Fanni o‘qitish muddati:	Kursantning o‘quv yuklamasi (soatlarda)										Nazorat usuli	
	Umumiy yuklamaning hajmi	Auditoriya mashg‘ulotlari (soatlarda)								Mustaqil tayyorgarlik	Oraliq nazorat	Yakuniy nazorat
		Jami	Ma’ ruzalar	Guruh mashg‘ ulotlari (mashqlari)	Amaliy mashg‘ ulotlar	Laboratoriya mashg‘ ulotlari	Seminarlar	Mustaqil ta’ lim	Kurs loyihasi (ishi)			
7	180	90	4		86			90			+	+
8	120	60	6		56			60				+
9	120	60	6		56			60			+	+
Jami soatlar	420	210	14		198			210				+

II. O‘QUV FANNI O‘QITILISH BO‘YICHA USLUBIY KO‘RSATMALAR

“Obyektga yo‘naltirilgan dasturlash” fanining asosiy maqsadi qo‘shilma va harbiy qismlarda avtomatlashtirishning qulay vositalaridan kompyuter texnologiyalaridan keng foydalangan holda dasturlashda ma’lumotlarning turli tarkibiy tuzilmalarini tadbqiq etish. C# dasturlash tilning sodda va murakkab tuzilmalarini qo‘llash. Algoritmnlarni baholash. Qo‘yilgan masalani yechish algoritmini tanlash, tanlovni asoslash, algoritmnini tadbqiq etish. C# dasturlash tilining asosiy konstruksiyalari, ma’lumotlar toifalari va tuzilmalarini qo‘llash. C# dasturlash tilida aniq algoritmnlarni tadbqiq etish. C# dasturlash tilining statik va dinamik imkoniyatlarni tadbqiq etish. Obyektga yo‘naltirilgan dasturlash asoslarini qo‘llash. Dasturiy ishlanmalarni ishlab chiqarish jarayonini ta’minlash. Ma’lumotlarning dinamik informasion tuzilmasini tadbqiq etish. Aniq loyihalarni ishlab chiqish va tadbqiq etish. Dasturiy ishlanmalarni testlash va sozlash. Murakkab dasturiy tizimnlarni tashkil etish. Natijalarni tahlil qilish.

Fan bo‘yicha o‘quv kursi ta’limning kredit-modul tizimi asosida ma’ruza, amaliy mashg‘ulotlari hamda mavzu bo‘yicha vazifalar va mustaqil topshiriqlarni o‘z ichiga oladi. Ma’ruza, amaliy ishlarga oid o‘quv materiallarda ko‘rsatilgan mavzular bo‘yicha nazariy va amaliy ma’lumotlar beriladi, amaliy ishlarni, mustaqil ishlarni bajarish va natijalarni hisoblash tartibi tushuntiriladi. Kurs bo‘yicha qo‘yilgan o‘quv materiallari kursantlar tomonidan mustaqil o‘rganiladi, testlar, amaliy ishlar individual tarzda bajariladi.

Fan mazmunini o‘zlashtirish davomida kursantlar quyidagilardan foydalanish imkoniga egadirlar: videoma’ruzalar; elektron shakldagi ma’ruza matnlari; har bir mavzuga doir prezentatsiya slaydlari; amaliy mashqlarni bajarishga doir uslubiy ko‘rsatmalar; har bir amaliy mashg‘ulot mavzusi yuzasidan topshiriqlar va mashqlar; turli shakldagi darsliklar va qo‘llanmalar.

Ma’ruza mashg‘uloti fan bo‘yicha umumiy nazariy bilimnlarni yetkazish, amaliy

mashg'ulotlar materiallarini o'zlashtirish uchun kerakli nazariy ma'lumotlar bilan tanishtirish maqsadiga ega bo'ladi. Ma'ruza mashg'ulotida o'qitishning faol va interfaol uslublardan keng foydalaniladi. Ma'ruzani o'qish uslubi ma'ruzachi tomonidan aniqlanadi, lekin bunda mashg'ulotda o'rganuvchilarning mashg'ulotlardagi faolligini oshirish, o'z fikrlarini erkin bayon etish malakalarini shakllantirishga qaratilgan usullardan foydalanishga ko'proq e'tibor qaratiladi.

Amaliy mashg'ulotlar C# dasturlash tili asosida algoritmlar, masalalar to'plamini ishlab chiqadi, kursantlarning dastur yaratishdan oldingi bilim ko'nikmalari mustahkamlanadi va amaliy bajarish hamda beriladigan topshiriqlarni mustaqil bajarish ko'nikmalarini hosil qilish maqsadida o'tkaziladi.

Amaliy mashg'ulotlar maxsus jihozlangan o'quv sinf xonalarida o'tkaziladi. Amaliy ko'nikmalar qo'shinlar stajirovkasi va amaliyotini o'tash mobaynida takomillashtiriladi.

Mashg'ulotlarni individuallashtirish va o'qitish sifatini oshirish maqsadida C# dasturlash tili mavzuning qamroviga qarab o'quv guruhlarini bir nechta kichik guruhlariga bo'lishga va alohida o'qituvchi rahbarligidagi o'quv nuqtalariga taqsimlashga ruxsat beriladi. Kursantlar C# dasturlash tilida ishlashga kerak bo'ladigan dasturiy muhitni o'rnatish, dasturiy va apparat vositalar sozlamalarini sozlash hamda ulardan amaliy foydalanish bo'yicha har bir elementlarni bir necha marta bajaradilar, undan keyin o'rnatish va sozlash amallarini mustaqil o'tkazadilar.

Mustaqil o'zlashtiriladigan mavzular bo'yicha kursantlar tomonidan mustaqil ish tayyorlanadi va uni taqdimoti tashkil qilinadi.

Ma'ruza va amaliy mashg'ulotlarining barcha mavzularini to'la o'zlashtirgan kursantlarga yakuniy nazoratda ishtirok etishga ruxsat etiladi. Kursant semestr oxirida yakuniy nazorat topshiradi.

Amaliy mashg'ulot o'tkazish maqsadida kursantlar zamonaviy kompyuterlarda Python dasturlash tilida dastur yaratishadi va dasturlarni tahlilini o'rganishadi.

Amaliy mashg'ulotlar zamonaviy kompyuterlar va multimedia vositalari bilan jihozlangan maxsus o'quv sinf xonalarida o'tkaziladi. Nazariy tajribani va amaliyotni o'tash mobaynida o'z qobiliyatini hamda ko'nikmalarini takomillashtiradi.

Mashg'ulotlarni individuallashtirish va o'qitishni sifatini oshirish maqsadida vositalarning soniga qarab guruhlar bir qancha guruhlariga bo'linadi va ular o'quv joylariga taqsimlanadi.

Amaliy mashg'ulotlarda kursantlar me'yorlarni bajarishda ishtirok etishi maqsadida bellashuv, musobaqa va sog'lom raqobat elementlarini kiritish lozim.

Harbiy tajribani va amaliyotni o'tash mobaynida o'z qobiliyat hamda ko'nikmalarni takomillashtiriladi.

O'quv-tarbiyaviy jarayonini jadallashtirishga qo'yilgan talablar oshishini inobatga olib mashg'ulotlarni tashkil etish va o'tkazish uslubiyatini doimo takomillashtirish lozim.

Mustaqil o'qish soatida kursantlar tavsiya etilgan adabiyotlarni o'rganib, konspektlarini to'ldirib, olgan bilimlarini mustahkamlaydi.

Guruh va yakka tartibdagi konsultatsiyalar kursantlarga o'qituvchilar tomonidan guruhli, amaliy mashg'ulotlarga va imtihonlarga yordam ko'rsatish maqsadida o'tkaziladi.

Kursantlarni bilimini aniqlash joriy va yakuniy nazoratlar baholari orqali amalga oshiriladi. Joriy nazorat o‘tkazish, kursantlar o‘quv materiallarni sifatli o‘zlashtirishlarini to‘liq tekshirish va ularni ishini rag‘batlantirish uchun amalga oshiriladi. U guruh va amaliy mashg‘ulotlar davomida o‘tkaziladi.

Kursantlarning bilimlarini tekshirish to‘rt balli tizimida amalga oshiriladi. Kursantlar bilim darajasining nazorati quyidagi ko‘rinishda amalga oshiriladi:

Joriy nazorat – mashg‘ulot jarayonida doimiy tizimli ravishda **savol-javob, test, amaliy ish usullari bilan olib boriladi.**

Yakuniy nazorat kursantlarni nazariy bilimlari va amaliy tayyorgarligi darajasini tekshirish maqsadida o‘tkaziladi. Sinov va imtihon orqali amalga oshiriladi.

Fan bo‘yicha kursantlarning bilim, ko‘nikma va malakalariga quyidagi talablar qo‘yiladi. Kursant:

Kursant bilimlarga ega bo‘lishi:

– dasturlash qismida dasturlash tilining tuzilmasi (C#), funksiyalari va asosiy parametrlari;

– dasturiy ta‘minot konfiguratsiyasini boshqarish;

– dasturiy ta‘minotni testlash va sifatini ta‘minlash usullari.

Kursant ko‘nikma va malakalarini egallashi:

– qo‘yilgan masalaga mos algoritmlarni tanlash;

– dastur strukturasini ishlab chiqish;

– dasturda xatoliklarni bartaraf etish va boshqarish;

– grafik foydalanuvchi interfeysini shakllantirish va boshqarish.

Kursant kompetensiyalarni egallashi lozim:

– Obyektga yo‘naltirilgan dasturlash bilan ishlash muhitlarni tadbiq qilish;

– Undagi dasturlash tilining sodda va murakkab tuzilmalarini qo‘llash;

– algoritmlarni baholash, qo‘yilgan masalani yechish algoritmini tanlash, tanlovni asoslash va algoritmni tadbiq etish;

– obyektga mo‘ljallangan dasturlash texnologiyalaridan foydalanish.

III. FANNI O‘QUV MASHG‘ULOT TURLARI BO‘YICHA O‘QITISH REJASI

T.r.	O‘quv mashg‘ulot raqami va turlari	Soatlar haimi	Mashg‘ulot mavzusi va o‘quv savollari	Mashg‘ulotning moddiy jihatdan ta‘minlanishi
7- semestr				
1	Ma‘ruza	4	1-Mavzu: C# dasturlash tiliga kirish 1-Mashg‘ulot: C# dasturlash tili va .NET platformasi bilan tanishish. O‘quv savollari: 1. C # tili va .NET platformasi haqida tushuncha. 2. Visual Studio dasturi bilan ishlashni boshlash. 3. Birinchi dastur “salom dunyo” yaratish.	Kompyuter. Interaktiv doska. Taqdimot materiallari.

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

2	Amaliy	2	1-Mavzu: C# dasturlash tili fundamental asoslari. 2-Mashg‘ulot: Arifmetik amallar. Consoleda kiritish va chiqarish. O‘quv savollari: 1. Konsol rejimida kiritish va chiqarish. 2. Arifmetik amallar(qo‘shish, ayirish, ko‘paytirish, bo‘lish, qoldiqli bo‘lish). 3. Bitli ammalar bilan ishlash.	Kompyuter. Multimedia proyektori. Taqdimot materiallari.
3	Amaliy	2	1-Mavzu: C# dasturlash tili fundamental asoslari. 3-Mashg‘ulot: O‘zlashtirish. Inkrement va dekrement amallari. O‘quv savollari: 1. O‘zlashtirish operatorlari. 2. Operatorlarning saralanganligi. 3. Inkrement va dekrementlar amallari.	Kompyuter. Multimedia proyektori. Taqdimot materiallari.
4	Ma’ruza	2	2-Mavzu: C# dasturlash tiliga fundamental asoslari. 1-Mashg‘ulot: C# dasturlash tili asoslari. Dastur strukturasi va o‘zgaruvchilar. O‘quv savollari: 1. Visual Studio dasturi tarkibi. 2. O‘zgaruvchilar haqida tushuncha. 3. Literallar haqida tushuncha va ularni dasturda qo‘llash. 4. Ma’lumotlar turlari.	Kompyuter. Interaktiv doska. Taqdimot materiallari.
5	Amaliy	4	2-Mavzu: C# dasturlash tili fundamental asoslari. 2-Mashg‘ulot: Malumotlari turlarini o‘zgartirish. O‘quv savollari: 1. Mantiqiy operatorlar ustida amallar. 2. Surish operatorlari(<<va>>). 3. Ma’mulot turlarini o‘zgartirish. 4. Aniq konversiyalar. 5. Ma’lumotlarni yo‘qotish va Checked kalit so‘z.	Kompyuter. Multimedia proyektori. Internet, USB-kamera, mikrofon va kolonka.
6	Amaliy	4	2-Mavzu: C# dasturlash tili fundamental asoslari. 3-Mashg‘ulot: Shart operatorlari. O‘quv savollari: 1. Taqqoslash amallari haqida tushuncha. 2. Mantiqiy operatorlari ustida amallar. 3. If / else konstruksiyasi haqida tushuncha va uning ustida amallar 4. Switch konstruksiyasi. 5. Ternar operatsiyalar.	Kompyuter. Multimedia proyektori. Internet, USB-kamera, mikrofon va kolonka.
7	Amaliy	4	2-Mavzu: C# dasturlash tili fundamental asoslari. 4-Mashg‘ulot: Sikl operatorlari. O‘quv savollari: 1. Sikllar haqida tushuncha. 2. For sikliga doir amallar. 3. Do bilan ishlash. 4. While sikli ishlash printsipi. 5. Continue va break operatorlari ustida amallar.	Kompyuter. Multimedia proyektori. Taqdimot materiallari.

OBYEKTGA YO'NALTIRILGAN DASTURLASH

8	Amaliy	4	2-Mavzu: C# dasturlash tili fundamental asoslari. 5-Mashg'ulot: Bir o'lchamli massivlar. O'quv savollari: 1. Massivlar haqida tushuncha. 2. Bir o'lchamli massivlarni yaratish va ular ustida amallar. 3. Bir o'lchamli massivlarni saralash. 4. Bir o'lchamli massivlar bilan ishlashda sikl operatorlaridan foydalanish.	Kompyuter. Multimedia proyektori. Taqdimot materiallari.
9	Amaliy	2	2-Mavzu: C# dasturlash tili fundamental asoslari. 6-Mashg'ulot: Ko'p o'lchamli massivlar. O'quv savollari: 1. Ko'p o'lchamli massivlar. 2. Matritsalar ustida amallar. 3. Rank, Dimension length, array length tushunchalari haqida. 4. Ko'p o'lchamli massivlarni saralash.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
10	Amaliy	4	2-Mavzu: C# dasturlash tili fundamental asoslari. 7-Mashg'ulot: Saralash algoritmlari. O'quv savollari: 1. Pufakchali saralash(bubble sort). 2. Sanab salarash(counting sort). 3. Quick sort(tezkor saralash).	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
11	Amaliy	4	2-Mavzu: C# dasturlash tili fundamental asoslari. 8-Mashg'ulot: Metodlar(funksiyalar). O'quv savollari: 1. Metodlar haqida tushuncha. 2. Metodlarni chaqirish. 3. Qiymat qaytaruvchi metodlar. 4. Metodlarni qisqartirish.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
12	Amaliy	4	2-Mavzu: C# dasturlash tili fundamental asoslari. 9-Mashg'ulot: Parametrli metodlar. O'quv savollari: 1. Formalli va faktli parametrlar. 2. Majbur bo'lmagan parametrli metodlar. 3. Nomlangan parametrli metodlar.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
13	Amaliy	4	2-Mavzu: C# dasturlash tili fundamental asoslari. 10-Mashg'ulot: Rekursiya. Rekursiv funksiyalar. O'quv savollari: 1. Rekursiya haqida tushuncha. 2. Rekursiv funksiyalar va ularning ishlash printsiipi. 3. Factorial ustida amallar. 4. Fibonacci sonlari.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
14	Ma'ruza	2	3-Mavzu: OOP tamoyillari 1-Mashg'ulot: Klasslar, strukturalar va obyektlar haqida tushuncha. O'quv savollari: 1. Klasslar, struktura va obyektlar haqida tushuncha. 2. Konstruktorlar va ularni yaratish. 3. This kalit so'zi.	Kompyuter. Interaktiv doska. Taqdimot materiallari..

OBYEKTGA YO'NALTIRILGAN DASTURLASH

			4. Obyektни ishga tushiruvchilari.	
15	Amaliy	2	3-Mavzu: OOP tamoyillari 2-Mashg'ulot: Klas kutubxonalarini yaratish va ruxsat modifikatorlari. O'quv savollari: 1. Sinf kutubxonasini yaratish. 2. Kirish modifikatorlari.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
16	Amaliy	4	3-Mavzu: OOP tamoyillari 3-Mashg'ulot: Xususiyatlar va metodlarning yuklamalari. O'quv savollari: 1. Xususiyatlar. 2. Ruxsat modifikatorlari. 3. Avtomatik xususiyatlar. 4. Xususiyatlarning qisqartirilgan yozuvi. 5. Yuklama metodlari.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
17	Amaliy	2	3-Mavzu: OOP tamoyillari 4-Mashg'ulot: Statik modifikatori. Konstantalar va o'qish uchun maydonlar. O'quv savollari: 1. Statik a'zolar va statik o'zgartiruvchi. 2. Statik xususiyatlar, usullari, statik konstruktor va statik klasslar. 3. Konstantalar, o'qish uchun maydonlar va strukturalarni o'qish. 4. Operator yuklamalari.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
18	Amaliy	2	3-Mavzu: OOP tamoyillari 5-Mashg'ulot: Null qiymati. Indeksatorlar. O'quv savollari: 1. Null qiymati va Shartli null operatori. 2. Indeksatorlar. 3. Get va set bloklari. 4. Indeksator yuklamalari.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
19	Amaliy	4	3-Mavzu: OOP tamoyillari 6-Mashg'ulot: Voris olish. Ma'lumot turlarini biridan ikkinchisiga o'zgartirish. O'quv savollari: 1. Vorislik va Tayanch sinfdan asosiy sinf a'zolariga kirish. 2. Base kalit so'zi. 3. Sinflardan olingan konstruktorlar va konstruktorni chaqirish tartibi. 4. Turni o'zgartirish. 5. O'zgartirish usullari.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
20	Amaliy	2	3-Mavzu: OOP tamoyillari 7-Mashg'ulot: Metodlar. O'quv savollari: 1. Virtual metod. 2. Yashirish metodi.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
21	Amaliy	2	3-Mavzu: OOP tamoyillari 8-Mashg'ulot: Abstrakt sinflar va ularning a'zolari. O'quv savollari:	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

			1. Abstrakt sinflar a’zolari. 2. Abstrakt metodlar. 3. Abstrakt xususiyatlar.	kursant uchun shaxsiy kompyuter.
22	Amaliy	2	3-Mavzu: OOP tamoyillari 9-Mashg‘ulot: System. Object sinfi va metodlari. O‘quv savollari: 1. GetHshCode metodi. 2. Obyekt turini usulini olish va GetType metodi. 3. Equals metodi.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
23	Amaliy	2	3-Mavzu: OOP tamoyillari 10-Mashg‘ulot: Umumlashtirish. O‘quv savollari: 1. Standart qiymatlar. 2. Umumiy sinf statik maydonlari. 3. Umumiy cheklovlar. 4. Umumiy turdagi merosxo‘rlik.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
24	Ma’ruza	2	4-Mavzu: Winforms komponentalari 1-Mashg‘ulot: Windows formalari bilan tanishish. O‘quv savollari: 1. Windows Forms haqida tushuncha. 2. Grafik dastur yaratish. 3. Dasturni ishga tushirish.	Kompyuter. Interaktiv doska. Taqdimot materiallari.
25	Amaliy	2	4-Mavzu: Winforms komponentalari 2-Mashg‘ulot: Shakllarni qo‘shish. Shakllarning o‘zaro ta’siri. O‘quv savollari: 1. Windows formda hodisalar. Shakl hodisalari. 2. To‘rtburchaklar bo‘lmagan shakllarni yaratish. Shaklni yopish.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
26	Amaliy	2	4-Mavzu: Winforms komponentalari 3-Mashg‘ulot: Windows Forms konteynerlari. O‘quv savollari: 1. Elementlarni dinamik qo‘shish. 2. GroupBox, Panel va FlowLayoutPanel elementlari. 3. TableLayoutPanel elementi.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
27	Amaliy	2	4-Mavzu: Winforms komponentalari 4-Mashg‘ulot: Elementlarning o‘lchamlari va ularni konteynerga joylashtirish. O‘quv savollari: 1. Joylashuv. 2. O‘lchamlarni sozlash. 3. Anchor xususiyatlari. 4. Dock xususiyati.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
28	Amaliy	2	4-Mavzu: Winforms komponentalari 5-Mashg‘ulot: TabControl va SplitContainer tab paneli. O‘quv savollari: 1. TabControl. 2. Koddagi yorliqlarni boshqarish. 3. SplitContainer.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

29	Amaliy	2	4-Mavzu: Winforms komponentalari 6-Mashg‘ulot: Boshqaruv elementlari: Tugma. O‘quv savollari: 1. Tugmani bezatish. 2. Tugma ustidagi rasm. 3. Standart tugmalar.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
30	Amaliy	2	4-Mavzu: Winforms komponentalari 7-Mashg‘ulot: Boshqaruv elementlari: Teglar va manzillar(silka). O‘quv savollari: 1. Label 2. LinkLabel 3. LinkClicked	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
31	Amaliy	2	4-Mavzu: Winforms komponentalari 8-Mashg‘ulot: Boshqaruv elementlari: MaskedTextBox elementi. O‘quv savollari: 1. Avtomatik to‘ldirilgan matnli maydon. 2. So‘zlar bo‘yicha ko‘chirish. 3. Parol kiritish, TextChanged hodisasi. 4. ToolStrip asboblari paneli.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
32	Amaliy	2	4-Mavzu: Winforms komponentalari 9-Mashg‘ulot: Boshqaruv elementlari: MaskedTextBox, RadioButton va CheckBox. O‘quv savollari: 1. MaskedTextBox elementi. 2. RadioButton elementi. 3. CheckBox elementi.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
33	Amaliy	2	4-Mavzu: Winforms komponentalari 10-Mashg‘ulot: Boshqaruv elementlari: ListBox. O‘quv savollari: 1. ListBox -dagi dasturiy elementlar. 2. ListBoxda SelectedIndexChanged hodisasi.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
34	Amaliy	2	4-Mavzu: Winforms komponentalari 11-Mashg‘ulot: Shakllarni qo‘shish. Shakllarning o‘zaro ta’siri O‘quv savollari: 1. Windows formda hodisalar. Shakl hodisalari. 2. To‘rtburchaklar bo‘lmagan shakllarni yaratish. Shaklni yopish.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
JAMI:				90 soat
8- semestr				
1	Ma’ruza	4	5-Mavzu: Winforms topologiyasi 1-Mashg‘ulot: Windows formalar ustida amallar. O‘quv savollari: 1. Windows Forms haqida tushuncha. 2. Grafik dastur yaratish. 3. Dasturni ishga tushirish.	Kompyuter. Interaktiv doska. Taqdimot materiallari.
2	Ma’ruza	2	6-Mavzu: WinForms komponentalari. 1-Mashg‘ulot: Shakllar bilan ishlash. O‘quv savollari: 1. Shakl asoslari va shakllarning asosiy xususiyatlari.	Kompyuter. Interaktiv doska. Taqdimot materiallari.

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

			2. Dasturiy xususiyatlarni sozlash. 3. Shaklning o‘lchamlarini sozlash. 4. Shaklning boshlang‘ich pozitsiyasi. 5. Shaklning rang va fonni.	
3	Amaliy	4	6-Mavzu: WinForms komponentalari. 2-Mashg‘ulot: Boshqaruv elementlari: ComboBox. O‘quv savollari: 1. ComboBox tashqi ko‘rinishini sozlash. 2. ComboBoxda SelectedIndexChanged hodisasi. 3. LinkLabel. 4. LinkClicked.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
4	Amaliy	4	6-Mavzu: WinForms komponentalari. 3-Mashg‘ulot: Boshqaruv elementlari: ListBox va ComboBoxda ma’lumotlarni bog‘lash. O‘quv savollari: 1. DataSource ma’lumotlar manbai. 2. DisplayMember: ob’ektning xususiyati. 3. ValueMember: ob’ektning xususiyati.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
5	Amaliy	4	6-Mavzu: WinForms komponentalari. 4-Mashg‘ulot: Boshqaruv elementlari: Checked-ListBox. O‘quv savollari: 1. Items xususiyatlari. 2. SetItemChecked va SetItemCheckState.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
6	Amaliy	2	6-Mavzu: WinForms komponentalari. 5-Mashg‘ulot: Boshqaruv elementlari: NumericUpDown va DomainUpDown. O‘quv savollari: 1. NumericUpDown hodisasi. 2. DomainUpDown hodisasi. 3. ImageList elementi	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
7	Amaliy	4	6-Mavzu: WinForms komponentalari. 6-Mashg‘ulot: Boshqaruv elementlari: ListView. O‘quv savollari: 1. ListViewItem hodisasi. 2. Element tasvirlari. 3. Ko‘rsatish turlari. 4. ListView. Amaliyot.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
8	Amaliy	4	6-Mavzu: WinForms komponentalari. 7-Mashg‘ulot: Boshqaruv elementlari: TreeView. O‘quv savollari: 1. Dasturiy tugunlarni boshqarish. 2. Tugunlarni yashirish va ochish. 3. CheckBox va tasvirni qo‘shish.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
9	Amaliy	4	6-Mavzu: WinForms komponentalari. 8-Mashg‘ulot: Boshqaruv elementlari: TrackBar, Timer va ProgressBar. O‘quv savollari: 1. TrackBar elementi. 2. Timer elementi. 3. ProgressBar elementidagi indikator hodisasi.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

10	Amaliy	4	6-Mavzu: WinForms komponentalari. 9-Mashg‘ulot: Boshqaruv elementlari: DateTime-Picker va MonthCalendar. O‘quv savollari: 1. DateTimePicker elementi. 2. MonthCalendar elementi.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
11	Amaliy	4	6-Mavzu: WinForms komponentalari. 10-Mashg‘ulot: Boshqaruv elementlari: PictureBox. O‘quv savollari: 1. ImageLocation, IntialImage xususiyatlari. 2. Rasm o‘lchami.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
12	Amaliy	4	6-Mavzu: WinForms komponentalari. 11-Mashg‘ulot: Boshqaruv elementlari: Web-Browser, NotifyIcon va MessageBox xabarlar oynasi. O‘quv savollari: 1. WebBrowser. 2. NotifyIcon. 3. MessageBox xabarlar oynasi.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
13	Amaliy	4	6-Mavzu: WinForms komponentalari. 12-Mashg‘ulot: Boshqaruv elementlari: OpenFileDialog, SaveFileDialog, FontDialog va ColorDialog. O‘quv savollari: 1. OpenFileDialog funksiyasi bilan ishlash. 2. SaveFileDialog funksiyasi bilan ishlash. 3. FontDialog va ColorDialog funksiya bilan ishlash.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
14	Amaliy	4	6-Mavzu: WinForms komponentalari. 13-Mashg‘ulot: ErrorProvider xususiyati. O‘quv savollari: 1. BlinkRate: xato belgisi 2. BlinkStyle: xato belgisi	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
15	Amaliy	4	6-Mavzu: WinForms komponentalari. 14-Mashg‘ulot: Menyular va asboblar paneli. O‘quv savollari: 1. ToolStrip asboblar paneli 2. MenuStrip yaratish	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
16	Amaliy	4	6-Mavzu: WinForms komponentalari. 15-Mashg‘ulot: StatusStrip va ContextMenuStrip menyulari. O‘quv savollari: 1. StatusStrip qator holati. 2. ContextMenuStrip kontekst menyusi.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
JAMI:				60 soat
9- semestr				
1	Ma’ruza	2	7-Mavzu: C# dasturlash tilida tarmoq nazariyasi 1-Mashg‘ulot: C# va .NET da tarmoq bilan ishlash asoslari. O‘quv savollari: 1. Tarmoq haqia tushuncha va tarmoq protokollari. 2. .NET da adreslar haqida tushuncha(IP adres, URI). 3. Addreslash sxemasi. 4. URI adresslar haqida tushuncha.	Kompyuter. Interaktiv doska. Taqdimot materiallari.

OBYEKTGA YO'NALTIRILGAN DASTURLASH

2	Ma'ruza	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 1-Mashg'ulot: Tarmoq konfiguratsiyasi va Soket klassi. O'quv savollari: 1. DNS. 2. Tarmoq konfiguratsiyasi va tarmoq trafigi haqida ma'lumot olish. 3. Soket klassi.	Kompyuter. Interaktiv doska. Taqdimot materiallari.
3	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 2-Mashg'ulot: HTTP protokoli. O'quv savollari: 1. HTTP protokoli bilan tanishish. 2. HTTP status kodlari. 3. HttpClient yaratish.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
4	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 3-Mashg'ulot: So'rovlar yuborish. O'quv savollari: 1. HttpClient bilan so'rovlarni yuborish. 2. SendAsync metodi orqali so'rovlar jo'natish. 3. JSON formatida ma'lumotlarni olish.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
5	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 4-Mashg'ulot: Sarlavhalarni yuborish va qabul qilish. O'quv savollari: 1. Sarlavhalarni yuborish va qabul qilish haqida tushuncha. 2. So'rov uchun "sarlavha" o'rnatish. 3. So'rovda ma'lumotlarni yuborish.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
6	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 5-Mashg'ulot: HttpClient orqali JSON malumotlarni yuborish. O'quv savollari: 1. HttpClient bilan json yuborish. 2. HttpClient-ning Web API bilan o'zaro ta'siri. 3. Malumotlar bilan ishlash.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
7	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 6-Mashg'ulot: FormUrlEncodedContent klassi O'quv savollari: 1. Shakllar va FormUrlEncodedContent sinfini yuborish. 2. Oqimlar va bayt massivini yuborish. 3. Baytlar massivi yuborish(ByteArrayContent).	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
8	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 7-Mashg'ulot: MultipartFormDataContent klassi. O'quv savollari:	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

			1. Fayllarni yuborish va MultipartFormDataContent klassi. 2. Birdaniga bir nechta fayllar yuborish. 3. HttpClient yordamida cookie-fayllarni yuborish va qabul qilish.	
9	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 8-Mashg‘ulot: Kukilarni yuborish va HttpListener. HTTP serveri. O‘quv savollari: 1. Kukilarni yuborish. 2. HttpListener. 3. So‘rov haqida ma’lumotlarni olish.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
10	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 9-Mashg‘ulot: Soketlarda TCP-klient. O‘quv savollari: 1. Soketlarda TCP mijozi. 2. Hostdan ma’lumotlarni olish. 3. Avialable xususiyati.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
11	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 10-Mashg‘ulot: Soketlarda TCP Server. O‘quv savollari: 1. Soketlarda TCP serveri haqida tushuncha. 2. NetworkStream. 3. Ma’lumotlarni o‘qish.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
12	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 11-Mashg‘ulot: TCP protokoli. O‘quv savollari: 1. TCP mijozi. TcpClient klassi. 2. Ma’lumot yuborish va qabul qilish. 3. TCP serveri. TcpListener sinfi.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
13	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 12-Mashg‘ulot: Ma’lumotlarni yuborish va qabul qilish. O‘quv savollari: 1. Ma’lumotlarni yuborish va qabul qilish. Soketlar orasidagi bir tomonlama aloqa. 2. Ma’lumotlarni olishning usullari. 3. Ma’lumotlarni yuborish va qabul qilish. TcpListener va TcpClient o‘rtasida bir tomonlama aloqa.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
14	Amaliy	2	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 13-Mashg‘ulot: HTTP protokoli. O‘quv savollari: 1. Ma’lumotlarni yuborish va qabul qilish. Ikki tomonlama aloqa. 2. Ko‘p tarmoqli TCP mijoz-server haqida tushuncha.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.

OBYEKTGA YO‘NALTIRILGAN DASTURLASH

			3. Ko‘p tarmoqli TCP mijoz-server ilovasi.	
15	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 14-Mashg‘ulot: NetworkStream klassi. O‘quv savollari: 1. NetworkStream va matn oqimlari. 2. NetworkStream va ikkilik oqimlar. 3. Konsol TCP suhbatlari.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
16	Amaliy	4	8-Mavzu: C# dasturlash tilida tarmoq amaliyoti komponentalari 15-Mashg‘ulot: UDP protokoli. O‘quv savollari: 1. UDP bilan ishlash uchun rozetkalardan foydalanish. 2. UdpClient. 3. Konsol UDP suhbatlari. 4. Translyatsiya.	Kompyuter. Interaktiv doska. Taqdimot materiallari. Har bir kursant uchun shaxsiy kompyuter.
JAMI:				60 soat

IV. MUSTAQIL TA’LIM VA MUSTAQIL ISHLAR

T/r	Mustaqil tayyorgarlik mavzulari	Soatlar hajmi
7- semestr. Mustaqil ta’lim (MT)		
1.	C# dasturlash tilida consoleda arifmetik amallar ustida misollar yechish	20
2.	C# dasturlash tilida consoleda string ma’lumotlar bilan ishlash	20
3.	SOLID va patternlarni o‘rganish	25
4.	DRY prinsipi va uni amalda qo‘llash	25
JAMI:		90 soat
8- semestr. Mustaqil ta’lim (MT)		
5.	Kalkulyator yaratish dizaynini yaratish	15
6.	Kalkulyator kodini yozish	15
7.	Bloknot (notepad) dizaynini yaratish	15
8.	Bloknot (notepad) kodini yozish	15
JAMI:		60 soat
9- semestr. Mustaqil ta’lim (MT)		
9.	Tarmoq orqali o‘zaro malumot almashish	15
10.	Tarmoqlararo fayllarni yuborish	15
11.	Mini chat dizaynini yaratish	15
12.	Mini chat dasturini yaratish	15
JAMI:		60 soat

Mustaqil o‘zlashtiriladigan mavzular bo‘yicha kursantlar tomonidan (referat, prezentatsiya, esse, mustaqil (ijodiy) ish, muammoli ma’ruza va boshqalar) tayyorlanadi va uni taqdimoti tashkil qilinadi.

V. FAN BO'YICHA KURSANTLAR BILIMINI BAHOLASH VA NAZORAT QILISH ME'ZONLARI

Baholash tartibi

Kursantlarning bilim saviyasi, ko'nikma va malakalarini nazorat qilishning reyting tizimi asosida kursantning har bir fan bo'yicha o'zlashtirish darajasi ballar orqali ifodalanadi.

Har bir fan bo'yicha kursantning semestr davomidagi o'zlashtirish ko'rsatkichi **100 ballik tizimda** butun sonlar bilan baholanadi.

Fanning xususiyatidan kelib chiqqan holda nazorat turlari bo'yicha quyidagicha taqsimlanadi:

joriy, oraliq va yakuniy nazorat:

joriy nazoratga - 40 ball;

oraliq nazoratga - 20 ball;

yakuniy nazoratga - 40 ball;

Fanning xususiyatidan kelib chiqqan holda joriy nazorat uchun ajratilgan maksimal ball kursantlarning bilim va ko'nikmalarini, mashg'ulotlardagi faolligini, bajarilgan amaliy topshiriqlarni kundalik mashg'ulotlar davomida joriy baholash hamda ular tomonidan bajarilgan mustaqil ta'lim topshiriqlarini baholashga quyidagicha bo'linadi:

Kursantlarning bilim va ko'nikmalari, mashg'ulotlardagi faolligi kundalik mashg'ulotlar davomida joriy baholash 5 ballik (0-5 ball) tizimda butun sonlar bilan baholanadi.

5 ball - kursant mavzuga oid materiallarga doir bilimlarini chuqur namoyon etib, ularni bilimi bilan va mantiqan to'g'ri bayon etsa, mustaqil xulosa va to'g'ri qaror qabul qilsa, ijodiy fikrlab mustaqil mushohada yurita olsa, mavzuning mohiyatini chuqur tushunib bilsa va ifodalay olganda;

4 ball - kursant mavzuga oid materiallari puxta bilib, ularni mantiqan to'g'ri bayon etsa, bergan javoblarida sezilarli noaniqliklarga yo'l qo'ymagan bo'lsa, mustaqil mushohada yuritsa, mavzuning mohiyatini tushunib bilsa va ifodalay olganda;

3 ball - kursant mavzuga oid materialning asosiy qismini bilib, tafsilotlarini o'zlashtirib olmagan, lekin bergan javoblarida qo'pol xatoliklarga yo'l qo'ymagan bo'lsa, to'g'ri qaror qabul qilishi uchun ayrim hollarda unga yordamchi (esga soluvchi) savollar berilishi zarur bo'lsa, mavzuning mohiyatini tushunsa va ifodalay olganda;

2 ball - kursant mavzuga materialning asosiy qismini bilmasa yoki bilib, tafsilotlarini o'zlashtirib olmagan, bergan javoblarida qo'pol xatoliklarga yo'l qo'ygan bo'lsa, olgan bilimni amalda qo'llay olishni mukammal bilmaganda.

0-1 ball - kursant mavzuga materialning asosiy qismini bilmasa yoki bilib, tafsilotlarini o'zlashtirib olmagan, bergan javoblarida pala-partishlik bo'lsa, qo'pol xatoliklarga yo'l qo'yganda;

joriy nazoratga maksimal 40 ball ajratilganda:

kundalik mashg'ulotlar davomida joriy baholashga - 30 ball;

mustaqil ta'lim topshiriqlarini baholashga - 10 ball;

Semestr yakunida kursantning kundalik mashg'ulotlar davomida **joriy baholash** bo'yicha yig'gan ballini hisoblashda uning mashg'ulotlar davomida va laboratoriya (hisob-grafika) ishlari bo'yicha olgan ballar yig'indisi kursant baholangan mashg'ulotlar soni yig'indisiga bo'linadi va mazkur nazorat turiga ajratilgan maksimal balldan kelib chiqqan holda belgilanadigan koeffitsiyentga ko'paytiriladi:

$$KJ = \frac{J}{M+L} * Q$$

jumladan:

KJ - kursantning kundalik mashg'ulotlar davomida joriy baholash bo'yicha to'plagan bali;

J - kursantning mashg'ulotlar davomida va hisob-grafika ishlari bo'yicha olgan ballari yig'indisi;

M - kursant baholangan mashg'ulotlar soni (faqat kursant baholangan mashg'ulotlar soni ko'rsatiladi);

L - o'tkazilgan hisob-grafika ishlari soni (ishchi o'quv dasturiga ko'ra semestrda rejalashtirilgan barcha laboratoriya (hisob-grafika) ishlari soni ko'rsatiladi) agar berilmasa $L=0$;

Q - ajratilgan maksimal balndan kelib chiqqan holda belgilanadigan koeffitsiyent (joriy nazoratning mazkur turiga ajratilgan maksimal ball 30 ball bo'lganda koeffitsiyent - 6).

Kursantlar tomonidan **mustaqil ta'lim** mavzulari bo'yicha bajarilgan mustaqil ta'lim topshiriqlarini baholash 5 ballik tizimda butun sonlar bilan quyidagicha baholanadi:

5 ball – topshiriqqa doir bilimlar to'liq bayon etilgan, amalda qo'llay olishini to'g'ri va ishonch bilan ifodalangan;

4 ball – topshiriqqa doir bilimlar bayon etilgan, amalda qo'llay olishida ayrim noaniqliklarga yo'l qo'ygan holda ifodalangan;

3 ball – topshiriqqa doir bilimlar bayon etilgan, amalda qo'llay olishida sezilarli noaniqliklarga yo'l qo'ygan holda ifodalangan;

2 ball – topshiriqqa doir bilimlar juda kam darajada bayon qilingan, amalda qo'llay olishida xatolarga yo'l qo'ygan holda ifodalangan;

0-1 ball - kursant mavzuga materialning asosiy qismini bilmasa yoki bilib, tafsilotlarini o'zlashtirib olmagan, bergan javoblarida pala-partishlik bo'lsa, qo'pol xatoliklarga yo'l qo'yganda;

Kursantlar har bir mustaqil ta'lim mavzusi bo'yicha navbatdagi mustaqil ta'lim mavzusi topshiriqlari berilgunga qadar semestrda rejalashtirilgan so'nggi mustaqil ta'lim mavzusi bo'yicha esa attestatsiya sessiyasi boshlangunga qadar baholanishlari lozim.

Semestr yakunida kursantning mustaqil ta'lim mavzulari bo'yicha yig'gan balni hisoblashda, uning mustaqil ta'lim topshiriqlari bo'yicha olgan ballari yig'indisi ishchi o'quv dasturiga ko'ra semestrda rejalashtirilgan mustaqil ta'lim mavzulari soniga bo'linadi va mazkur nazorat turiga ajratilgan maksimal balndan kelib chiqqan holda belgilanadigan koeffitsiyentga ko'paytiriladi:

$$MJ = \frac{MI}{MT} * Q$$

bu yerda:

MJ - kursantning mustaqil ta'lim mavzulari bo'yicha to'plagan balli;

MI - kursantning mustaqil ta'lim topshiriqlari bo'yicha olgan ballar yig'indisi;

MT - mustaqil ta'lim mavzulari soni (ishchi o'quv dasturiga ko'ra semestrda rejalashtirilgan barcha mustaqil ta'lim mavzulari soni ko'rsatiladi);

Q - ajratilgan maksimal balldan kelib chiqqan holda belgilanadigan koeffitsiyent (joriy nazoratning mazkur turiga ajratilgan maksimal ball 10 ball bo'lganda koeffitsiyent - 2).

Semestr yakunida kursantning joriy baholash bo'yicha to'plagan umumiy bali, uning kundalik mashg'ulotlar davomida joriy baholash bo'yicha va mustaqil ta'lim mavzulari bo'yicha to'plagan ballari yig'indisi bo'yicha chiqariladi:

$$JB = KJ + MJ$$

bu yerda:

JB - semestr yakunida kursantning joriy baholash bo'yicha to'plagan umumiy balli;

KJ - kursantning kundalik mashg'ulotlar davomida joriy baholash bo'yicha to'plagan balli;

MJ - kursantning mustaqil ta'lim mavzulari bo'yicha to'plagan balli.

Kursantning joriy baholash bo'yicha to'plagan umumiy ballini guruh jurnali, reyting qaydnomasi va reyting daftarchasiga qayd etishda yaxlitlanib, butun son ko'rinishida qo'yiladi. Bunda, 0,5 va undan katta bo'lgan o'nliklar yuqoriga, 0,4 va undan kichik bo'lgan o'nliklar pastga yaxlitlanadi.

Oraliq nazoratlarda kursantlarning bilimiga belgilanadigan umumiy ball har bir savolga berilgan javoblar uchun qo'yilgan alohida ballar yig'indisiga asosan chiqariladi. Yozma ish shaklida qabul qilinadigan oraliq nazoratlarda 2 ta savol, har bir savolga 10 balidan baholanganda kursant tomonidan to'plangan butun bo'lmagan ballar yuqoriga yaxlitlanadi.

Kursantlarning bilim va amaliy ko'nikmalari darajasining **yakuniy nazoratini** o'tkazishda yakuniy nazorat biletlaridagi 4 ta savolning har biri 10 ballik (0-10 ball) tizimda butun sonlar bilan baholanadi.

Yakuniy (oraliq) nazoratni baholash quyidagi mezonlarga asoslanadi:

A'lo – 9-10 ball – kursant dasturiy materiallarga doir bilimlarini chuqur namoyon etib, ularni bilimi bilan va mantiqan to'g'ri bayon etsa, mustaqil xulosa va to'g'ri qaror qabul qilsa, ijodiy fikrlab mustaqil mushohada yurita olsa, olgan bilimini amalda qo'llay olishni namoyon qilsa, fanning mohiyatini chuqur tushunib bilsa va ifodalay olsa hamda fan bo'yicha yetarli darajada tasavvurga ega deb topilganda;

Yaxshi - 7-8 ball – kursant dasturiy materiallari puxta bilib, ularni mantiqan to'g'ri bayon etsa, bergan javoblarida sezilarli noaniqliklarga yo'l qo'ymagan bo'lsa, mustaqil mushohada yuritsa, olgan bilimini amalda qo'llay olishni namoyon qilsa, fanning mohiyatini tushunib bilsa va ifodalay olsa hamda fan bo'yicha tasavvurga ega deb topilganda;

qoniqarli - 5-6 ball – kursant dasturiy materialning asosiy qismini bilib, tafsilotlarini o'zlashtirib olmagan, lekin bergan javoblarida qo'pol xatoliklarga yo'l qo'ymagan bo'lsa, to'g'ri qaror qabul qilishi uchun ayrim hollarda unga yordamchi (esga soluvchi) savollar berilishi zarur bo'lsa, olgan bilimini amalda qo'llay olishni bilsa, fanning mohiyatini tushunsa va ifodalay olsa hamda fan bo'yicha tasavvurga ega deb topilganda;

Qoniqarsiz 0-4 ball – kursant dasturiy materialning asosiy qismini bilmasa yoki bilib, tafsilotlarini o'zlashtirib olmagan, bergan javoblarida qo'pol xatoliklarga yo'l qo'ygan bo'lsa, olgan bilimini amalda qo'llay olishni mukammal bilmasa.

Yakuniy nazoratlarda kursantlarning bilimiga belgilanadigan umumiy ball har bir savolga berilgan javoblar uchun qo'yilgan alohida ballar yig'indisiga asosan chiqariladi.

Kursantning semestr davomida fan bo'yicha to'plagan umumiy bali har bir nazorat turidan belgilangan qoidalariga muvofiq to'plagan ballari yig'indisiga teng.

Kursantlar tegishli fan bo'yicha yakuniy nazorat o'tkaziladigan muddatga qadar joriy va oraliq nazoratlari topshirgan bo'lishlari shart.

Kursantlar fan bo'yicha yakuniy nazorat o'tkaziladigan muddatga qadar joriy nazoratlarni topshirgan bo'lishlari shart.

Joriy nazorat turlari bo'yicha 55 va undan yuqori ballni to'plagan kursant fanni o'zlashtirgan deb hisoblanadi va ushbu fan bo'yicha yakuniy nazoratga kirmasligiga yo'l qo'yiladi.

Fan bo'yicha joriy va oraliq nazoratlarga ajratilgan umumiy ballning **55 foizi (33 ball)** saralash ball hisoblanib, ushbu foizdan kam ball to'plagan kursantlar yakuniy nazoratga **kiritilmaydi**.

Kursantning bilimini (og'zaki javobi, yozma ishi, amaliy harakatlari, bo'linmani boshqarish mobaynida bajargan harakatlari va shu kabi boshqa ishlar bajarganligini) baholashda quyidagi namunaviy mezonlar tavsiya etiladi:

86-100 ball (a'lo), agar kursant dasturiy materiallarga doir bilimlarini chuqur namoyon etib, ularni bilimi bilan va mantiqan to'g'ri bayon etsa, mustaqil xulosa va to'g'ri qaror qabul qilsa, ijodiy fikrlab mustaqil mushohada yurita olsa, olgan bilimini amalda qo'llay olishni namoyon qilsa, fanning mohiyatini chuqur tushunib bilsa va ifodalay olsa hamda fan bo'yicha yetarli darajada tasavvurga ega deb topilganda;

71-85 ball (yaxshi), agar kursant dasturiy materiallari puxta bilib, ularni mantiqan to'g'ri bayon etsa, bergan javoblarida sezilarli noaniqliklarga yo'l qo'ymagan bo'lsa, mustaqil mushohada yuritsa, olgan bilimini amalda qo'llay olishni namoyon qilsa, fanning mohiyatini tushunib bilsa va ifodalay olsa hamda fan bo'yicha tasavvurga ega deb topilganda;

55-70 ball (qoniqarli), agar kursant dasturiy materialning asosiy qismini bilib, tafsilotlarini o'zlashtirib olmagan, lekin bergan javoblarida qo'pol xatoliklarga yo'l qo'ymagan bo'lsa, to'g'ri qaror qabul qilishi uchun ayrim hollarda unga yordamchi (esga soluvchi) savollar berilishi zarur bo'lsa, olgan bilimini amalda qo'llay olishni bilsa, fanning mohiyatini tushunsa va ifodalay olsa hamda fan bo'yicha tasavvurga ega deb topilganda;

0-54 ball (qoniqarsiz), agar kursant dasturiy materialning asosiy qismini bilmasa yoki bilib, tafsilotlarini o'zlashtirib olmagan, bergan javoblarida qo'pol xatoliklarga yo'l qo'ygan bo'lsa, olgan bilimini amalda qo'llay olishni mukammal bilmasa.

Fan bo'yicha o'tkazilgan joriy va yakuniy nazorat turlari bo'yicha to'plangan ballar yig'indisi 55 balldan kam bo'lsa, kursant akademik qarzdor hisoblanadi.

VI. ASOSIY VA QO'SHIMCHA O'QUV ADABIYOTLAR HAMDA AXBOROT MANBAALARI

Asosiy adabiyotlar

1. Джозеф Албахари, Бен Албахари. C# 7.0 Карманный справочник: Учебное пособие, Перевод с англ. под ред. Ю.Н. Артеменко 2017 г. 174 с.
2. Програмируем на C# 8.0: Разработка приложений – СПб: Питер, 2021 г. 944 с.

Tavsiya qilinadigan qo'shimcha adabiyotlar

Программирование на C# для начинающих. Основные сведения /Алексей Васильев. – Москва, : Эксмо, 2018 г. 592 с.

A.A. Raximov, O.Sh. Abdiroziqov, B.K. Yusupov "Dasturlash texnologiyalari" fanidan Darslik. AKT va AHI. 84 bet, Toshkent, 2020 y.

Tavsiya qilinadigan Internet saytlar

1. <https://metanit.com/sharp/tutorial> – C# dasturlash bo'yicha darslar web sayt
2. www.stackoverflow.com – Dasturlash bo'yicha muammoli savollarga javoblar
3. www.mycsharp.ru - C# dasturlash bo'yicha darslar web sayt
4. <https://docs.microsoft.com/en-us/dotnet/csharp/> - Microsoft kompaniyasi C# guide

**O‘ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA MUDOFAA
UNIVERSITETINING
AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**

**“AXBOROT TEXNOLOGIYALARI VA SUN’IY INTELLEKT”
KAFEDRASI**



**“OBYEKTGA YO‘NALTIRILGAN DASTURLASH”
FANIDAN**

**ASOSIY VA QO‘SHIMCHA
ADABIYOTLAR VA AXBOROT
MANBAALARI (4-ILOVA)**

ASOSIY VA QO'SHIMCHA O'QUV ADABIYOTLAR HAMDA AXBOROT MANBAALARI

Asosiy darslik va o'quv qo'llanmalar

1. O'zbekiston Respublikasining Mudofaa doktrinasini. - T., 2018 y. – B. 54-86.
2. O'zbekiston Respublika Konstitutsiyasi. – Toshkent: O'zbekiston, NMIU, 2017 y. 75 b. (41 modda).
3. Джозеф Албахари, Бен Албахари. С# 7.0 Карманный справочник: Учебное пособие, Перевод с англ. под ред. Ю.Н. Артеменко 2017 г. 174 с.
4. Программируем на С# 8.0: Разработка приложений – СПб: Питер, 2021 г. 944 с.
5. A.A. Raximov. B.K. Yusupov, O.Sh. Abdiroziqov. "Dasturlash texnologiyalari" fanidan amaliy mashg'ulotlarni bajarish bo'yicha uslubiy ko'rsatma. AKT va AHI. 74 bet, Toshkent, 2020 y.

Qo'shimcha adabiyotlar

1. O'zbekiston Respublikasi Prezidentining 2019 yil 17 yanvardagi PQ-4122 sonli "Axborot-kommunikatsiya texnologiyalari va aloqa sohasida ofitser kadrlar tayyorlash tizimini yanada takomillashtirish chora-tadbirlari to'g'risidagi" qarori.
2. Программирование на С# для начинающих. Основные сведения /Алексей Васильев. – Москва, : Эксмо, 2018 г. 592 с.
3. A.A. Raximov. B.K. Yusupov, O.Sh. Abdiroziqov. "Dasturlash texnologiyalari" fanidan o'quv qo'llanma. AKT va AHI. 83 bet, Toshkent, 2020 y.

Internet saytlari

1. <https://mentanit.com/sharp/windowsforms/>
2. <https://dot-net.uz/>
3. <https://dotnet.microsoft.com/>
4. <https://code.mu/>
5. <https://w3school.ru/>
6. <https://codeforces.com777>
7. <https://robocontest.uz/>
8. [https:// https://www.c-sharpcorner.com//](https://www.c-sharpcorner.com//)

